

emUSB-Host

CPU independent USB Host
stack for embedded applications

User Guide & Reference Manual

Document: UM10001
Software Version: 2.48.1
Revision: 0
Date: September 29, 2025



A product of SEGGER Microcontroller GmbH

www.segger.com

Disclaimer

The information written in this document is assumed to be accurate without guarantee. The information in this manual is subject to change for functional or performance improvements without notice. SEGGER Microcontroller GmbH (SEGGER) assumes no responsibility for any errors or omissions in this document. SEGGER disclaims any warranties or conditions, express, implied or statutory for the fitness of the product for a particular purpose. It is your sole responsibility to evaluate the fitness of the product for any specific use.

Copyright notice

You may not extract portions of this manual or modify the PDF file in any way without the prior written permission of SEGGER. The software described in this document is furnished under a license and may only be used or copied in accordance with the terms of such a license.

© 2010-2025 SEGGER Microcontroller GmbH, Monheim am Rhein / Germany

Trademarks

Names mentioned in this manual may be trademarks of their respective companies.

Brand and product names are trademarks or registered trademarks of their respective holders.

Contact address

SEGGER Microcontroller GmbH

Ecolab-Allee 5
D-40789 Monheim am Rhein

Germany

Tel. +49 2173-99312-0
Fax. +49 2173-99312-28
E-mail: ticket_emusb@segger.com*
Internet: www.segger.com

*By sending us an email your (personal) data will automatically be processed. For further information please refer to our privacy policy which is available at <https://www.segger.com/legal/privacy-policy/>.

Manual versions

This manual describes the current software version. If you find an error in the manual or a problem in the software, please inform us and we will try to assist you as soon as possible. Contact us for further information on topics or functions that are not yet documented.

As of version 2.00 the history has been reset. Older history entries can be found in older versions of this document.

Print date: September 29, 2025

Software	Date	By	Description
2.48.1	2025-09-29	RH	Chapter "FT232": • Added function <code>USBH_FT232_ConfigurePollDelay()</code>
2.48.0	2025-08-12	RH	Chapter "Video": • Added function <code>USBH_VIDEO_GetStreamState()</code> • Added function <code>USBH_VIDEO_RestartStream()</code> Chapter "Human Interface Device (HID) class": • Added function <code>USBH_HID_SuspendResume()</code>
2.46.0	2025-06-13	SR	Added new chapter <i>CH34X Device Driver</i> .
2.44.0	2024-02-25	RH	Update to latest software version.
2.42.0	2024-06-26	RH	Added section <i>PSOC Edge E84 driver</i> .
2.40.0	2024-03-18	RH	Changed API of AUDIO class for easier use. Changed function <code>USBH_Cypress_PSoC_DMA_Add()</code> .
2.36.5	2023-11-14	RH	Updated section <i>Cypress PSoC 6 driver</i>
2.36.4	2023-09-28	RH	Added function <code>USBH_LAN_SetOnAddRemove()</code>
2.36.3	2023-06-21	RH	Added missing AUDIO functions. Added function <code>USBH_ConfigOvercurrentDetection()</code>
2.36.2	2023-01-16	RH	Update to latest software version.
2.36.1	2022-11-15	YR	Update to latest software version.
2.36.0	2022-07-28	YR	Added new chapter "Video". Updated section <i>Cypress PSoC 6 driver</i> .
2.34.0	2022-04-13	RH	Update to latest software version.
2.32.1	2021-12-03	YR	Update to latest software version.
2.32.0	2021-09-24	RH	Chapter "USB Host Core": • Added function <code>USBH_SetOnPortEvent()</code>
2.30.0	2021-07-14	RH SR	Chapter "Compile-time configuration" rewritten. Chapter "HID": • Added function <code>USBH_HID_GetReportEP0()</code> Added new chapter "FT260".
2.28.0	2021-04-09	YR	Chapter "USB Host Core": • Added function <code>USBH_GetIADInfo()</code> Chapter "HID": • Added function <code>USBH_HID_SetOnRCStateChange()</code> Chapter "Debugging": • Logging functionality was reworked, see function <code>USBH_ConfigMsgFilter()</code> for details. Update to latest software version.
2.26.1	2020-12-15	YR	Chapter "Printer class": • Added function <code>USBH_PRINTER_AddCustomDeviceMask()</code> Chapter "CDC": • Added function <code>USBH_CDC_GetMaxTransferSize()</code> Chapter "Bulk": • Added function <code>USBH_BULK_GetMaxTransferSize()</code> Update to latest software version.
2.26.0	2020-11-20	RH	Update to latest software version.
2.24.7	2020-05-29	YR	Update to latest software version.
2.24.5	2020-03-10	RH	Update to latest software version.
2.24.4	2020-01-17	YR	Chapter "Printer class": • Added function <code>USBH_PRINTER_Receive()</code>

Software	Date	By	Description
			- Added function USBH_PRINTER_WriteEx() - Added structure USBH_PRINTER_DEVICE_INFO
2.24.3	2019-11-29	YR	Chapter "USB Host Core": - Added function USBH_GetStringDescriptor() Chapter "Printer class": - Added function USBH_PRINTER_SendVendorRequest()
2.24b	2019-10-04	YR	Chapter "CCID": Added function USBH_CCID_GetResponse().
2.24a	2019-09-03	RH	Update to latest software version.
2.24	2019-06-18	RH	Added function USBH_SetHubPortPower(). Added function USBH_HID_SetOnExKeyboardStateChange().
2.22a	2019-04-23	YR	Update to latest software version.
2.22	2019-04-12	PC	Chapter "MIDI Device Driver": - Added function USBH_MIDI_RdData(). - Added function USBH_MIDI_WrData().
2.20	2019-01-30	YR	Added CP210x chapter.
2.18	2019-01-16	RH	Added AUDIO chapter.
2.16	2019-01-08	RH	Added MIDI chapter. Chapter "USB Host Core": - Added function USBH_GetNumRootPortConnections() - Added function USBH_GetDeviceDescriptorPtr() - Added function USBH_GetStringDescriptorASCII() - Added function USBH_GetSerialNumberASCII() Chapter "HID": - Added function USBH_HID_AddNotification(). - Added function USBH_HID_RemoveNotification().
2.14a	2018-11-14	YR	Chapter "Bulk": Added function USBH_BULK_Receive()
2.14	2018-11-08	YR	Added CCID chapter. Chapter "USB Host Core": - Added function USBH_ConfigPortPowerPinEx() Chapter "FT232": - Added function USBH_BULK_GetEndpointInfo() Chapter "Bulk": - Added function USBH_FT232_AddCustomDeviceMask() Chapter "Configuration": - Added function USBH_EHCI_Config_IgnoreOverCurrent()
2.12	2018-07-09	YR	Update to latest software version.
2.10	2018-06-18	YR	Added LAN chapter. Chapter "USB Host Core": - Added function USBH_SetRootPortPower() - Added function USBH_HUB_SuspendResume() Chapter "CDC": - Added function USBH_CDC_SuspendResume(). Chapter "Bulk": - Added function USBH_BULK_SetupRequest(). Added section <i>LPC54xxx High Speed driver</i> .
2.08a	2018-05-18	RH	Update to latest software version.
2.08	2018-05-15	RH	Added section <i>Updating emUSB-Host</i> . Added section <i>ATSAMx7 driver</i> .
2.07	2018-02-20	RH	Chapter "HID": Added function USBH_HID_SetOnGenericEvent().
2.06a	2018-01-18	RH	Chapter "HID": - Function USBH_HID_GetReportDescriptor() replaced by new function USBH_HID_GetReportDesc(). - Update description for USBH_HID_GetReport(). - Added "DeviceType" to USBH_HID_DEVICE_INFO.
2.06	2018-01-08	RH	Section "Synopsys DWC2 driver" - Added STM32H7 driver specific functions. Chapter "HID": - Added function USBH_HID_SetIndicators(). - Added function USBH_HID_GetIndicators(). - Added function USBH_HID_SetReportEx().

Software	Date	By	Description
2.04	2017-12-08	RH	Added vendor (BULK) class driver.
2.02	2017-11-30	RH	Update RAM usage values.
2.00b	2017-10-16	YR	Update to latest software version.
2.00a	2017-09-21	YR	Update to latest software version. Updated Performance & resource usage chapter.
2.00	2017-09-15	RH	Major revision of the manual. • Manual converted to text processor emDoc. • Chapter "Running emUSB-Host on target hardware" revised. • Chapter "Configuring emUSB-Host" revised. • All API function descriptions synchronized with source code.

About this document

Assumptions

This document assumes that you already have a solid knowledge of the following:

- The software tools used for building your application (compiler, linker, Integrated Development Environment).
- The C programming language.
- The target processor.

How to use this manual

This manual explains all the functions and macros that the product offers. It assumes you have a working knowledge of the C language.

Typographic conventions for syntax

This manual uses the following typographic conventions:

Style	Used for
Body	Body text.
Keyword	Text that you enter at the command prompt or that appears on the display (that is system functions, file- or pathnames).
<i>Parameter</i>	Parameters in API functions.
Sample	Sample code in program examples.
<i>Sample comment</i>	Comments in program examples.
<i>Reference</i>	Reference to chapters, sections, tables and figures or other documents.
Emphasis	Very important sections.

Table of contents

1	Introduction	25
1.1	What is emUSB-Host	26
1.2	emUSB-Host features	26
1.3	Basic concepts	27
1.4	Tasks and interrupt usage	28
1.5	Development environment (compiler)	29
1.6	Use of undocumented functions	30
2	USB Background information	31
2.1	Short Overview	32
2.2	Important USB Standard Versions	32
2.3	USB System Architecture	33
2.4	Transfer Types	34
2.5	Setup phase / Enumeration	34
2.6	Product / Vendor IDs	34
2.7	Predefined device classes	34
3	Running emUSB-Host on target hardware	35
3.1	Integrating emUSB-Host	36
3.2	Take a running project	36
3.3	Add emUSB-Host files	36
3.4	Configuring debugging output	36
3.5	Add hardware dependent configuration	37
3.6	Prepare and run the application	37
3.7	Write your own application	38
3.8	Updating emUSB-Host	39
4	Example applications	40
4.1	Overview	41
4.2	Mouse and keyboard events (USBH_HID_Start.c)	42
4.3	Mass storage handling (USBH_MSD_Start.c)	43
4.4	Printer interaction (USBH_Printer_Start.c)	44
4.5	Serial communication (USBH_CDC_Start.c)	45
4.6	Media Transfer Protocol (USBH_MTP_Start.c)	46
4.7	FTDI devices (USBH_FT232_Start.c)	47
5	USB Host Core	48
5.1	Target API	49
5.1.1	USBH_AssignMemory()	52

5.1.2	USBH_AssignTransferMemory()	53
5.1.3	USBH_CloseInterface()	54
5.1.4	USBH_Config_SetV2PHandler()	55
5.1.5	USBH_ConfigPowerOnGoodTime()	56
5.1.6	USBH_ConfigSupportExternalHubs()	57
5.1.7	USBH_ConfigTransferBufferSize()	58
5.1.8	USBH_CreateInterfaceList()	59
5.1.9	USBH_DestroyInterfaceList()	61
5.1.10	USBH_Exit()	62
5.1.11	USBH_GetCurrentConfigurationDescriptor()	63
5.1.12	USBH_GetDeviceDescriptor()	64
5.1.13	USBH_GetDeviceDescriptorPtr()	65
5.1.14	USBH_GetEndpointDescriptor()	66
5.1.15	USBH_GetFrameNumber()	67
5.1.16	USBH_GetInterfaceDescriptor()	68
5.1.17	USBH_GetInterfaceId()	69
5.1.18	USBH_GetInterfaceIdByHandle()	70
5.1.19	USBH_GetInterfaceInfo()	71
5.1.20	USBH_GetInterfaceSerial()	72
5.1.21	USBH_GetIADInfo()	73
5.1.22	USBH_GetPortInfo()	74
5.1.23	USBH_GetSerialNumber()	75
5.1.24	USBH_GetSerialNumberASCII()	76
5.1.25	USBH_GetStringDescriptorASCII()	77
5.1.26	USBH_GetStringDescriptor()	78
5.1.27	USBH_GetSpeed()	79
5.1.28	USBH_GetStatusStr()	80
5.1.29	USBH_ISRTask()	81
5.1.30	USBH_Init()	82
5.1.31	USBH_MEM_GetMaxUsed()	83
5.1.32	USBH_SetRootPortPower()	84
5.1.33	USBH_SetHubPortPower()	85
5.1.34	USBH_HUB_SuspendResume()	86
5.1.35	USBH_GetNumRootPortConnections()	87
5.1.36	USBH_OpenInterface()	88
5.1.37	USBH_RegisterEnumErrorNotification()	89
5.1.38	USBH_RegisterPnPNotification()	90
5.1.39	USBH_RestartEnumError()	91
5.1.40	USBH_SetCacheConfig()	92
5.1.41	USBH_SetOnSetPortPower()	93
5.1.42	USBH_ConfigPortPowerPinEx()	94
5.1.43	USBH_SetOnPortEvent()	95
5.1.44	USBH_ConfigOvercurrentDetection()	96
5.1.45	USBH_SubmitUrb()	97
5.1.46	USBH_IsoDataCtrl()	99
5.1.47	USBH_Task()	100
5.1.48	USBH_IsRunning()	101
5.1.49	USBH_UnregisterEnumErrorNotification()	102
5.1.50	USBH_UnregisterPnPNotification()	103
5.2	Data structures	104
5.2.1	USBH_BULK_INT_REQUEST	105
5.2.2	USBH_CONTROL_REQUEST	106
5.2.3	USBH_ISO_REQUEST	107
5.2.4	USBH_ENDPOINT_REQUEST	108
5.2.5	USBH_ENUM_ERROR	109
5.2.6	USBH_EP_MASK	110
5.2.7	USBH_HEADER	111
5.2.8	USBH_INTERFACE_INFO	112
5.2.9	USBH_INTERFACE_MASK	114
5.2.10	USBH_PNP_NOTIFICATION	116

5.2.11	USBH_PORT_INFO	117
5.2.12	USBH_PORT_EVENT	118
5.2.13	USBH_SET_INTERFACE	119
5.2.14	USBH_SET_POWER_STATE	120
5.2.15	USBH_URB	121
5.2.16	SEGGER_CACHE_CONFIG	122
5.2.17	USBH_ISO_DATA_CTRL	123
5.2.18	USBH_IAD_INFO	124
5.3	Enumerations	125
5.3.1	USBH_DEVICE_EVENT	126
5.3.2	USBH_FUNCTION	127
5.3.3	USBH_PNP_EVENT	129
5.3.4	USBH_POWER_STATE	130
5.3.5	USBH_SPEED	131
5.3.6	USBH_PORT_EVENT_TYPE	132
5.4	Function Types	133
5.4.1	USBH_NOTIFICATION_FUNC	134
5.4.2	USBH_ON_COMPLETION_FUNC	135
5.4.3	USBH_ON_ENUM_ERROR_FUNC	136
5.4.4	USBH_ON_PNP_EVENT_FUNC	137
5.4.5	USBH_ON_SETPORTPOWER_FUNC	138
5.4.6	USBH_CHECK_ADDRESS_FUNC	139
5.4.7	USBH_ON_PORT_EVENT_FUNC	140
5.5	USBH_STATUS	141
6	Human Interface Device (HID) class	144
6.1	Introduction	145
6.1.1	Overview	145
6.1.2	Example code	145
6.2	API Functions	146
6.2.1	USBH_HID_CancelIo()	147
6.2.2	USBH_HID_Close()	148
6.2.3	USBH_HID_Exit()	149
6.2.4	USBH_HID_GetDeviceInfo()	150
6.2.5	USBH_HID_GetNumDevices()	151
6.2.6	USBH_HID_GetReport()	152
6.2.7	USBH_HID_GetReportDesc()	153
6.2.8	USBH_HID_Init()	154
6.2.9	USBH_HID_Open()	155
6.2.10	USBH_HID_AddNotification()	156
6.2.11	USBH_HID_RemoveNotification()	157
6.2.12	USBH_HID_RegisterNotification()	158
6.2.13	USBH_HID_SetOnKeyboardStateChange()	159
6.2.14	USBH_HID_SetOnExKeyboardStateChange()	160
6.2.15	USBH_HID_SetOnMouseStateChange()	161
6.2.16	USBH_HID_SetOnRCStateChange()	162
6.2.17	USBH_HID_SetOnGenericEvent()	163
6.2.18	USBH_HID_SetReport()	164
6.2.19	USBH_HID_SetReportEx()	165
6.2.20	USBH_HID_GetReportEP0()	166
6.2.21	USBH_HID_SetIndicators()	167
6.2.22	USBH_HID_GetIndicators()	168
6.2.23	USBH_HID_ConfigureAllowLEDUpdate()	169
6.2.24	USBH_HID_GetInterfaceHandle()	170
6.2.25	USBH_HID_SuspendResume()	171
6.3	Data structures	172
6.3.1	USBH_HID_DEVICE_INFO	173
6.3.2	USBH_HID_KEYBOARD_DATA	174
6.3.3	USBH_HID_MOUSE_DATA	175

6.3.4	USBH_HID_RC_DATA	176
6.3.5	USBH_HID_GENERIC_DATA	177
6.3.6	USBH_HID_REPORT_INFO	178
6.3.7	USBH_HID_RW_CONTEXT	179
6.4	Function Types	180
6.4.1	USBH_HID_ON_KEYBOARD_FUNC	181
6.4.2	USBH_HID_ON_MOUSE_FUNC	182
6.4.3	USBH_HID_ON_RC_FUNC	183
6.4.4	USBH_HID_ON_GENERIC_FUNC	184
6.4.5	USBH_HID_USER_FUNC	185
7	Printer class (Add-On)	186
7.1	Introduction	187
7.1.1	Overview	187
7.1.2	Features	187
7.1.3	Example code	187
7.2	API Functions	188
7.2.1	USBH_PRINTER_Close()	189
7.2.2	USBH_PRINTER_ConfigureTimeout()	190
7.2.3	USBH_PRINTER_ExecSoftReset()	191
7.2.4	USBH_PRINTER_Exit()	192
7.2.5	USBH_PRINTER_GetDeviceId()	193
7.2.6	USBH_PRINTER_GetNumDevices()	194
7.2.7	USBH_PRINTER_GetPortStatus()	195
7.2.8	USBH_PRINTER_Init()	196
7.2.9	USBH_PRINTER_Open()	197
7.2.10	USBH_PRINTER_OpenByIndex()	198
7.2.11	USBH_PRINTER_Receive()	199
7.2.12	USBH_PRINTER_WriteEx()	200
7.2.13	USBH_PRINTER_RegisterNotification()	201
7.2.14	USBH_PRINTER_AddNotification()	202
7.2.15	USBH_PRINTER_RemoveNotification()	203
7.2.16	USBH_PRINTER_Write()	204
7.2.17	USBH_PRINTER_Read()	205
7.2.18	USBH_PRINTER_SendVendorRequest()	206
7.2.19	USBH_PRINTER_AddCustomDeviceMask()	207
7.3	Data structures	208
7.3.1	USBH_PRINTER_DEVICE_INFO	209
8	Mass Storage Device (MSD) class	210
8.1	Introduction	211
8.1.1	Overview	211
8.1.2	Features	212
8.1.3	Requirements	212
8.1.4	Example code	212
8.1.5	Supported Protocols	212
8.2	API Functions	213
8.2.1	USBH_MSD_Exit()	214
8.2.2	USBH_MSD_GetStatus()	215
8.2.3	USBH_MSD_GetUnits()	216
8.2.4	USBH_MSD_GetUnitInfo()	217
8.2.5	USBH_MSD_GetPortInfo()	218
8.2.6	USBH_MSD_Init()	219
8.2.7	USBH_MSD_ReadSectors()	220
8.2.8	USBH_MSD_WriteSectors()	221
8.2.9	USBH_MSD_UseAheadCache()	222
8.2.10	USBH_MSD_SetAheadBuffer()	223
8.3	Data Structures	224
8.3.1	USBH_MSD_UNIT_INFO	225

8.3.2	USBH_MSD_AHEAD_BUFFER	226
8.4	Function Types	227
8.4.1	USBH_MSD_LUN_NOTIFICATION_FUNC	228
9	MTP Device Driver (Add-On)	229
9.1	Introduction	230
9.1.1	Overview	230
9.1.2	Features	230
9.1.3	Example code	230
9.2	API Functions	231
9.2.1	USBH_MTP_Init()	233
9.2.2	USBH_MTP_Exit()	234
9.2.3	USBH_MTP_RegisterNotification()	235
9.2.4	USBH_MTP_Open()	236
9.2.5	USBH_MTP_Close()	237
9.2.6	USBH_MTP_GetDeviceInfo()	238
9.2.7	USBH_MTP_GetNumStorages()	239
9.2.8	USBH_MTP_Reset()	240
9.2.9	USBH_MTP_SetTimeouts()	241
9.2.10	USBH_MTP_GetLastErrorCode()	242
9.2.11	USBH_MTP_GetStorageInfo()	243
9.2.12	USBH_MTP_Format()	244
9.2.13	USBH_MTP_GetNumObjects()	245
9.2.14	USBH_MTP_GetObjectList()	246
9.2.15	USBH_MTP_GetObjectInfo()	247
9.2.16	USBH_MTP_CreateObject()	248
9.2.17	USBH_MTP_DeleteObject()	250
9.2.18	USBH_MTP_RenameObject()	251
9.2.19	USBH_MTP_ReadFile()	252
9.2.20	USBH_MTP_GetDevicePropDesc()	254
9.2.21	USBH_MTP_GetDevicePropValue()	255
9.2.22	USBH_MTP_GetObjectPropsSupported()	256
9.2.23	USBH_MTP_GetObjectPropDesc()	257
9.2.24	USBH_MTP_GetObjectPropValue()	259
9.2.25	USBH_MTP_SetObjectProperty()	260
9.2.26	USBH_MTP_CheckLock()	261
9.2.27	USBH_MTP_SetEventCallback()	262
9.2.28	USBH_MTP_ConfigEventSupport()	263
9.2.29	USBH_MTP_GetEventSupport()	264
9.3	Data structures	265
9.3.1	USBH_MTP_DEVICE_INFO	266
9.3.2	USBH_MTP_STORAGE_INFO	267
9.3.3	USBH_MTP_OBJECT	268
9.3.4	USBH_MTP_OBJECT_INFO	269
9.3.5	USBH_MTP_CREATE_INFO	270
9.3.6	USBH_MTP_OBJECT_PROP_DESC	271
9.4	Function Types	272
9.4.1	USBH_SEND_DATA_FUNC	273
9.4.2	USBH_RECEIVE_DATA_FUNC	274
9.4.3	USBH_EVENT_CALLBACK	275
9.5	Enums	276
9.5.1	USBH_MTP_DEVICE_PROPERTIES	277
9.5.2	USBH_MTP_OBJECT_PROPERTIES	278
9.5.3	USBH_MTP_RESPONSE_CODES	281
9.5.4	USBH_MTP_OBJECT_FORMAT	282
9.5.5	USBH_MTP_EVENT_CODES	284
10	CDC Device Driver (Add-On)	285
10.1	Introduction	286

10.1.1	Overview	286
10.1.2	Features	286
10.1.3	Example code	286
10.2	API Functions	287
10.2.1	USBH_CDC_Init()	289
10.2.2	USBH_CDC_Exit()	290
10.2.3	USBH_CDC_AddNotification()	291
10.2.4	USBH_CDC_RemoveNotification()	292
10.2.5	USBH_CDC_RegisterNotification()	293
10.2.6	USBH_CDC_ConfigureDefaultTimeout()	294
10.2.7	USBH_CDC_Open()	295
10.2.8	USBH_CDC_Close()	296
10.2.9	USBH_CDC_AllowShortRead()	297
10.2.10	USBH_CDC_GetDeviceInfo()	298
10.2.11	USBH_CDC_SetTimeouts()	299
10.2.12	USBH_CDC_Read()	300
10.2.13	USBH_CDC_Write()	301
10.2.14	USBH_CDC_GetMaxTransferSize()	302
10.2.15	USBH_CDC_ReadAsync()	303
10.2.16	USBH_CDC_WriteAsync()	305
10.2.17	USBH_CDC_CancelRead()	307
10.2.18	USBH_CDC_CancelWrite()	308
10.2.19	USBH_CDC_SetCommParas()	309
10.2.20	USBH_CDC_SetDtr()	310
10.2.21	USBH_CDC_ClrDtr()	311
10.2.22	USBH_CDC_SetRts()	312
10.2.23	USBH_CDC_ClrRts()	313
10.2.24	USBH_CDC_GetQueueStatus()	314
10.2.25	USBH_CDC_SetBreak()	315
10.2.26	USBH_CDC_SetBreakOn()	316
10.2.27	USBH_CDC_SetBreakOff()	317
10.2.28	USBH_CDC_GetSerialState()	318
10.2.29	USBH_CDC_SetOnSerialStateChange()	319
10.2.30	USBH_CDC_SetOnIntStateChange()	320
10.2.31	USBH_CDC_GetSerialNumber()	321
10.2.32	USBH_CDC_AddDevice()	322
10.2.33	USBH_CDC_RemoveDevice()	323
10.2.34	USBH_CDC_SetConfigFlags()	324
10.2.35	USBH_CDC_SuspendResume()	325
10.2.36	USBH_CDC_GetInterfaceHandle()	326
10.3	Data structures	327
10.3.1	USBH_CDC_DEVICE_INFO	328
10.3.2	USBH_CDC_SERIALSTATE	329
10.3.3	USBH_CDC_RW_CONTEXT	330
10.4	Type definitions	331
10.4.1	USBH_CDC_ON_COMPLETE_FUNC	332
10.4.2	USBH_CDC_SERIAL_STATE_CALLBACK	333
11	FT232 Device Driver (Add-On)	334
11.1	Introduction	335
11.1.1	Features	335
11.1.2	Example code	335
11.1.3	Compatibility	335
11.1.4	Further reading	335
11.2	API Functions	336
11.2.1	USBH_FT232_Init()	338
11.2.2	USBH_FT232_Exit()	339
11.2.3	USBH_FT232_AddNotification()	340
11.2.4	USBH_FT232_RemoveNotification()	341

11.2.5	USBH_FT232_RegisterNotification()	342
11.2.6	USBH_FT232_AddCustomDeviceMask()	343
11.2.7	USBH_FT232_ConfigureDefaultTimeout()	344
11.2.8	USBH_FT232_Open()	345
11.2.9	USBH_FT232_Close()	346
11.2.10	USBH_FT232_GetDeviceInfo()	347
11.2.11	USBH_FT232_ResetDevice()	348
11.2.12	USBH_FT232_SetTimeouts()	349
11.2.13	USBH_FT232_ConfigurePollDelay()	350
11.2.14	USBH_FT232_Read()	351
11.2.15	USBH_FT232_Write()	352
11.2.16	USBH_FT232_AllowShortRead()	353
11.2.17	USBH_FT232_SetBaudRate()	354
11.2.18	USBH_FT232_SetDataCharacteristics()	355
11.2.19	USBH_FT232_SetFlowControl()	356
11.2.20	USBH_FT232_SetDtr()	357
11.2.21	USBH_FT232_ClrDtr()	358
11.2.22	USBH_FT232_SetRts()	359
11.2.23	USBH_FT232_ClrRts()	360
11.2.24	USBH_FT232_GetModemStatus()	361
11.2.25	USBH_FT232_SetChars()	362
11.2.26	USBH_FT232_Purge()	363
11.2.27	USBH_FT232_GetQueueStatus()	364
11.2.28	USBH_FT232_SetBreakOn()	365
11.2.29	USBH_FT232_SetBreakOff()	366
11.2.30	USBH_FT232_SetLatencyTimer()	367
11.2.31	USBH_FT232_GetLatencyTimer()	368
11.2.32	USBH_FT232_SetBitMode()	369
11.2.33	USBH_FT232_GetBitMode()	370
11.2.34	USBH_FT232_GetInterfaceHandle()	371
11.3	Data structures	372
11.3.1	USBH_FT232_DEVICE_INFO	373
12	FT260 Device Driver (Add-On)	374
12.1	Introduction	375
12.1.1	Features	375
12.1.2	Example code	375
12.1.3	Compatibility	375
12.1.4	Further reading	375
12.2	API Functions	376
12.2.1	USBH_FT260_Init()	378
12.2.2	USBH_FT260_AddSupportItem()	379
12.2.3	USBH_FT260_RemoveSupportItem()	380
12.2.4	USBH_FT260_AddNotification()	381
12.2.5	USBH_FT260_RemoveNotification()	382
12.2.6	USBH_FT260_Open()	383
12.2.7	USBH_FT260_Close()	384
12.2.8	USBH_FT260_SetClock()	385
12.2.9	USBH_FT260_SetWakeupInterrupt()	386
12.2.10	USBH_FT260_SetInterruptTriggerType()	387
12.2.11	USBH_FT260_SelectGpio2Function()	388
12.2.12	USBH_FT260_SelectGpioAFunction()	389
12.2.13	USBH_FT260_SelectGpioGFunction()	390
12.2.14	USBH_FT260_SetSuspendOutPolarity()	391
12.2.15	USBH_FT260_GetChipVersion()	392
12.2.16	USBH_FT260_EnableI2CPin()	393
12.2.17	USBH_FT260_SetUartToGPIOPin()	394
12.2.18	USBH_FT260_EnableDcdRiPin()	395
12.2.19	USBH_FT260_I2CMaster_Init()	396

12.2.20	USBH_FT260_I2CMaster_Read()	397
12.2.21	USBH_FT260_I2CMaster_Write()	398
12.2.22	USBH_FT260_I2CMaster_ReadReg()	399
12.2.23	USBH_FT260_I2CMaster_WriteReg()	400
12.2.24	USBH_FT260_I2CMaster_GetStatus()	401
12.2.25	USBH_FT260_I2CMaster_Reset()	402
12.2.26	USBH_FT260_UART_Init()	403
12.2.27	USBH_FT260_UART_SetBaudRate()	404
12.2.28	USBH_FT260_UART_SetFlowControl()	405
12.2.29	USBH_FT260_UART_SetDataCharacteristics()	406
12.2.30	USBH_FT260_UART_SetBreak()	407
12.2.31	USBH_FT260_UART_SetXonXoffChar()	408
12.2.32	USBH_FT260_UART_GetConfig()	409
12.2.33	USBH_FT260_UART_Read()	410
12.2.34	USBH_FT260_UART_Write()	411
12.2.35	USBH_FT260_UART_Reset()	412
12.2.36	USBH_FT260_UART_GetDcdRiStatus()	413
12.2.37	USBH_FT260_UART_EnableRiWakeup()	414
12.2.38	USBH_FT260_UART_SetRiWakeupConfig()	415
12.2.39	USBH_FT260_GetInterruptFlag()	416
12.2.40	USBH_FT260_CleanInterruptFlag()	417
12.2.41	USBH_FT260_GPIO_Set()	418
12.2.42	USBH_FT260_GPIO_Get()	419
12.2.43	USBH_FT260_GPIO_SetDir()	420
12.2.44	USBH_FT260_GPIO_Read()	421
12.2.45	USBH_FT260_GPIO_Write()	422
12.2.46	USBH_FT260_GPIO_Set_OD()	423
12.2.47	USBH_FT260_GPIO_Reset_OD()	424
12.3	Data structures	425
12.3.1	USBH_FT260_VID_PID_PAIR	426
12.3.2	USBH_FT260_UART_CONFIG	427
12.3.3	USBH_FT260_GPIO_REPORT	428
13	BULK Device Driver (Add-On)	429
13.1	Introduction	430
13.1.1	Overview	430
13.1.2	Features	430
13.1.3	Example code	430
13.2	API Functions	431
13.2.1	USBH_BULK_Init()	432
13.2.2	USBH_BULK_Exit()	433
13.2.3	USBH_BULK_RegisterNotification()	434
13.2.4	USBH_BULK_RemoveNotification()	435
13.2.5	USBH_BULK_AddNotification()	436
13.2.6	USBH_BULK_RegisterNotification()	438
13.2.7	USBH_BULK_Open()	439
13.2.8	USBH_BULK_Close()	440
13.2.9	USBH_BULK_AllowShortRead()	441
13.2.10	USBH_BULK_GetDeviceInfo()	442
13.2.11	USBH_BULK_GetEndpointInfo()	443
13.2.12	USBH_BULK_Read()	444
13.2.13	USBH_BULK_Receive()	445
13.2.14	USBH_BULK_Write()	446
13.2.15	USBH_BULK_GetMaxTransferSize()	447
13.2.16	USBH_BULK_ReadAsync()	448
13.2.17	USBH_BULK_WriteAsync()	450
13.2.18	USBH_BULK_Cancel()	452
13.2.19	USBH_BULK_GetNumBytesInBuffer()	453
13.2.20	USBH_BULK_SetupRequest()	454

13.2.21	USBH_BULK_IsoDataCtrl()	455
13.2.22	USBH_BULK_GetInterfaceHandle()	456
13.3	Data structures	457
13.3.1	USBH_BULK_DEVICE_INFO	458
13.3.2	USBH_BULK_EP_INFO	459
13.3.3	USBH_BULK_RW_CONTEXT	460
13.3.4	USBH_ISO_DATA_CTRL	461
13.4	Type definitions	462
13.4.1	USBH_BULK_ON_COMPLETE_FUNC	463
14	LAN component (Add-On)	464
14.1	Introduction	465
14.1.1	Overview	465
14.1.2	Features	465
14.1.3	Example code	465
14.2	IP_Config_USBH_LAN.c in detail	466
14.3	API Functions	467
14.3.1	USBH_LAN_Init()	468
14.3.2	USBH_LAN_RegisterDriver()	469
14.3.3	USBH_LAN_Exit()	470
14.3.4	USBH_LAN_SetOnAddRemove()	471
14.4	Function Types	472
14.4.1	USBH_LAN_ADD_REMOVE_CALLBACK	473
15	CCID Device Driver (Add-On)	474
15.1	Introduction	475
15.1.1	Overview	475
15.1.2	Features	475
15.1.3	Example code	475
15.2	API Functions	476
15.2.1	USBH_CCID_Init()	477
15.2.2	USBH_CCID_Exit()	478
15.2.3	USBH_CCID_AddNotification()	479
15.2.4	USBH_CCID_RemoveNotification()	480
15.2.5	USBH_CCID_Open()	481
15.2.6	USBH_CCID_Close()	482
15.2.7	USBH_CCID_SetTimeout()	483
15.2.8	USBH_CCID_SetOnSlotChange()	484
15.2.9	USBH_CCID_GetDeviceInfo()	485
15.2.10	USBH_CCID_GetSerialNumber()	486
15.2.11	USBH_CCID_GetNumSlots()	487
15.2.12	USBH_CCID_GetCSDesc()	488
15.2.13	USBH_CCID_Cmd()	489
15.2.14	USBH_CCID_PowerOn()	490
15.2.15	USBH_CCID_PowerOnATR()	491
15.2.16	USBH_CCID_PowerOff()	492
15.2.17	USBH_CCID_GetSlotStatus()	493
15.2.18	USBH_CCID_APDU()	494
15.2.19	USBH_CCID_GetResponse()	495
15.2.20	USBH_CCID_GetParameters()	496
15.2.21	USBH_CCID_ResetParameters()	497
15.2.22	USBH_CCID_SetParameters()	498
15.2.23	USBH_CCID_Abort()	499
15.2.24	USBH_CCID_GetInterfaceHandle()	500
15.3	Data structures	501
15.3.1	USBH_CCID_DEVICE_INFO	502
15.3.2	USBH_CCID_CMD_PARA	503
15.4	Type definitions	504
15.4.1	USBH_CCID_SLOT_CHANGE_CALLBACK	505

16	MIDI Device Driver (Add-On)	506
16.1	Introduction	507
16.1.1	Overview	507
16.1.2	Features	507
16.1.3	Example code	507
16.2	API Functions	509
16.2.1	USBH_MIDI_Init()	510
16.2.2	USBH_MIDI_Exit()	511
16.2.3	USBH_MIDI_AddNotification()	512
16.2.4	USBH_MIDI_RemoveNotification()	513
16.2.5	USBH_MIDI_ConfigureDefaultTimeout()	514
16.2.6	USBH_MIDI_Open()	515
16.2.7	USBH_MIDI_Close()	516
16.2.8	USBH_MIDI_SetBuffer()	517
16.2.9	USBH_MIDI_SetTimeouts()	518
16.2.10	USBH_MIDI_GetDeviceInfo()	519
16.2.11	USBH_MIDI_RdData()	520
16.2.12	USBH_MIDI_RdEvent()	521
16.2.13	USBH_MIDI_WrData()	522
16.2.14	USBH_MIDI_WrEvent()	523
16.2.15	USBH_MIDI_Send()	524
16.2.16	USBH_MIDI_GetQueueStatus()	525
16.3	Data structures	526
16.3.1	USBH_MIDI_DEVICE_INFO	527
17	AUDIO Device Driver (Add-On)	528
17.1	Overview	529
17.1.1	Using an audio device	530
17.1.2	Example code	530
17.2	API Functions	531
17.2.1	USBH_AUDIO_Init()	533
17.2.2	USBH_AUDIO_Exit()	534
17.2.3	USBH_AUDIO_AddNotification()	535
17.2.4	USBH_AUDIO_RemoveNotification()	536
17.2.5	USBH_AUDIO_GetIndex()	537
17.2.6	USBH_AUDIO_Open()	538
17.2.7	USBH_AUDIO_Close()	539
17.2.8	USBH_AUDIO_GetInterfaceInfo()	540
17.2.9	USBH_AUDIO_GetInterfaceHandle()	541
17.2.10	USBH_AUDIO_GetDescriptorList()	542
17.2.11	USBH_AUDIO_FreeDescriptorList()	543
17.2.12	USBH_AUDIO_GetAltInfo()	544
17.2.13	USBH_AUDIO_FindStreamConfiguration()	545
17.2.14	USBH_AUDIO_GetFeatureUnitInfo()	546
17.2.15	USBH_AUDIO_GetSelectorUnitInfo()	547
17.2.16	USBH_AUDIO_GetMixerUnitInfo()	548
17.2.17	USBH_AUDIO_GetSampleFrequency()	549
17.2.18	USBH_AUDIO_SetSampleFrequency()	550
17.2.19	USBH_AUDIO_GetVolume()	551
17.2.20	USBH_AUDIO_SetVolume()	552
17.2.21	USBH_AUDIO_GetMute()	553
17.2.22	USBH_AUDIO_SetMute()	554
17.2.23	USBH_AUDIO_GetFeatureUnitControl()	555
17.2.24	USBH_AUDIO_SetFeatureUnitControl()	556
17.2.25	USBH_AUDIO_GetMixerUnitControl()	557
17.2.26	USBH_AUDIO_SetMixerUnitControl()	558
17.2.27	USBH_AUDIO_GetSelectorUnitControl()	559
17.2.28	USBH_AUDIO_SetSelectorUnitControl()	560
17.2.29	USBH_AUDIO_GetOutPacketSize()	561

17.2.30	USBH_AUDIO_OpenOutputStream()	562
17.2.31	USBH_AUDIO_OpenInputStream()	563
17.2.32	USBH_AUDIO_CloseStream()	564
17.2.33	USBH_AUDIO_Write()	565
17.2.34	USBH_AUDIO_GetNumQueueEntries()	566
17.2.35	USBH_AUDIO_GetFreeQueueEntries()	567
17.2.36	USBH_AUDIO_CheckBufferIdle()	568
17.2.37	USBH_AUDIO_Receive()	569
17.2.38	USBH_AUDIO_Ack()	570
17.2.39	USBH_AUDIO_Read()	571
17.2.40	USBH_AUDIO_GetInterfaceHandle()	572
17.3	Data structures	573
17.3.1	USBH_AUDIO_INTERFACE_INFO	574
17.3.2	USBH_AUDIO_DESCRIPTOR	575
17.3.3	USBH_AUDIO_TERMINAL_INFO	576
17.3.4	USBH_AUDIO_ALT_INFO	577
17.3.5	USBH_AUDIO_STREAM_MASK	578
17.3.6	USBH_AUDIO_FU_INFO	579
17.3.7	USBH_AUDIO_SU_INFO	580
17.3.8	USBH_AUDIO_MU_INFO	581
17.3.9	USBH_AUDIO_VOLUME_INFO	582
18	CP210X Device Driver (Add-On)	583
18.1	Introduction	584
18.1.1	Features	584
18.1.2	Example code	584
18.1.3	Compatibility	584
18.1.4	Further reading	584
18.2	API Functions	585
18.2.1	USBH_CP210X_Init()	586
18.2.2	USBH_CP210X_Exit()	587
18.2.3	USBH_CP210X_AddNotification()	588
18.2.4	USBH_CP210X_RemoveNotification()	589
18.2.5	USBH_CP210X_Open()	590
18.2.6	USBH_CP210X_Close()	591
18.2.7	USBH_CP210X_GetDeviceInfo()	592
18.2.8	USBH_CP210X_AllowShortRead()	593
18.2.9	USBH_CP210X_SetBaudRate()	594
18.2.10	USBH_CP210X_Read()	595
18.2.11	USBH_CP210X_Write()	596
18.2.12	USBH_CP210X_SetDataCharacteristics()	597
18.2.13	USBH_CP210X_SetModemHandshaking()	598
18.2.14	USBH_CP210X_GetModemStatus()	599
18.2.15	USBH_CP210X_Purge()	600
18.2.16	USBH_CP210X_SetBreak()	601
18.2.17	USBH_CP210X_GetInterfaceHandle()	602
18.3	Data structures	603
18.3.1	USBH_CP210X_DEVICE_INFO	604
19	CH34X Device Driver	605
19.1	Introduction	606
19.1.1	Features	606
19.1.2	Example code	606
19.1.3	Compatibility	606
19.1.4	Further reading	606
19.2	API Functions	607
19.2.1	USBH_CH34X_Init()	608
19.2.2	USBH_CH34X_Exit()	609
19.2.3	USBH_CH34X_AddCustomDeviceMask()	610

19.2.4	USBH_CH34X_AddNotification()	611
19.2.5	USBH_CH34X_RemoveNotification()	612
19.2.6	USBH_CH34X_Open()	613
19.2.7	USBH_CH34X_Close()	614
19.2.8	USBH_CH34X_GetDeviceInfo()	615
19.2.9	USBH_CH34X_AllowShortRead()	616
19.2.10	USBH_CH34X_SetBaudRate()	617
19.2.11	USBH_CH34X_Read()	618
19.2.12	USBH_CH34X_Write()	619
19.2.13	USBH_CH34X_SetDataCharacteristics()	620
19.2.14	USBH_CH34X_SetModemHandshaking()	621
19.2.15	USBH_CH34X_GetModemStatus()	622
19.2.16	USBH_CH34X_Reset()	623
19.2.17	USBH_CH34X_SetBreak()	624
19.2.18	USBH_CH34X_GetInterfaceHandle()	625
19.3	Data structures	626
19.3.1	USBH_CH34X_DEVICE_INFO	627
20	Video Device Driver (Add-On)	628
20.1	Introduction	629
20.1.1	Overview	629
20.1.2	Example code	629
20.2	API Functions	630
20.2.1	USBH_VIDEO_Init()	631
20.2.2	USBH_VIDEO_Exit()	632
20.2.3	USBH_VIDEO_AddNotification()	633
20.2.4	USBH_VIDEO_RemoveNotification()	634
20.2.5	USBH_VIDEO_Open()	635
20.2.6	USBH_VIDEO_Close()	636
20.2.7	USBH_VIDEO_GetIndex()	637
20.2.8	USBH_VIDEO_GetInterfaceInfo()	638
20.2.9	USBH_VIDEO_GetTermUnitInfo()	639
20.2.10	USBH_VIDEO_TermUnitID2Index()	640
20.2.11	USBH_VIDEO_GetInputHeader()	641
20.2.12	USBH_VIDEO_GetFormatInfo()	642
20.2.13	USBH_VIDEO_GetFrameInfo()	643
20.2.14	USBH_VIDEO_GetColorMatchingInfo()	644
20.2.15	USBH_VIDEO_OpenStream()	645
20.2.16	USBH_VIDEO_Ack()	646
20.2.17	USBH_VIDEO_GetStreamState()	647
20.2.18	USBH_VIDEO_RestartStream()	648
20.2.19	USBH_VIDEO_CloseStream()	649
20.2.20	USBH_VIDEO_GetControlVal()	650
20.2.21	USBH_VIDEO_SetControlVal()	651
20.2.22	USBH_VIDEO_ReadStatus()	652
20.3	Data structures	653
20.3.1	USBH_VIDEO_INTERFACE_INFO	654
20.3.2	USBH_VIDEO_TERM_UNIT_INFO	655
20.3.3	USBH_VIDEO_INPUT_HEADER_INFO	656
20.3.4	USBH_VIDEO_FORMAT_INFO	657
20.3.5	USBH_VIDEO_FRAME_INFO	658
20.3.6	USBH_VIDEO_COLOR_INFO	659
20.3.7	USBH_VIDEO_PAYLOAD_HEADER	660
20.3.8	USBH_VIDEO_STREAM_CONFIG	661
20.3.9	USBH_VIDEO_GET_CONTROL	662
20.3.10	USBH_VIDEO_SET_CONTROL	663
20.4	Function Types	664
20.4.1	USBH_VIDEO_RX_CALLBACK	665
20.4.2	USBH_VIDEO_PAYLOAD_CALLBACK	666

21	USB On-The-Go (Add-On)	667
21.1	Introduction	668
21.1.1	Overview	668
21.1.2	Features	668
21.1.3	Example code	668
21.1.4	OTG Driver	669
21.2	API Functions	670
21.2.1	USB_OTG_Init()	671
21.2.2	USB_OTG_DeInit()	672
21.2.3	USB_OTG_GetSessionState()	673
21.2.4	USB_OTG_GetIdPin()	674
21.2.5	USB_OTG_SetIdPinFunc()	675
21.2.6	USB_OTG_AddDriver()	676
21.2.7	USB_OTG_IDPIN_FUNC	677
21.2.7.1	USB_OTG_X_Config()	678
22	Configuring emUSB-Host	679
22.1	Runtime configuration	680
22.1.1	Memory pools	680
22.1.2	USBH_X_Config()	681
22.2	Compile-time configuration	683
22.2.1	Compile-time switches for debugging	683
22.2.1.1	USBH_DEBUG	683
22.2.1.2	USBH_LOG_BUFFER_SIZE	683
22.2.2	Use of standard C-library functions	683
22.2.3	General USB configuration	684
22.2.3.1	USBH_SUPPORT_ISO_TRANSFER	684
22.2.3.2	USBH_MAX_NUM_HOST_CONTROLLERS	684
22.2.3.3	USBH_SUPPORT_VIRTUALMEM	684
22.2.3.4	USBH_REO_FREE_MEM_LIST	684
22.2.3.5	USBH_USE_APP_MEM_PANIC	684
22.2.3.6	USBH_MAX_INTERFACES_IN_IAD	685
22.2.4	USB device enumeration behavior	685
22.2.4.1	USBH_WAIT_AFTER_RESET	685
22.2.4.2	USBH_HUB_WAIT_AFTER_RESET	685
22.2.4.3	WAIT_AFTER_SETADDRESS	685
22.2.4.4	USBH_RESET_RETRY_COUNTER	685
22.2.4.5	USBH_DELAY_FOR_REENUM	686
22.2.4.6	USBH_DELAY_BETWEEN_ENUMERATIONS	686
22.2.4.7	USBH_DEFAULT_SETUP_TIMEOUT	686
22.2.5	URB handling	686
22.2.5.1	USBH_URB_QUEUE_SIZE	686
22.2.5.2	USBH_URB_QUEUE_RETRY_INTV	686
22.2.5.3	USBH_SUPPORT_HUB_CLEAR_TT_BUFFER	687
22.2.6	Mass storage class configuration	687
22.2.6.1	USBH_MSD_MAX_DEVICES	687
22.2.6.2	USBH_MSD_MAX_SECTORS_AT_ONCE	687
22.2.6.3	USBH_MSD_EP0_TIMEOUT	687
22.2.6.4	USBH_MSD_CBW_WRITE_TIMEOUT	687
22.2.6.5	USBH_MSD_CSW_READ_TIMEOUT	687
22.2.6.6	USBH_MSD_COMMAND_TIMEOUT	688
22.2.6.7	USBH_MSD_DATA_READ_TIMEOUT	688
22.2.6.8	USBH_MSD_DATA_WRITE_TIMEOUT	688
22.2.6.9	USBH_MSD_MAX_TEST_READY_RETRIES	688
22.2.6.10	USBH_MSD_MAX_READY_WAIT_TIME	688
22.2.6.11	USBH_MSD_TEST_UNIT_READY_DELAY	689
22.2.7	HID class configuration	689
22.2.7.1	USBH_HID_MAX_REPORTS	689
22.2.7.2	USBH_HID_DISABLE_INTERFACE_PROTOCOL_CHECK	689

22.2.7.3	USBH_HID_EP0_TIMEOUT	689
22.2.8	CDC-ACM class configuration	689
22.2.8.1	USBH_CDC_DISABLE_AUTO_DETECT	689
22.2.8.2	USBH_CDC_EP0_TIMEOUT	690
22.2.9	BULK (Vendor) class configuration	690
22.2.9.1	USBH_BULK_EP0_TIMEOUT	690
22.2.9.2	USBH_BULK_MAX_NUM_EPS	690
22.2.10	Printer class configuration	690
22.2.10.1	USBH_PRINTER_EP0_TIMEOUT	690
22.2.11	AUDIO class configuration	690
22.2.11.1	USBH_AUDIO_EP0_TIMEOUT	690
22.2.12	FT232 class configuration	691
22.2.12.1	USBH_FT232_EP0_TIMEOUT	691
22.2.13	CP210X class configuration	691
22.2.13.1	USBH_CP210X_EP0_TIMEOUT	691
22.3	Host controller specifics	692
22.3.1	EHCI driver	693
22.3.1.1	EHCI driver specific configuration functions	693
22.3.1.2	USBH_EHCI_Add()	695
22.3.1.3	USBH_EHCI_EX_Add()	696
22.3.1.4	USBH_RT1050_Add()	697
22.3.1.5	USBH_RT1170_Add()	698
22.3.1.6	USBH_IMX6U5_Add()	699
22.3.1.7	USBH_EHCI_Config_SetM2MEndianMode()	700
22.3.1.8	USBH_EHCI_Config_UseTransferBuffer()	701
22.3.1.9	USBH_EHCI_Config_IgnoreOverCurrent()	702
22.3.2	Synopsys DWC2 driver	703
22.3.2.1	High-speed driver	703
22.3.2.2	Restrictions	703
22.3.2.3	Synopsys driver specific configuration functions	704
22.3.2.4	USBH_STM32_Add()	706
22.3.2.5	USBH_STM32F2_FS_Add()	707
22.3.2.6	USBH_STM32F2_HS_Add()	708
22.3.2.7	USBH_STM32F2_HS_AddEx()	709
22.3.2.8	USBH_STM32F2_HS_SetCheckAddress()	710
22.3.2.9	USBH_STM32F7_FS_Add()	711
22.3.2.10	USBH_STM32F7_HS_Add()	712
22.3.2.11	USBH_STM32F7_HS_AddEx()	713
22.3.2.12	USBH_STM32F7_HS_SetCheckAddress()	714
22.3.2.13	USBH_STM32H7_HS_Add()	715
22.3.2.14	USBH_STM32H7_HS_AddEx()	716
22.3.2.15	USBH_STM32H7_HS_SetCheckAddress()	717
22.3.2.16	USBH_XMC4xxx_FS_Add()	718
22.3.3	PSOC Edge E84 driver	719
22.3.3.1	Restrictions	719
22.3.3.2	Synopsys driver specific configuration functions	719
22.3.3.3	USBH_Infineon_PSOCE84_DMA_Add()	720
22.3.3.4	USBH_Infineon_PSOCE84_SetCheckAddress()	721
22.3.4	OHCI driver	722
22.3.4.1	OHCI driver specific configuration functions	722
22.3.4.2	USBH_OHCI_Add()	723
22.3.4.3	USBH_OHCI_ZC_Add()	724
22.3.4.4	USBH_OHCI_ZC_SetCheckZeroCopyAddress()	725
22.3.4.5	USBH_OHCI_LPC546_Add()	726
22.3.5	Kinetis USBOTG FS driver	727
22.3.5.1	KinetisFS driver specific configuration functions	727
22.3.5.2	USBH_KINETIS_FS_Add()	728
22.3.6	Renesas driver	729
22.3.6.1	Restrictions	729
22.3.6.2	Overcurrent detection	729

22.3.6.3	Renesas driver specific configuration functions	729
22.3.6.4	USBH_RX11_Add()	731
22.3.6.5	USBH_RX23_Add()	732
22.3.6.6	USBH_RX62_Add()	733
22.3.6.7	USBH_RX63_Add()	734
22.3.6.8	USBH_RX64_Add()	735
22.3.6.9	USBH_RX65_Add()	736
22.3.6.10	USBH_RX71_FS_Add()	737
22.3.6.11	USBH_RX71_HS_Add()	738
22.3.6.12	USBH_RZA1_Add()	739
22.3.6.13	USBH_Synergy_FS_Add()	740
22.3.6.14	USBH_Synergy_HS_Add()	741
22.3.7	ATSAMx7 driver	742
22.3.7.1	Restrictions	742
22.3.7.2	ATSAMx7 driver specific configuration functions	742
22.3.7.3	USBH_SAMx7_Add()	743
22.3.8	LPC54xxx/LPC55xxx High-Speed driver	744
22.3.8.1	Restrictions	744
22.3.8.2	LPC54xxx/LPC55xxx driver specific configuration functions	744
22.3.8.3	USBH_LPC54xxx_Add()	745
22.3.8.4	USBH_LPC55xxx_Add()	746
22.3.9	Cypress PSoC 6 driver	747
22.3.9.1	Restrictions	747
22.3.9.2	PSoC 6 driver specific configuration functions	747
22.3.9.3	USBH_Cypress_PSoC_Add()	747
22.3.9.4	USBH_Cypress_PSoC_DMA_Add()	747
22.3.10	ST full-speed driver	749
22.3.10.1	ST full-speed driver specific configuration functions	749
22.3.10.2	USBH_STM32H5xx_Add()	749
23	Support	750
23.1	Contacting support	751
23.1.1	Where can I find the license number?	751
24	Debugging	752
24.1	Message output	753
24.2	API functions	754
24.2.1	USBH_ConfigMsgFilter()	755
24.2.2	USBH_Log()	756
24.2.3	USBH_Warn()	757
24.2.4	USBH_Panic()	758
24.2.5	USBH_Logf_Application()	759
24.2.6	USBH_Warnf_Application()	760
24.2.7	USBH_sprintf_Application()	761
24.2.8	USBH_Puts()	762
25	OS integration	763
25.1	General information	764
25.1.1	Operating system support supplied with this release	764
25.2	OS layer API functions	765
25.2.1	USBH_OS_Delay()	766
25.2.2	USBH_OS_DisableInterrupt()	767
25.2.3	USBH_OS_EnableInterrupt()	768
25.2.4	USBH_OS_GetTime32()	769
25.2.5	USBH_OS_Init()	770
25.2.6	USBH_OS_DeInit()	771
25.2.7	USBH_OS_Lock()	772
25.2.8	USBH_OS_Unlock()	773

25.2.9	USBH_OS_SignalNetEvent()	774
25.2.10	USBH_OS_WaitNetEvent()	775
25.2.11	USBH_OS_SignalISREx()	776
25.2.12	USBH_OS_WaitISR()	777
25.2.13	USBH_OS_AllocEvent()	778
25.2.14	USBH_OS_FreeEvent()	779
25.2.15	USBH_OS_SetEvent()	780
25.2.16	USBH_OS_ResetEvent()	781
25.2.17	USBH_OS_WaitEvent()	782
25.2.18	USBH_OS_WaitEventTimed()	783
26	Performance & resource usage	784
26.1	Memory footprint	785
26.1.1	ROM usage	785
26.1.2	RAM usage	786
26.2	Performance	787
27	Glossary	788
28	FAQ	790

Chapter 1

Introduction

This chapter provides an introduction to using emUSB-Host. It explains the basic concepts behind emUSB-Host.

1.1 What is emUSB-Host

emUSB-Host is a CPU-independent USB Host stack. emUSB-Host is a high-performance library that has been optimized for speed, versatility and small memory footprint.

1.2 emUSB-Host features

emUSB-Host is written in ANSI C and can be used on virtually any CPU. Here is a list of emUSB-Host features:

- ISO/ANSI C source code.
- High performance.
- Small footprint.
- No configuration required.
- Runs out-of-the-box.
- Control, bulk, interrupt and isochronous transfers.
- Very simple host controller driver structure.
- USB Mass Storage Device Class available.
- Works seamlessly with embOS, emFile (for MSD) and emNET (for LAN).
- Support for class drivers.
- Support for external USB hub devices.
- Support for devices with alternate settings.
- Support for multi-interface devices.
- Support for multi-configuration devices.
- Royalty-free.

1.3 Basic concepts

emUSB-Host consists of three layers: a driver for hardware access, the emUSB-Host core and a USB class driver. For a functional emUSB-Host, the core component and at least one of the hardware drivers is necessary. emUSB-Host handles all USB operations independently in a separate task(s) beside the target application task. This implicitly means that an RTOS is required. A recommendation is using embOS since it perfectly fits the requirements of emUSB Host and works seamlessly with emUSB-Host, not requiring any integration work.

1.4 Tasks and interrupt usage

emUSB-Host uses two dedicated tasks. One of the tasks processes the interrupts generated by the USB host controller. The function `USBH_ISRTask()` must run as this task with the highest priority. The other task manages the internal software timers. Its routine must be the `USBH_Task()` function. The priorities of both tasks have to be higher than the priority of any other application task which uses emUSB-Host. To recap:

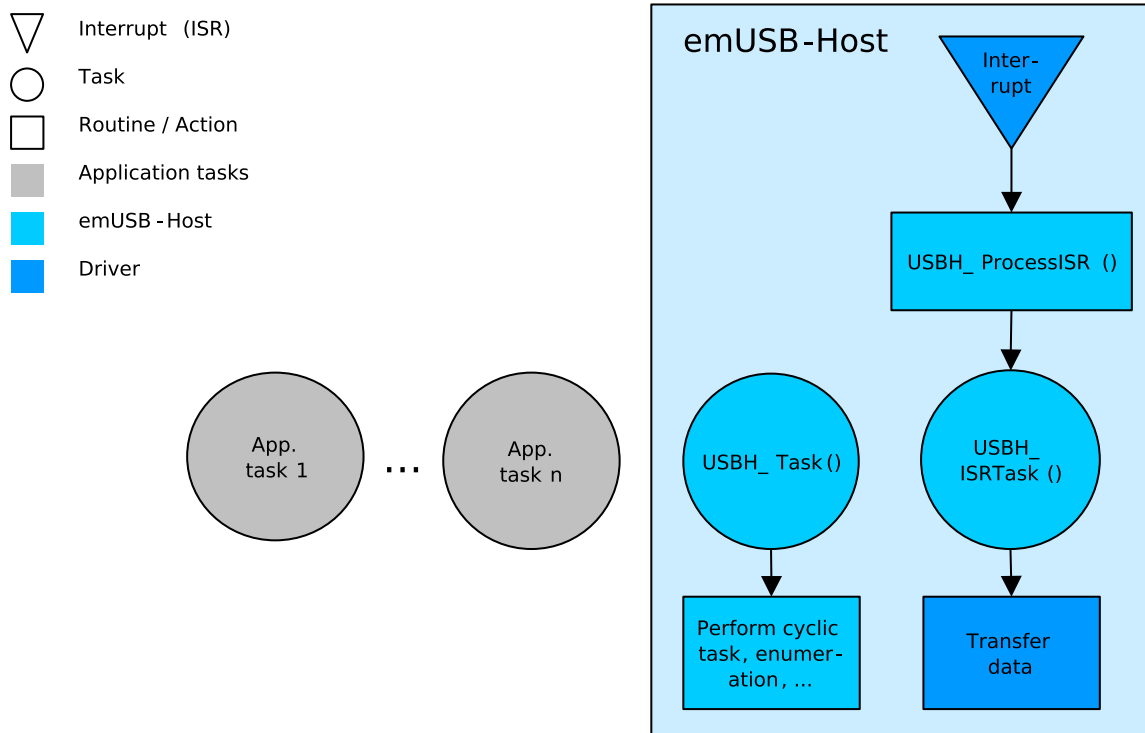
- `USBH_ISRTask` runs with the highest priority
- `USBH_Task` runs with a priority lower than `USBH_ISRTask`
- All application tasks run with a priority lower than `USBH_Task`

emUSBH API functions must not be called from interrupt functions, only from any task context.

Especially when using MSD it is easy to forget that the file system functions actually call emUSB-Host functions underneath. Therefore a task operating on the file system of a connected USB medium is considered an application task and must have a lower priority than `USBH_Task`.

Tasks which do not use emUSB-Host in any way can run at a higher priority than `USBH_ISRTask`. Even if a different high-priority task blocks the CPU for extended periods of time, USB communication should not be affected. USB communication is host-controlled, there are no timeouts on the device side and the host is free to delay the communication depending on how busy it is.

Your application must properly configure these two tasks at startup. The examples in the Application folder show how to do this.



1.5 Development environment (compiler)

The CPU used is of no importance; only an ANSI-compliant C compiler complying with at least one of the following international standard is required:

- ISO/IEC 9899:1999 (C99)
- ISO/IEC 14882:1998 (C++)

If your compiler has some limitations, let us know and we will inform you if these will be a problem when compiling the software. Any compiler for 16/32/64-bit CPUs or DSPs that we know of can be used. A C++ compiler is not required, but can be used. The application program can therefore also be programmed in C++ if desired.

1.6 Use of undocumented functions

Functions, variables and data-types which are not explained in this manual are considered internal. They are in no way required to use the software. Your application should not use and rely on any of the internal elements, as only the documented API functions are guaranteed to remain unchanged in future versions of the software. If you feel that it is necessary to use undocumented (internal) functions, please get in touch with SEGGER support in order to find a solution.

Chapter 2

USB Background information

This is a short introduction to USB. The fundamentals of USB are explained and links to additional resources are given.

2.1 Short Overview

The Universal Serial Bus (USB) is an external bus architecture for connecting peripherals to a host computer. It is an industry standard - maintained by the USB Implementers Forum - and because of its many advantages it enjoys a huge industry-wide acceptance. Over the years, a number of USB-capable peripherals appeared on the market, for example printers, keyboards, mice, digital cameras etc. Among the top benefits of USB are:

- Excellent plug-and-play capabilities allow devices to be added to the host system without reboots ("hot-plug"). Plugged-in devices are identified by the host and the appropriate drivers are loaded instantly.
- USB allows easy extensions of host systems without requiring host-internal extension cards.
- Device bandwidths may range from a few Kbytes/second to hundreds of Mbytes/ second.
- A wide range of packet sizes and data transfer rates are supported.
- USB provides internal error handling. Together with the hot-plug capability mentioned before this greatly improves robustness.
- The provisions for powering connected devices dispense the need for extra power supplies for many low power devices.
- Several transfer modes are supported which ensures the wide applicability of USB.

These benefits have not only led to broad market acceptance, but have also produced several other advantages, such as low costs of USB cables and connectors or a wide range of USB stack implementations. Last but not least, the major operating systems such as Microsoft Windows XP, Mac OS X, or Linux provide excellent USB support.

2.2 Important USB Standard Versions

USB 1.1 (September 1998)

This standard version supports isochronous and asynchronous data transfers. It has dual speed data transfer of 1.5 Mbit/s for low speed and 12 Mbit/s for full-speed devices. The maximum cable length between host and device is five meters. Up to 500 mA of electric current may be distributed to low power devices.

USB 2.0 (April 2000)

As all previous USB standards, USB 2.0 is fully forward and backward compatible. Existing cables and connectors may be reused. A new high-speed transfer speed of 480 Mbit/s (40 times faster than USB 1.1 at full-speed) was added.

USB 3.0 (November 2008)

As all previous USB standards, USB 3.0 is fully forward and backward compatible. Existing cables and connectors may be reused but the new speed can only be used with new USB 3.0 cables and devices. The new speed class is named USB Super-Speed, which offers a maximum rate of 5 Gbit/s.

USB 3.1 (July 2013)

As all previous USB standards, USB 3.1 is fully forward and backward compatible. The new specification replaces the 3.0 standard and introduces new transfer speeds of up to 10 Gbit/s.

2.3 USB System Architecture

A USB system is composed of three parts - a host side, a device side and a physical bus. The physical bus is represented by the USB cable and connects the host and the device. The USB system architecture is asymmetric. Every single host can be connected to multiple devices in a tree-like fashion using special hub devices. You can connect up to 127 devices to a single host, but the count must include the hub devices as well.

USB Host

A USB host consists of a USB host controller hardware and a layered software stack. This host stack contains:

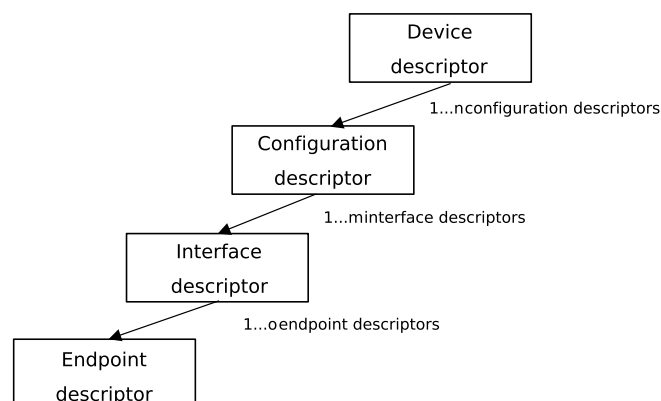
- A driver for the USB host controller hardware.
- The USB host stack which implements the high level functions used by the USB class drivers (including enumeration and hub support).
- One or more USB class drivers providing generic access to certain types of USB devices such as printers or mass storage devices.

USB Device

Two types of devices exist: Hubs and functions. Hubs usually provide four or more additional USB attachment points. Functions provide capabilities to the host and are able to transmit or receive data or control information over the USB bus. Every peripheral USB device represents at least one function but may implement more than one function. A USB printer for instance may provide file system like access in addition to printing. In this guide we treat the term USB device as synonymous with functions and will not consider hubs. Each USB device contains configuration information which describes its capabilities and resource requirements. Before it can be used a USB device must be configured by the host. When a new device is connected for the first time, the host enumerates it, requests the configuration from the device, and performs the actual configuration. For example, if a memory stick is connected to a USB host, it will appear as a USB mass storage device, and the host will use a standard MSD class implementation to access the device.

Descriptors

A device reports its attributes via descriptors. Descriptors are data structures with a standard defined format. A USB device has one device descriptor which contains information applicable to the device and all of its configurations. It also contains the number of configurations supported by the device. For each configuration, a configuration descriptor contains configuration-specific information. The configuration descriptor also contains the number of interfaces provided by the configuration. An interface groups the endpoints into logical units. Each interface descriptor contains information about the number of endpoints. Each endpoint has its own endpoint descriptor which states the endpoint's address, transfer types etc.



2.4 Transfer Types

The USB standard defines four transfer types: control, isochronous, interrupt, and bulk. Control transfers are used in the setup phase. The application can basically select one of the other three transfer types. For most embedded applications, bulk is the best choice because it allows the highest possible data rates.

Control transfers

Typically used for configuring a device when attached to the host. It may also be used for other device-specific purposes, including control of other pipes on the device.

Isochronous transfers

Typically used for applications which need guaranteed speed. Isochronous transfer is fast but with possible data loss. A typical use is for audio data which requires a constant data rate.

Interrupt transfers

Typically used by devices that need guaranteed quick responses (bounded latency).

Bulk transfers

Typically used by devices that generate or consume data in relatively large and burstly quantities. Bulk transfer has wide dynamic latitude in transmission constraints. It can use all remaining available bandwidth, but with no guarantees on bandwidth or latency. Because the USB bus is normally not very busy, there is typically 90% or more of the bandwidth available for USB transfers.

2.5 Setup phase / Enumeration

The host first needs to get information from the target before the target can start communicating with the host. This information is gathered in the initial setup phase. The information is contained in the descriptors. The most important part of target device identification are the Product and Vendor IDs. During the setup phase, the host also assigns an address to the device. This part of the setup is called enumeration.

2.6 Product / Vendor IDs

Each USB device can be identified by its a Vendor and Product ID. A USB host does not have a Vendor and Product ID.

2.7 Predefined device classes

The USB Implementers Forum has defined device classes for different purposes. In general, every device class defines a protocol for a particular type of application such as a mass storage device (MSD), human interface device (HID), etc.

Chapter 3

Running emUSB-Host on target hardware

This chapter explains how to integrate and run emUSB-Host on your target hardware.

3.1 Integrating emUSB-Host

We assume that you are familiar with the tools you have selected for your project (compiler, project manager, linker, etc.). You should therefore be able to add files, add directories to the include search path, and so on. In this document the Embedded Studio IDE is used for all examples and screenshots, but every other ANSI C toolchain can also be used. It is also possible to use makefiles; in this case, when we say “add to the project”, this translates into “add to the makefile”.

Procedure to follow

Integration of emUSB-Host is a relatively simple process, which consists of the following steps:

- Take a running project for your target hardware.
- Add emUSB-Host files to the project.
- Add hardware dependent configuration to the project.
- Prepare and run the application.

3.2 Take a running project

The project to start with should include the setup for basic hardware (e.g. CPU, PLL, DDR SDRAM) and initialization of the RTOS. emUSB-Host is designed to be used with embOS, SEGGER’s real-time operating system. We recommend to start with an embOS sample project and include emUSB-Host into this project.

3.3 Add emUSB-Host files

Add all necessary source files from the USBH folder to your project. You may simply add all files and let the linker drop everything not needed for your configuration. But there are some source files containing dependencies to emFile or emNet. If you don’t have these middleware components, remove the respective files from your project.

Add RTOS layer

Additionally add the RTOS interface layer to your project. Choose a file from the folder Sample/USBH/OS that matches your RTOS. For embOS use USBH_OS_embOS.c.

Configuring the include path

The include path is the path in which the compiler looks for include files. In cases where the included files (typically header files, .h) do not reside in the same folder as the C file to compile, an include path needs to be set. In order to build the project with all added files, you will need to add the following directories to your include path:

- Config
- Inc
- SEGGER
- USBH

3.4 Configuring debugging output

While developing and testing emUSB-Host, we recommend to use the DEBUG configuration of emUSB-Host. This is enabled by setting the preprocessor symbol DEBUG to 1 (or USBH_DEBUG to 2). The DEBUG configuration contains many additional run-time checks and generate debug output messages which are very useful to identify problems that may occur during development. In case of a fatal problem (e.g. an invalid configuration) the program will end up in the function *USBH_Panic()* with a appropriate error message that describes the cause of the problem.

Add the file `USBH_ConfigIO.c` found in the folder `Config` to your project and configure it to match the message output method used by your debugging tools. If possible use RTT.

To later compile a release configuration, which has a significant smaller code footprint, simply set the preprocessor symbol `DEBUG` (or `USBH_DEBUG`) to 0.

3.5 Add hardware dependent configuration

To perform target hardware dependent runtime configuration, the emUSB-Host stack calls a function named `USBH_X_Config`. Typical tasks that may be done inside this function are:

- Assign memory to be used by the emUSB-Host stack.
- Select an appropriate driver for the USB host controller.
- Configure I/O pins of the MCU for USB.
- Configure PLL and clock divider necessary for USB operation.
- Install an interrupt service routine for USB and set interrupt priority.

Details can be found in *Runtime configuration* on page 680.

Sample configurations for popular evaluation boards are supplied with the driver shipment. They can be found in files called `USBH_Config_<TargetName>.c` in the folders `BSP/<BoardName>/Setup`.

Add the appropriate configuration file to your project. If there is no configuration file for your target hardware, take a file for a similar hardware and modify it if necessary.

If the file needs modifications, we recommend to copy it into the directory `Config` for easy updates to later versions of emUSB-Host.

Add BSP file

Some targets require CPU specific functions for initialization, mainly for installing an interrupt service routine. They are contained in the file `BSP_USB.c`. USB interrupt priority can also be configured in `BSP_USB.c`.

Sample `BSP_USB.c` files for popular evaluation boards are supplied with the driver shipment. They can be found in the folders `BSP/<BoardName>/Setup`.

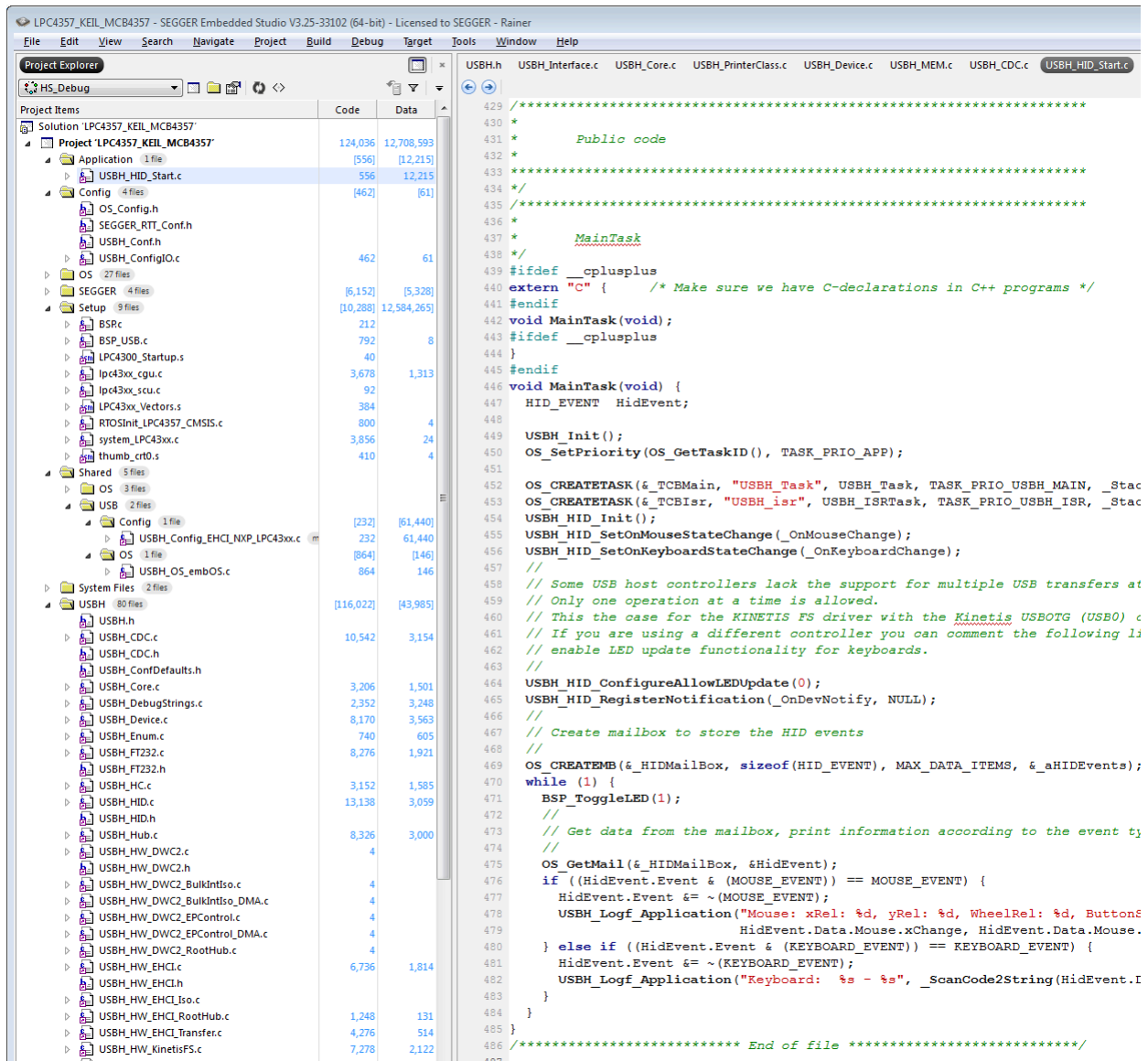
Add the appropriate `BSP_USB.c` file to your project. If there is no BSP file for your target hardware, take a file for a similar hardware and modify it if necessary.

If the file needs modifications, we recommend to copy it into the directory `Config` for easy updates to later versions of emUSB-Host.

Note that a `BSP_USB.c` file is not always required, because for some target hardware all runtime configuration is done in `USBH_X_Config`.

3.6 Prepare and run the application

Choose a sample application from the folder `Application` and add it to your project. Sample applications are described in *Example applications* on page 40. Compile and run the application on the target hardware.



3.7 Write your own application

Take one of the sample applications as a starting point to write your own application. In order to use emUSB-Host, the application has to:

- Initialize the USB core stack by calling `USBH_Init()`.
- Create two separate tasks that call the functions `USBH_Task()` and `USBH_ISRTask()`, respectively. Task priority requirements described in *Tasks and interrupt usage* on page 28 must be considered.
- Initialize the USB class drivers needed by calling the `USBH_<class>_Init()` function(s).

3.8 Updating emUSB-Host

If an existing project should be updated to a later emUSB-Host version, only files have to be replaced. You should have received the emUSB-Host update as a zip file. Unzip this file to the location of your choice and replace all emUSB-Host files in your project with the newer files from the emUSB-Host update shipment.

In general, all files from the following directories have to be updated:

- USBH
- Inc
- SEGGER
- Doc
- Sample/USBH/OS

Some files may contain modification required for project specific customization. These files should reside in the folder Config and must not be overwritten. This includes:

- USBH_Conf.h
- USBH_ConfigIO.c
- BSP_USB.c
- USBH_Config_<TargetName>.c

Chapter 4

Example applications

In this chapter, you will find a description of each emUSB-Host example application.

4.1 Overview

File	Description
USBH_HID_Start.c	Demonstrates the handling of mouse and keyboard events.
USBH_MSD_Start.c	Demonstrates how to handle mass storage devices.
USBH_Printer_Start.c	Shows how to interact with a printer.
USBH_CDC_Start.c	Demonstrates communication with CDC devices.
USBH_MTP_Start.c	Shows how to interact with smart phones and other MTP-enabled devices.
USBH_FT232_Start.c	Demonstrates communication with FTDI serial adapters.

The example applications for the target-side are supplied in source code in the Application folder of your shipment.

4.2 Mouse and keyboard events (USBH_HID_Start.c)

This example application displays in the terminal I/O of the debugger the events generated by a mouse and a keyboard connected over USB. A message in the form:

```
6:972 MainTask - Mouse: xRel: 0, yRel: 0, WheelRel: 0, ButtonState: 1
```

is generated each time the mouse generates an event. An event is generated when the mouse is moved, a button is pressed or the scroll-wheel is rolled. The message indicates the change in position over the vertical and horizontal axis, the scroll-wheel displacement and the status of all buttons. In case of a keyboard these two messages are generated when a key is pressed and then released:

```
386:203 MainTask - Keyboard: Key e/E - pressed  
386:287 MainTask - Keyboard: Key e/E - released
```

The keycode is displayed followed by its status.

4.3 Mass storage handling (USBH_MSD_Start.c)

This demonstrates the handling of mass storage devices. A small test is run as soon as a mass storage device is connected to host. The results of the test are displayed in the terminal I/O window of the debugger. If the medium is not formatted only the message "Medium is not formatted." is shown and the application waits for a new device to be connected. In case the medium is formatted the file system is mounted and the total disk space is displayed. The test goes on and creates a file named TestFile.txt in the root directory of the disk followed by a listing of the files in the root directory. The value returned by OS_GetTime() is stored in the created file. At the end of test the file system is unmounted and information about the mass storage device is displayed like Vendor ID and name. Information similar to the following is shown when a memory stick is connected:

```
<...>
**** Device added
Running sample on "msd:0:"

Reading volume information...

**** Volume information for msd:0:
125105536 KBytes total disk space
125105536 KBytes available free space

32768 bytes per cluster
3909548 clusters available on volume
3909548 free cluster available on volume

Creating file msd:0:\TestFile.txt...Ok
Contents of msd:0:
TESTFILE.TXT Attributes: A--- Size: 21

**** Unmount ****

Test with the following device was successful:
VendorId: 0x1234
ProductId: 0x5678
VendorName: XXXXXXXX
ProductName: XXXXXXXXXXXXXXXX
Revision: 1.00
NumSectors: 250272768
BytesPerSector: 512
TotalSize: 122203 MByte
HighspeedCapable: No
ConnectedToRootHub: Yes
SelfPowered: No
Reported Imax: 500 mA
Connected to Port: 1
PortSpeed: FullSpeed
```

4.4 Printer interaction (USBH_Printer_Start.c)

This example shows how to communicate with a printer connected over USB. As soon as a printer connects over USB the message "**** Device added" is displayed on the terminal I/O window of the debugger followed by the device ID of the printer and the port status. After that the ASCII text "Hello World" and a form feed is sent to the printer.

Terminal output:

```
**** Device added
Device Id = MFG:Hewlett-Packard;CMD:PJL,PML,POSTSCRIPT,PCLXL,PCL;MDL:HP
LaserJet P2015 Series;CLS:PRINTER;DES:Hewlett-Packard LaserJet P2015
Series;MEM:MEM=23MB;COMMENT:RES=1200x1;
PortStatus = 0x18 ->NoError=1, Select/OnLine=1, PaperEmpty=0
Printing Hello World to printer
Printing completed

**** Device removed
```

4.5 Serial communication (USBH_CDC_Start.c)

This example shows how to communicate with a CDC-enabled device. Since CDC is just a transport protocol it is not possible to write a generic sample which will work with all devices. This sample is designed to be used with a emUSB-Device CDC counterpart, the "USB_CDC_Echo.c" sample. It can also be used with any other device, but it may not be able to demonstrate continuous communication. The sample works as follows:

- When a CDC is connected the sample prints generic information about the device.
- After that the sample writes data onto the device.
- The sample reads data from the device and in case it has received any - sends it back.

With the emUSB-Device "USB_CDC_Echo.c" sample this causes a simple, continuous ping-pong of messages.

Terminal output:

```
**** Device added
<...>
0:663 USBH_isr - INIT: USBH_ISRTask started

**** Device added
Vendor Id = 0x1234
Product Id = 0x5678
Serial no. = 123456789
Speed      = HighSpeed
Started communication...
<...>
**** Device removed
```

4.6 Media Transfer Protocol (USBH_MTP_Start.c)

This example shows how to communicate with a MTP-enabled device. The sample demonstrates most of the emUSB-Host MTP API. When a MTP device is connected the sample prints generic information about the device. If the device is locked (e.g. pin code on a smart phone) the sample will wait for the user to unlock it. The sample will then iterate over the storages made available by the device, print information about it, print the file and folder list in the root directory and create a new file under it called "SEGGER_Test.txt".

Terminal output:

```
<...>
**** Device added
Vendor Id      = 0x1234
Product Id     = 0x1234
Serial no.     = 1
Speed         = FullSpeed
Manufacturer   : XXXXXX
Model          : XXXXXXXXXXXXXXXXXXXXXXXX
DeviceVersion  : 8.10.12397.0
MTP SerialNumber : 844848fb44583cbaa1ecae45545b3

USBH_MTP_CheckLock returns USBH_STATUS_ERROR
Please unlock the device to proceed.
<...>
_cbOnUserEvent: MTP Event received! EventCode: 0x4004, Para1: 0x00010001
                , Para2: 0x00000000, Para3: 0x00000000.
<...>
USBH_MTP_CheckLock returns USBH_STATUS_SUCCESS
Found storage with ID: 0
StorageType      = 0x0003
FilesystemType   = 0x0002
AccessCapability = 0x0000
MaxCapacity      = 3959422976 bytes
FreeSpaceInBytes = 1033814016 bytes
FreeSpaceInImages = 0x00000000
StorageDescription : Phone
VolumeLabel      : MTP Volume - 65537

Found 9 objects in directory 0xFFFFFFFF

Processing object 0x00000001 in directory 0xFFFFFFFF...
StorageID        = 0x00010001
ObjectFormat     = 0x3001
ParentObject     = 0xFFFFFFFF
ProtectionStatus = 0x0001
Filename         : Documents
CaptureDate      : 20140522T0
ModificationDate : 20160707T1

Processing object 0x00000002 in directory 0xFFFFFFFF...
StorageID        = 0x00010001
ObjectFormat     = 0x3001
ParentObject     = 0xFFFFFFFF
ProtectionStatus = 0x0001
Filename         : Downloads
CaptureDate      : 20140522T0
ModificationDate : 20160707T1
<...>
Creating new object with 135 bytes in folder 0xFFFFFFFF
with name SEGGER_Test.txt.
Created new object in folder 0xFFFFFFFF, ID: 0x000013F9.

Connection to MTP device closed.
<...>
**** Device removed
```

4.7 FTDI devices (USBH_FT232_Start.c)

This example shows how to communicate with a FTDI FT232 adapters. When a FT232 is connected the sample prints generic information about the device. After that it receives data from the connected FT232 adapter and sends it back. The sample is easily tested by using two identical FT232 adapters connected to each other via a null modem cable. One of the devices should be connected to emUSB-Host. The other to a PC. You can use any PC terminal emulator to send data from one adapter to the other, which will be then received by emUSB-Host and sent back. Baudrate and other serial setting should match between the sample and the PC for this to work.

Terminal output:

```
**** Device added
<...>
3:213 MainTask - Vendor Id = 0x0403
3:213 MainTask - Product Id = 0x6001
3:213 MainTask - bcdDevice = 0x0600
<...>
**** Device removed
```

Chapter 5

USB Host Core

In this chapter, you will find a description of all API functions as well as all required data and function types.

5.1 Target API

This section describes the functions that can be used by the target application.

Function	Description
General	
USBH_Init()	Initializes the emUSB-Host stack.
USBH_Exit()	Shuts down and de-initializes the emUSB-Host stack.
USBH_ISRTask()	Processes the events triggered from the interrupt handler.
USBH_Task()	Manages the internal software timers.
USBH_IsRunning()	Returns whether the stack is running or not.
USBH_GetFrameNumber()	Retrieves the current frame number.
USBH_GetStatusStr()	Converts the result status into a string.
USBH_MEM_GetMaxUsed()	Returns the maximum used memory since initialization of the memory pool.
USBH_SetRootPortPower()	Set port of the root hub to a given state.
USBH_SetHubPortPower()	Set port of an external hub to a given power state.
USBH_HUB_SuspendResume()	Prepares hubs for suspend (stops the interrupt endpoint) or re-starts the interrupt endpoint functionality after a resume.
USBH_GetNumRootPortConnections()	Determine how many devices are directly connected to the host controllers root hub ports.
Runtime configuration	
USBH_AssignMemory()	Assigns a memory area that will be used by the memory management functions for allocating memory.
USBH_AssignTransferMemory()	Assigns a memory area for a heap that will be used for allocating DMA memory.
USBH_Config_SetV2PHandler()	Sets a virtual address to physical address translator.
USBH_ConfigPowerOnGoodTime()	Configures the power on time that the host waits after connecting a device before starting to communicate with the device.
USBH_ConfigSupportExternalHubs()	Enable support for external USB hubs.
USBH_ConfigTransferBufferSize()	Configures the size of a copy buffer that can be used if the USB controller has limited access to the system memory or the system is using cached (data) memory.
USBH_SetCacheConfig()	Configures cache related functionality that might be required by the stack for several purposes such as cache handling in drivers.
USBH_SetOnSetPortPower()	Sets a callback for the set-port-power driver function.

Function	Description
<code>USBH_ConfigPortPowerPinEx()</code>	Setups how the port-power pin should be set in order to enable port for this port.
<code>USBH_SetOnPortEvent()</code>	Sets a callback to report port events to the application.
<code>USBH_ConfigOvercurrentDetection()</code>	Configures how the driver detects an overcurrent condition.
Information about interfaces	
<code>USBH_CreateInterfaceList()</code>	Generates a list of available interfaces matching a given criteria.
<code>USBH_DestroyInterfaceList()</code>	Destroy a device list created by <code>USBH_CreateInterfaceList()</code> and free the related resources.
<code>USBH_GetInterfaceId()</code>	Returns the interface id for a specified interface.
<code>USBH_GetInterfaceInfo()</code>	Obtain information about a specified interface.
<code>USBH_GetInterfaceSerial()</code>	Retrieves the serial number of the device containing the given interface.
<code>USBH_GetIADInfo()</code>	Obtains information about the corresponding Interface Association Descriptor for an interface ID (if one is available).
USB interface handling	
<code>USBH_CloseInterface()</code>	Close an interface handle that was opened with <code>USBH_OpenInterface()</code> .
<code>USBH_GetCurrentConfigurationDescriptor()</code>	Retrieves the current configuration descriptor of the device containing the given interface.
<code>USBH_GetDeviceDescriptor()</code>	Obsolete function, use <code>USBH_GetDeviceDescriptorPtr()</code> .
<code>USBH_GetDeviceDescriptorPtr()</code>	Returns a pointer to the device descriptor structure of the device containing the given interface.
<code>USBH_GetEndpointDescriptor()</code>	Retrieves an endpoint descriptor of the device containing the given interface.
<code>USBH_GetInterfaceDescriptor()</code>	Retrieves the interface descriptor of the given interface.
<code>USBH_GetInterfaceIdByHandle()</code>	Get the interface ID for a given index.
<code>USBH_GetSerialNumber()</code>	Retrieves the serial number of the device containing the given interface.
<code>USBH_GetSerialNumberASCII()</code>	Retrieves the serial number of the device containing the given interface.
<code>USBH_GetStringDescriptorASCII()</code>	Retrieves a string from a string descriptor from the device containing the given interface.
<code>USBH_GetStringDescriptor()</code>	Retrieves the raw string descriptor from the device containing the given interface.
<code>USBH_GetSpeed()</code>	Returns the operating speed of the device.
<code>USBH_OpenInterface()</code>	Opens the specified interface.

Function	Description
USBH_GetPortInfo()	Obtains information about a connected USB device.
USBH_SubmitUrb()	Submits an URB.
USBH_IsoDataCtrl()	Acknowledge ISO data received from an IN EP or provide data for OUT EPs.
Notification	
USBH_RegisterEnumErrorNotification()	Registers a notification for a port enumeration error.
USBH_RegisterPnPNotification()	Registers a notification function for PnP events.
USBH_RestartEnumError()	Restarts the enumeration process for all devices that have failed to enumerate.
USBH_UnregisterEnumErrorNotification()	Removes a registered notification for a port enumeration error.
USBH_UnregisterPnPNotification()	Removes a previously registered notification for PnP events.

5.1.1 USBH_AssignMemory()

Description

Assigns a memory area that will be used by the memory management functions for allocating memory. This function must be called in the initialization phase.

Prototype

```
void USBH_AssignMemory(void * pMem,  
                       U32    NumBytes);
```

Parameters

Parameter	Description
pMem	Pointer to the memory area.
NumBytes	Size of the memory area in bytes.

Additional information

emUSB-Host comes with its own dynamic memory allocator optimized for its needs. This function is used to set up up a memory area for the heap. The best place to call it is in the `USBH_X_Config()` function.

For some USB host controllers additionally a separate memory heap for DMA memory must be provided by calling `USBH_AssignTransferMemory()`.

5.1.2 USBH_AssignTransferMemory()

Description

Assigns a memory area for a heap that will be used for allocating DMA memory. This function must be called in the initialization phase.

The memory area provided to this function must fulfill the following requirements:

- Not cachable/bufferable.
- Fast access to avoid timeouts.
- USB-Host controller must have full read/write access.
- Cache aligned

If the physical address is not equal to the virtual address of the memory area (address translation by an MMU), additionally a mapping function must be installed using `USBH_Config_SetV2PHandler()`.

Prototype

```
void USBH_AssignTransferMemory(void * pMem,  
                               U32   NumBytes);
```

Parameters

Parameter	Description
<code>pMem</code>	Pointer to the memory area (virtual address).
<code>NumBytes</code>	Size of the memory area in bytes.

Additional information

Use of this function is required only in systems in which “normal” default memory does not fulfill all of these criteria. In simple microcontroller systems without cache, MMU and external RAM, use of this function is not required. If no transfer memory is assigned, memory assigned with `USBH_AssignMemory()` is used instead.

5.1.3 USBH_CloseInterface()

Description

Close an interface handle that was opened with `USBH_OpenInterface()`.

Prototype

```
void USBH_CloseInterface(USBH_INTERFACE_HANDLE hInterface);
```

Parameters

Parameter	Description
<code>hInterface</code>	Handle to a valid interface, returned by <code>USBH_OpenInterface()</code> .

Additional information

Each handle must be closed one time. Calling this function with an invalid handle leads to undefined behavior.

5.1.4 USBH_Config_SetV2PHandler()

Description

Sets a virtual address to physical address translator. Is required, if the physical address is not equal to the virtual address of the memory used for DMA access (address translation by an MMU). See USBH_AssignTransferMemory.

Prototype

```
void USBH_Config_SetV2PHandler(USBH_V2P_FUNC * pfV2PHandler);
```

Parameters

Parameter	Description
pfV2PHandler	Handler to be called to convert virtual address.

5.1.5 USBH_ConfigPowerOnGoodTime()

Description

Configures the power on time that the host waits after connecting a device before starting to communicate with the device. The default value is 300 ms.

Prototype

```
void USBH_ConfigPowerOnGoodTime(unsigned PowerGoodTime);
```

Parameters

Parameter	Description
PowerGoodTime	Time the stack should wait before doing any other operation (in ms).

Additional information

If you are dealing with problematic devices which have long initialization sequences it is advisable to increase this timeout.

5.1.6 USBH_ConfigSupportExternalHubs()

Description

Enable support for external USB hubs.

Prototype

```
void USBH_ConfigSupportExternalHubs(U8 OnOff);
```

Parameters

Parameter	Description
OnOff	1 - Enable support for external hubs 0 - Disable support for external hubs.

Additional information

This function should not be called if no external hub support is required to avoid the code for external hubs to be linked into the application.

5.1.7 USBH_ConfigTransferBufferSize()

Description

Configures the size of a copy buffer that can be used if the USB controller has limited access to the system memory or the system is using cached (data) memory. Transfer buffers of this size are allocated for each used endpoint. If this function is not called, a driver specific default size is used. The value configured for this function is used for bulk and interrupt endpoints only.

Prototype

```
void USBH_ConfigTransferBufferSize(U32 HCIndex,  
                                   U32 Size);
```

Parameters

Parameter	Description
HCIndex	Index of the host controller.
Size	Size of the buffer in bytes. Must be a multiple of the maximum packet size (512 for high speed, 64 for full speed).

5.1.8 USBH_CreateInterfaceList()

Description

Generates a list of available interfaces matching a given criteria.

Prototype

```
USBH_INTERFACE_LIST_HANDLE USBH_CreateInterfaceList
                                (const USBH_INTERFACE_MASK * pInterfaceMask,
                                 unsigned int                 * pInterfaceCount);
```

Parameters

Parameter	Description
<code>pInterfaceMask</code>	Pointer to a caller provided structure, that allows to select interfaces to be included in the list. If this pointer is NULL all available interfaces are returned.
<code>pInterfaceCount</code>	Pointer to a variable that receives the number of interfaces in the list created.

Return value

On success it returns a handle to the interface list. In case of an error it returns NULL.

Additional information

The generated interface list is stored in the emUSB-Host and must be deleted by a call to `USBH_DestroyInterfaceList()`. The list contains a snapshot of interfaces available at the point of time where the function is called. This enables the application to have a fixed relation between the index and a USB interface in a list. The list is not updated if a device is removed or connected. A new list must be created to capture the current available interfaces. Hub devices are not added to the list!

Example

```

/*****
 *
 *      _ListJLinkDevices
 *
 *      Function description
 *      Generates a list of JLink devices connected to host.
 */
static void _ListJLinkDevices(void) {
    USBH_INTERFACE_MASK IfaceMask;
    unsigned int IfaceCount;
    USBH_INTERFACE_LIST_HANDLE hIfaceList;

    memset(&IfaceMask, 0, sizeof(IfaceMask));
    //
    // We want a list of all SEGGER J-Link devices connected to our host.
    // The devices are selected by their Vendor and Product ID.
    // Other identification information is not taken into account.
    //
    IfaceMask.Mask = USBH_INFO_MASK_VID | USBH_INFO_MASK_PID;
    IfaceMask.VendorId = 0x1366;
    IfaceMask.ProductId = 0x0101;
    hIfaceList = USBH_CreateInterfaceList(&IfaceMask, &IfaceCount);
    if (hIfaceList == NULL) {
        USBH_Warnf_Application("Cannot create the interface list!");
    } else {
        if (IfaceCount == 0) {
            USBH_Logf_Application("No devices found.");
        } else {

```

```
    unsigned int i;
    USBH_INTERFACE_ID IfaceId;
    //
    // Traverse the list of devices and display information about each of them
    //
    for (i = 0; i < IfaceCount; ++i) {
        //
        // An interface is addressed by its ID
        //
        IfaceId = USBH_GetInterfaceId(hIfaceList, i);
        _ShowIfaceInfo(IfaceId);
    }
    //
    // Ensure the list is properly cleaned up
    //
    USBH_DestroyInterfaceList(hIfaceList);
}
}
```

5.1.9 USBH_DestroyInterfaceList()

Description

Destroy a device list created by `USBH_CreateInterfaceList()` and free the related resources.

Prototype

```
void USBH_DestroyInterfaceList(USBH_INTERFACE_LIST_HANDLE hInterfaceList);
```

Parameters

Parameter	Description
<code>hInterfaceList</code>	Valid handle to a interface list, returned by <code>USBH_CreateInterfaceList()</code> .

5.1.10 USBH_Exit()

Description

Shuts down and de-initializes the emUSB-Host stack. All resources will be freed within this function. This includes also the removing and deleting of all host controllers.

Before this function can be used, the exit functions of all initialized USB classes (e.g. USBH_CDC_Exit(), USBH_MSD_Exit(), ...) must be called.

Calling USBH_Exit() will cause the functions USBH_Task() and USBH_ISRTask() to return.

Prototype

```
void USBH_Exit(void);
```

Additional information

After this function call, no other function of the USB stack should be called.

5.1.11 USBH_GetCurrentConfigurationDescriptor()

Description

Retrieves the current configuration descriptor of the device containing the given interface.

Prototype

```
USBH_STATUS USBH_GetCurrentConfigurationDescriptor
(
    USBH_INTERFACE_HANDLE hInterface,
    U8 * pDescriptor,
    unsigned * pBufferSize);
```

Parameters

Parameter	Description
hInterface	Valid handle to an interface, returned by <code>USBH_OpenInterface()</code> .
pDescriptor	Pointer to a buffer where the descriptor is stored.
pBufferSize	in Size of the buffer pointed to by pDescriptor. out Number of bytes copied into the buffer.

Return value

`USBH_STATUS_SUCCESS` on success. Other values indicate an error.

Additional information

The function returns a copy of the current configuration descriptor, that was stored during the device enumeration. If the given buffer size is too small the configuration descriptor returned is truncated.

5.1.12 USBH_GetDeviceDescriptor()

Description

Obsolete function, use `USBH_GetDeviceDescriptorPtr()`. Retrieves the current device descriptor of the device containing the given interface.

Prototype

```
USBH_STATUS USBH_GetDeviceDescriptor(USBH_INTERFACE_HANDLE hInterface,  
                                     U8 * pDescriptor,  
                                     unsigned * pBufferSize);
```

Parameters

Parameter	Description
<code>hInterface</code>	Valid handle to an interface, returned by <code>USBH_OpenInterface()</code> .
<code>pDescriptor</code>	Pointer to a buffer where the descriptor is stored.
<code>pBufferSize</code>	<code>in</code> Size of the buffer pointed to by <code>pDescriptor</code>. <code>out</code> Number of bytes copied into the buffer.

Return value

`USBH_STATUS_SUCCESS` on success. Other values indicate an error.

Additional information

The function returns a copy of the current device descriptor, that was stored during the device enumeration. If the given buffer size is too small the device descriptor returned is truncated.

5.1.13 USBH_GetDeviceDescriptorPtr()

Description

Returns a pointer to the device descriptor structure of the device containing the given interface.

Prototype

```
USBH_DEVICE_DESCRIPTOR *USBH_GetDeviceDescriptorPtr  
    (USBH_INTERFACE_HANDLE hInterface);
```

Parameters

Parameter	Description
hInterface	Valid handle to an interface, returned by <code>USBH_OpenInterface()</code> .

Return value

Pointer to the current device descriptor information (read only), that was stored during the device enumeration. The pointer gets invalid, when the interface is closed using `USBH_CloseInterface()`.

5.1.14 USBH_GetEndpointDescriptor()

Description

Retrieves an endpoint descriptor of the device containing the given interface.

Prototype

```
USBH_STATUS USBH_GetEndpointDescriptor
(
    USBH_INTERFACE_HANDLE hInterface,
    U8 AlternateSetting,
    const USBH_EP_MASK * pMask,
    U8 * pBuffer,
    unsigned * pBufferSize);
```

Parameters

Parameter	Description
<code>hInterface</code>	Valid handle to an interface, returned by <code>USBH_OpenInterface()</code> .
<code>AlternateSetting</code>	Specifies the alternate setting for the interface. The function returns endpoint descriptors that are inside the specified alternate setting.
<code>pMask</code>	Pointer to a structure of type <code>USBH_EP_MASK</code> , that specifies the endpoint selection pattern.
<code>pBuffer</code>	Pointer to a buffer where the descriptor is stored.
<code>pBufferSize</code>	<code>in</code> Size of the buffer pointed to by <code>pBuffer</code>. <code>out</code> Number of bytes copied into the buffer.

Return value

`USBH_STATUS_SUCCESS` on success. Other values indicate an error.

Additional information

The endpoint descriptor is extracted from the current configuration descriptor, that was stored during the device enumeration. If the given buffer size is too small the endpoint descriptor returned is truncated.

5.1.15 USBH_GetFrameNumber()

Description

Retrieves the current frame number.

Prototype

```
USBH_STATUS USBH_GetFrameNumber(USBH_INTERFACE_HANDLE hInterface,  
                                U32 * pFrameNumber);
```

Parameters

Parameter	Description
hInterface	Valid handle to an interface, returned by <code>USBH_OpenInterface()</code> .
pFrameNumber	Pointer to a variable that receives the frame number.

Return value

`USBH_STATUS_SUCCESS` on success. Other values indicate an error.

Additional information

The frame number is transferred on the bus with 11 bits. This frame number is returned as a 16 or 32 bit number related to the implementation of the host controller. The last 11 bits are equal to the current frame. The frame number is increased each millisecond if the host controller is running in full-speed mode, or each 125 microsecond if the host controller is running in high-speed mode. The returned frame number is related to the bus where the device is connected. The frame numbers between different host controllers can be different.

CAUTION: The functionality is not implemented for all host drivers. For some host controllers the function may always return a frame number of 0.

5.1.16 USBH_GetInterfaceDescriptor()

Description

Retrieves the interface descriptor of the given interface.

Prototype

```
USBH_STATUS USBH_GetInterfaceDescriptor(USBH_INTERFACE_HANDLE hInterface,  
                                         U8 AlternateSetting,  
                                         U8 * pBuffer,  
                                         unsigned * pBufferSize);
```

Parameters

Parameter	Description
hInterface	Valid handle to an interface, returned by <code>USBH_OpenInterface()</code> .
AlternateSetting	Specifies the alternate setting for this interface.
pBuffer	Pointer to a buffer where the descriptor is stored.
pBufferSize	in Size of the buffer pointed to by pBuffer. out Number of bytes copied into the buffer.

Return value

USBH_STATUS_SUCCESS on success. Other values indicate an error.

Additional information

The interface descriptor is extracted from the current configuration descriptor, that was stored during the device enumeration. The interface descriptor belongs to the interface that is identified by [hInterface](#). If the interface has different alternate settings the interface descriptors of each alternate setting can be requested.

If the given buffer size is too small the interface descriptor returned is truncated.

5.1.17 USBH_GetInterfaceId()

Description

Returns the interface id for a specified interface.

Prototype

```
USBH_INTERFACE_ID USBH_GetInterfaceId(USBH_INTERFACE_LIST_HANDLE hInterfaceList,  
                                       unsigned int                Index);
```

Parameters

Parameter	Description
hInterfaceList	Valid handle to a interface list, returned by <code>USBH_CreateInterfaceList()</code> .
Index	Specifies the zero based index for an interface in the list.

Return value

On success the interface Id for the interface specified by [Index](#) is returned. If the interface index does not exist the function returns 0.

Additional information

The interface ID identifies a USB interface as long as the device is connected to the host. If the device is removed and re-connected a new interface ID is assigned. The interface ID is even valid if the interface list is deleted. The function can return an interface ID even if the device is removed between the call to the function `USBH_CreateInterfaceList()` and the call to this function. If this is the case, the function `USBH_OpenInterface()` fails.

Example

See *USBH_CreateInterfaceList* on page 59.

5.1.18 USBH_GetInterfaceIdByHandle()

Description

Get the interface ID for a given index. A returned value of zero indicates an error.

Prototype

```
USBH_STATUS USBH_GetInterfaceIdByHandle(USBH_INTERFACE_HANDLE hInterface,  
                                         USBH_INTERFACE_ID * pInterfaceId);
```

Parameters

Parameter	Description
hInterface	Handle to a valid interface, returned by <code>USBH_OpenInterface()</code> .
pInterfaceId	Pointer to a variable that will receive the interface id.

Return value

`USBH_STATUS_SUCCESS` on success. Any other value means error.

Additional information

Returns the interface ID if the handle to the interface is available. This may be useful if a Plug and Play notification is received and the application checks if it is related to a given handle. The application can avoid calls to this function if the interface ID is stored in the device context of the application.

5.1.19 USBH_GetInterfaceInfo()

Description

Obtain information about a specified interface.

Prototype

```
USBH_STATUS USBH_GetInterfaceInfo(USBH_INTERFACE_ID    InterfaceID,  
                                  USBH_INTERFACE_INFO * pInterfaceInfo);
```

Parameters

Parameter	Description
InterfaceID	ID of the interface to query.
pInterfaceInfo	Pointer to a caller allocated structure that will receive the interface information on success.

Return value

USBH_STATUS_SUCCESS on success. Any other value means error.

Additional information

Can be used to identify a USB interface without having to open it. More detailed information can be requested after the USB interface is opened.

If the interface belongs to a device which is no longer connected to the host USBH_STATUS_DEVICE_REMOVED is returned and [pInterfaceInfo](#) is not filled.

5.1.20 USBH_GetInterfaceSerial()

Description

Retrieves the serial number of the device containing the given interface.

Prototype

```
USBH_STATUS USBH_GetInterfaceSerial(USBH_INTERFACE_ID  InterfaceID,
                                     U32                BuffSize,
                                     U8                  * pSerialNumber,
                                     U32                * pSerialNumberSize);
```

Parameters

Parameter	Description
InterfaceID	ID of the interface to query.
BuffSize	Size of the buffer pointed to by pSerialNumber.
pSerialNumber	Pointer to a buffer where the serial number is stored.
pSerialNumberSize	out Number of bytes copied into the buffer.

Return value

USBH_STATUS_SUCCESS on success. Other values indicate an error.

Additional information

The serial number is returned as a UNICODE string in USB little endian format. The number of valid bytes is returned in pSerialNumberSize. The string is not zero terminated. The returned data does not contain a USB descriptor header and is encoded in the first language Id. This string is a copy of the serial number string that was requested during the enumeration. If the device does not support a USB serial number string the function returns USBH_STATUS_SUCCESS and a length of 0. If the given buffer size is too small the serial number returned is truncated.

5.1.21 USBH_GetIADInfo()

Description

Obtains information about the corresponding Interface Association Descriptor for an interface ID (if one is available).

Prototype

```
USBH_STATUS USBH_GetIADInfo(USBH_INTERFACE_ID  InterfaceID,  
                           USBH_IAD_INFO      * pIADInfo);
```

Parameters

Parameter	Description
InterfaceID	ID of an interface of the device to query.
pIADInfo	Pointer to a caller allocated structure that will receive the IAD information on success.

Return value

USBH_STATUS_SUCCESS on success. Any other value means error.

5.1.22 USBH_GetPortInfo()

Description

Obtains information about a connected USB device.

Prototype

```
USBH_STATUS USBH_GetPortInfo(USBH_INTERFACE_ID  InterfaceID,  
                             USBH_PORT_INFO      * pPortInfo);
```

Parameters

Parameter	Description
InterfaceID	ID of an interface of the device to query.
pPortInfo	Pointer to a caller allocated structure that will receive the port information on success.

Return value

USBH_STATUS_SUCCESS on success. Any other value means error.

5.1.23 USBH_GetSerialNumber()

Description

Retrieves the serial number of the device containing the given interface. The serial number is returned as a UNICODE string in little endian format. The number of valid bytes is returned in `pBufferSize`. The string is not zero terminated. The returned data does not contain a USB descriptor header and is encoded in the first language Id. This string is a copy of the serial number string that was requested during the enumeration. If the device does not support a USB serial number string the function returns `USBH_STATUS_SUCCESS` and a length of 0. If the given buffer size is too small the serial number returned is truncated.

Prototype

```
USBH_STATUS USBH_GetSerialNumber(USBH_INTERFACE_HANDLE hInterface,  
                                U8 * pBuffer,  
                                unsigned * pBufferSize);
```

Parameters

Parameter	Description
<code>hInterface</code>	Valid handle to an interface, returned by <code>USBH_OpenInterface()</code> .
<code>pBuffer</code>	Pointer to a buffer where the serial number is stored.
<code>pBufferSize</code>	<code>in</code> Size of the buffer pointed to by <code>pBuffer</code>. <code>out</code> Number of bytes copied into the buffer.

Return value

`USBH_STATUS_SUCCESS` on success. Other values indicate an error.

5.1.24 USBH_GetSerialNumberASCII()

Description

Retrieves the serial number of the device containing the given interface. The serial number is returned as 0 terminated string. The returned data does not contain a USB descriptor header and is encoded in the first language Id. This string is a copy of the serial number string that was requested during the enumeration. Non-ASCII characters are replaced by '@'. If the device does not support a USB serial number string the function returns USBH_STATUS_SUCCESS and a zero length string. If the given buffer size is too small the serial number returned is truncated. The maximum string length returned is BuffSize - 1.

Prototype

```
USBH_STATUS USBH_GetSerialNumberASCII(USBH_INTERFACE_HANDLE hInterface,
                                     char * pBuffer,
                                     unsigned BufferSize);
```

Parameters

Parameter	Description
hInterface	Valid handle to an interface, returned by USBH_OpenInterface().
pBuffer	Pointer to a buffer where the serial number is stored.
BufferSize	Size of the buffer pointed to by pBuffer.

Return value

USBH_STATUS_SUCCESS on success. Other values indicate an error.

5.1.25 USBH_GetStringDescriptorASCII()

Description

Retrieves a string from a string descriptor from the device containing the given interface. The string returned is 0-terminated. The returned data does not contain a USB descriptor header and is encoded in the first language Id. Non-ASCII characters are replaced by '@'. If the given buffer size is too small the string is truncated. The maximum string length returned is `BufferSize - 1`.

Prototype

```
USBH_STATUS USBH_GetStringDescriptorASCII(USBH_INTERFACE_HANDLE hInterface,  
                                           U8 StringIndex,  
                                           char * pBuffer,  
                                           unsigned BufferSize);
```

Parameters

Parameter	Description
<code>hInterface</code>	Valid handle to an interface, returned by <code>USBH_OpenInterface()</code> .
<code>StringIndex</code>	Index of the string.
<code>pBuffer</code>	Pointer to a buffer where the string is stored.
<code>BufferSize</code>	Size of the buffer pointed to by <code>pBuffer</code> .

Return value

`USBH_STATUS_SUCCESS` on success. Other values indicate an error.

5.1.26 USBH_GetStringDescriptor()

Description

Retrieves the raw string descriptor from the device containing the given interface. First two bytes of descriptor are type (always USB_STRING_DESCRIPTOR_TYPE) and length. The rest contains a UTF-16 LE string. If the given buffer size is too small the string is truncated.

Prototype

```
USBH_STATUS USBH_GetStringDescriptor(USBH_INTERFACE_HANDLE hInterface,  
                                     U8 StringIndex,  
                                     U16 LangID,  
                                     U8 * pBuffer,  
                                     unsigned * pNumBytes);
```

Parameters

Parameter	Description
hInterface	Valid handle to an interface, returned by USBH_OpenInterface() .
StringIndex	Index of the string.
LangID	Language index. The default language of a device has the ID 0. See "Universal Serial Bus Language Identifiers (LANGIDs) version 1.0" for more details. This document is available on usb.org .
pBuffer	Pointer to a buffer where the string is stored.
pNumBytes	in Size of the buffer pointed to by pBuffer. out Number of bytes copied into the buffer.

Return value

USBH_STATUS_SUCCESS on success. Other values indicate an error.

5.1.27 USBH_GetSpeed()

Description

Returns the operating speed of the device.

Prototype

```
USBH_STATUS USBH_GetSpeed(USBH_INTERFACE_HANDLE hInterface,  
                           USBH_SPEED * pSpeed);
```

Parameters

Parameter	Description
hInterface	Valid handle to an interface, returned by <code>USBH_OpenInterface()</code> .
pSpeed	Pointer to a variable that will receive the speed information.

Return value

`USBH_STATUS_SUCCESS` on success. Other values indicate an error.

Additional information

A high speed device can operate in full or high speed mode.

5.1.28 USBH_GetStatusStr()

Description

Converts the result status into a string.

Prototype

```
char *USBH_GetStatusStr(USBH_STATUS x);
```

Parameters

Parameter	Description
x	Result status to convert.

Return value

Pointer to a string which contains the result status in text form.

5.1.29 USBH_ISRTask()

Description

Processes the events triggered from the interrupt handler. This function must run as a separate task in order to use the emUSBH stack. The function only returns, if the USBH stack is shut down (if `USBH_Exit()` was called). In order for the emUSB-Host to work reliably, the task should have the highest priority of all tasks dealing with USB.

Prototype

```
void USBH_ISRTask(void);
```

Additional information

This function waits for events from the interrupt handler of the host controller and processes them.

When `USBH_Exit()` is used in the application this function should not be directly started as a task, as it returns when `USBH_Exit()` is called. A wrapper function can be used in this case, see `USBH_IsRunning()` for a sample.

Note

Task priority requirements described in *Tasks and interrupt usage* on page 28 must be considered.

5.1.30 USBH_Init()

Description

Initializes the emUSB-Host stack.

Prototype

```
void USBH_Init(void);
```

Additional information

Has to be called one time during startup before any other function. The library initializes or allocates global resources within this function.

5.1.31 USBH_MEM_GetMaxUsed()

Description

Returns the maximum used memory since initialization of the memory pool.

Prototype

```
U32 USBH_MEM_GetMaxUsed(int Idx);
```

Parameters

Parameter	Description
<code>Idx</code>	Index of memory pool. • 0 - normal memory • 1 - transfer memory.

Return value

Maximum used memory in bytes.

Additional information

This function only works in a debug configuration of emUSB-Host. If compiled as release configuration, this function always returns 0.

5.1.32 USBH_SetRootPortPower()

Description

Set port of the root hub to a given state. Following state are available:

- `USBH_NORMAL_POWER`: The port operates normally.
- `USBH_POWER_OFF`: The port is switched off.
- `USBH_SUSPEND`: Set the port to suspend mode.
- `USBH_TEST_MODE_J`: Enter test mode.
- `USBH_TEST_MODE_K`: Enter test mode.
- `USBH_TEST_MODE_SE0_NAK`: Enter test mode.
- `USBH_TEST_MODE_PACKET`: Enter test mode.
- `USBH_TEST_MODE_FORCE_ENABLE`: Enter test mode.

Before setting a port into suspend state, the application must ensure that no transaction is pending on that port.

Before setting a port into test mode, the application must ensure that no device is connected to that port. Depending of the USB controller hardware a power cycle of the device may be required in order to leave test mode.

Prototype

```
void USBH_SetRootPortPower(U32          HCIndex,  
                           U8          Port,  
                           USBH_POWER_STATE State);
```

Parameters

Parameter	Description
<code>HCIndex</code>	Index of the host controller.
<code>Port</code>	<code>Port</code> number of the root hub. Ports are counted starting with 1. if set to 0, the new state is set to all ports of the root hub.
<code>State</code>	New state of the port.

5.1.33 USBH_SetHubPortPower()

Description

Set port of an external hub to a given power state.

Prototype

```
USBH_STATUS USBH_SetHubPortPower(USBH_INTERFACE_ID InterfaceID,  
                                U8 Port,  
                                USBH_POWER_STATE State);
```

Parameters

Parameter	Description
InterfaceID	Interface ID of the external hub. May be retrieved using USBH_GetPortInfo() .
Port	Port number of the hub. Ports are counted starting with 1.
State	New power state of the port (USBH_NORMAL_POWER, USBH_POWER_OFF or USBH_SUSPEND).

Return value

USBH_STATUS_SUCCESS on success. Any other value means error.

5.1.34 USBH_HUB_SuspendResume()

Description

Prepares hubs for suspend (stops the interrupt endpoint) or re-starts the interrupt endpoint functionality after a resume.

This function may be used, if a port of a host controller is set to suspend mode via the function `USBH_SetRootPortPower()`. The application must make sure that no transactions are running on that port while it is suspended. If there may be any external hubs connected to that port, then polling of the interrupt endpoints of these hubs must be stopped while suspending. To achieve this, `USBH_HUB_SuspendResume()` should be called with `State = 0` before `USBH_SetRootPortPower(x, y, USBH_SUSPEND)` and with `State = 1` after resume with `USBH_SetRootPortPower(x, y, USBH_NORMAL_POWER)`.

All hubs connected to the given port of a host controller (directly or indirectly) are handled by the function.

Prototype

```
void USBH_HUB_SuspendResume(U32 HCIndex,  
                             U8  Port,  
                             U8  State);
```

Parameters

Parameter	Description
<code>HCIndex</code>	Index of the host controller.
<code>Port</code>	<code>Port</code> number of the roothub. Ports are counted starting with 1. if set to 0, the function applies to all ports of the root hub.
<code>State</code>	0 - Prepare for suspend. 1 - Return from resume.

5.1.35 USBH_GetNumRootPortConnections()

Description

Determine how many devices are directly connected to the host controllers root hub ports. All physically connected devices are counted, irrespective of the identification or enumeration of these devices. Devices connected via a hub are not counted.

Prototype

```
unsigned USBH_GetNumRootPortConnections(U32 HCIndex);
```

Parameters

Parameter	Description
HCIndex	Index of the host controller.

Return value

Number of devices physically connected to the host controllers root hub ports.

5.1.36 USBH_OpenInterface()

Description

Opens the specified interface.

Prototype

```
USBH_STATUS USBH_OpenInterface(USBH_INTERFACE_ID      InterfaceID,  
                               U8                      Exclusive,  
                               USBH_INTERFACE_HANDLE * pInterfaceHandle);
```

Parameters

Parameter	Description
InterfaceID	Specifies the interface to open by its interface Id. The interface ID can be obtained by a call to <code>USBH_GetInterfaceId()</code> .
Exclusive	Specifies if the interface should be opened exclusive or not. If the value is nonzero the function succeeds only if no other application has an open handle to this interface.
pInterfaceHandle	Pointer where the handle to the opened interface is stored.

Return value

USBH_STATUS_SUCCESS on success. Any other value means error.

Additional information

The handle returned by this function via the [pInterfaceHandle](#) parameter is used by the functions that perform data transfers. The returned handle must be closed with `USBH_CloseInterface()` when it is no longer required.

If the interface is allocated exclusive no other application can open it.

5.1.37 USBH_RegisterEnumErrorNotification()

Description

Registers a notification for a port enumeration error.

Prototype

```
USBH_ENUM_ERROR_HANDLE USBH_RegisterEnumErrorNotification  
    (void * pContext,  
     USBH_ON_ENUM_ERROR_FUNC * pfEnumErrorCallback);
```

Parameters

Parameter	Description
<code>pContext</code>	A user defined pointer that is passed unchanged to the notification callback function.
<code>pfEnumErrorCallback</code>	A pointer to a notification function of type <code>USBH_ON_ENUM_ERROR_FUNC</code> that is called if a port enumeration error occurs.

Return value

On success a valid handle to the added notification is returned. A NULL is returned in case of an error.

Additional information

To remove the notification `USBH_UnregisterEnumErrorNotification()` must be called. The `pfOnEnumError` callback routine is called in the context of the process where the interrupt status of a host controller is processed. The callback routine must not block.

5.1.38 USBH_RegisterPnPNotification()

Description

Registers a notification function for PnP events.

Prototype

```
USBH_NOTIFICATION_HANDLE USBH_RegisterPnPNotification  
    (const USBH_PNP_NOTIFICATION * pPnPNotification);
```

Parameters

Parameter	Description
<code>pPnPNotification</code>	Pointer to a caller provided structure.

Return value

On success a valid handle to the added notification is returned. A NULL is returned in case of an error.

Additional information

An application can register any number of notifications. The user notification routine is called in the context of a notify timer that is global for all USB bus PnP notifications. If this function is called while the bus driver has already enumerated devices that match the `USBH_INTERFACE_MASK` the callback function passed in the `USBH_PNP_NOTIFICATION` structure is called for each matching interface.

5.1.39 USBH_RestartEnumError()

Description

Restarts the enumeration process for all devices that have failed to enumerate.

Prototype

```
void USBH_RestartEnumError(void);
```

Additional information

If any problem occur during enumeration of a device, the device is reset and enumeration is retried. To avoid an endless enumeration loop on broken devices there is a maximum retry count of 5 (USBH_RESET_RETRY_COUNTER). After the retry count is expired, the port where the device is connected to is finally disabled. Calling USBH_RestartEnumError() resets the retry counts and restarts enumeration on disabled ports.

5.1.40 USBH_SetCacheConfig()

Description

Configures cache related functionality that might be required by the stack for several purposes such as cache handling in drivers.

Prototype

```
void USBH_SetCacheConfig(const SEGGER_CACHE_CONFIG * pConfig,  
                        unsigned ConfSize);
```

Parameters

Parameter	Description
pConfig	Pointer to an element of SEGGER_CACHE_CONFIG .
ConfSize	Size of the passed structure in case library and header size of the structure differs.

Additional information

This function has to called in USBH_X_Config().

5.1.41 USBH_SetOnSetPortPower()

Description

Sets a callback for the set-port-power driver function. The user callback is called when the ports are added to the host driver instance, this occurs during initialization, or when the ports are removed (during de-initialization). Using this function is necessary if the port power is not controlled directly through the USB controller but is provided from an external source.

Prototype

```
void USBH_SetOnSetPortPower(USBH_ON_SETPORTPOWER_FUNC * pfOnSetPortPower);
```

Parameters

Parameter	Description
<code>pfOnSetPortPower</code>	Pointer to a user-provided callback function of type <code>USBH_ON_SETPORTPOWER_FUNC</code> .

Additional information

The callback function should not block.

5.1.42 USBH_ConfigPortPowerPinEx()

Description

Setups how the port-power pin should be set in order to enable port for this port. In normal case low means power enable. This feature must be supported by the USBH driver.

Prototype

```
USBH_STATUS USBH_ConfigPortPowerPinEx(U32 HCIndex,  
                                       U8  SetHighIsPowerOn);
```

Parameters

Parameter	Description
HCIndex	Index of the host controller.
SetHighIsPowerOn	Select which logical voltage level enables the port. 1 - To enable port power, set the pin high. 0 - To enable port power, set the pin low.

Return value

USBH_STATUS_SUCCESS	Configuration set.
USBH_STATUS_ERROR	Invalid HCIndex .

5.1.43 USBH_SetOnPortEvent()

Description

Sets a callback to report port events to the application.

Prototype

```
void USBH_SetOnPortEvent(USBH_ON_PORT_EVENT_FUNC * pfOnPortEvent);
```

Parameters

Parameter	Description
<code>pfOnPortEvent</code>	Pointer to a user-provided callback function of type <code>USBH_ON_PORT_EVENT_FUNC</code> .

Additional information

The callback function should not block.

5.1.44 USBH_ConfigOvercurrentDetection()

Description

Configures how the driver detects an overcurrent condition. Not supported by all drivers.

Prototype

```
USBH_STATUS USBH_ConfigOvercurrentDetection(U32 HCIndex,  
                                             U32 OvercurrentDetectionMode);
```

Parameters

Parameter	Description
HCIndex	Index of the host controller.
OvercurrentDetectionMode	Value is driver and hardware specific.

Return value

USBH_STATUS_SUCCESS on success, other values indicate error.

Additional information

See *Host controller specifics* on page 692 for valid values of [OvercurrentDetectionMode](#) for the actual driver.

5.1.45 USBH_SubmitUrb()

Description

Submits an URB. Interface function for all asynchronous requests.

Prototype

```
USBH_STATUS USBH_SubmitUrb(USBH_INTERFACE_HANDLE hInterface,
                           USBH_URB * pUrb);
```

Parameters

Parameter	Description
<code>hInterface</code>	Handle to a interface.
<code>pUrb</code>	Pointer to a caller allocated structure. in The URB which should be submitted. out Submitted URB with the appropriate status and the received data if any. The storage for the URB must be permanent as long as the request is pending. The host controller can define special alignment requirements for the URB or the data transfer buffer.

Return value

The request can fail for different reasons. In that case the return value is different from `USBH_STATUS_PENDING` or `USBH_STATUS_SUCCESS`. If the function returns `USBH_STATUS_PENDING` the completion function is called later. In all other cases the completion routine is not called. If the function returns `USBH_STATUS_SUCCESS`, the request was processed immediately. On error the request cannot be processed.

Additional information

If the status `USBH_STATUS_PENDING` is returned the ownership of the URB is passed to the driver. The storage of the URB must not be freed nor modified as long as the ownership is assigned to the driver. The driver passes the URB back to the application by calling the completion routine. An URB that transfers data can be pending for a long time. Please make sure that the URB is not located in the stack. Otherwise the structure may be corrupted in memory. Either use `USBH_Malloc()` or use global/static memory.

Notes

A pending URB transactions may be aborted with an abort request by using `USBH_SubmitUrb` with a new URB where `Urb->Header.Function = USBH_FUNCTION_ABORT_ENDPOINT` and `Urb->Request.EndpointRequest.Endpoint = EndpointAddressToAbort`. Otherwise this operation will last until the device has responded to the request or the device has been disconnected.

Example (asynchronous operation)

```
static U8      _Buffer[512];
static USBH_URB _Urb;

static void _OnUrbCompletion(USBH_URB * pUrb) {
    if (pUrb->Header.Status == USBH_SUCCESS) {
        ProcessData(pUrb->BulkIntRequest.pBuffer, pUrb->BulkIntRequest.Length);
    } else {
        // error handling ...
    }
}
```

```
//
// Start IN transfer on interface 'hInterface' for endpoint 'Ep'
//
_Urb.Header.Function          = USBH_FUNCTION_BULK_REQUEST;
_Urb.Header.pfOnCompletion    = _OnUrbCompletion;
_Urb.Header.pContext          = NULL;
_Urb.BulkIntRequest.pBuffer   = &_amp;_Buffer[0];
_Urb.BulkIntRequest.Length    = sizeof(_Buffer);
_Urb.BulkIntRequest.Endpoint  = Ep;
Status = USBH_SubmitUrb(hInterface, pUrb);
if (Status != USBH_STATUS_PENDING) {
    // error handling ...
}
```

Example (synchronous operation)

```
static U8          _Buffer[512];
static USBH_URB    _Urb;

static void _OnUrbCompletion(USBH_URB * pUrb) {
    USBH_OS_EVENT_OBJ *pEvent;

    pEvent = (USBH_OS_EVENT_OBJ *)pUrb->Header.pContext;
    USBH_OS_SetEvent(pEvent);
}

USBH_OS_EVENT_OBJ *pEvent;
//
// Start IN transfer on interface 'hInterface' for endpoint 'Ep'
//
pEvent = USBH_OS_AllocEvent();
_Urb.Header.Function          = USBH_FUNCTION_BULK_REQUEST;
_Urb.Header.pfOnCompletion    = _OnUrbCompletion;
_Urb.Header.pContext          = pEvent;
_Urb.BulkIntRequest.pBuffer   = &_amp;_Buffer[0];
_Urb.BulkIntRequest.Length    = sizeof(_Buffer);
_Urb.BulkIntRequest.Endpoint  = Ep;
Status = USBH_SubmitUrb(hInterface, pUrb);
if (Status != USBH_STATUS_PENDING) {
    // error handling ...
} else {
    USBH_OS_WaitEvent(pEvent);
    if (_Urb.Header.Status == USBH_SUCCESS) {
        ProcessData(_Urb.BulkIntRequest.pBuffer, _Urb.BulkIntRequest.Length);
    } else {
        // error handling ...
    }
}
USBH_OS_FreeEvent(pEvent);
```

5.1.46 USBH_IsoDataCtrl()

Description

Acknowledge ISO data received from an IN EP or provide data for OUT EPs.

On order to start ISO OUT transfers after calling `USBH_SubmitUrb()`, initially the output packet queue must be filled. For that purpose this function must be called repeatedly until it does not return `USBH_STATUS_NEED_MORE_DATA` any more.

Prototype

```
USBH_STATUS USBH_IsoDataCtrl(const USBH_URB          * pUrb,  
                             USBH_ISO_DATA_CTRL * pIsoData);
```

Parameters

Parameter	Description
<code>pUrb</code>	Pointer to the an active URB running ISO transfers.
<code>pIsoData</code>	ISO data structure.

Return value

`USBH_STATUS_SUCCESS` or `USBH_STATUS_NEED_MORE_DATA` on success or error code on failure.

5.1.47 USBH_Task()

Description

Manages the internal software timers. This function must run as a separate task in order to use the emUSBH stack. The function only returns, if the USBH stack is shut down (if USBH_Exit() was called).

Prototype

```
void USBH_Task(void);
```

Additional information

The function iterates over the list of active timers and invokes the registered callback functions in case the timer expired.

When USBH_Exit() is used in the application this function should not be directly started as a task, as it returns when USBH_Exit() is called. A wrapper function can be used in this case, see USBH_IsRunning() for a sample.

Note

Task priority requirements described in *Tasks and interrupt usage* on page 28 must be considered.

5.1.48 USBH_IsRunning()

Description

Returns whether the stack is running or not.

Prototype

```
int USBH_IsRunning(void);
```

Return value

0	USBH is not running
1	USBH is running

Example

```

/*****
 *
 *      _USBH_Task
 *
 * Function description
 * Wrapper task for emUSBH USBH_Task.
 * Before the function is called, the task stays in a loop to
 * check whether the emUSBH stack is running.
 */
static void _USBH_Task(void) {
    while (1) {
        //
        // Wait until USBH is Ready
        //
        while (USBH_IsRunning() == 0) {
            OS_Delay(10);
        }
        USBH_Task();
    }
}

/*****
 *
 *      _USBH_ISRTask
 *
 * Function description
 * Wrapper task for emUSBH USBH_ISRTask.
 * Before the function is called, the task stays in a loop to
 * check whether the emUSBH stack is running.
 */
static void _USBH_ISRTask(void) {
    while (1) {
        //
        // Wait until USBH is Ready
        //
        while (USBH_IsRunning() == 0) {
            OS_Delay(10);
        }
        USBH_ISRTask();
    }
}

```

5.1.49 USBH_UnregisterEnumErrorNotification()

Description

Removes a registered notification for a port enumeration error.

Prototype

```
void USBH_UnregisterEnumErrorNotification(USBH_ENUM_ERROR_HANDLE hEnumError);
```

Parameters

Parameter	Description
<code>hEnumError</code>	A valid handle for the notification previously returned from <code>USBH_RegisterEnumErrorNotification()</code> .

Additional information

Must be called for a port enumeration error notification that was successfully registered by a call to `USBH_RegisterEnumErrorNotification()`.

5.1.50 USBH_UnregisterPnPNotification()

Description

Removes a previously registered notification for PnP events.

Prototype

```
void USBH_UnregisterPnPNotification(USBH_NOTIFICATION_HANDLE hNotification);
```

Parameters

Parameter	Description
<code>hNotification</code>	A valid handle for a PnP notification previously registered by a call to <code>USBH_RegisterPnPNotification()</code> .

Additional information

Must be called for to unregister a PnP notification that was successfully registered by a call to `USBH_RegisterPnPNotification()`.

5.2 Data structures

The table below lists the available data structures.

Structure	Description
USBH_BULK_INT_REQUEST	Defines parameters for a BULK or INT transfer request.
USBH_CONTROL_REQUEST	Defines parameters for a CONTROL transfer request.
USBH_ISO_REQUEST	Defines parameters for a ISO transfer request.
USBH_ENDPOINT_REQUEST	Defines parameter for an endpoint operation.
USBH_ENUM_ERROR	Is used as a notification parameter for the <code>USBH_ON_ENUM_ERROR_FUNC</code> callback function.
USBH_EP_MASK	Is used as an input parameter to get an endpoint descriptor.
USBH_HEADER	Common parameters for all URB based requests.
USBH_INTERFACE_INFO	This structure contains information about a USB interface and the related device and is returned by the function <code>USBH_GetInterfaceInfo()</code> .
USBH_INTERFACE_MASK	Data structure that defines conditions to select USB interfaces.
USBH_PNP_NOTIFICATION	Is used as an input parameter for the <code>USBH_RegisterEnumErrorNotification()</code> function.
USBH_PORT_INFO	Information about a connected USB device returned by <code>USBH_GetPortInfo()</code> .
USBH_PORT_EVENT	Information about an event occurred on a port of a root hub or external hub.
USBH_SET_INTERFACE	Defines parameters for a control request to set an alternate interface setting.
USBH_SET_POWER_STATE	Defines parameters to set or reset suspend mode for a device.
USBH_URB	This data structure is used to submit an URB.
SEGGER_CACHE_CONFIG	Used to pass cache configuration and callback function pointers to the stack.
USBH_ISO_DATA_CTRL	This data structure is used to provide or acknowledge ISO data.
USBH_IAD_INFO	Information about an Interface Association Descriptor returned by <code>USBH_GetIADInfo()</code> .

5.2.1 USBH_BULK_INT_REQUEST

Description

Defines parameters for a BULK or INT transfer request. Used with USBH_FUNCTION_BULK_REQUEST and USBH_FUNCTION_INT_REQUEST.

Type definition

```
typedef struct {  
    U8      Endpoint;  
    void *  pBuffer;  
    U32     Length;  
} USBH_BULK_INT_REQUEST;
```

Structure members

Member	Description
Endpoint	Specifies the endpoint address with direction bit.
pBuffer	Pointer to a caller provided buffer.
Length	in length of data / size of buffer (in bytes). out Bytes transferred.

5.2.2 USBH_CONTROL_REQUEST

Description

Defines parameters for a CONTROL transfer request. Used with `USBH_FUNCTION_CONTROL_REQUEST`.

Type definition

```
typedef struct {  
    USBH_SETUP_PACKET Setup;  
    void * pBuffer;  
    U32 Length;  
} USBH_CONTROL_REQUEST;
```

Structure members

Member	Description
Setup	The setup packet, direction of data phase, the length field must be valid!
pBuffer	Pointer to the caller provided storage, can be NULL. This buffer is used in the data phase to transfer the data. The direction of the data transfer depends from the Type field in the Setup . See the USB specification for details.
Length	Returns the number of bytes transferred in the data phase. Output value only: Set by the driver.

Additional information

A control request consists of a setup phase, an optional data phase, and a handshake phase. The data phase is limited to a length of 4096 bytes. The [Setup](#) data structure must be filled in properly. The length field in the [Setup](#) must contain the size of the Buffer. The caller must provide the storage for the Buffer.

With this request any setup packet can be submitted. Some standard requests, like `SetAddress` can be sent but would lead to a breakdown of the communication. It is not allowed to set the following standard requests:

SetAddress: It is assigned by the USB stack during enumeration or USB reset.

Clear Feature Endpoint Halt: Use `USBH_FUNCTION_RESET_ENDPOINT` instead. The function `USBH_FUNCTION_RESET_ENDPOINT` resets the data toggle bit in the host controller structures.

SetConfiguration

5.2.3 USBH_ISO_REQUEST

Description

Defines parameters for a ISO transfer request. Used with `USBH_FUNCTION_ISO_REQUEST`.

Only `Endpoint` must be set to submit an ISO URB. All other members are set by the driver, before the completion routine is called.

For every packet send or received, the members of this structure are filled, `Header.Status` is set to `USBH_STATUS_SUCCESS` and the callback function is called. This is repeated until the URB is explicitly canceled. The URB is finally terminated, if `Header.Status` $\neq$ `USBH_STATUS_SUCCESS`.

Type definition

```
typedef struct {
    U8      Endpoint;
    U8      NBuffers;
    U16     Length;
    const void * pData;
    USBH_STATUS Status;
} USBH_ISO_REQUEST;
```

Structure members

Member	Description
<code>Endpoint</code>	<code>in</code> Specifies the endpoint address with direction bit.
<code>NBuffers</code>	<code>out</code> Number of buffers used by the driver.
<code>Length</code>	<code>out</code> <code>Length</code> of the data packet received (IN EPs only).
<code>pData</code>	<code>out</code> Pointer to the data packet received (IN EPs only).
<code>Status</code>	<code>out</code> <code>Status</code> of the transaction.

5.2.4 USBH_ENDPOINT_REQUEST

Description

Defines parameter for an endpoint operation. Used with USBH_FUNCTION_ABORT_ENDPOINT and USBH_FUNCTION_RESET_ENDPOINT.

Type definition

```
typedef struct {  
    U8 Endpoint;  
} USBH_ENDPOINT_REQUEST;
```

Structure members

Member	Description
Endpoint	Specifies the endpoint address with direction bit.

5.2.5 USBH_ENUM_ERROR

Description

Is used as a notification parameter for the USBH_ON_ENUM_ERROR_FUNC callback function. This data structure does not contain detailed information about the device that fails at enumeration because this information is not available in all phases of the enumeration.

Type definition

```
typedef struct {
    unsigned    Flags;
    int         PortNumber;
    USBH_STATUS Status;
    int         ExtendedErrorInformation;
} USBH_ENUM_ERROR;
```

Structure members

Member	Description
Flags	Additional flags to determine the location and the type of the error. • USBH_ENUM_ERROR_EXTHUBPORT_FLAG means the device is connected to an external hub. • USBH_ENUM_ERROR_RETRY_FLAG the bus driver retries the enumeration of this device automatically. • USBH_ENUM_ERROR_STOP_ENUM_FLAG the bus driver does not restart the enumeration for this device because all retries have failed. The application can force the bus driver to restart the enumeration by calling the function USBH_RestartEnumError. • USBH_ENUM_ERROR_DISCONNECT_FLAG means the device has been disconnected during the enumeration. If the hub port reports a disconnect state the device cannot be re-enumerated by the bus driver automatically. Also the function USBH_RestartEnumError cannot re-enumerate the device. • USBH_ENUM_ERROR_ROOT_PORT_RESET means an error during the USB reset of a root hub port occurs. • USBH_ENUM_ERROR_HUB_PORT_RESET means an error during a reset of an external hub port occurs.
PortNumber	Port number of the parent port where the USB device is connected. A flag in the PortFlags field determines if this is an external hub port.
Status	Status of the failed operation.
ExtendedErrorInformation	Internal information used for debugging.

5.2.6 USBH_EP_MASK

Description

Is used as an input parameter to get an endpoint descriptor. The comparison with the mask is true if each member that is marked as valid by a flag in the mask member is equal to the value stored in the endpoint. E.g. if the mask is 0 the first endpoint is returned. If [Mask](#) is set to USBH_EP_MASK_INDEX the zero based index can be used to address all endpoints.

Type definition

```
typedef struct {
    U32  Mask;
    U8   Index;
    U8   Address;
    U8   Type;
    U8   Direction;
} USBH_EP_MASK;
```

Structure members

Member	Description
Mask	This member contains the information which fields are valid. It is an OR combination of the following flags: • USBH_EP_MASK_INDEX The Index is used for comparison. • USBH_EP_MASK_ADDRESS The Address field is used for comparison. • USBH_EP_MASK_TYPE The Type field is used for comparison. • USBH_EP_MASK_DIRECTION The Direction field is used for comparison.
Index	If valid, this member contains the zero based index of the endpoint in the interface.
Address	If valid, this member contains an endpoint address with direction bit.
Type	If valid, this member contains the type of the endpoint: • USB_EP_TYPE_CONTROL • USB_EP_TYPE_BULK • USB_EP_TYPE_ISO • USB_EP_TYPE_INT
Direction	If valid, this member specifies a direction. It is one of the following values: • USB_IN_DIRECTION From device to host • USB_OUT_DIRECTION From host to device

5.2.7 USBH_HEADER

Description

Common parameters for all URB based requests.

Type definition

```
typedef struct {
    USBH_FUNCTION          Function;
    USBH_STATUS            Status;
    USBH_ON_COMPLETION_FUNC * pfOnCompletion;
    void                  * pContext;
    USBH_ON_COMPLETION_FUNC * pfOnInternalCompletion;
    void                  * pInternalContext;
    U32                    HcFlags;
    USBH_ON_COMPLETION_USER_FUNC * pfOnUserCompletion;
    void                  * pUserContext;
    USB_DEVICE             * pDevice;
} USBH_HEADER;
```

Structure members

Member	Description
Function	Function code defines the operation of the URB.
Status	After completion this member contains the status for the request.
pfOnCompletion	Caller provided pointer to the completion function. This completion function is called if the function USBH_SubmitUrb() returns USBH_STATUS_PENDING. If a different status code is returned the completion function is never called.
pContext	This member can be used by the caller to store a context passed to the completion routine.
pfOnInternalCompletion	Internal use.
pInternalContext	Internal use.
HcFlags	Internal use.
pfOnUserCompletion	Internal use.
pUserContext	Internal use.
pDevice	Internal use.

5.2.8 USBH_INTERFACE_INFO

Description

This structure contains information about a USB interface and the related device and is returned by the function `USBH_GetInterfaceInfo()`.

Type definition

```
typedef struct {
    USBH_INTERFACE_ID  InterfaceId;
    USBH_DEVICE_ID     DeviceId;
    U16                VendorId;
    U16                ProductId;
    U16                bcdDevice;
    U8                 Interface;
    U8                 Class;
    U8                 SubClass;
    U8                 Protocol;
    unsigned int       OpenCount;
    U8                 ExclusiveUsed;
    USBH_SPEED         Speed;
    U8                 SerialNumberSize;
    U8                 NumConfigurations;
    U8                 CurrentConfiguration;
    U8                 HCIndex;
    U8                 AlternateSetting;
    U8                 iManufacturer;
    U8                 iProduct;
    U8                 iSerialNumber;
    U8                 iConfiguration;
    U8                 iInterface;
} USBH_INTERFACE_INFO;
```

Structure members

Member	Description
InterfaceId	The unique interface Id. This ID is assigned if the USB device was successful enumerated. It is valid until the device is removed for the host. If the device is reconnected a different interface ID is assigned to each interface.
DeviceId	The unique device Id. This ID is assigned if the USB device was successfully enumerated. It is valid until the device is removed from the host. If the device is reconnected a different device ID is assigned. The relation between the device ID and the interface ID can be used by an application to detect which USB interfaces belong to a device.
VendorId	The Vendor ID of the device.
ProductId	The Product ID of the device.
bcdDevice	The BCD coded device version.
Interface	The USB interface number.
Class	The interface class.
SubClass	The interface sub class.
Protocol	The interface protocol.
OpenCount	Number of open handles for this interface.
ExclusiveUsed	If not 0, this interface is used exclusively.
Speed	Operation speed of the device.

Member	Description
SerialNumberSize	The size of the serial number in bytes, 0 means not available or error during request. The serial number itself can be retrieved using <code>USBH_GetInterfaceSerial()</code> .
NumConfigurations	Number of different configuration of the device.
CurrentConfiguration	Currently selected configuration, zero-based: 0... (NumConfigurations -1)
HCIndex	Index of the host controller the device is connected to.
AlternateSetting	The current alternate setting for this interface.
iManufacturer	String descriptor index for the device manufacturer name (0 if not available).
iProduct	String descriptor index for the device product name (0 if not available).
iSerialNumber	String descriptor index for the device serial number (0 if not available).
iConfiguration	String descriptor index for this configuration's description (0 if not available).
iInterface	String descriptor index for this interface's description (0 if not available).

5.2.9 USBH_INTERFACE_MASK

Description

Data structure that defines conditions to select USB interfaces. Can be used to register notifications. Members that are not selected with **Mask** do not need to be initialized.

Type definition

```
typedef struct {
    U16      Mask;
    U16      VendorId;
    U16      ProductId;
    U16      bcdDevice;
    U8       Interface;
    U8       Class;
    U8       SubClass;
    U8       Protocol;
    const U16 * pVendorIds;
    const U16 * pProductIds;
    U16      NumIds;
} USBH_INTERFACE_MASK;
```

Structure members

Member	Description
Mask	Contains an OR combination of the following flags. If the flag is set the related member of this structure is compared to the properties of the USB interface. USBH_INFO_MASK_VID Compare the Vendor ID (VID) of the device. USBH_INFO_MASK_PID Compare the Product ID (PID) of the device. USBH_INFO_MASK_DEVICE Compare the bcdDevice value of the device. USBH_INFO_MASK_INTERFACE Compare the interface number. USBH_INFO_MASK_CLASS Compare the class of the interface. USBH_INFO_MASK_SUBCLASS Compare the sub class of the interface. USBH_INFO_MASK_PROTOCOL Compare the protocol of the interface. USBH_INFO_MASK_VID_ARRAY Compare the Vendor ID (VID) of the device to a list if ids. USBH_INFO_MASK_PID_ARRAY Compare the Product ID (PID) of the device to a list if ids. If both USBH_INFO_MASK_VID_ARRAY and USBH_INFO_MASK_PID_ARRAY are selected, then the VendorId/ProductId of the device is compared to pairs pVendorIds[i]/pProductIds[i].
VendorId	Vendor ID to compare with.
ProductId	Product ID to compare with.
bcdDevice	BCD coded device version to compare with.
Interface	Interface number to compare with.
Class	Class code to compare with.
SubClass	Sub class code to compare with.
Protocol	Protocol stored in the interface to compare with.
pVendorIds	Points to an array of Vendor IDs.

Member	Description
<code>pProductIds</code>	Points to an array of Product IDs.
<code>NumIds</code>	Number of ids in <code>*pVendorIds</code> and <code>*pProductIds</code> . When only <code>USBH_INFO_MASK_VID_ARRAY</code> is set this is the size of the <code>pVendorIds</code> array. When only <code>USBH_INFO_MASK_PID_ARRAY</code> is set this is the size of the <code>pProductIds</code> array. When both are set this is the size for both arrays (the arrays have to be the same size when both flags are set).

5.2.10 USBH_PNP_NOTIFICATION

Description

Is used as an input parameter for the `USBH_RegisterEnumErrorNotification()` function.

Type definition

```
typedef struct {  
    USBH_ON_PNP_EVENT_FUNC * pfPnpNotification;  
    void * pContext;  
    USBH_INTERFACE_MASK InterfaceMask;  
} USBH_PNP_NOTIFICATION;
```

Structure members

Member	Description
<code>pfPnpNotification</code>	The notification function that is called from the USB stack if a PnP event occurs.
<code>pContext</code>	Pointer to a context, that is passed to the notification function.
<code>InterfaceMask</code>	Mask for the interfaces for which the PnP notification should be called.

5.2.11 USBH_PORT_INFO

Description

Information about a connected USB device returned by `USBH_GetPortInfo()`.

Type definition

```
typedef struct {
    U8          IsHighSpeedCapable;
    U8          IsRootHub;
    U8          IsSelfPowered;
    U8          HCIndex;
    U16         MaxPower;
    U16         PortNumber;
    USBH_SPEED  PortSpeed;
    USBH_DEVICE_ID DeviceId;
    USBH_DEVICE_ID HubDeviceId;
    USBH_INTERFACE_ID HubInterfaceId;
    U32         PortStatus;
} USBH_PORT_INFO;
```

Structure members

Member	Description
IsHighSpeedCapable	1: Port supports high-speed, full-speed and low-speed communication. 0: Port supports only full-speed and low-speed communication.
IsRootHub	1: RootHub, device is directly connected to the host. 0: Device is connected via an external hub to the host.
IsSelfPowered	1: Device is externally powered 0: Device is powered by USB host controller.
HCIndex	Index of the host controller the device is connected to.
MaxPower	Max power the USB device consumes from USB host controller / USB hub in mA.
PortNumber	Port number of the hub or roothub. Ports are counted starting with 1.
PortSpeed	The port speed is the speed with which the device is connected. Can be either <code>USBH_LOW_SPEED</code> or <code>USBH_FULL_SPEED</code> or <code>USBH_HIGH_SPEED</code> .
DeviceId	The unique device Id. This ID is assigned if the USB device was successfully enumerated. It is valid until the device is removed from the host. If the device is reconnected a different device ID is assigned. The relation between the device ID and the interface ID can be used by an application to detect which USB interfaces belong to a device.
HubDeviceId	The unique device ID of the hub, if the device is connected via an external hub. If IsRootHub = 1, then HubDeviceId is zero.
HubInterfaceId	Interface ID of the hub, if the device is connected via an external hub. If IsRootHub = 1, then HubInterfaceId is zero.
PortStatus	Contents of the port status register of the hub.

5.2.12 USBH_PORT_EVENT

Description

Information about an event occurred on a port of a root hub or external hub.

Type definition

```
typedef struct {  
    USBH_PORT_EVENT_TYPE  Event;  
    U8                    HCIndex;  
    U8                    PortNumber;  
    USBH_INTERFACE_ID     HubInterfaceId;  
} USBH_PORT_EVENT;
```

Structure members

Member	Description
Event	Event detected on the port.
HCIndex	Index of the host controller the port is connected to.
PortNumber	Port number of the hub or root hub. Ports are counted starting with 1.
HubInterfaceId	Interface ID of the hub, if the device is connected via an external hub. If the port belongs to a root hub, then HubInterfaceId is zero.

5.2.13 USBH_SET_INTERFACE

Description

Defines parameters for a control request to set an alternate interface setting. Used with `USBH_FUNCTION_SET_INTERFACE`.

Type definition

```
typedef struct {  
    U8  AlternateSetting;  
} USBH_SET_INTERFACE;
```

Structure members

Member	Description
AlternateSetting	Number of alternate interface setting (zero based).

5.2.14 USBH_SET_POWER_STATE

Description

Defines parameters to set or reset suspend mode for a device. Used with `USBH_FUNCTION_SET_POWER_STATE`.

Type definition

```
typedef struct {  
    USBH_POWER_STATE  PowerState;  
} USBH_SET_POWER_STATE;
```

Structure members

Member	Description
PowerState	New power state of the device.

Additional information

If the device is switched to suspend, there must be no pending requests on the device.

5.2.15 USBH_URB

Description

This data structure is used to submit an URB. The URB is the basic structure for all asynchronous operations on the USB stack. All requests that exchange data with the device are using this data structure. The caller has to provide the memory for this structure. The memory must be permanent until the completion function is called.

Prototype

```
struct _USBH_URB {
    USBH_HEADER Header;
    union {
        USBH_CONTROL_REQUEST    ControlRequest;
        USBH_BULK_INT_REQUEST    BulkIntRequest;
        USBH_ISO_REQUEST         IsoRequest;
        USBH_ENDPOINT_REQUEST    EndpointRequest;
        USBH_SET_INTERFACE       SetInterface;
        USBH_SET_POWER_STATE     SetPowerState;
    } Request;
};
```

Member	Description
Header	Contains the URB header of type USBH_HEADER. The most important parameters are the function code and the callback function.
Request	A union that contains information depending on the specific request of the USBH_HEADER. See description of the individual sub structures.

5.2.16 SEGGER_CACHE_CONFIG

Description

Used to pass cache configuration and callback function pointers to the stack.

Prototype

```
typedef struct {  
    int CacheLineSize;  
    void (*pfDMB)      (void);  
    void (*pfClean)    (void *p, unsigned long NumBytes);  
    void (*pfInvalidate)(void *p, unsigned long NumBytes);  
} SEGGER_CACHE_CONFIG;
```

Member	Description
CacheLineSize	Cache line size of the CPU in bytes. Most Systems such as ARM9 use a 32 bytes cache line size.
pfDMB	Unused.
pfClean	Pointer to a callback function that executes a clean operation on cached memory. The parameter 'p' is always cache aligned. 'NumBytes' must be rounded up by the function to the next multiple of the cache line size, if necessary.
pfInvalidate	Pointer to a callback function that executes an invalidate operation on cached memory. The parameter 'p' is always cache aligned. 'NumBytes' must be rounded up by the function to the next multiple of the cache line size, if necessary.

Additional information

For further information about how this structure is used please refer to *USBH_SetCacheConfig* on page 92.

5.2.17 USBH_ISO_DATA_CTRL

Description

This data structure is used to provide or acknowledge ISO data. Used with function `USBH_IsoDataCtrl()`.

Type definition

```
typedef struct {  
    U32      Length;  
    const U8 * pData;  
    U32      Length2;  
    const U8 * pData2;  
    const U8 * pBuffer;  
} USBH_ISO_DATA_CTRL;
```

Structure members

Member	Description
Length	in Length of the first data part to be transferred via ISO OUT EP in bytes. The ISO packet send has size ' Length ' + ' Length2 '.
pData	in Pointer to the first data part to be transferred via ISO OUT EP. The ISO packet send is constructed by concatenating both data parts ' pData ' and ' pData2 '.
Length2	in Length of the second data part to be transferred via ISO OUT EP in bytes (optional).
pData2	in Pointer to the second data part to be transferred via ISO OUT EP.
pBuffer	out Buffer used by the driver.

5.2.18 USBH_IAD_INFO

Description

Information about an Interface Association Descriptor returned by `USBH_GetIADInfo()`.

Type definition

```
typedef struct {  
    USBH_INTERFACE_ID  aInterfaceIDs[];  
    U8                  NumIDs;  
    U8                  FunctionClass;  
    U8                  FunctionSubClass;  
    U8                  FunctionProtocol;  
    U8                  iFunction;  
} USBH_IAD_INFO;
```

Structure members

Member	Description
aInterfaceIDs	Interface IDs which are combined by the IAD
NumIDs	Number of valid interface IDs inside aInterfaceIDs .
FunctionClass	Class code.
FunctionSubClass	Subclass code.
FunctionProtocol	Protocol code.
iFunction	Index of string descriptor describing this function.

5.3 Enumerations

The table below lists the available enumerations.

Structure	Description
USBH_DEVICE_EVENT	Enum containing the device events.
USBH_FUNCTION	Is used as a member for the USBH_HEADER data structure.
USBH_PNP_EVENT	Is used as a parameter for the PnP notification.
USBH_POWER_STATE	Enumerates the power states of a device.
USBH_SPEED	Enum containing operation speed values of a device.
USBH_PORT_EVENT_TYPE	Defines an event type for USB ports.

5.3.1 USBH_DEVICE_EVENT

Description

Enum containing the device events. Enumerates the types of device events. It is used by the USBH_NOTIFICATION_FUNC callback to indicate which type of event occurred.

Type definition

```
typedef enum {  
    USBH_DEVICE_EVENT_ADD,  
    USBH_DEVICE_EVENT_REMOVE  
} USBH_DEVICE_EVENT;
```

Enumeration constants

Constant	Description
USBH_DEVICE_EVENT_ADD	Indicates that a device was connected to the host and new interface is available.
USBH_DEVICE_EVENT_REMOVE	Indicates that a device has been removed.

5.3.2 USBH_FUNCTION

Description

Is used as a member for the USBH_HEADER data structure. All function codes use the API function USBH_SubmitUrb() and are handled asynchronously.

Type definition

```
typedef enum {
    USBH_FUNCTION_CONTROL_REQUEST,
    USBH_FUNCTION_BULK_REQUEST,
    USBH_FUNCTION_INT_REQUEST,
    USBH_FUNCTION_ISO_REQUEST,
    USBH_FUNCTION_RESET_DEVICE,
    USBH_FUNCTION_RESET_ENDPOINT,
    USBH_FUNCTION_ABORT_ENDPOINT,
    USBH_FUNCTION_SET_INTERFACE,
    USBH_FUNCTION_SET_POWER_STATE,
    USBH_FUNCTION_CONFIGURE_EPS
} USBH_FUNCTION;
```

Enumeration constants

Constant	Description
USBH_FUNCTION_CONTROL_REQUEST	Is used to send an URB with a control request. It uses the data structure USBH_CONTROL_REQUEST. A control request includes standard, class and vendor defines requests. The standard requests SetAddress, SetConfiguration and SetInterface can not be submitted by this request. These requests require a special handling in the driver. See USBH_FUNCTION_SET_INTERFACE for details.
USBH_FUNCTION_BULK_REQUEST	Is used to transfer data to or from a bulk endpoint. It uses the data structure USBH_BULK_INT_REQUEST.
USBH_FUNCTION_INT_REQUEST	Is used to transfer data to or from an interrupt endpoint. It uses the data structure USBH_BULK_INT_REQUEST. The interval is defined by the endpoint descriptor.
USBH_FUNCTION_ISO_REQUEST	Is used to transfer data to or from an ISO endpoint. It uses the data structure USBH_ISO_REQUEST. ISO transfer may not be supported by all host controllers.
USBH_FUNCTION_RESET_DEVICE	Sends a USB reset to the device. This removes the device and all its interfaces from the USB stack. The application should abort all pending requests and close all handles to this device. All handles become invalid. The USB stack then starts a new enumeration of the device. All interfaces will get new interface Ids. This request can be part of an error recovery or part of special class protocols like DFU. This function uses only the URB header.
USBH_FUNCTION_RESET_ENDPOINT	Clears an error condition on a special endpoint. If a data transfer error occurs that cannot be handled in hardware the driver stops the endpoint and does not allow further data transfers before the endpoint is reset with this function. On a bulk or interrupt endpoint the host driver sends a Clear Feature Endpoint Halt request. This informs the device about the hardware error. The driver resets the data toggle bit for this endpoint. This request expects that no pending URBs are scheduled on this endpoint. Pending URBs must be aborted with the URB based function USBH_FUNCTION_ABORT_ENDPOINT . This function uses the data structure USBH_ENDPOINT_REQUEST.

Constant	Description
<code>USBH_FUNCTION_ABORT_ENDPOINT</code>	Aborts all pending requests on an endpoint. The host controller calls the completion function with a status code <code>USBH_STATUS_CANCELED</code> . The completion of the URBs may be delayed. The application should wait until all pending requests have been returned by the driver before the handle is closed or <code>USBH_FUNCTION_RESET_ENDPOINT</code> is called.
<code>USBH_FUNCTION_SET_INTERFACE</code>	Selects a new alternate setting for the interface. There must be no pending requests on any endpoint to this interface. The interface handle does not become invalid during this operation. The number of endpoints may be changed. This request uses the data structure <code>USBH_SET_INTERFACE</code> .
<code>USBH_FUNCTION_SET_POWER_STATE</code>	Is used to set the power state for a device. There must be no pending requests for this device if the device is set to the suspend state. The request uses the data structure <code>USBH_SET_POWER_STATE</code> . After the enumeration the device is in normal power state.
<code>USBH_FUNCTION_CONFIGURE</code>	Internal use.

5.3.3 USBH_PNP_EVENT

Description

Is used as a parameter for the PnP notification.

Type definition

```
typedef enum {  
    USBH_ADD_DEVICE,  
    USBH_REMOVE_DEVICE  
} USBH_PNP_EVENT;
```

Enumeration constants

Constant	Description
USBH_ADD_DEVICE	Indicates that a device was connected to the host and a new interface is available.
USBH_REMOVE_DEVICE	Indicates that a device has been removed.

5.3.4 USBH_POWER_STATE

Description

Enumerates the power states of a device. Is used as a member in the USBH_SET_POWER_STATE data structure.

Type definition

```
typedef enum {  
    USBH_NORMAL_POWER,  
    USBH_SUSPEND,  
    USBH_POWER_OFF,  
    USBH_TEST_MODE_J,  
    USBH_TEST_MODE_K,  
    USBH_TEST_MODE_SE0_NAK,  
    USBH_TEST_MODE_PACKET,  
    USBH_TEST_MODE_FORCE_ENABLE,  
    USBH_POWER_MODE_MAX  
} USBH_POWER_STATE;
```

Enumeration constants

Constant	Description
USBH_NORMAL_POWER	The device is switched to normal operation.
USBH_SUSPEND	The device is switched to USB suspend mode.
USBH_POWER_OFF	The device is powered off.
USBH_TEST_MODE_J	Internal use.
USBH_TEST_MODE_K	Internal use.
USBH_TEST_MODE_SE0_NAK	Internal use.
USBH_TEST_MODE_PACKET	Internal use.
USBH_TEST_MODE_FORCE_ENABLE	Internal use.
USBH_POWER_MODE_MAX	Internal use.

5.3.5 USBH_SPEED

Description

Enum containing operation speed values of a device. Is used as a member in the USBH_INTERFACE_INFO data structure and to get the operation speed of a device.

Type definition

```
typedef enum {  
    USBH_SPEED_UNKNOWN,  
    USBH_LOW_SPEED,  
    USBH_FULL_SPEED,  
    USBH_HIGH_SPEED,  
    USBH_SUPER_SPEED  
} USBH_SPEED;
```

Enumeration constants

Constant	Description
USBH_SPEED_UNKNOWN	The speed is unknown.
USBH_LOW_SPEED	The device operates in low-speed mode.
USBH_FULL_SPEED	The device operates in full-speed mode.
USBH_HIGH_SPEED	The device operates in high-speed mode.
USBH_SUPER_SPEED	The device operates in SuperSpeed mode.

5.3.6 USBH_PORT_EVENT_TYPE

Description

Defines an event type for USB ports. Currently only one event type is defined. May be extended in the future.

Type definition

```
typedef enum {
    USBH_PORT_EVENT_OVER_CURRENT
} USBH_PORT_EVENT_TYPE;
```

Enumeration constants

Constant	Description
USBH_PORT_EVENT_OVER_CURRENT	An over current condition has been detected on the hub port.

5.4 Function Types

The table below lists the available function types.

Type	Description
USBH_NOTIFICATION_FUNC	Type of user callback set in <code>USBH_PRINTER_RegisterNotification()</code> , <code>USBH_HID_RegisterNotification()</code> , <code>USBH_CDC_AddNotification()</code> , <code>USBH_FT232_RegisterNotification()</code> and <code>USBH_MTP_RegisterNotification()</code> .
USBH_ON_COMPLETION_FUNC	Is called by the library when an URB request completes.
USBH_ON_ENUM_ERROR_FUNC	Is called by the library if an error occurs at enumeration stage.
USBH_ON_PNP_EVENT_FUNC	Is called by the library if a PnP event occurs and if a PnP notification was registered.
USBH_ON_SETPORTPOWER_FUNC	Callback set by <code>USBH_SetOnSetPortPower()</code> .
USBH_CHECK_ADDRESS_FUNC	Checks if an address can be used for transfers.
USBH_ON_PORT_EVENT_FUNC	Callback set by <code>USBH_SetOnPortEvent()</code> .

5.4.1 USBH_NOTIFICATION_FUNC

Description

Type of user callback set in `USBH_PRINTER_RegisterNotification()`, `USBH_HID_RegisterNotification()`, `USBH_CDC_AddNotification()`, `USBH_FT232_RegisterNotification()` and `USBH_MTP_RegisterNotification()`.

Type definition

```
typedef void (USBH_NOTIFICATION_FUNC)(void * pContext,  
                                       U8    DevIndex,  
                                       USBH_DEVICE_EVENT Event);
```

Parameters

Parameter	Description
<code>pContext</code>	Pointer to a context passed by the user in the call to one of the register functions.
<code>DevIndex</code>	Zero based index of the device that was added or removed. First device has index 0, second one has index 1, etc
<code>Event</code>	Enum <code>USBH_DEVICE_EVENT</code> which gives information about the event that occurred.

5.4.2 USBH_ON_COMPLETION_FUNC

Description

Is called by the library when an URB request completes.

Type definition

```
typedef void (USBH_ON_COMPLETION_FUNC)(USBH_URB * pUrb);
```

Parameters

Parameter	Description
<code>pUrb</code>	Contains the URB that was completed.

Additional information

Is called in the context of the `USBH_Task()` or `USBH_ISRTask()`.

5.4.3 USBH_ON_ENUM_ERROR_FUNC

Description

Is called by the library if an error occurs at enumeration stage.

Type definition

```
typedef void (USBH_ON_ENUM_ERROR_FUNC)(          void          * pContext,  
                                         const USBH_ENUM_ERROR * pEnumError);
```

Parameters

Parameter	Description
<code>pContext</code>	Is a user defined pointer that was passed to <code>USBH_RegisterEnumErrorNotification()</code> .
<code>pEnumError</code>	Pointer to a structure containing information about the error occurred. This structure is temporary and must not be accessed after the functions returns.

Additional information

Is called in the context of `USBH_Task()` function or of a `ProcessInterrupt` function of a host controller. Before this function is called it must be registered with `USBH_RegisterEnumErrorNotification()`. If a device is not successfully enumerated the function `USBH_RestartEnumError()` can be called to re-start a new enumeration in the context of this function. This callback mechanism is part of the enhanced error recovery.

5.4.4 USBH_ON_PNP_EVENT_FUNC

Description

Is called by the library if a PnP event occurs and if a PnP notification was registered.

Type definition

```
typedef void (USBH_ON_PNP_EVENT_FUNC)(void * pContext,  
                                     USBH_PNP_EVENT Event,  
                                     USBH_INTERFACE_ID InterfaceId);
```

Parameters

Parameter	Description
pContext	Is the user defined pointer that was passed to <code>USBH_RegisterEnumErrorNotification()</code> . The library does not dereference this pointer.
Event	Enum <code>USBH_DEVICE_EVENT</code> specifies the PnP event.
InterfaceId	Contains the interface ID of the removed or added interface.

Additional information

Is called in the context of `USBH_Task()` function. In the context of this function all other API functions of the USB host stack can be called. The removed or added interface can be identified by the interface Id. The client can use this information to find the related USB Interface and close all handles if it was in use, to open it or to collect information about the interface.

5.4.5 USBH_ON_SETPORTPOWER_FUNC

Description

Callback set by `USBH_SetOnSetPortPower()`. Is called when port power should be changed.

Type definition

```
typedef void (USBH_ON_SETPORTPOWER_FUNC)(U32 HostControllerIndex,  
                                          U8  Port,  
                                          U8  PowerOn);
```

Parameters

Parameter	Description
<code>HostControllerIndex</code>	Index of the host controller. This corresponds to the return value of the respective <code>USBH_<DriverName>_Add</code> call.
<code>Port</code>	1-based port index.
<code>PowerOn</code>	0 - power off 1 - power on

5.4.6 USBH_CHECK_ADDRESS_FUNC

Description

Checks if an address can be used for transfers. The function must return 0, if access is allowed for the given address, 1 otherwise.

Type definition

```
typedef int USBH_CHECK_ADDRESS_FUNC(const void * pMem);
```

Parameters

Parameter	Description
<code>pMem</code>	Pointer to the memory.

Return value

- 0 - Memory can be used for access.
- 1 - Access not allowed for the given address.

5.4.7 USBH_ON_PORT_EVENT_FUNC

Description

Callback set by `USBH_SetOnPortEvent()`. Is called when an event occurs on a port.

Type definition

```
typedef void (USBH_ON_PORT_EVENT_FUNC)(const USBH_PORT_EVENT * pPortEvent);
```

Parameters

Parameter	Description
<code>pPortEvent</code>	Pointer to information about the port even.

5.5 USBH_STATUS

Description

Status codes returned by most of the API functions.

Type definition

```
typedef enum {
    USBH_STATUS_SUCCESS,
    USBH_STATUS_CRC,
    USBH_STATUS_BITSTUFFING,
    USBH_STATUS_DATATOGGLE,
    USBH_STATUS_STALL,
    USBH_STATUS_NOTRESPONDING,
    USBH_STATUS_PID_CHECK,
    USBH_STATUS_UNEXPECTED_PID,
    USBH_STATUS_DATA_OVERRUN,
    USBH_STATUS_DATA_UNDERRUN,
    USBH_STATUS_XFER_SIZE,
    USBH_STATUS_DMA_ERROR,
    USBH_STATUS_BUFFER_OVERRUN,
    USBH_STATUS_BUFFER_UNDERRUN,
    USBH_STATUS_OHCI_NOT_ACCESSED1,
    USBH_STATUS_OHCI_NOT_ACCESSED2,
    USBH_STATUS_HC_ERROR,
    USBH_STATUS_FRAME_ERROR,
    USBH_STATUS_SPLIT_ERROR,
    USBH_STATUS_NEED_MORE_DATA,
    USBH_STATUS_CHANNEL_NAK,
    USBH_STATUS_ERROR,
    USBH_STATUS_INVALID_PARAM,
    USBH_STATUS_PENDING,
    USBH_STATUS_DEVICE_REMOVED,
    USBH_STATUS_CANCELED,
    USBH_STATUS_HC_STOPPED,
    USBH_STATUS_BUSY,
    USBH_STATUS_NO_CHANNEL,
    USBH_STATUS_DEVICE_SUSPENDED,
    USBH_STATUS_INVALID_DESCRIPTOR,
    USBH_STATUS_ENDPOINT_HALTED,
    USBH_STATUS_TIMEOUT,
    USBH_STATUS_PORT,
    USBH_STATUS_INVALID_HANDLE,
    USBH_STATUS_NOT_OPENED,
    USBH_STATUS_ALREADY_ADDED,
    USBH_STATUS_ENDPOINT_INVALID,
    USBH_STATUS_NOT_FOUND,
    USBH_STATUS_NOT_SUPPORTED,
    USBH_STATUS_ISO_DISABLED,
    USBH_STATUS_LENGTH,
    USBH_STATUS_COMMAND_FAILED,
    USBH_STATUS_INTERFACE_PROTOCOL,
    USBH_STATUS_INTERFACE_SUB_CLASS,
    USBH_STATUS_WRITE_PROTECT,
    USBH_STATUS_INTERNAL_BUFFER_NOT_EMPTY,
    USBH_STATUS_BAD_RESPONSE,
    USBH_STATUS_DEVICE_ERROR,
    USBH_STATUS_MTP_OPERATION_NOT_SUPPORTED,
    USBH_STATUS_MEMORY,
    USBH_STATUS_RESOURCES
} USBH_STATUS;
```

Enumeration constants

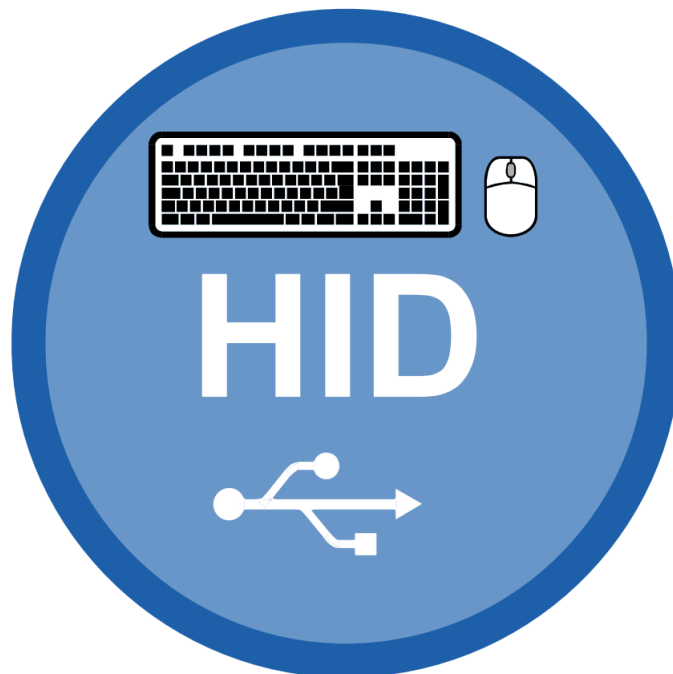
Constant	Description
USBH_STATUS_SUCCESS	Operation successfully completed.
USBH_STATUS_CRC	Data packet received from device contained a CRC error.
USBH_STATUS_BITSTUFFING	Data packet received from device contained a bit stuffing violation.
USBH_STATUS_DATATOGGLE	Data packet received from device had data toggle PID that did not match the expected value.
USBH_STATUS_STALL	Endpoint was stalled by the device.
USBH_STATUS_NOTRESPONDING	USB device did not respond to the request (did not respond to IN token or did not provide a handshake to the OUT token).
USBH_STATUS_PID_CHECK	Check bits on PID from endpoint failed on data PID (IN) or handshake (OUT).
USBH_STATUS_UNEXPECTED_PID	Receive PID was not valid when encountered or PID value is not defined.
USBH_STATUS_DATA_OVERRUN	The amount of data returned by the device exceeded either the size of the maximum data packet allowed from the endpoint or the remaining buffer size (Babble error).
USBH_STATUS_DATA_UNDERRUN	The endpoint returned less than maximum packet size and that amount was not sufficient to fill the specified buffer.
USBH_STATUS_XFER_SIZE	Size exceeded the maximum transfer size supported by the driver.
USBH_STATUS_DMA_ERROR	Direct memory access error.
USBH_STATUS_BUFFER_OVERRUN	During an IN transfer, the host controller received data from the device faster than it could be written to system memory.
USBH_STATUS_BUFFER_UNDERRUN	During an OUT transfer, the host controller could not retrieve data from system memory fast enough to keep up with data USB data rate.
USBH_STATUS_OHCI_NOT_ACCESSED1	Exclusive to OHCI. This code is set before the TD is placed on a list to be processed by the HC. (Binary for this code is 111x (1111 or 1110 depending on implementation))
USBH_STATUS_OHCI_NOT_ACCESSED2	Exclusive to OHCI. This code is set before the TD is placed on a list to be processed by the HC.
USBH_STATUS_HC_ERROR	Error reported by the host controller.
USBH_STATUS_FRAME_ERROR	An interrupt transfer could not be scheduled within a micro frame.
USBH_STATUS_SPLIT_ERROR	Error while using split transactions.
USBH_STATUS_NEED_MORE_DATA	The transfer could not be started yet. More data is required.
USBH_STATUS_CHANNEL_NAK	Internal use.
USBH_STATUS_ERROR	Unspecified error occurred.
USBH_STATUS_INVALID_PARAM	An invalid parameter was provided.
USBH_STATUS_PENDING	The operation was started asynchronously.
USBH_STATUS_DEVICE_REMOVED	The device was detached from the host.

Constant	Description
USBH_STATUS_CANCELED	The operation was canceled by user request.
USBH_STATUS_HC_STOPPED	Host controller stopped.
USBH_STATUS_BUSY	The endpoint, interface or device has pending requests and therefore the operation can not be executed.
USBH_STATUS_NO_CHANNEL	Transfer request can't be processed, because there is no free channel in the USB controller.
USBH_STATUS_DEVICE_SUSPENDED	The device was detached from the host.
USBH_STATUS_INVALID_DESCRIPTOR	A device provided an invalid descriptor.
USBH_STATUS_ENDPOINT_HALTED	The endpoint has been halted. A pipe will be halted when a data transmission error (CRC, bit stuff, DATA toggle) occurs.
USBH_STATUS_TIMEOUT	The operation was aborted due to a timeout.
USBH_STATUS_PORT	Operation on a USB port failed.
USBH_STATUS_INVALID_HANDLE	An invalid handle was provided to the function.
USBH_STATUS_NOT_OPENED	The device or interface was not opened.
USBH_STATUS_ALREADY_ADDED	Item has already been added.
USBH_STATUS_ENDPOINT_INVALID	Invalid endpoint for the requested operation.
USBH_STATUS_NOT_FOUND	Requested information not available.
USBH_STATUS_NOT_SUPPORTED	The operation is not supported by the connected device.
USBH_STATUS_ISO_DISABLED	The support for isochronous transfers is disabled in the USB stack, see USBH_SUPPORT_ISO_TRANSFER.
USBH_STATUS_LENGTH	The operation detected a length error.
USBH_STATUS_COMMAND_FAILED	This error is reported if the MSD command code was sent successfully but the status returned from the device indicates a command error.
USBH_STATUS_INTERFACE_PROTOCOL	The used MSD interface protocol is not supported. The interface protocol is defined by the interface descriptor.
USBH_STATUS_INTERFACE_SUB_CLASS	The used MSD interface sub class is not supported. The interface sub class is defined by the interface descriptor.
USBH_STATUS_WRITE_PROTECT	The MSD medium is write protected.
USBH_STATUS_INTERNAL_BUFFER_NOT_FOUND	Internal use.
USBH_STATUS_BAD_RESPONSE	Device responded unexpectedly.
USBH_STATUS_DEVICE_ERROR	Device indicated a failure.
USBH_STATUS_MTP_OPERATION_NOT_SUPPORTED	The requested MTP operation is not supported by the connected device.
USBH_STATUS_MEMORY	Memory could not be allocated.
USBH_STATUS_RESOURCES	Not enough resources (e.g endpoints, events, handles, ...)

Chapter 6

Human Interface Device (HID) class

This chapter describes the emUSB-Host Human interface device class driver and its usage. The HID class is part of the BASE package. The HID-class code is linked in only if registered by the application program.



6.1 Introduction

The emUSB-Host HID class software allows accessing USB Human Interface Devices. It implements the USB Human interface Device class protocols specified by the USB Implementers Forum. The entire API of this class driver is prefixed with the "USBH_HID_" text. This chapter describes the architecture, the features and the programming interface of this software component.

6.1.1 Overview

Two types of HIDs are currently supported: Keyboard and Mouse. For both, the application can set a callback routine which is invoked whenever a message from either one is received.

Types of HIDs:

- "True" HIDs: Mouse & Keyboard
- Devices using the HID protocol for data transfer

6.1.2 Example code

Example code which is provided in the `USBH_HID_Start.c` file. It outputs mouse and keyboard events to the terminal I/O of debugger.

6.2 API Functions

This chapter describes the emUSB-Host HID API functions.

Function	Description
USBH_HID_CancelIo()	Cancels any pending read/write operation.
USBH_HID_Close()	Closes a handle to opened HID device.
USBH_HID_Exit()	Releases all resources, closes all handles to the USB stack and unregisters all notification functions.
USBH_HID_GetDeviceInfo()	Retrieves information about an opened HID device.
USBH_HID_GetNumDevices()	Returns the number of available devices.
USBH_HID_GetReport()	Reads a report from a HID device.
USBH_HID_GetReportDesc()	Returns the data of a report descriptor in raw form.
USBH_HID_Init()	Initializes and registers the HID device driver with emUSB-Host.
USBH_HID_Open()	Opens a device given by an index.
USBH_HID_AddNotification()	Adds a callback in order to be notified when a device is added or removed.
USBH_HID_RemoveNotification()	Removes a callback added via USBH_HID_AddNotification .
USBH_HID_RegisterNotification()	Obsolete function, use USBH_HID_AddNotification() .
USBH_HID_SetOnKeyboardStateChange()	Sets a callback to be called in case of keyboard events.
USBH_HID_SetOnExKeyboardStateChange()	Sets a callback to be called in case of keyboard events.
USBH_HID_SetOnMouseStateChange()	Sets a callback to be called in case of mouse events.
USBH_HID_SetOnRCStateChange()	Sets a callback to be called in case of remote control events.
USBH_HID_SetOnGenericEvent()	Sets a callback to be called in case of generic HID events.
USBH_HID_SetReport()	Sends an output report to a HID device.
USBH_HID_SetReportEx()	Sends an output or feature report to a HID device.
USBH_HID_GetReportEP0()	Reads a report from a HID device via control request.
USBH_HID_SetIndicators()	Sets the indicators (usually LEDs) on a keyboard.
USBH_HID_GetIndicators()	Retrieves the indicator (LED) status.
USBH_HID_ConfigureAllowLEDUpdate()	Sets whether the keyboard LED should be updated or not.
USBH_HID_GetInterfaceHandle()	Return the handle to the (open) USB interface.
USBH_HID_SuspendResume()	Prepares a HID device for suspend (stops the interrupt endpoint) or re-starts the interrupt endpoint functionality after a resume.

6.2.1 USBH_HID_CancelIo()

Description

Cancels any pending read/write operation.

Prototype

```
USBH_STATUS USBH_HID_CancelIo(USBH_HID_HANDLE hDevice);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to the HID device.

Return value

USBH_STATUS_SUCCESS : Operation successfully canceled. Any other value means error.

6.2.2 USBH_HID_Close()

Description

Closes a handle to opened HID device.

Prototype

```
USBH_STATUS USBH_HID_Close(USBH_HID_HANDLE hDevice);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to the opened device.

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

6.2.3 USBH_HID_Exit()

Description

Releases all resources, closes all handles to the USB stack and unregisters all notification functions.

Prototype

```
void USBH_HID_Exit(void);
```

6.2.4 USBH_HID_GetDeviceInfo()

Description

Retrieves information about an opened HID device.

Prototype

```
USBH_STATUS USBH_HID_GetDeviceInfo(USBH_HID_HANDLE      hDevice,  
                                   USBH_HID_DEVICE_INFO * pDevInfo);
```

Parameters

Parameter	Description
hDevice	Handle to an opened HID device.
pDevInfo	Pointer to a USBH_HID_DEVICE_INFO buffer.

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

6.2.5 USBH_HID_GetNumDevices()

Description

Returns the number of available devices. It also retrieves the information about a device.

Prototype

```
int USBH_HID_GetNumDevices(USBH_HID_DEVICE_INFO * pDevInfo,  
                           U32 NumItems);
```

Parameters

Parameter	Description
pDevInfo	Pointer to an array of USBH_HID_DEVICE_INFO structures.
NumItems	Number of items that pDevInfo can hold.

Return value

Number of devices available.

6.2.6 USBH_HID_GetReport()

Description

Reads a report from a HID device.

Prototype

```
USBH_STATUS USBH_HID_GetReport(USBH_HID_HANDLE    hDevice,
                                U8                  * pBuffer,
                                U32                  BufferSize,
                                USBH_HID_USER_FUNC * pFunc,
                                USBH_HID_RW_CONTEXT * pRWContext);
```

Parameters

Parameter	Description
hDevice	Handle to an opened HID device.
pBuffer	Pointer to a buffer to read.
BufferSize	Size of the buffer.
pFunc	[Optional] Callback function of type USBH_HID_USER_FUNC invoked when the read operation finishes (asynchronous operation). It can be the NULL pointer, the function is executed synchronously.
pRWContext	[Optional] Pointer to a USBH_HID_RW_CONTEXT structure which will be filled with data after the transfer has been completed and passed as a parameter to the callback function (pFunc). If pFunc ≠ NULL, this parameter is required. If pFunc = NULL, only the member pRWContext->NumBytesTransferred is set by the function.

Return value

USBH_STATUS_SUCCESS	Success on synchronous operation (pFunc = NULL).
USBH_STATUS_PENDING	Request was submitted successfully and the application is informed via callback (pFunc ≠ NULL). Any other value means error.

6.2.7 USBH_HID_GetReportDesc()

Description

Returns the data of a report descriptor in raw form.

Prototype

```
USBH_STATUS USBH_HID_GetReportDesc(      USBH_HID_HANDLE  hDevice,
                                         const U8          ** ppReportDescriptor,
                                         unsigned          * pNumBytes);
```

Parameters

Parameter	Description
hDevice	Handle to an opened device.
ppReportDescriptor	Returns a pointer to the report descriptor which is stored in an internal data structure of the USB stack. The report descriptor must not be changed. The pointer becomes invalid after the device is closed.
pNumBytes	Returns the size of the report descriptor in bytes.

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

6.2.8 USBH_HID_Init()

Description

Initializes and registers the HID device driver with emUSB-Host.

Prototype

```
U8 USBH_HID_Init(void);
```

Return value

- | | |
|---|---------------------------------------|
| 1 | Success. |
| 0 | Could not register HID device driver. |

Additional information

This function can be called multiple times, but only the first call initializes the module. Any further calls only increase the initialization counter. This is useful for cases where the module is initialized from different places which do not interact with each other. To de-initialize the module `USBH_HID_Exit` has to be called the same number of times as this function was called.

6.2.9 USBH_HID_Open()

Description

Opens a device given by an index.

Prototype

```
USBH_HID_HANDLE USBH_HID_Open(unsigned Index);
```

Parameters

Parameter	Description
Index	Device index.

Return value

≠ USBH_HID_INVALID_HANDLE	Handle to a HID device.
= USBH_HID_INVALID_HANDLE	Device not available.

Additional information

The index of a new connected device is provided to the callback function registered with `USBH_HID_AddNotification()`.

6.2.10 USBH_HID_AddNotification()

Description

Adds a callback in order to be notified when a device is added or removed.

Prototype

```
USBH_STATUS USBH_HID_AddNotification(USBH_NOTIFICATION_HOOK * pHook,  
                                     USBH_NOTIFICATION_FUNC * pfNotification,  
                                     void * pContext);
```

Parameters

Parameter	Description
<code>pHook</code>	Pointer to a user provided USBH_NOTIFICATION_HOOK structure, which is initialized and used by this function. The memory area must be valid, until the notification is removed.
<code>pfNotification</code>	Pointer to a function the stack should call when a device is connected or disconnected.
<code>pContext</code>	Pointer to a user context that is passed to the callback function.

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

6.2.11 USBH_HID_RemoveNotification()

Description

Removes a callback added via USBH_HID_AddNotification.

Prototype

```
USBH_STATUS USBH_HID_RemoveNotification(const USBH_NOTIFICATION_HOOK * pHook);
```

Parameters

Parameter	Description
<code>pHook</code>	Pointer to a user provided USBH_NOTIFICATION_HOOK variable.

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

6.2.12 USBH_HID_RegisterNotification()

Description

Obsolete function, use `USBH_HID_AddNotification()`. Registers a notification callback in order to inform user about adding or removing a device.

Prototype

```
void USBH_HID_RegisterNotification(USBH_NOTIFICATION_FUNC * pfNotification,  
                                  void * pContext);
```

Parameters

Parameter	Description
<code>pfNotification</code>	Pointer to a callback function of type <code>USBH_NOTIFICATION_FUNC</code> the emUSB-Host calls when a HID device is attached/removed.
<code>pContext</code>	Application specific pointer. The pointer is not dereferenced by the emUSB-Host. It is passed to the callback function. Any value the application chooses is permitted, including <code>NULL</code> .

6.2.13 USBH_HID_SetOnKeyboardStateChange()

Description

Sets a callback to be called in case of keyboard events. Handles all keyboards that do not use report ids. These are all keyboards that can be used in boot mode (with a PC BIOS).

Prototype

```
void USBH_HID_SetOnKeyboardStateChange(USBH_HID_ON_KEYBOARD_FUNC * pfOnChange);
```

Parameters

Parameter	Description
<code>pfOnChange</code>	Callback that shall be called when a keyboard change notification is available.

6.2.14 USBH_HID_SetOnExKeyboardStateChange()

Description

Sets a callback to be called in case of keyboard events. Handles also keyboards that use report ids. In contrast to the function `USBH_HID_SetOnKeyboardStateChange()`, some unusual Apple keyboards are supported, too.

Prototype

```
void USBH_HID_SetOnExKeyboardStateChange(USBH_HID_ON_KEYBOARD_FUNC * pfOnChange);
```

Parameters

Parameter	Description
<code>pfOnChange</code>	Callback that shall be called when a keyboard change notification is available.

6.2.15 USBH_HID_SetOnMouseStateChange()

Description

Sets a callback to be called in case of mouse events.

Prototype

```
void USBH_HID_SetOnMouseStateChange(USBH_HID_ON_MOUSE_FUNC * pfOnChange);
```

Parameters

Parameter	Description
<code>pfOnChange</code>	Callback that shall be called when a mouse change notification is available.

6.2.16 USBH_HID_SetOnRCStateChange()

Description

Sets a callback to be called in case of remote control events. Remote control interfaces are often a part of a USB audio device, the HID interface is used to tell the host about changes in volume, mute, for music track control and similar.

Prototype

```
void USBH_HID_SetOnRCStateChange(USBH_HID_ON_RC_FUNC * pfOnChange);
```

Parameters

Parameter	Description
pfOnChange	Callback that shall be called when a remote control change notification is available.

6.2.17 USBH_HID_SetOnGenericEvent()

Description

Sets a callback to be called in case of generic HID events.

Prototype

```
void USBH_HID_SetOnGenericEvent(      U32          NumUsages,
                                     const U32          * pUsages,
                                     USBH_HID_ON_GENERIC_FUNC * pfOnEvent);
```

Parameters

Parameter	Description
NumUsages	Number of usage codes provided by the caller.
pUsages	List of usage codes of fields from the report to be monitored. Each usage code must contain the Usage Page in the high order 16 bits and the Usage ID in the the low order 16 bits. pUsages must point to a static memory area that remains valid until the USBH_HID module is shut down.
pfOnEvent	Callback that shall be called when a report is received that contains at least one field with usage code from the list.

6.2.18 USBH_HID_SetReport()

Description

Sends an output report to a HID device. This function assumes report IDs are not used.

Prototype

```
USBH_STATUS USBH_HID_SetReport(      USBH_HID_HANDLE    hDevice,
                                     const U8                * pBuffer,
                                     U32                    BufferSize,
                                     USBH_HID_USER_FUNC     * pfFunc,
                                     USBH_HID_RW_CONTEXT    * pRWContext);
```

Parameters

Parameter	Description
hDevice	Handle to an opened HID device.
pBuffer	Pointer to a buffer containing the data to be sent. In case the device has more than one report descriptor the first byte inside the buffer must contain a valid ID matching one of the report descriptors.
BufferSize	Size of the buffer.
pfFunc	[Optional] Callback function of type USBH_HID_USER_FUNC invoked when the send operation finishes. It can be the NULL pointer.
pRWContext	[Optional] Pointer to a USBH_HID_RW_CONTEXT structure which will be filled with data after the transfer has been completed and passed as a parameter to the pfFunc function.

Return value

USBH_STATUS_SUCCESS	Success.
USBH_STATUS_PENDING	Request was submitted and application is informed via callback. Any other value means error.

6.2.19 USBH_HID_SetReportEx()

Description

Sends an output or feature report to a HID device. Optionally sends out a report ID. Output reports are send via the OUT endpoint of the device if present, or using a control request otherwise.

Prototype

```
USBH_STATUS USBH_HID_SetReportEx(    USBH_HID_HANDLE    hDevice,
                                     const U8                * pBuffer,
                                     U32                    BufferSize,
                                     USBH_HID_USER_FUNC      * pfFunc,
                                     USBH_HID_RW_CONTEXT      * pRWContext,
                                     unsigned                 Flags);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to an opened HID device.
<code>pBuffer</code>	Pointer to a buffer containing the data to be sent. In case the device has more than one report descriptor the first byte inside the buffer must contain a valid ID matching one of the report descriptors.
<code>BufferSize</code>	Size of the buffer.
<code>pfFunc</code>	[Optional] Callback function of type <code>USBH_HID_USER_FUNC</code> invoked when the send operation finishes. It can be the NULL pointer.
<code>pRWContext</code>	[Optional] Pointer to a <code>USBH_HID_RW_CONTEXT</code> structure which will be filled with data after the transfer has been completed and passed as a parameter to the <code>pfOnComplete</code> function.
<code>Flags</code>	A bitwise OR-combination of flags • <code>USBH_HID_USE_REPORT_ID</code>: Enables report ID usage. The first byte in the buffer pointed to by <code>pBuffer</code> is used as report ID. • <code>USBH_HID_OUTPUT_REPORT</code>: Send an output report (default). • <code>USBH_HID_FEATURE_REPORT</code>: Send a feature report.

Return value

<code>USBH_STATUS_SUCCESS</code>	Success.
<code>USBH_STATUS_PENDING</code>	Request was submitted and application is informed via callback. Any other value means error.

6.2.20 USBH_HID_GetReportEP0()

Description

Reads a report from a HID device via control request.

Prototype

```
USBH_STATUS USBH_HID_GetReportEP0(USBH_HID_HANDLE hDevice,
                                     U8 ReportID,
                                     unsigned Flags,
                                     U8 * pBuffer,
                                     U32 Length,
                                     U32 * pNumBytesRead);
```

Parameters

Parameter	Description
hDevice	Handle to an opened HID device.
ReportID	ID of the report requested from the device.
Flags	- USBH_HID_INPUT_REPORT: Request for an input report (default). - USBH_HID_FEATURE_REPORT: Request for a feature report.
pBuffer	Pointer to a buffer to read.
Length	Requested length of the report.
pNumBytesRead	[O] Actual length of the report read.

Return value

USBH_STATUS_SUCCESS Success. Any other value means error.

6.2.21 USBH_HID_SetIndicators()

Description

Sets the indicators (usually LEDs) on a keyboard.

Prototype

```
USBH_STATUS USBH_HID_SetIndicators(USBH_HID_HANDLE hDevice,  
                                   U8 IndicatorMask);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device.
IndicatorMask	Binary mask of the following items USBH_HID_IND_NUM_LOCK USBH_HID_IND_CAPS_LOCK USBH_HID_IND_SCROLL_LOCK USBH_HID_IND_COMPOSE USBH_HID_IND_KANA USBH_HID_IND_SHIFT

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

6.2.22 USBH_HID_GetIndicators()

Description

Retrieves the indicator (LED) status.

Prototype

```
USBH_STATUS USBH_HID_GetIndicators(USBH_HID_HANDLE    hDevice,  
                                   U8                  * pIndicatorMask);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device.
pIndicatorMask	Binary mask of the following items USBH_HID_IND_NUM_LOCK USBH_HID_IND_CAPS_LOCK USBH_HID_IND_SCROLL_LOCK USBH_HID_IND_COMPOSE USBH_HID_IND_KANA USBH_HID_IND_SHIFT

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

6.2.23 USBH_HID_ConfigureAllowLEDUpdate()

Description

Sets whether the keyboard LED should be updated or not. (Default is disabled).

Prototype

```
void USBH_HID_ConfigureAllowLEDUpdate(unsigned AllowLEDUpdate);
```

Parameters

Parameter	Description
AllowLEDUpdate	• 0 - Disable LED Update. • 1 - Allow LED Update.

6.2.24 USBH_HID_GetInterfaceHandle()

Description

Return the handle to the (open) USB interface. Can be used to call USBH core functions like USBH_GetStringDescriptor().

Prototype

```
USBH_INTERFACE_HANDLE USBH_HID_GetInterfaceHandle(USBH_HID_HANDLE hDevice);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to an open device returned by USBH_HID_Open().

Return value

Handle to an open interface.

6.2.25 USBH_HID_SuspendResume()

Description

Prepares a HID device for suspend (stops the interrupt endpoint) or re-starts the interrupt endpoint functionality after a resume.

Prototype

```
USBH_STATUS USBH_HID_SuspendResume(USBH_HID_HANDLE hDevice,  
                                     U8               State);
```

Parameters

Parameter	Description
hDevice	Handle to an open device returned by <code>USBH_HID_Open()</code> .
State	0 - Prepare for suspend. 1 - Return from resume.

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

Additional information

The application must make sure that no transactions are running when setting a device into suspend mode. This function is used in combination with `USBH_SetRootPortPower()`. To suspend: Call this function before `USBH_SetRootPortPower(x, y, USBH_SUSPEND)` with [State](#) = 0. To resume: Call this function after `USBH_SetRootPortPower(x, y, USBH_NORMAL_POWER)` with [State](#) = 1.

6.3 Data structures

This chapter describes the emUSB-Host HID API structures.

Structure	Description
USBH_HID_DEVICE_INFO	Structure containing information about a HID device.
USBH_HID_KEYBOARD_DATA	Structure containing information about a keyboard event.
USBH_HID_MOUSE_DATA	Structure containing information about a mouse event.
USBH_HID_RC_DATA	Structure containing information about a remote control event.
USBH_HID_GENERIC_DATA	Structure containing information from a HID report event.
USBH_HID_REPORT_INFO	Structure containing information about a HID report.
USBH_HID_RW_CONTEXT	Contains information about a completed, asynchronous transfers.

6.3.1 USBH_HID_DEVICE_INFO

Description

Structure containing information about a HID device.

Type definition

```
typedef struct {
    U16          InputReportSize;
    U16          OutputReportSize;
    U16          ProductId;
    U16          VendorId;
    unsigned     DevIndex;
    USBH_INTERFACE_ID  InterfaceID;
    unsigned     NumReportInfos;
    USBH_HID_REPORT_INFO  ReportInfo[];
    U8           DeviceType;
    U8           InterfaceNo;
} USBH_HID_DEVICE_INFO;
```

Structure members

Member	Description
InputReportSize	Deprecated. = ReportInfo [0]. InputReportSize for compatibility.
OutputReportSize	Deprecated. = ReportInfo [0]. OutputReportSize for compatibility.
ProductId	The Product ID of the device.
VendorId	The Vendor ID of the device.
DevIndex	Device index.
InterfaceID	Interface ID of the HID device.
NumReportInfos	Number of entries in ReportInfo .
ReportInfo	Size and Report Ids of all reports of the interface.
DeviceType	Device type. • USBH_HID_VENDOR - Vendor device • USBH_HID_MOUSE - Mouse device • USBH_HID_KEYBOARD - Keyboard device
InterfaceNo	Index of the interface (from USB descriptor).

6.3.2 USBH_HID_KEYBOARD_DATA

Description

Structure containing information about a keyboard event.

Type definition

```
typedef struct {  
    unsigned      Code;  
    unsigned      Value;  
    USBH_INTERFACE_ID  InterfaceID;  
} USBH_HID_KEYBOARD_DATA;
```

Structure members

Member	Description
Code	Contains the keycode.
Value	Keyboard state info. Refer to sample code for more information.
InterfaceID	ID of the interface that caused the event.

6.3.3 USBH_HID_MOUSE_DATA

Description

Structure containing information about a mouse event.

Type definition

```
typedef struct {  
    int          xChange;  
    int          yChange;  
    int          wheelChange;  
    int          ButtonState;  
    USBH_INTERFACE_ID  InterfaceID;  
} USBH_HID_MOUSE_DATA;
```

Structure members

Member	Description
xChange	Change of x-position since last event.
yChange	Change of y-position since last event.
WheelChange	Change of wheel-position since last event (if wheel is present).
ButtonState	Each bit corresponds to one button on the mouse. If the bit is set, the corresponding button is pressed. Typically, • bit 0 corresponds to the left mouse button • bit 1 corresponds to the right mouse button • bit 2 corresponds to the middle mouse button.
InterfaceID	ID of the interface that caused the event.

6.3.4 USBH_HID_RC_DATA

Description

Structure containing information about a remote control event.

Type definition

```
typedef struct {
    U8          VolumeIncrement;
    U8          VolumeDecrement;
    I8          Mute;
    U8          PlayPause;
    U8          ScanNextTrack;
    U8          ScanPreviousTrack;
    U8          Repeat;
    U8          RandomPlay;
    USBH_INTERFACE_ID InterfaceID;
} USBH_HID_RC_DATA;
```

Structure members

Member	Description
VolumeIncrement	Indicates that the volume increment button was pressed (1 - pressed, 0 - unpressed).
VolumeDecrement	Indicates that the volume decrement button was pressed (1 - pressed, 0 - unpressed).
Mute	Indicates that the mute button was pressed (1 - pressed, 0 - unpressed OR -1 for "off", 0 for "no change" and 1 for "on", which selection variant is used depends on the device, but the second variant is rarely used).
PlayPause	Indicates that the play/pause button was pressed (1 - pressed, 0 - unpressed).
ScanNextTrack	Indicates that the scan next track button was pressed (1 - pressed, 0 - unpressed).
ScanPreviousTrack	Indicates that the scan previous track button was pressed (1 - pressed, 0 - unpressed).
Repeat	Indicates that the repeat button was pressed (1 - pressed, 0 - unpressed).
RandomPlay	Indicates that the random play button was pressed (1 - pressed, 0 - unpressed).
InterfaceID	ID of the interface that caused the event.

6.3.5 USBH_HID_GENERIC_DATA

Description

Structure containing information from a HID report event.

Type definition

```
typedef struct {
    U32          Usage;
    USBH_ANY_SIGNED Value;
    U8          Valid;
    U8          Signed;
    U8          ReportID;
    USBH_ANY_SIGNED LogicalMin;
    USBH_ANY_SIGNED LogicalMax;
    USBH_ANY_SIGNED PhysicalMin;
    USBH_ANY_SIGNED PhysicalMax;
    U8          PhySigned;
    U8          NumBits;
    U16         BitPosStart;
} USBH_HID_GENERIC_DATA;
```

Structure members

Member	Description
Usage	HID usage code. Copied from the array given to <code>USBH_HID_SetOnGenericEvent()</code> . Set to 0, if the usage code was not found in any report descriptor.
Value	Value of the field extracted from the report.
Valid	= 1 if Value field contains valid value.
Signed	= 1 if Value is signed, = 0 if unsigned.
ReportID	ID of the report containing the field.
LogicalMin	Logical minimum from report descriptor. Contains signed value, if Signed = 1, unsigned value otherwise.
LogicalMax	Logical maximum from report descriptor. Contains signed value, if Signed = 1, unsigned value otherwise.
PhysicalMin	Physical minimum from report descriptor. Contains signed value, if PhySigned = 1, unsigned value otherwise.
PhysicalMax	Physical maximum from report descriptor. Contains signed value, if PhySigned = 1, unsigned value otherwise.
PhySigned	= 1 if PhysicalMin / PhysicalMax are signed, = 0 if unsigned.
NumBits	Internal use.
BitPosStart	Internal use.

6.3.6 USBH_HID_REPORT_INFO

Description

Structure containing information about a HID report.

Type definition

```
typedef struct {  
    U8    ReportId;  
    U16   InputReportSize;  
    U16   OutputReportSize;  
} USBH_HID_REPORT_INFO;
```

Structure members

Member	Description
ReportId	Report Id
InputReportSize	Size of input report in bytes.
OutputReportSize	Size of output report in bytes.

6.3.7 USBH_HID_RW_CONTEXT

Description

Contains information about a completed, asynchronous transfers. Is passed to the USBH_HID_ON_COMPLETE_FUNC user callback when using asynchronous write and read. When this structure is passed to USBH_HID_GetReport() or USBH_HID_SetReport() its member need not to be initialized.

Type definition

```
typedef struct {
    void * pUserContext;
    USBH_STATUS Status;
    U32 NumBytesTransferred;
    void * pUserBuffer;
    U32 UserBufferSize;
} USBH_HID_RW_CONTEXT;
```

Structure members

Member	Description
pUserContext	Pointer to a user context. Can be arbitrarily used by the application.
Status	Result status of the asynchronous transfer.
NumBytesTransferred	Number of bytes transferred.
pUserBuffer	Pointer to the buffer provided to USBH_HID_GetReport() or USBH_HID_SetReport().
UserBufferSize	Size of the buffer as provided to USBH_HID_GetReport() or USBH_HID_SetReport().

6.4 Function Types

This chapter describes the emUSB-Host HID API function types.

Type	Description
USBH_HID_ON_KEYBOARD_FUNC	Function called on every keyboard event.
USBH_HID_ON_MOUSE_FUNC	Function called on every mouse event.
USBH_HID_ON_RC_FUNC	Function called on every remote control event.
USBH_HID_ON_GENERIC_FUNC	Function called on every generic HID event.
USBH_HID_USER_FUNC	Function called on completion of <code>USBH_HID_GetReport()</code> or <code>USBH_HID_SetReport()</code> .

6.4.1 USBH_HID_ON_KEYBOARD_FUNC

Description

Function called on every keyboard event.

Type definition

```
typedef void (USBH_HID_ON_KEYBOARD_FUNC)(USBH_HID_KEYBOARD_DATA * pKeyData);
```

Parameters

Parameter	Description
pKeyData	Pointer to a USBH_HID_KEYBOARD_DATA structure.

6.4.2 USBH_HID_ON_MOUSE_FUNC

Description

Function called on every mouse event.

Type definition

```
typedef void (USBH_HID_ON_MOUSE_FUNC)(USBH_HID_MOUSE_DATA * pMouseData);
```

Parameters

Parameter	Description
pMouseData	Pointer to a USBH_HID_MOUSE_DATA structure.

6.4.3 USBH_HID_ON_RC_FUNC

Description

Function called on every remote control event.

Type definition

```
typedef void (USBH_HID_ON_RC_FUNC)(USBH_HID_RC_DATA * pMouseData);
```

Parameters

Parameter	Description
pMouseData	Pointer to a USBH_HID_RC_DATA structure.

6.4.4 USBH_HID_ON_GENERIC_FUNC

Description

Function called on every generic HID event.

Type definition

```
typedef void (USBH_HID_ON_GENERIC_FUNC)
(
    USBH_INTERFACE_ID      InterfaceID,
    unsigned                NumGenericInfos,
    const USBH_HID_GENERIC_DATA * pGenericData);
```

Parameters

Parameter	Description
InterfaceID	Interface ID of the HID device that generated the event.
NumGenericInfos	Number of USBH_HID_GENERIC_DATA structures provided.
pGenericData	Pointer to an array of USBH_HID_GENERIC_DATA structures.

6.4.5 USBH_HID_USER_FUNC

Description

Function called on completion of USBH_HID_GetReport() or USBH_HID_SetReport().

Type definition

```
typedef void (USBH_HID_USER_FUNC)(USBH_HID_RW_CONTEXT * pContext);
```

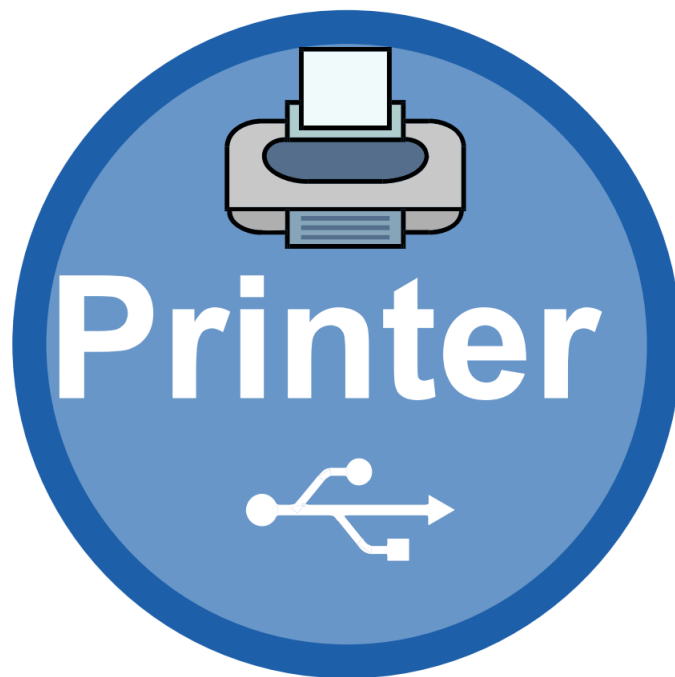
Parameters

Parameter	Description
<code>pContext</code>	Pointer to a USBH_HID_RW_CONTEXT structure.

Chapter 7

Printer class (Add-On)

This chapter describes the emUSB-Host printer class software component and how to use it. The printer class is an optional extension to emUSB-Host.



7.1 Introduction

The printer class software component of emUSB-Host allows the communication to USB printing devices. It implements the USB printer class protocol specified by the USB Implementers Forum.

This chapter describes the architecture, the features and the programming interface of this software component. To improve the readability of application code, all the functions and data types of this API are prefixed with the "USBH_PRINTER_" text.

In the following text the word "printer" is used to refer to any USB device that produces a hard copy of data sent to it.

7.1.1 Overview

A printer connected to the emUSB-Host is automatically configured and added to an internal list. The application receives a notification each time a printer is added or removed over a callback. In order to communicate to a printer the application should open a handle to it. The printers are identified by an index. The first connected printer gets assigned the index 0, the second index 1, and so on. You can use this index to identify a printer in a call to `USBH_PRINTER_OpenByIndex()` function.

7.1.2 Features

The following features are provided:

- Handling of multiple printers at the same time.
- Notifications about printer connection status.
- Ability to query the printer operating status and its device ID.

7.1.3 Example code

An example application which uses the API is provided in the `USBH_Printer_Start.c` file of your shipment. This example displays information about the printer and its connection status in the I/O terminal of the debugger. In addition the text "Hello World" is printed out at the top of the current page when the first printer connects.

7.2 API Functions

This chapter describes the emUSB-Host Printer API functions.

Function	Description
USBH_PRINTER_Close()	Closes a handle to an opened printer.
USBH_PRINTER_ConfigureTimeout()	Sets up the default timeout the host waits until the data transfer will be aborted.
USBH_PRINTER_ExecSoftReset()	Flushes all send and receive buffers.
USBH_PRINTER_Exit()	Unregisters and de-initializes the PRINTER device driver from emUSB-Host.
USBH_PRINTER_GetDeviceId()	Ask the USB printer to send the IEEE.1284 ID string.
USBH_PRINTER_GetNumDevices()	Returns the number of available devices.
USBH_PRINTER_GetPortStatus()	Returns the status of printer.
USBH_PRINTER_Init()	Initialize the Printer device class driver.
USBH_PRINTER_Open()	Opens a handle to a printer.
USBH_PRINTER_OpenByIndex()	Opens a device given by an index.
USBH_PRINTER_Receive()	Reads one packet from the device.
USBH_PRINTER_WriteEx()	Writes data to the printer device.
USBH_PRINTER_RegisterNotification()	This function is deprecated, please use function USBH_PRINTER_AddNotification() ! Sets a callback in order to be notified when a device is added or removed.
USBH_PRINTER_AddNotification()	Adds a callback in order to be notified when a device is added or removed.
USBH_PRINTER_RemoveNotification()	Removes a callback added via USBH_PRINTER_AddNotification() .
USBH_PRINTER_Write()	Sends data to a printer.
USBH_PRINTER_Read()	Receives data from a printer.
USBH_PRINTER_SendVendorRequest()	Sends custom vendor requests to the printer.
USBH_PRINTER_AddCustomDeviceMask()	This function allows the PRINTER module to receive notifications about devices which do not use the correct class and subclass IDs for the printer class.

7.2.1 USBH_PRINTER_Close()

Description

Closes a handle to an opened printer.

Prototype

```
USBH_STATUS USBH_PRINTER_Close(USBH_PRINTER_HANDLE hDevice);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to the opened device.

Return value

= USBH_STATUS_SUCCESS	Successful.
≠ USBH_STATUS_SUCCESS	An error occurred.

7.2.2 USBH_PRINTER_ConfigureTimeout()

Description

Sets up the default timeout the host waits until the data transfer will be aborted.

Prototype

```
void USBH_PRINTER_ConfigureTimeout(U32 Timeout);
```

Parameters

Parameter	Description
Timeout	Timeout given in ms.

7.2.3 USBH_PRINTER_ExecSoftReset()

Description

Flushes all send and receive buffers.

Prototype

```
USBH_STATUS USBH_PRINTER_ExecSoftReset(USBH_PRINTER_HANDLE hDevice);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to the opened printer.

Return value

USBH_STATUS_SUCCESS	Reset executed.
USBH_STATUS_ERROR	An error occurred.

7.2.4 USBH_PRINTER_Exit()

Description

Unregisters and de-initializes the PRINTER device driver from emUSB-Host.

Prototype

```
void USBH_PRINTER_Exit(void);
```

Additional information

Before this function is called any notifications added via `USBH_PRINTER_AddNotification()` must be removed via `USBH_PRINTER_RemoveNotification()`. This function will release resources that were used by this device driver. It has to be called if the application is closed. This has to be called before `USBH_Exit()` is called. No more functions of this module may be called after calling `USBH_PRINTER_Exit()`. The only exception is `USBH_PRINTER_Init()`, which would in turn reinitialize the module and allows further calls.

7.2.5 USBH_PRINTER_GetDeviceId()

Description

Ask the USB printer to send the IEEE.1284 ID string.

Prototype

```
USBH_STATUS USBH_PRINTER_GetDeviceId(USBH_PRINTER_HANDLE hDevice,
                                     U8 * pData,
                                     unsigned NumBytes);
```

Parameters

Parameter	Description
hDevice	Handle to the opened printer device.
pData	Pointer to a caller allocated buffer.
NumBytes	Number of bytes allocated for the read buffer.

Return value

USBH_STATUS_SUCCESS : Device ID read. Any other status : An error occurred.

7.2.6 USBH_PRINTER_GetNumDevices()

Description

Returns the number of available devices.

Prototype

```
int USBH_PRINTER_GetNumDevices(void);
```

Return value

Number of devices available

7.2.7 USBH_PRINTER_GetPortStatus()

Description

Returns the status of printer.

Prototype

```
USBH_STATUS USBH_PRINTER_GetPortStatus(USBH_PRINTER_HANDLE hDevice,  
                                         U8 * pStatus);
```

Parameters

Parameter	Description
hDevice	Handle to the opened printer.
pStatus	Pointer to a caller allocated variable.

Return value

= USBH_STATUS_SUCCESS Status retrieved successfully.
≠ USBH_STATUS_SUCCESS An error occurred.

Additional information

The returned status is to be interpreted as follows:

Bit(s)	Fields	Explanations
7 .. 6	Reserved	Reserved for future use; device shall return these bits set to zero.
5	Paper Empty	1 = Paper Empty, 0 = Paper Not Empty
4	Select	1 = Selected, 0 = Not Selected
3	Not Error	1 = No error, 0 = Error
2 .. 0	Reserved	Reserved for future use; device shall return these bits set to zero.

7.2.8 USBH_PRINTER_Init()

Description

Initialize the Printer device class driver.

Prototype

```
U8 USBH_PRINTER_Init(void);
```

Return value

- | | |
|---|--|
| 1 | Success |
| 0 | Could not register class device driver |

7.2.9 USBH_PRINTER_Open()

Description

Opens a handle to a printer. The printer is identified by its name.

Prototype

```
USBH_PRINTER_HANDLE USBH_PRINTER_Open(const char * sName);
```

Parameters

Parameter	Description
sName	Pointer to a name of the device eg. prt001 for device 0.

Return value

≠ 0 Handle to a printing device
= 0 Device not available or error occurred.

Additional information

It is recommended to use USBH_PRINTER_OpenByIndex(). It is slightly faster.

7.2.10 USBH_PRINTER_OpenByIndex()

Description

Opens a device given by an index.

Prototype

```
USBH_PRINTER_HANDLE USBH_PRINTER_OpenByIndex(unsigned Index);
```

Parameters

Parameter	Description
Index	Index of the device that shall be opened. In general this means: the first connected device is 0, second device is 1 etc.

Return value

≠ USBH_PRINTER_INVALID_HANDLE
= USBH_PRINTER_INVALID_HANDLE

Handle to the device.
Device could not be opened (removed or not available).

7.2.11 USBH_PRINTER_Receive()

Description

Reads one packet from the device. The size of the buffer provided by the caller must be at least the maximum packet size of the IN endpoint. The maximum packet size of the IN endpoint can be retrieved using `USBH_PRINTER_GetDeviceInfo()`.

Prototype

```
USBH_STATUS USBH_PRINTER_Receive(USBH_PRINTER_HANDLE hDevice,  
                                U8 * pData,  
                                U32 * pNumBytesRead,  
                                U32 Timeout);
```

Parameters

Parameter	Description
hDevice	Handle to an open device returned by <code>USBH_PRINTER_Open()</code> .
pData	Pointer to a buffer to store the data read.
pNumBytesRead	Pointer to a variable which receives the number of bytes read from the device.
Timeout	Timeout in ms. 0 means infinite timeout.

Return value

`USBH_STATUS_SUCCESS` on success or error code on failure.

Additional information

This function does not access the buffer used by the function `USBH_PRINTER_Read()`. Data contained in this buffer are not returned by `USBH_PRINTER_Receive()`. Intermixing calls to `USBH_PRINTER_Read()` and `USBH_PRINTER_Receive()` for the same endpoint should be avoided or used with care.

7.2.12 USBH_PRINTER_WriteEx()

Description

Writes data to the printer device. The function blocks until all data has been written or until the timeout has been reached.

Prototype

```
USBH_STATUS USBH_PRINTER_WriteEx(      USBH_PRINTER_HANDLE  hDevice,
                                     const U8          * pData,
                                     U32          NumBytes,
                                     U32          * pNumBytesWritten,
                                     U32          Timeout);
```

Parameters

Parameter	Description
hDevice	Handle to an open device returned by USBH_PRINTER_Open().
pData	Pointer to data to be sent.
NumBytes	Number of bytes to send.
pNumBytesWritten	Pointer to a variable which receives the number of bytes written to the device. Can be NULL.
Timeout	Timeout in ms. 0 means infinite timeout.

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

Additional information

If the function returns an error code (including USBH_STATUS_TIMEOUT) it already may have written part of the data. The number of bytes written successfully is always stored in the variable pointed to by pNumBytesWritten.

7.2.13 USBH_PRINTER_RegisterNotification()

Description

This function is deprecated, please use function `USBH_PRINTER_AddNotification!` Sets a callback in order to be notified when a device is added or removed.

Prototype

```
void USBH_PRINTER_RegisterNotification(USBH_NOTIFICATION_FUNC * pfNotification,  
                                       void * pContext);
```

Parameters

Parameter	Description
<code>pfNotification</code>	Pointer to a function the stack should call when a device is connected or disconnected.
<code>pContext</code>	Pointer to a user context that is passed to the callback function.

Additional information

This function is deprecated, please use function `USBH_PRINTER_AddNotification`.

7.2.14 USBH_PRINTER_AddNotification()

Description

Adds a callback in order to be notified when a device is added or removed.

Prototype

```
USBH_STATUS USBH_PRINTER_AddNotification(USBH_NOTIFICATION_HOOK * pHook,  
                                         USBH_NOTIFICATION_FUNC * pfNotification,  
                                         void * pContext);
```

Parameters

Parameter	Description
<code>pHook</code>	Pointer to a user provided USBH_NOTIFICATION_HOOK variable.
<code>pfNotification</code>	Pointer to a function the stack should call when a device is connected or disconnected.
<code>pContext</code>	Pointer to a user context that is passed to the callback function.

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

7.2.15 USBH_PRINTER_RemoveNotification()

Description

Removes a callback added via USBH_PRINTER_AddNotification.

Prototype

```
USBH_STATUS USBH_PRINTER_RemoveNotification(const USBH_NOTIFICATION_HOOK * pHook);
```

Parameters

Parameter	Description
<code>pHook</code>	Pointer to a user provided USBH_NOTIFICATION_HOOK variable.

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

7.2.16 USBH_PRINTER_Write()

Description

Sends data to a printer.

Prototype

```
USBH_STATUS USBH_PRINTER_Write(      USBH_PRINTER_HANDLE hDevice,  
                                   const U8 * pData,  
                                   unsigned NumBytes);
```

Parameters

Parameter	Description
hDevice	Handle to the opened printer.
pData	Pointer to data to be sent.
NumBytes	Number of bytes to send.

Return value

= USBH_STATUS_SUCCESS Data sent.
≠ USBH_STATUS_SUCCESS An error occurred.

Additional information

This function does not alter the data it sends to printer. Data in ASCII form is typically printed out correctly by the majority of printers. For complex graphics the data passed to this function must be properly formatted according to the protocol the printer understands, like Hewlett Packard PLC, IEEE 1284.1, Adobe Postscript or Microsoft Windows Printing System (WPS).

7.2.17 USBH_PRINTER_Read()

Description

Receives data from a printer.

Prototype

```
USBH_STATUS USBH_PRINTER_Read(USBH_PRINTER_HANDLE hDevice,  
                               U8 * pData,  
                               unsigned NumBytes);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to the opened printer.
<code>pData</code>	Pointer to a caller allocated buffer.
<code>NumBytes</code>	Size of the receive buffer in bytes.

Return value

= USBH_STATUS_SUCCESS Data received.
≠ USBH_STATUS_SUCCESS An error occurred.

Additional information

Not all printers support read operation. For the normal usage of a printer, reading from the printer is normally not required. Some printers do not even provide an IN Endpoint for read operations.

Typically a read operation can be used to feedback status information from the printer to the host. This type of feedback requires usually a command to be sent to the printer first. Which type of information can be read from the printer depends very much on the model.

7.2.18 USBH_PRINTER_SendVendorRequest()

Description

Sends custom vendor requests to the printer.

Prototype

```

USBH_STATUS USBH_PRINTER_SendVendorRequest(USBH_PRINTER_HANDLE hDevice,
                                             U8 RequestType,
                                             U8 Request,
                                             U16 wValue,
                                             void * pData,
                                             U32 * pNumBytes,
                                             U32 Timeout);

```

Parameters

Parameter	Description
hDevice	Handle to the opened printer device.
RequestType	This parameter is a bitmap containing the following values: • bit 7 transfer direction: • 0 = OUT (Host to Device) • 1 = IN (Device to Host) • bits 6..5 request type: • 0 = Standard • 1 = Class • 2 = Vendor • 3 = Reserved • bits 4..0 recipient: • 0 = Device • 1 = Interface • 2 = Endpoint • 3 = Other
Request	Request code in the setup request.
wValue	wValue in the setup request.
pData	Additional data for the setup request.
pNumBytes	• in Number of bytes to be received/sent in pData. • out Number of bytes processed.
Timeout	Timeout in ms. 0 means infinite timeout.

Return value

= USBH_STATUS_SUCCESS Successful.
 ≠ USBH_STATUS_SUCCESS An error occurred.

Additional information

wLength which is normally part of the setup packet will be determined given by the [pNumBytes](#) and [pData](#). In case no pBuffer is given, wLength will be 0.

7.2.19 USBH_PRINTER_AddCustomDeviceMask()

Description

This function allows the PRINTER module to receive notifications about devices which do not use the correct class and subclass IDs for the printer class.

Prototype

```
USBH_STATUS USBH_PRINTER_AddCustomDeviceMask(const U16 * pVendorIds,
                                              const U16 * pProductIds,
                                              U16    NumIds);
```

Parameters

Parameter	Description
pVendorIds	Array of vendor IDs.
pProductIds	Array of product IDs.
NumIds	Number of elements in both arrays, each index in both arrays is used as a pair to create a filter.

Return value

USBH_STATUS_SUCCESS	Success.
USBH_STATUS_ERROR	Notification could not be registered.

7.3 Data structures

This chapter describes the emUSB-Host Printer class data structures.

Structure	Description
USBH_PRINTER_DEVICE_INFO	

7.3.1 USBH_PRINTER_DEVICE_INFO

Type definition

```
typedef struct {  
    U16  VendorId;  
    U16  ProductId;  
    U16  bcdDevice;  
    U16  MaxPacketSize_OUT;  
    U16  MaxPacketSize_IN;  
    U8   acSerialNo[];  
} USBH_PRINTER_DEVICE_INFO;
```

Structure members

Member	Description
VendorId	The printer's vendor ID.
ProductId	The printer's product ID.
bcdDevice	Binary Coded Decimal device version.
MaxPacketSize_OUT	Maximum packet size of the bulk OUT EP.
MaxPacketSize_IN	Maximum packet size of the bulk IN EP. If this value is zero it means that the printer does not have an IN endpoint.
acSerialNo	The printer's serial number.

Chapter 8

Mass Storage Device (MSD) class

This chapter describes the emUSB-Host Mass storage device class driver and its usage. The MSD class is part of the BASE package. The MSD class code is only linked in if registered by the application program.

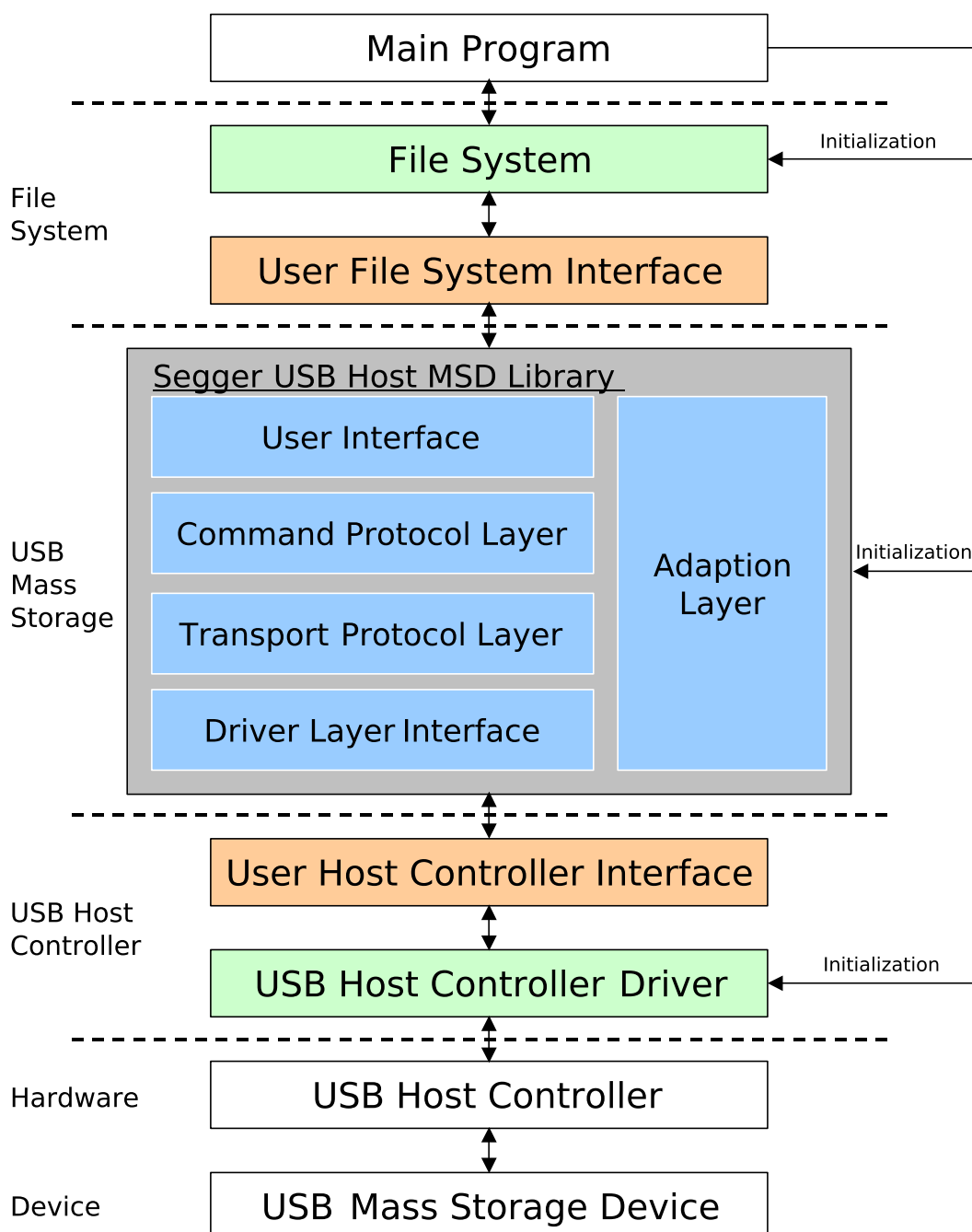


8.1 Introduction

The emUSB-Host MSD class software allows accessing USB Mass Storage Devices. It implements the USB Mass Storage Device class protocols specified by the USB Implementers Forum. The entire API of this class driver is prefixed "USBH_MSD_". This chapter describes the architecture, the features and the programming interface of the class driver.

8.1.1 Overview

A mass storage device connected to the emUSB-Host is added to the file system as a device. All operations on the device, such as formatting, reading / writing of files and directories are performed through the API of the file system. With *emFile*, the device name of the first MSD is "msd:0:". The structure of MSD component is shown in the following diagram:



8.1.2 Features

The following features are provided:

- The command block specification and protocol implementation used by the connected device will be automatically detected.
- It is independent of the file system. An interface to *emFile* is provided.

8.1.3 Requirements

To use the MSD class driver to perform file and directory operations, a file system (typically *emFile*) is required.

8.1.4 Example code

Example code which is provided in the file `USBH_MSD_Start.c`. The example shows the capacity of the connected device, shows files in the root directory and creates and writes to a file.

8.1.5 Supported Protocols

The following table contains an overview about the implemented command protocols.

Command block specification	Implementation	Related documents
SCSI transparent command set	All necessary commands for accessing flash devices.	Mass Storage Class Specification Overview Revision 1.2., SCSI-2 Specification September 1993 Rev.10 (X3T9.2 Project 275D)

The following table contains an overview about the implemented transport protocols.

Protocol implementation	Implementation	Related documents
Bulk-Only transport	All commands implemented	Universal Serial Bus Mass Storage Class Bulk-Only Transport Rev.1.0.

8.2 API Functions

This chapter describes the emUSB-Host MSD API functions.

Function	Description
USBH_MSD_Exit()	Releases all resources, closes all handles to the USB bus driver and un-register all notification functions.
USBH_MSD_GetStatus()	Checks the Status of a device.
USBH_MSD_GetUnits()	Returns available units for a device.
USBH_MSD_GetUnitInfo()	Returns basic information about the logical unit (LUN).
USBH_MSD_GetPortInfo()	Retrieves the port information about a USB MSC device using a unit ID.
USBH_MSD_Init()	Initializes the USB Mass Storage Class Driver.
USBH_MSD_ReadSectors()	Reads sectors from a USB Mass Storage device.
USBH_MSD_WriteSectors()	Writes sectors to a USB Mass Storage device.
USBH_MSD_UseAheadCache()	Enables the read-ahead-cache functionality.
USBH_MSD_SetAheadBuffer()	Sets a user provided buffer for the read-ahead-cache functionality.

8.2.1 USBH_MSD_Exit()

Description

Releases all resources, closes all handles to the USB bus driver and un-register all notification functions. Has to be called if the application is closed before the `USBH_Exit` is called.

Prototype

```
void USBH_MSD_Exit(void);
```

8.2.2 USBH_MSD_GetStatus()

Description

Checks the Status of a device. Therefore it performs a "Test **Unit** Ready" command to test if the device is still connected and if a logical unit is assigned.

Prototype

```
USBH_STATUS USBH_MSD_GetStatus(U8 Unit);
```

Parameters

Parameter	Description
Unit	0-based Unit Id. See USBH_MSD_GetUnits().

Return value

= USBH_STATUS_SUCCESS	Device is ready for operation.
≠ USBH_STATUS_SUCCESS	An error occurred.

8.2.3 USBH_MSD_GetUnits()

Description

Returns available units for a device.

Prototype

```
USBH_STATUS USBH_MSD_GetUnits(U8    DevIndex,  
                               U32 * pUnitMask);
```

Parameters

Parameter	Description
DevIndex	Index of the MSD device returned by USBH_MSD_LUN_NOTIFICATION_FUNC.
pUnitMask	out Pointer to a U32 variable which will receive the LUN mask.

Return value

= USBH_STATUS_SUCCESS Device is ready for operation.
≠ USBH_STATUS_SUCCESS An error occurred.

Additional information

The mask corresponds to the unit IDs. The device has unit ID n, if bit n of the mask is set. E.g. a mask of 0x0000000C means unit ID 2 and unit ID 3 are available for the device.

8.2.4 USBH_MSD_GetUnitInfo()

Description

Returns basic information about the logical unit (LUN).

Prototype

```
USBH_STATUS USBH_MSD_GetUnitInfo(U8 Unit,  
                                USBH_MSD_UNIT_INFO * pInfo);
```

Parameters

Parameter	Description
<code>Unit</code>	0-based <code>Unit</code> Id. See <code>USBH_MSD_GetUnits()</code> .
<code>pInfo</code>	<code>out</code> Pointer to a caller provided structure of type <code>USBH_MSD_UNIT_INFO</code> . It receives the information about the LUN in case of success.

Return value

<code>= USBH_STATUS_SUCCESS</code>	Device is ready for operation.
<code>≠ USBH_STATUS_SUCCESS</code>	An error occurred.

8.2.5 USBH_MSD_GetPortInfo()

Description

Retrieves the port information about a USB MSC device using a unit ID.

Prototype

```
USBH_STATUS USBH_MSD_GetPortInfo(U8 Unit,  
                                  USBH_PORT_INFO * pPortInfo);
```

Parameters

Parameter	Description
<code>Unit</code>	0-based <code>Unit</code> Id. See <code>USBH_MSD_GetUnits()</code> .
<code>pPortInfo</code>	<code>out</code> Pointer to a caller provided structure of type <code>USBH_PORT_INFO</code> . It receives the information about the LUN in case of success.

Return value

<code>= USBH_STATUS_SUCCESS</code>	Success, <code>pPortInfo</code> contains valid port information.
<code>≠ USBH_STATUS_SUCCESS</code>	An error occurred.

8.2.6 USBH_MSD_Init()

Description

Initializes the USB Mass Storage Class Driver.

Prototype

```
int USBH_MSD_Init(USBH_MSD_LUN_NOTIFICATION_FUNC * pLunNotification,
                  void * pContext);
```

Parameters

Parameter	Description
<code>pLunNotification</code>	Pointer to a function that shall be called when a new device notification is received. The function is called when a device is attached and ready or when it is removed.
<code>pContext</code>	Pointer to a context that should be passed to <code>pLunNotification</code> .

Return value

1 Success.
0 Initialization failed.

Additional information

Performs basic initialization of the library. Has to be called before any other library function is called.

Example:

```

/*****
 *
 *      _cbOnAddRemoveDevice
 *
 * Function description
 *      Callback, called when a device is added or removed.
 *      Call in the context of the USBH_Task.
 *      The functionality in this routine should not block!
 */
static void _cbOnAddRemoveDevice(void * pContext, U8 DevIndex, USBH_MSD_EVENT Event) {
    switch (Event) {
        case USBH_MSD_EVENT_ADD:
            USBH_Logf_Application("**** Device added\n");
            _MSDReady = 1;
            _CurrentDevIndex = DevIndex;
            break;
        case USBH_MSD_EVENT_REMOVE:
            USBH_Logf_Application("**** Device removed\n");
            _MSDReady = 0;
            _CurrentDevIndex = 0xff;
            break;
        default: /* USBH_MSD_EVENT_ERROR */
            USBH_Logf_Application("**** Device error\n");
            break;
    }
}

<...>
USBH_MSD_Init(_cbOnAddRemoveDevice, NULL);
<...>

```

8.2.7 USBH_MSD_ReadSectors()

Description

Reads sectors from a USB Mass Storage device. To read file and folders use the file system functions. This function allows to read sectors raw.

Prototype

```
USBH_STATUS USBH_MSD_ReadSectors(U8      Unit,  
                                  U32     SectorAddress,  
                                  U32     NumSectors,  
                                  U8      * pBuffer);
```

Parameters

Parameter	Description
Unit	0-based Unit Id. See USBH_MSD_GetUnits().
SectorAddress	Index of the first sector to read. The first sector has the index 0.
NumSectors	Number of sectors to read.
pBuffer	Pointer to a caller allocated buffer.

Return value

- = USBH_STATUS_SUCCESS Sectors successfully read.
- ≠ USBH_STATUS_SUCCESS An error occurred.

8.2.8 USBH_MSD_WriteSectors()

Description

Writes sectors to a USB Mass Storage device. To write files and folders use the file system functions. This function allows to write sectors raw.

Prototype

```
USBH_STATUS USBH_MSD_WriteSectors(    U8    Unit,
                                      U32    SectorAddress,
                                      U32    NumSectors,
                                      const U8 * pBuffer);
```

Parameters

Parameter	Description
Unit	0-based Unit Id. See USBH_MSD_GetUnits().
SectorAddress	Index of the first sector to write. The first sector has the index 0.
NumSectors	Number of sectors to write.
pBuffer	Pointer to the data.

Return value

- = USBH_STATUS_SUCCESS

≠ USBH_STATUS_SUCCESS
- Sectors successfully written.

An error occurred.

8.2.9 USBH_MSD_UseAheadCache()

Description

Enables the read-ahead-cache functionality.

Prototype

```
void USBH_MSD_UseAheadCache(int OnOff);
```

Parameters

Parameter	Description
OnOff	1 : on, 0 - off.

Additional information

The read-ahead-cache is a functionality which makes sure that read accesses to an MSD will always read a minimal amount of sectors (normally at least four). The rest of the sectors which have not been requested directly will be stored in a cache and subsequent reads will be supplied with data from the cache instead of the actual device.

This functionality is mainly used as a workaround for certain MSD devices which crash when single sectors are being read directly from the device too often. Enabling the cache will cause a slight drop in performance, but will make sure that all MSD devices which are affected by the aforementioned issue do not crash. Unless `USBH_MSD_SetAheadBuffer()` was used before calling this function with a "1" as parameter the function will try to allocate a buffer for eight sectors (4096 bytes) from the emUSB-Host memory pool.

8.2.10 USBH_MSD_SetAheadBuffer()

Description

Sets a user provided buffer for the read-ahead-cache functionality.

Prototype

```
void USBH_MSD_SetAheadBuffer(const USBH_MSD_AHEAD_BUFFER * pAheadBuf);
```

Parameters

Parameter	Description
<code>pAheadBuf</code>	Pointer to a <code>USBH_MSD_AHEAD_BUFFER</code> structure which holds the buffer information.

Additional information

This function has to be called before enabling the read-ahead-cache with `USBH_MSD_UseAheadCache()`. The buffer should have space for at least four sectors (2048 bytes), but eight sectors (4096 bytes) are suggested for better performance. The buffer size must be a multiple of 512.

8.3 Data Structures

This chapter describes the used emUSB-Host MSD API structures.

Function	Description
USBH_MSD_UNIT_INFO	Contains logical unit information.
USBH_MSD_AHEAD_BUFFER	Structure describing the read-ahead-cache buffer.

8.3.1 USBH_MSD_UNIT_INFO

Description

Contains logical unit information.

Type definition

```
typedef struct {  
    U32    TotalSectors;  
    U16    BytesPerSector;  
    int    WriteProtectFlag;  
    U16    VendorId;  
    U16    ProductId;  
    char    acVendorName[];  
    char    acProductName[];  
    char    acRevision[];  
} USBH_MSD_UNIT_INFO;
```

Structure members

Member	Description
TotalSectors	Contains the number of total sectors available on the LUN.
BytesPerSector	Contains the number of bytes per sector.
WriteProtectFlag	Nonzero if the device is write protected.
VendorId	USB Vendor ID.
ProductId	USB Product ID.
acVendorName	LUN's vendor identification string.
acProductName	LUN's product identification string.
acRevision	LUN's revision string.

8.3.2 USBH_MSD_AHEAD_BUFFER

Description

Structure describing the read-ahead-cache buffer.

Type definition

```
typedef struct {  
    U8 * pBuffer;  
    U32 Size;  
} USBH_MSD_AHEAD_BUFFER;
```

Structure members

Member	Description
pBuffer	Pointer to a buffer.
Size	Size of the buffer in bytes.

8.4 Function Types

This chapter describes the used emUSB-Host MSD API function types.

Type	Description
USBH_MSD_LUN_NOTIFICATION_FUNC	This callback function is called when a logical unit is either added or removed.

8.4.1 USBH_MSD_LUN_NOTIFICATION_FUNC

Description

This callback function is called when a logical unit is either added or removed. To get detailed information `USBH_MSD_GetStatus()` has to be called. The LUN indexes must be used to get access to a specified unit of the device.

Type definition

```
typedef void USBH_MSD_LUN_NOTIFICATION_FUNC(void * pContext,  
                                             U8      DevIndex,  
                                             USBH_MSD_EVENT Event);
```

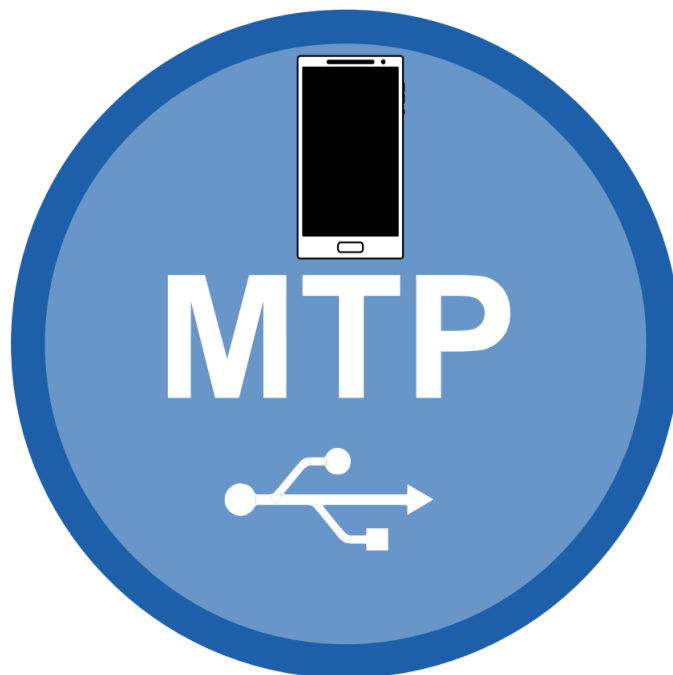
Parameters

Parameter	Description
<code>pContext</code>	Pointer to a context that was set by the user when the <code>USBH_MSD_Init()</code> was called.
<code>DevIndex</code>	Zero based index of the device that was attached or removed.
<code>Event</code>	Gives information about the event that has occurred. The following events are currently available: • <code>USBH_MSD_EVENT_ADD</code> A device was attached. • <code>USBH_MSD_EVENT_REMOVE</code> A device was removed. • <code>USBH_MSD_EVENT_ERROR</code> A new device could not be added because of errors.

Chapter 9

MTP Device Driver (Add-On)

This chapter describes the optional emUSB-Host add-on “MTP device driver”. It allows communication with MTP USB devices.



9.1 Introduction

The MTP driver software component of emUSB-Host allows communication with MTP devices such as Android or Windows smartphones, media players, cameras and so on. A file system is not required to use emUSB-Host MTP. This chapter provides an explanation of the functions available to application developers via the MTP driver software. All the functions and data types of this add-on are prefixed with the "USBH_MTP_" text.

9.1.1 Overview

An MTP device connected to the emUSB-Host is automatically configured and added to an internal list. If the MTP module has been registered, it is notified via a callback when an MTP device has been added or removed. The driver then can notify the application program when a callback function has been registered via `USBH_MTP_RegisterNotification()`. In order to communicate with such a device, the application has to call `USBH_MTP_Open()`, passing the device index. MTP devices are identified by an index. The first connected device gets assigned the index 0, the second index 1, and so on.

9.1.2 Features

The following features are provided:

- Compatibility with different MTP devices.

9.1.3 Example code

An example application which demonstrates the API is provided in the `USBH_MTP_Start.c` file.

9.2 API Functions

This chapter describes the emUSB-Host MTP driver API functions.

Function	Description
USBH_MTP_Init()	Initializes and registers the MTP device driver with emUSB-Host.
USBH_MTP_Exit()	Unregisters and de-initializes the MTP device driver from emUSB-Host.
USBH_MTP_RegisterNotification()	Sets a callback in order to be notified when a device is added or removed.
USBH_MTP_Open()	Opens a device using the given index.
USBH_MTP_Close()	Closes a handle to an opened device.
USBH_MTP_GetDeviceInfo()	Retrieves basic information about the MTP device.
USBH_MTP_GetNumStorages()	Retrieves the number of storages the device has.
USBH_MTP_Reset()	Executes the MTP reset command on the device.
USBH_MTP_SetTimeouts()	Sets timeouts for read and write transactions for a device.
USBH_MTP_GetLastErrorCode()	Returns the error code for the last executed operation.
USBH_MTP_GetStorageInfo()	Retrieves information about a storage on the device.
USBH_MTP_Format()	Formats (deletes all data!) on a device storage.
USBH_MTP_GetNumObjects()	Retrieves the number of objects inside a single directory.
USBH_MTP_GetObjectList()	Retrieves a list of object IDs from a directory.
USBH_MTP_GetObjectInfo()	Retrieves the ObjectInfo dataset for a specific object.
USBH_MTP_CreateObject()	Writes a new object onto the device.
USBH_MTP_DeleteObject()	Deletes an object from the device.
USBH_MTP_RenameObject()	Changes the name of an object.
USBH_MTP_ReadFile()	Reads a file from the device.
USBH_MTP_GetDevicePropDesc()	Retrieves the description of a MTP property from the device.
USBH_MTP_GetDevicePropValue()	Retrieves the value of a property of a specific Device.
USBH_MTP_GetObjectPropsSupported()	Retrieves a list of supported properties for a given object format.
USBH_MTP_GetObjectPropDesc()	Retrieves information about an MTP object property used by the device.
USBH_MTP_GetObjectPropValue()	Retrieves the value of a property of a specific object.
USBH_MTP_SetObjectProperty()	Sets the property of an object to the specified value.
USBH_MTP_CheckLock()	Determines whether the device is locked by a pin/password/etc.

Function	Description
USBH_MTP_SetEventCallback()	Sets a callback for MTP events, e.g.
USBH_MTP_ConfigEventSupport()	Turns MTP event support on or off.
USBH_MTP_GetEventSupport()	Returns the event support configuration, see USBH_MTP_ConfigEventSupport() for details.

9.2.1 USBH_MTP_Init()

Description

Initializes and registers the MTP device driver with emUSB-Host.

Prototype

```
USBH_STATUS USBH_MTP_Init(void);
```

Return value

USBH_STATUS_SUCCESS	Success.
USBH_STATUS_MEMORY	Can not init MTP module, out of memory.

9.2.2 USBH_MTP_Exit()

Description

Unregisters and de-initializes the MTP device driver from emUSB-Host.

Prototype

```
void USBH_MTP_Exit(void);
```

Additional information

This function will release resources that were used by this device driver. It has to be called if the application is closed. This has to be called before `USBH_Exit()` is called. No more functions of this module may be called after calling `USBH_MTP_Exit()`. The only exception is `USBH_MTP_Init()`, which would in turn re-init the module and allow further calls.

9.2.3 USBH_MTP_RegisterNotification()

Description

Sets a callback in order to be notified when a device is added or removed.

Prototype

```
void USBH_MTP_RegisterNotification(USBH_NOTIFICATION_FUNC * pfNotification,  
                                   void * pContext);
```

Parameters

Parameter	Description
<code>pfNotification</code>	Pointer to a function the stack should call when a device is connected or disconnected.
<code>pContext</code>	Pointer to a user context that should be passed to the callback function.

Additional information

Only one notification function can be set for all devices. To unregister, call this function with the `pfNotification` parameter set to NULL.

9.2.4 USBH_MTP_Open()

Description

Opens a device using the given index.

Prototype

```
USBH_MTP_DEVICE_HANDLE USBH_MTP_Open(U8 Index);
```

Parameters

Parameter	Description
Index	Index of the device that should be opened. In general this means: the first connected device is 0, second device is 1 etc.

Return value

≠ 0 Handle to the device
= 0 Device not available or removed.

9.2.5 USBH_MTP_Close()

Description

Closes a handle to an opened device.

Prototype

```
USBH_STATUS USBH_MTP_Close(USBH_MTP_DEVICE_HANDLE hDevice);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to the opened device.

Return value

= USBH_STATUS_SUCCESS	Successful.
≠ USBH_STATUS_SUCCESS	An error occurred.

9.2.6 USBH_MTP_GetDeviceInfo()

Description

Retrieves basic information about the MTP device.

Prototype

```
USBH_STATUS USBH_MTP_GetDeviceInfo(USBH_MTP_DEVICE_HANDLE hDevice,  
                                   USBH_MTP_DEVICE_INFO * pDevInfo);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to the opened device.
<code>pDevInfo</code>	out Pointer to a <code>USBH_MTP_DEVICE_INFO</code> structure where the information related to the device will be stored.

Return value

= <code>USBH_STATUS_SUCCESS</code>	Successful.
≠ <code>USBH_STATUS_SUCCESS</code>	An error occurred.

9.2.7 USBH_MTP_GetNumStorages()

Description

Retrieves the number of storages the device has.

Prototype

```
USBH_STATUS USBH_MTP_GetNumStorages(USBH_MTP_DEVICE_HANDLE hDevice,  
                                     U8 * pNumStorages);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to the opened device.
<code>pNumStorages</code>	out Pointer to a variable where the number of storages reported by the device will be stored.

Return value

= USBH_STATUS_SUCCESS Successful.
≠ USBH_STATUS_SUCCESS An error occurred.

Additional information

This function may return zero storages when the device is locked. Unfortunately this is not always the case and can not be used as a criteria to check whether a device is locked (e.g. Windows Phones will return the correct number of storages even if they are locked.)

See USBH_MTP_CheckLock() for further information.

9.2.8 USBH_MTP_Reset()

Description

Executes the MTP reset command on the device. This command sets the device in the default state. "Default state" can mean different things for different manufacturers. This MTP command is rarely supported by devices. This command will close all sessions on the device side. Therefore the host application should call `USBH_MTP_Close()` after a successful call to this function.

Prototype

```
USBH_STATUS USBH_MTP_Reset(USBH_MTP_DEVICE_HANDLE hDevice);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to the opened device.

Return value

<code>= USBH_STATUS_SUCCESS</code>	Successful.
<code>≠ USBH_STATUS_SUCCESS</code>	An error occurred.

9.2.9 USBH_MTP_SetTimeouts()

Description

Sets timeouts for read and write transactions for a device. The timeouts are valid for single transactions, not for whole API calls.

Prototype

```
USBH_STATUS USBH_MTP_SetTimeouts(USBH_MTP_DEVICE_HANDLE hDevice,  
                                U32 ReadTimeout,  
                                U32 WriteTimeout);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device.
ReadTimeout	Timeout for all transactions which read from the device.
WriteTimeout	Timeout for all transactions which write to the device.

Return value

= USBH_STATUS_SUCCESS Successful.
≠ USBH_STATUS_SUCCESS An error occurred.

Additional information

It is advised to set the timeouts to at least 10 seconds, as this is the time many Android devices may require to respond to certain commands.

9.2.10 USBH_MTP_GetLastErrorCode()

Description

Returns the error code for the last executed operation.

Prototype

```
U16 USBH_MTP_GetLastErrorCode(USBH_MTP_DEVICE_HANDLE hDevice);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device.

Return value

= 0 Last operation completed without an error code.
≠ 0 Error code. See USBH_MTP_RESPONSE_CODES for a list of MTP error codes.

9.2.11 USBH_MTP_GetStorageInfo()

Description

Retrieves information about a storage on the device.

Prototype

```
USBH_STATUS USBH_MTP_GetStorageInfo(USBH_MTP_DEVICE_HANDLE hDevice,  
                                     U8 StorageIndex,  
                                     USBH_MTP_STORAGE_INFO * pStorageInfo);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device.
StorageIndex	Zero-based index of the storage, see <code>USBH_MTP_GetNumStorages()</code> .
pStorageInfo	out Pointer to a <code>USBH_MTP_STORAGE_INFO</code> structure to store information related to the storage.

Return value

- = `USBH_STATUS_SUCCESS` Successful.
- ≠ `USBH_STATUS_SUCCESS` An error occurred.

Notes

This operation is always supported by MTP devices.

9.2.12 USBH_MTP_Format()

Description

Formats (deletes all data!) on a device storage.

Prototype

```
USBH_STATUS USBH_MTP_Format(USBH_MTP_DEVICE_HANDLE hDevice,  
                             U8                      StorageIndex);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device.
StorageIndex	Zero-based index of the storage, see <code>USBH_MTP_GetNumStorages()</code> .

Return value

= <code>USBH_STATUS_SUCCESS</code>	Successful.
≠ <code>USBH_STATUS_SUCCESS</code>	An error occurred.

9.2.13 USBH_MTP_GetNumObjects()

Description

Retrieves the number of objects inside a single directory.

Prototype

```
USBH_STATUS USBH_MTP_GetNumObjects(USBH_MTP_DEVICE_HANDLE hDevice,
                                   U8 StorageIndex,
                                   U32 DirObjectID,
                                   U32 * pNumObjects);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device.
StorageIndex	Zero-based index of the storage, see USBH_MTP_GetNumStorages().
DirObjectID	Object ID for the directory.
pNumObjects	out Pointer to a variable where the number of objects inside the directory will be stored.

Return value

- = USBH_STATUS_SUCCESS Successful.
- ≠ USBH_STATUS_SUCCESS An error occurred.

9.2.14 USBH_MTP_GetObjectList()

Description

Retrieves a list of object IDs from a directory. The number of objects inside a directory can be found out beforehand by using `USBH_MTP_GetNumObjects`.

Prototype

```
USBH_STATUS USBH_MTP_GetObjectList(USBH_MTP_DEVICE_HANDLE hDevice,
                                   U8 StorageIndex,
                                   U32 DirObjectID,
                                   USBH_MTP_OBJECT * pBuffer,
                                   U32 * pNumObjects);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to the opened device.
<code>StorageIndex</code>	Zero-based index of the storage, see <code>USBH_MTP_GetNumStorages()</code> .
<code>DirObjectID</code>	Object ID for the directory.
<code>pBuffer</code>	out Pointer to an array of <code>USBH_MTP_OBJECT</code> structures.
<code>pNumObjects</code>	in/out The application should specify the size of the buffer in <code>USBH_MTP_OBJECT</code> units. The MTP module will read object IDs up to the specified value. If there are less objects in the folder the number of objects read will be stored in this variable. If there are more objects in the folder than specified the data for the surplus objects is discarded by the module.

Return value

= `USBH_STATUS_SUCCESS` Successful.
 ≠ `USBH_STATUS_SUCCESS` An error occurred.

Example

```
static USBH_MTP_OBJECT _aObjBuffer[10];
<...>
Status = USBH_MTP_GetNumObjects(hDevice, StorageIndex, DirObjectID, &NumObjectsDir);
if (Status == USBH_STATUS_SUCCESS) {
    USBH_Logf_Application("Found %d objects in directory 0x%0.8X \n",
                          NumObjectsDir, DirObjectID);
    NumObjects = USBH_MIN(NumObjectsDir, NumObjectsFree);
    //
    // Retrieve a list of object IDs from the root directory.
    //
    Status = USBH_MTP_GetObjectList(hDevice,
                                    StorageIndex,
                                    DirObjectID,
                                    _aObjBuffer,
                                    &NumObjects);

    if (Status == USBH_STATUS_SUCCESS) {
        <...>
    } else {
        <...>
    }
} else {
    <...>
}
```

9.2.15 USBH_MTP_GetObjectInfo()

Description

Retrieves the ObjectInfo dataset for a specific object.

Prototype

```
USBH_STATUS USBH_MTP_GetObjectInfo(USBH_MTP_DEVICE_HANDLE hDevice,
                                   U32 ObjectID,
                                   USBH_MTP_OBJECT_INFO * pObjInfo);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device.
ObjectID	Object ID to retrieve information for.
pObjInfo	out Pointer to a USBH_MTP_OBJECT_INFO structure where the data will be stored.

Return value

- = USBH_STATUS_SUCCESS Successful.
- ≠ USBH_STATUS_SUCCESS An error occurred.

9.2.16 USBH_MTP_CreateObject()

Description

Writes a new object onto the device. MTP does not allow files to be written in chunks, therefore a callback mechanism is implemented to allow the embedded host to write files of any size onto the MTP device. As soon as the contents of the first buffer have been written or the file has been completely written onto the device - the registered callback is called. Inside the callback the user can either put new data into the previously used buffer or change the buffer by modifying the `pNextBuffer` parameter inside the `USBH_SEND_DATA_FUNC` callback. Using two (or more) buffers and switching between them has the advantage that the MTP module can write continuously to the device.

Prototype

```
USBH_STATUS USBH_MTP_CreateObject(USBH_MTP_DEVICE_HANDLE hDevice,
                                  U8 StorageIndex,
                                  USBH_MTP_CREATE_INFO * pInfo);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to the opened device.
<code>StorageIndex</code>	Zero-based index of the storage, see <code>USBH_MTP_GetNumStorages()</code> .
<code>pInfo</code>	in/out Pointer to a <code>USBH_MTP_CREATE_INFO</code> structure where parameters for the new object are stored.

Return value

= `USBH_STATUS_SUCCESS` Successful. On success the member `ObjectID` of the `USBH_MTP_CREATE_INFO` will contain the new object ID provided by the device.

≠ `USBH_STATUS_SUCCESS` An error occurred.

Example

```
static U8 _acBufWrite[1024*64];
const U16 _sFileName[] = L"SEGGER.txt";
U32 FileSize = 1024 * 1024;

/*****
 *
 *      _SendData
 *
 *      Function description
 *      In this sample application the file data is simply generated
 *      through a memset, in a real application data can for example
 *      be read from the host's file system.
 */
static void _SendData(void * pUserContext,
                     U32 NumBytesSentTotal,
                     U32 * pNumBytesToSend,
                     void ** pNextBuffer) {
    U32 NumBytesToSend;
    int r;

    NumBytesToSend = *(U32*)&pUserContext - NumBytesSentTotal;
    NumBytesToSend = USBH_MIN(sizeof(_acBufWrite), NumBytesToSend);
    if (NumBytesToSend) {
        USBH_MEMSET(_acBufWrite, 0xA5, NumBytesToSend);
    }
}
```



```
<...>
USBH_MTP_CREATE_INFO CreateInfo;

CreateInfo.FileNameSize    = strlen(_sFileName) + 1; // File name length
                                                                // + terminating character
CreateInfo.sFileName       = _sFileName;              // File name Unicode string
CreateInfo.ObjectSize      = FileSize;                // Size of the file in bytes
CreateInfo.ParentObjectID  = ObjectID;                // Object ID of the parent
                                                                // folder
CreateInfo.pfGetData       = _SendData;               // Callback function
CreateInfo.pUserBuf        = _acBufWrite;             // User buffer containing
                                                                // the data
CreateInfo.UserBufSize     = sizeof(_acBufWrite);     // Size of the user buffer
CreateInfo.isFolder        = FALSE;                   // Not a folder
CreateInfo.pUserContext    = (void*)FileSize;         // User context is passed
                                                                // to the callback

Status = USBH_MTP_CreateObject(hDevice, StorageIndex, &CreateInfo);
<...>
```

9.2.17 USBH_MTP_DeleteObject()

Description

Deletes an object from the device.

Prototype

```
USBH_STATUS USBH_MTP_DeleteObject(USBH_MTP_DEVICE_HANDLE hDevice,  
                                   U32                      ObjectID);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device.
ObjectID	Object ID to be deleted.

Return value

= USBH_STATUS_SUCCESS	Successful.
≠ USBH_STATUS_SUCCESS	An error occurred.

9.2.18 USBH_MTP_RenameObject()

Description

Changes the name of an object.

Prototype

```
USBH_STATUS USBH_MTP_RenameObject(      USBH_MTP_DEVICE_HANDLE hDevice,
                                         U32 ObjectID,
                                         const U16 * sNewName,
                                         U32 NumChars);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device.
ObjectID	Object ID to retrieve the property from.
sNewName	Pointer to a Unicode string containing the new file name.
NumChars	Length of the new file name in U16 units.

Return value

- = USBH_STATUS_SUCCESSSuccessful.
- ≠ USBH_STATUS_SUCCESSAn error occurred.

9.2.19 USBH_MTP_ReadFile()

Description

Reads a file from the device. MTP does not allow files to be read in chunks, therefore a callback mechanism is implemented to allow embedded devices with limited memory to be able to read files of any size from an MTP device. The callback is called as soon as the user provided buffer is full or the file has been completely read. In the callback the user can either process the data in the user buffer or change the user buffer by writing the pNextBuffer parameter inside the USBH_RECEIVE_DATA_FUNC callback and process the data in the first buffer in another task. The second method has the advantage that the callback can return immediately and the MTP module can continue reading from the device.

Prototype

```
USBH_STATUS USBH_MTP_ReadFile(USBH_MTP_DEVICE_HANDLE hDevice,
                               U32 ObjectID,
                               USBH_RECEIVE_DATA_FUNC * pfReadData,
                               void * pUserContext,
                               U8 * pUserBuf,
                               U32 UserBufSize);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to the opened device.
<code>ObjectID</code>	Object ID of the file to read.
<code>pfReadData</code>	Pointer to a user-provided USBH_RECEIVE_DATA_FUNC callback function which will be called when the file is being received.
<code>pUserContext</code>	Pointer to a user context which is passed to the callback function.
<code>pUserBuf</code>	Pointer to a buffer where the data will be stored. This parameter can be NULL. In this case the callback is called directly and a buffer has to be set from there.
<code>UserBufSize</code>	Size of the user buffer.

Return value

= USBH_STATUS_SUCCESS Successful.
 ≠ USBH_STATUS_SUCCESS An error occurred.

Example

```

/*****
 *
 * _ReceiveData
 *
 * Function description
 * This function is responsible for receiving the data provided by
 * the USBH_MTP_ReadFile function.
 */
static void _ReceiveData(void * pUserContext,
                        U32 NumBytesRemaining,
                        U32 NumBytesInBuffer,
                        void ** pNextBuffer,
                        U32 * pNextBufferSize) {
    printf("Reading file 0x%8lx, reading chunk of %lu still have to "
           "read %lu bytes for the current file.\n",
           *(U32*)pUserContext,
           NumBytesInBuffer,
```

```
        NumBytesRemaining);  
    }  
  
<...>  
  
    Status = USBH_MTP_ReadFile(hDevice, // Handle to the device  
                               ObjectID, // Object ID to read  
                               &_ReceiveData, // Callback function  
                               &ObjectID, // User context passed to the callback  
                               _acReadBuf, // User buffer to use  
                               sizeof(_acReadBuf)); // User buffer size
```

9.2.20 USBH_MTP_GetDevicePropDesc()

Description

Retrieves the description of a MTP property from the device. The description includes the size of a property, which is highly important as the same properties can have different sizes on different devices.

Prototype

```
USBH_STATUS USBH_MTP_GetDevicePropDesc(USBH_MTP_DEVICE_HANDLE    hDevice,  
                                         U16                      DevicePropCode,  
                                         USBH_MTP_DEVICE_PROP_DESC * pDesc);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device.
DevicePropCode	Device property code, see USBH_MTP_DEVICE_PROPERTIES.
pDesc	Pointer to a USBH_MTP_DEVICE_PROP_DESC structure where the information should be saved.

Return value

= USBH_STATUS_SUCCESS	Successful.
≠ USBH_STATUS_SUCCESS	An error occurred.

9.2.21 USBH_MTP_GetDevicePropValue()

Description

Retrieves the value of a property of a specific Device. The property description has to be retrieved via USBH_MTP_GetDevicePropDesc prior to calling this function.

Prototype

```
USBH_STATUS USBH_MTP_GetDevicePropValue
(
    USBH_MTP_DEVICE_HANDLE    hDevice,
    const USBH_MTP_DEVICE_PROP_DESC * pDesc,
    void * pData,
    U32 BufferSize);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device.
pDesc	Pointer to a USBH_MTP_DEVICE_PROP_DESC structure which has the property size and code.
pData	Pointer to a buffer where the value should be stored.
BufferSize	Size of the buffer, if the value is longer than the size of the buffer the value will be truncated.

Return value

- = USBH_STATUS_SUCCESS
- Successful.
- ≠ USBH_STATUS_SUCCESS
- An error occurred.

9.2.22 USBH_MTP_GetObjectPropsSupported()

Description

Retrieves a list of supported properties for a given object format.

Prototype

```
USBH_STATUS USBH_MTP_GetObjectPropsSupported
(
    (USBH_MTP_DEVICE_HANDLE  hDevice,
     U32                      ObjectFormatCode,
     U16                      * pBuffer,
     U32                      * pNumProps);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device.
ObjectFormatCode	MTP Format Code ID for the MTP property which should be queried. (See USBH_MTP_OBJECT_FORMAT for a list of valid format codes).
pBuffer	out Pointer to an array of U16 values, this array will receive the list of property codes.
pNumProps	in/out The application should specify the size of the buffer in U16 values. The MTP module will read property codes up to the specified value. If there are less codes delivered by the device the number of codes read will be stored in this variable. If there are more codes delivered by device the surplus codes are discarded by the module.

Return value

= USBH_STATUS_SUCCESS Successful.
 ≠ USBH_STATUS_SUCCESS An error occurred.

Additional information

Unfortunately there is no way to ask the device how many properties a format has before requesting the list or to request a partial list, therefore the buffer should be big enough to contain all of them.

9.2.23 USBH_MTP_GetObjectPropDesc()

Description

Retrieves information about an MTP object property used by the device. This is especially important because the application needs to know the data type (the size) of the property before retrieving it.

Prototype

```
USBH_STATUS USBH_MTP_GetObjectPropDesc
(
    USBH_MTP_DEVICE_HANDLE hDevice,
    U16 ObjectPropCode,
    U32 ObjectFormatCode,
    USBH_MTP_OBJECT_PROP_DESC * pDesc);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device.
ObjectPropCode	Object property code, see USBH_MTP_OBJECT_PROPERTIES .
ObjectFormatCode	MTP Format Code ID for the MTP property which should be queried. (See USBH_MTP_OBJECT_FORMAT for a list of valid format codes).
pDesc	in Pointer to a USBH_MTP_OBJECT_PROP_DESC structure where the MTP property descriptor will be stored.

Return value

= [USBH_STATUS_SUCCESS](#) Successful.
 ≠ [USBH_STATUS_SUCCESS](#) An error occurred.

Example

```

/*****
 *
 * _DataTypeToBytes
 *
 * Function description
 * Returns the number of bytes required for the given data type.
 */
static unsigned _DataTypeToBytes(U16 DataType) {
    unsigned NumBytes;

    switch (DataType) {
        case USBH_MTP_DATA_TYPE_INT8:
        case USBH_MTP_DATA_TYPE_UINT8:
            NumBytes = 1;
            break;
        case USBH_MTP_DATA_TYPE_INT16:
        case USBH_MTP_DATA_TYPE_UINT16:
            NumBytes = 2;
            break;
        case USBH_MTP_DATA_TYPE_INT32:
        case USBH_MTP_DATA_TYPE_UINT32:
            NumBytes = 4;
            break;
        case USBH_MTP_DATA_TYPE_INT64:
        case USBH_MTP_DATA_TYPE_UINT64:
            NumBytes = 8;
            break;
        case USBH_MTP_DATA_TYPE_STR:
            NumBytes = 256;
    }
}
```

```

        break;
    case USBH_MTP_DATA_TYPE_UINT128:
        NumBytes = 16;
        break;
    default:
        NumBytes = 0; // Error, invalid data type.
        break;
    }
    return NumBytes;
}

USBH_MTP_OBJECT_PROP_DESC ObjPropDesc;
U8 * pPropertyBuffer;

//
// Check in which format the property
// USBH_MTP_OBJECT_PROP_STORAGE_ID is stored.
//
Status = USBH_MTP_GetObjectPropDesc(hDevice,
                                     USBH_MTP_OBJECT_PROP_STORAGE_ID,
                                     USBH_MTP_OBJECT_FORMAT_UNDEFINED,
                                     &ObjPropDesc);
if (Status == USBH_STATUS_SUCCESS) {
    //
    // Now that we know the format - memory can be allocated
    // and the property can be retrieved.
    //
    NumBytes = _DataTypeToBytes(ObjPropDesc.DataType);
    pPropertyBuffer = malloc(NumBytes);
    if (pPropertyBuffer) {
        Status = USBH_MTP_GetObjectPropValue(hDevice,
                                              CreateInfo.ObjectID,
                                              &ObjPropDesc,
                                              pPropertyBuffer,
                                              NumBytes);

        if (Status == USBH_STATUS_SUCCESS) {
            <...do something with the value...>
        } else {
            <...>
        }
        free(pPropertyBuffer);
    } else {
        <...>
    }
} else {
    <...>
}
}

```

9.2.24 USBH_MTP_GetObjectPropValue()

Description

Retrieves the value of a property of a specific object. The property description has to be retrieved via USBH_MTP_GetObjectPropDesc prior to calling this function.

Prototype

```
USBH_STATUS USBH_MTP_GetObjectPropValue
(
    USBH_MTP_DEVICE_HANDLE    hDevice,
    U32                        ObjectID,
    const USBH_MTP_OBJECT_PROP_DESC * pDesc,
    void                       * pData,
    U32                        BufferSize);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device.
ObjectID	Object ID to retrieve the property from.
pDesc	Pointer to a USBH_MTP_OBJECT_PROP_DESC structure which has the property size and code.
pData	Pointer to a buffer where the value should be stored.
BufferSize	Size of the buffer, if the value is longer than the size of the buffer the value will be truncated.

Return value

- = USBH_STATUS_SUCCESS Successful.
- ≠ USBH_STATUS_SUCCESS An error occurred.

Example

See USBH_MTP_GetObjectPropDesc().

9.2.25 USBH_MTP_SetObjectProperty()

Description

Sets the property of an object to the specified value.

Prototype

```
USBH_STATUS USBH_MTP_SetObjectProperty(    USBH_MTP_DEVICE_HANDLE    hDevice,
                                           U32                        ObjectID,
                                           const USBH_MTP_OBJECT_PROP_DESC * pDesc,
                                           const void * pData,
                                           U32                        NumBytes);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to the opened device.
<code>ObjectID</code>	Object ID to retrieve the property from.
<code>pDesc</code>	Pointer to a <code>USBH_MTP_OBJECT_PROP_DESC</code> structure which should be retrieved earlier via <code>USBH_MTP_GetObjectPropDesc()</code> .
<code>pData</code>	Pointer to a buffer where the new value is stored.
<code>NumBytes</code>	Size of the value inside the buffer in bytes.

Return value

= `USBH_STATUS_SUCCESS` Successful.
 ≠ `USBH_STATUS_SUCCESS` An error occurred.

Example

```
USBH_MTP_OBJECT_PROP_DESC ObjPropDesc;
const U16 _sNewDateCreated[] = L"20010101T011642.0-0700";

//
// Change "DateCreated"
//
ObjPropDesc.PropertyCode = USBH_MTP_OBJECT_PROP_DATE_CREATED;
ObjPropDesc.DataType = USBH_MTP_DATA_TYPE_STR;
Status = USBH_MTP_SetObjectProperty(hDevice,
                                    CreateInfo.ObjectID,
                                    &ObjPropDesc,
                                    (void *)_sNewDateCreated,
                                    sizeof(_sNewDateCreated));
```

9.2.26 USBH_MTP_CheckLock()

Description

Determines whether the device is locked by a pin/password/etc.

Prototype

```
USBH_STATUS USBH_MTP_CheckLock(USBH_MTP_DEVICE_HANDLE hDevice);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to the opened device.

Return value

USBH_STATUS_SUCCESS	The device is not locked.
USBH_STATUS_ERROR	The device is locked.
USBH_STATUS_BUSY	The endpoint is busy with a different operation.

Any other value means the device is locked and reports the specific error through which it was determined.

Additional information

Some devices (mainly Windows Phone) may re-enumerate when they are unlocked by the user, which will cause this function to correctly report `USBH_STATUS_DEVICE_REMOVED`. The application should open a new handle to the device and call this function again.

On some devices (mainly Android) the storage count of the device (the value which you get from `USBH_MTP_GetNumStorages()`) will be updated automatically when this function is called and the user has unlocked the device (normally from zero to the real value).

9.2.27 USBH_MTP_SetEventCallback()

Description

Sets a callback for MTP events, e.g. StoreAdded, ObjectAdded, etc.

Prototype

```
USBH_STATUS USBH_MTP_SetEventCallback(USBH_MTP_DEVICE_HANDLE hDevice,  
                                       USBH_EVENT_CALLBACK * cbOnUserEvent);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device.
cbOnUserEvent	Pointer to a user provided function of type USBH_EVENT_CALLBACK.

Return value

= USBH_STATUS_SUCCESS Successful.
≠ USBH_STATUS_SUCCESS An error occurred.

Additional information

The callback should not block. See the description of USBH_EVENT_CALLBACK for additional information.

9.2.28 USBH_MTP_ConfigEventSupport()

Description

Turns MTP event support on or off. Should be called after USBH_MTP_Init(). When turning MTP support on or off only newly connected devices will be affected. Event support is off by default.

Prototype

```
void USBH_MTP_ConfigEventSupport(U8 OnOff);
```

Parameters

Parameter	Description
OnOff	Turn events on or off.

Additional information

Calling this function will not affect devices which are already open. To make sure open devices are affected you have to close them (USBH_MTP_Close()) and open them again (USBH_MTP_Open()). Some MTP devices do not behave as they should when re-opening a session and refuse to communicate, in such a case it is advised to re-enumerate them.

9.2.29 USBH_MTP_GetEventSupport()

Description

Returns the event support configuration, see `USBH_MTP_ConfigEventSupport()` for details.

Prototype

```
U8 USBH_MTP_GetEventSupport(void);
```

Return value

- 1 Event support enabled.
- 0 Event support disabled.

9.3 Data structures

This chapter describes the emUSB-Host HID API structures.

Structure	Description
USBH_MTP_DEVICE_INFO	Contains information about an MTP compatible device.
USBH_MTP_STORAGE_INFO	Contains information about an MTP storage.
USBH_MTP_OBJECT	Contains basic information about an MTP object.
USBH_MTP_OBJECT_INFO	Contains extended information about an MTP object.
USBH_MTP_CREATE_INFO	Contains information needed to create a new MTP object.
USBH_MTP_OBJECT_PROP_DESC	Contains information about the data-type and accessibility of an object property.

9.3.1 USBH_MTP_DEVICE_INFO

Description

Contains information about an MTP compatible device.

Type definition

```
typedef struct {  
    U16      VendorId;  
    U16      ProductId;  
    U8       acSerialNo[];  
    USBH_SPEED Speed;  
    const U16 * sManufacturer;  
    const U16 * sModel;  
    const U16 * sDeviceVersion;  
    const U16 * sSerialNumber;  
} USBH_MTP_DEVICE_INFO;
```

Structure members

Member	Description
VendorId	Vendor identification number.
ProductId	Product identification number.
acSerialNo	Serial number string.
Speed	The USB speed of the device, see USBH_SPEED.
sManufacturer	Pointer to a Unicode string "manufacturer".
sModel	Pointer to a Unicode string "model".
sDeviceVersion	Pointer to a Unicode string "device version".
sSerialNumber	Pointer to a Unicode string "serial number".

9.3.2 USBH_MTP_STORAGE_INFO

Description

Contains information about an MTP storage.

Type definition

```
typedef struct {
    U16  StorageType;
    U16  FileSystemType;
    U16  AccessCapability;
    U8   MaxCapacity[];
    U8   FreeSpaceInBytes[];
    U32  FreeSpaceInImages;
    U16  sStorageDescription[];
    U16  sVolumeLabel[];
} USBH_MTP_STORAGE_INFO;
```

Structure members

Member	Description
StorageType	0x0000 Undefined 0x0001 Fixed ROM 0x0002 Removable ROM 0x0003 Fixed RAM 0x0004 Removable RAM Note: This value is often unreliable as many devices return 0x0003 for everything.
FileSystemType	0x0000 Undefined 0x0001 Generic flat 0x0002 Generic hierarchical 0x0003 DCF
AccessCapability	0x0000 Read-write 0x0001 Read-only without object deletion 0x0002 Read-only with object deletion
MaxCapacity	An U64 little-endian value which designates the maximum capacity of the storage in bytes. The value is declared as an array of U8, you will have to cast it into a U64 in your application. If the storage is read-only, this field is optional and may contain zero.
FreeSpaceInBytes	An U64 little-endian value which designates the free space on the storage in bytes. The value is declared as an array of U8, you will have to cast it into a U64 in your application. If the storage is read-only, this field is optional and may contain zero.
FreeSpaceInImages	Value describing the number of images which could still be fit into the storage. This is a PTP relevant value, for MTP it is normally zero or 0xFFFFFFFF.
sStorageDescription	Unicode string describing the storage.
sVolumeLabel	Unicode string which contains the volume label.

9.3.3 USBH_MTP_OBJECT

Description

Contains basic information about an MTP object.

Type definition

```
typedef struct {  
    U32  ObjectID;  
    U16  ObjectFormat;  
    U16  AssociationType;  
} USBH_MTP_OBJECT;
```

Structure members

Member	Description
ObjectID	Unique ID for the object, provided by the device.
ObjectFormat	MTP Object format, see USBH_MTP_OBJECT_FORMAT
AssociationType	MTP association type, see USBH_MTP_ASSOCIATION_TYPES

9.3.4 USBH_MTP_OBJECT_INFO

Description

Contains extended information about an MTP object.

Type definition

```
typedef struct {
    U32  StorageID;
    U16  ObjectFormat;
    U16  ProtectionStatus;
    U32  ParentObject;
    U16  AssociationType;
    U16  sFilename[];
    U16  sCaptureDate[];
    U16  sModificationDate[];
} USBH_MTP_OBJECT_INFO;
```

Structure members

Member	Description
StorageID	ID of the storage where the Object is located.
ObjectFormat	MTP Object format, see USBH_MTP_OBJECT_FORMAT.
ProtectionStatus	0x0000 - No protection 0x0001 - Read-only 0x8002 - Read-only data 0x8003 - Non-transferable data
ParentObject	ObjectID of the parent Object. For “root” use the define USBH_MTP_ROOT_OBJECT_ID.
AssociationType	MTP association type, see USBH_MTP_ASSOCIATION_TYPES
sFilename	File name buffer.
sCaptureDate	CaptureDate string buffer.
sModificationDate	ModificationDate string buffer.

9.3.5 USBH_MTP_CREATE_INFO

Description

Contains information needed to create a new MTP object.

Type definition

```
typedef struct {
    U32          ObjectID;
    U32          ParentObjectID;
    U32          ObjectSize;
    USBH_SEND_DATA_FUNC * pfGetData;
    U32          FileNameSize;
    const U16    * sFileName;
    U8           isFolder;
    U8           * pUserBuf;
    U32          UserBufSize;
    void         * pUserContext;
} USBH_MTP_CREATE_INFO;
```

Structure members

Member	Description
ObjectID	Filled by the MTP Module after the Object has been created. This ID is provided by the device.
ParentObjectID	The ObjectID of the parent object. For "root" use <code>USBH_MTP_ROOT_OBJECT_ID</code> .
ObjectSize	Size of the file in bytes.
pfGetData	Pointer to a user-provided callback which will provide the data. See <code>USBH_SEND_DATA_FUNC</code> .
FileNameSize	The length of the file-name string (including termination).
sFileName	Pointer to the Unicode file-name string (must include zero-termination).
isFolder	Flag indicating whether the new object should be a folder or not. When creating a folder pfGetData , pUserBuf , UserBufSize can be set to NULL.
pUserBuf	Pointer to a user buffer where the data is located. Can be NULL, in this case the callback is called immediately and the buffer has to be set inside the callback.
UserBufSize	Size of the user buffer in bytes.
pUserContext	User context which is passed to the callback.

9.3.6 USBH_MTP_OBJECT_PROP_DESC

Description

Contains information about the data-type and accessibility of an object property.

Type definition

```
typedef struct {
    U16  PropertyCode;
    U16  DataType;
    U8   GetSet;
} USBH_MTP_OBJECT_PROP_DESC;
```

Structure members

Member	Description
PropertyCode	MTP object property code, see USBH_MTP_OBJECT_PROPERTIES.
DataType	Data type of the property.
GetSet	0 - Read-only, 1 - Read-write.

Additional information

Type	Code	Size in bytes
USBH_MTP_DATA_TYPE_INT8	0x0001	1
USBH_MTP_DATA_TYPE_UINT8	0x0002	1
USBH_MTP_DATA_TYPE_INT16	0x0003	2
USBH_MTP_DATA_TYPE_UINT16	0x0004	2
USBH_MTP_DATA_TYPE_INT32	0x0005	4
USBH_MTP_DATA_TYPE_UINT32	0x0006	4
USBH_MTP_DATA_TYPE_INT64	0x0007	8
USBH_MTP_DATA_TYPE_UINT64	0x0008	8
USBH_MTP_DATA_TYPE_UINT128	0x000A	16
USBH_MTP_DATA_TYPE_AUINT8	0x4002	Variable size.
USBH_MTP_DATA_TYPE_STR	0xFFFF	Variable size.

9.4 Function Types

This chapter describes the emUSB-Host MTP API function types.

Type	Description
USBH_SEND_DATA_FUNC	Definition of the callback which has to be specified when using <code>USBH_MTP_CreateObject()</code> .
USBH_RECEIVE_DATA_FUNC	Definition of the callback which has to be specified when using <code>USBH_MTP_ReadFile()</code> .
USBH_EVENT_CALLBACK	Definition of the callback which can be set via <code>USBH_MTP_SetEventCallback()</code> .

9.4.1 USBH_SEND_DATA_FUNC

Description

Definition of the callback which has to be specified when using `USBH_MTP_CreateObject()`.

Type definition

```
typedef void USBH_SEND_DATA_FUNC(void * pUserContext,  
                                U32   NumBytesSentTotal,  
                                U32   * pNumBytesToSend,  
                                U8   * * ppNextBuffer);
```

Parameters

Parameter	Description
<code>pUserContext</code>	User context which is passed to the callback.
<code>NumBytesSentTotal</code>	This value contains the total number of bytes which have already been transferred
<code>pNumBytesToSend</code>	The user has to set this value to the number of bytes which are inside the buffer.
<code>ppNextBuffer</code>	The user can change this pointer to a different buffer. If this parameter remains NULL after the callback returns, the previous buffer is re-used (the application should put new data into the buffer first).

9.4.2 USBH_RECEIVE_DATA_FUNC

Description

Definition of the callback which has to be specified when using USBH_MTP_ReadFile().

Type definition

```
typedef void USBH_RECEIVE_DATA_FUNC(void * pUserContext,  
                                     U32  NumBytesRemaining,  
                                     U32  NumBytesInBuffer,  
                                     void ** ppNextBuffer,  
                                     U32  * pNextBufferSize);
```

Parameters

Parameter	Description
<code>pUserContext</code>	User context which is passed to the callback.
<code>NumBytesRemaining</code>	This value contains the total number of bytes which still have to be read.
<code>NumBytesInBuffer</code>	The number of bytes which have been read in this transaction.
<code>ppNextBuffer</code>	The user can change this pointer to a different buffer. If this parameter remains NULL after the callback returns, the previous buffer is re-used (the application should copy the data out of the buffer first, as it will be overwritten on the next transaction).
<code>pNextBufferSize</code>	Size of the next buffer. This only needs to be changed when the <code>pNextBuffer</code> parameter is changed.

9.4.3 USBH_EVENT_CALLBACK

Description

Definition of the callback which can be set via `USBH_MTP_SetEventCallback()`.

Type definition

```
typedef void USBH_EVENT_CALLBACK(U16 EventCode,  
                                U32 Para1,  
                                U32 Para2,  
                                U32 Para3);
```

Parameters

Parameter	Description
EventCode	Code of the MTP event, see <code>USBH_MTP_EVENT_CODES</code> .
Para1	First parameter passed with the event.
Para2	Second parameter passed with the event.
Para3	Third parameter passed with the event.

Additional information

The events `USBH_MTP_EVENT_STORE_ADDED` and `USBH_MTP_EVENT_STORE_REMOVED` are handled by the MTP module before being passed to the callback. The storage information for the device is updated automatically when one of these events is received. All events are passed to the callback, this includes vendor specific events which are not present in the `USBH_MTP_EVENT_CODES` enum. Parameters which are not used with a specific event (e.g. `USBH_MTP_EVENT_STORE_ADDED` has only one parameter) will be passed as zero. The callback should not block.

9.5 Enums

This chapter describes the emUSB-Host MTP API enums.

Enum	Description
USBH_MTP_DEVICE_PROPERTIES	Device properties describe conditions or setting relevant to the device itself.
USBH_MTP_OBJECT_PROPERTIES	Object properties identify settings or state conditions of files and folders (objects).
USBH_MTP_RESPONSE_CODES	Possible response codes reported by the device upon completion of an operation.
USBH_MTP_OBJECT_FORMAT	Identifiers describing the format type of a given object.
USBH_MTP_EVENT_CODES	Events are described by a 16-bit code.

9.5.1 USBH_MTP_DEVICE_PROPERTIES

Description

Device properties describe conditions or setting relevant to the device itself. The properties are unrelated to objects.

Type definition

```
typedef enum {
    USBH_MTP_DEVICE_PROP_UNDEFINED,
    USBH_MTP_DEVICE_PROP_BATTERY_LEVEL,
    USBH_MTP_DEVICE_PROP_FUNCTIONAL_MODE,
    USBH_MTP_DEVICE_PROP_IMAGE_SIZE,
    USBH_MTP_DEVICE_PROP_COMPRESSION_SETTING,
    USBH_MTP_DEVICE_PROP_WHITE_BALANCE,
    USBH_MTP_DEVICE_PROP_RGB_GAIN,
    USBH_MTP_DEVICE_PROP_F_NUMBER,
    USBH_MTP_DEVICE_PROP_FOCAL_LENGTH,
    USBH_MTP_DEVICE_PROP_FOCUS_DISTANCE,
    USBH_MTP_DEVICE_PROP_FOCUS_MODE,
    USBH_MTP_DEVICE_PROP_EXPOSURE_METERING_MODE,
    USBH_MTP_DEVICE_PROP_FLASH_MODE,
    USBH_MTP_DEVICE_PROP_EXPOSURE_TIME,
    USBH_MTP_DEVICE_PROP_EXPOSURE_PROGRAM_MODE,
    USBH_MTP_DEVICE_PROP_EXPOSURE_INDEX,
    USBH_MTP_DEVICE_PROP_EXPOSURE_BIAS_COMPENSATION,
    USBH_MTP_DEVICE_PROP_DATETIME,
    USBH_MTP_DEVICE_PROP_CAPTURE_DELAY,
    USBH_MTP_DEVICE_PROP_STILL_CAPTURE_MODE,
    USBH_MTP_DEVICE_PROP_CONTRAST,
    USBH_MTP_DEVICE_PROP_SHARPNESS,
    USBH_MTP_DEVICE_PROP_DIGITAL_ZOOM,
    USBH_MTP_DEVICE_PROP_EFFECT_MODE,
    USBH_MTP_DEVICE_PROP_BURST_NUMBER,
    USBH_MTP_DEVICE_PROP_BURST_INTERVAL,
    USBH_MTP_DEVICE_PROP_TIMELAPSE_NUMBER,
    USBH_MTP_DEVICE_PROP_TIMELAPSE_INTERVAL,
    USBH_MTP_DEVICE_PROP_FOCUS_METERING_MODE,
    USBH_MTP_DEVICE_PROP_UPLOAD_URL,
    USBH_MTP_DEVICE_PROP_ARTIST,
    USBH_MTP_DEVICE_PROP_COPYRIGHT_INFO,
    USBH_MTP_DEVICE_PROP_SYNCHRONIZATION_PARTNER,
    USBH_MTP_DEVICE_PROP_DEVICE_FRIENDLY_NAME,
    USBH_MTP_DEVICE_PROP_VOLUME,
    USBH_MTP_DEVICE_PROP_SUPPORTEDFORMATSORDERED,
    USBH_MTP_DEVICE_PROP_DEVICEICON,
    USBH_MTP_DEVICE_PROP_PLAYBACK_RATE,
    USBH_MTP_DEVICE_PROP_PLAYBACK_OBJECT,
    USBH_MTP_DEVICE_PROP_PLAYBACK_CONTAINER,
    USBH_MTP_DEVICE_PROP_SESSION_INITIATOR_VERSION_INFO,
    USBH_MTP_DEVICE_PROP_PERCEIVED_DEVICE_TYPE
} USBH_MTP_DEVICE_PROPERTIES;
```

9.5.2 USBH_MTP_OBJECT_PROPERTIES

Description

Object properties identify settings or state conditions of files and folders (objects).

Type definition

```
typedef enum {
    USBH_MTP_OBJECT_PROP_STORAGE_ID,
    USBH_MTP_OBJECT_PROP_OBJECT_FORMAT,
    USBH_MTP_OBJECT_PROP_PROTECTION_STATUS,
    USBH_MTP_OBJECT_PROP_OBJECT_SIZE,
    USBH_MTP_OBJECT_PROP_ASSOCIATION_TYPE,
    USBH_MTP_OBJECT_PROP_ASSOCIATION_DESC,
    USBH_MTP_OBJECT_PROP_OBJECT_FILE_NAME,
    USBH_MTP_OBJECT_PROP_DATE_CREATED,
    USBH_MTP_OBJECT_PROP_DATE_MODIFIED,
    USBH_MTP_OBJECT_PROP_KEYWORDS,
    USBH_MTP_OBJECT_PROP_PARENT_OBJECT,
    USBH_MTP_OBJECT_PROP_ALLOWED_FOLDER_CONTENTS,
    USBH_MTP_OBJECT_PROP_HIDDEN,
    USBH_MTP_OBJECT_PROP_SYSTEM_OBJECT,
    USBH_MTP_OBJECT_PROP_PERSISTENT_UNIQUE_OBJECT_IDENTIFIER,
    USBH_MTP_OBJECT_PROP_SYNCID,
    USBH_MTP_OBJECT_PROP_PROPERTY_BAG,
    USBH_MTP_OBJECT_PROP_NAME,
    USBH_MTP_OBJECT_PROP_CREATED_BY,
    USBH_MTP_OBJECT_PROP_ARTIST,
    USBH_MTP_OBJECT_PROP_DATE_AUTHORED,
    USBH_MTP_OBJECT_PROP_DESCRIPTION,
    USBH_MTP_OBJECT_PROP_URL_REFERENCE,
    USBH_MTP_OBJECT_PROP_LANGUAGELOCALE,
    USBH_MTP_OBJECT_PROP_COPYRIGHT_INFORMATION,
    USBH_MTP_OBJECT_PROP_SOURCE,
    USBH_MTP_OBJECT_PROP_ORIGIN_LOCATION,
    USBH_MTP_OBJECT_PROP_DATE_ADDED,
    USBH_MTP_OBJECT_PROP_NON_CONSUMABLE,
    USBH_MTP_OBJECT_PROP_CORRUPTUNPLAYABLE,
    USBH_MTP_OBJECT_PROP_PRODUCERSERIALNUMBER,
    USBH_MTP_OBJECT_PROP_REPRESENTATIVE_SAMPLE_FORMAT,
    USBH_MTP_OBJECT_PROP_REPRESENTATIVE_SAMPLE_SIZE,
    USBH_MTP_OBJECT_PROP_REPRESENTATIVE_SAMPLE_HEIGHT,
    USBH_MTP_OBJECT_PROP_REPRESENTATIVE_SAMPLE_WIDTH,
    USBH_MTP_OBJECT_PROP_REPRESENTATIVE_SAMPLE_DURATION,
    USBH_MTP_OBJECT_PROP_REPRESENTATIVE_SAMPLE_DATA,
    USBH_MTP_OBJECT_PROP_WIDTH,
    USBH_MTP_OBJECT_PROP_HEIGHT,
    USBH_MTP_OBJECT_PROP_DURATION,
    USBH_MTP_OBJECT_PROP_RATING,
    USBH_MTP_OBJECT_PROP_TRACK,
    USBH_MTP_OBJECT_PROP_GENRE,
    USBH_MTP_OBJECT_PROP_CREDITS,
    USBH_MTP_OBJECT_PROP_LYRICS,
    USBH_MTP_OBJECT_PROP_SUBSCRIPTION_CONTENT_ID,
    USBH_MTP_OBJECT_PROP_PRODUCED_BY,
    USBH_MTP_OBJECT_PROP_USE_COUNT,
    USBH_MTP_OBJECT_PROP_SKIP_COUNT,
    USBH_MTP_OBJECT_PROP_LAST_ACCESSED,
    USBH_MTP_OBJECT_PROP_PARENTAL_RATING,
    USBH_MTP_OBJECT_PROP_META_GENRE,
    USBH_MTP_OBJECT_PROP_COMPOSER,
    USBH_MTP_OBJECT_PROP_EFFECTIVE_RATING,
    USBH_MTP_OBJECT_PROP_SUBTITLE,
    USBH_MTP_OBJECT_PROP_ORIGINAL_RELEASE_DATE,
    USBH_MTP_OBJECT_PROP_ALBUM_NAME,
    USBH_MTP_OBJECT_PROP_ALBUM_ARTIST,
```

```
USBH_MTP_OBJECT_PROP_MOOD,  
USBH_MTP_OBJECT_PROP_DRM_STATUS,  
USBH_MTP_OBJECT_PROP_SUB_DESCRIPTION,  
USBH_MTP_OBJECT_PROP_IS_CROPPED,  
USBH_MTP_OBJECT_PROP_IS_COLOUR_CORRECTED,  
USBH_MTP_OBJECT_PROP_IMAGE_BIT_DEPTH,  
USBH_MTP_OBJECT_PROP_FNUMBER,  
USBH_MTP_OBJECT_PROP_EXPOSURE_TIME,  
USBH_MTP_OBJECT_PROP_EXPOSURE_INDEX,  
USBH_MTP_OBJECT_PROP_TOTAL_BITRATE,  
USBH_MTP_OBJECT_PROP_BITRATE_TYPE,  
USBH_MTP_OBJECT_PROP_SAMPLE_RATE,  
USBH_MTP_OBJECT_PROP_NUMBER_OF_CHANNELS,  
USBH_MTP_OBJECT_PROP_AUDIO_BITDEPTH,  
USBH_MTP_OBJECT_PROP_SCAN_TYPE,  
USBH_MTP_OBJECT_PROP_AUDIO_WAVE_CODEC,  
USBH_MTP_OBJECT_PROP_AUDIO_BITRATE,  
USBH_MTP_OBJECT_PROP_VIDEO_FOURCC_CODEC,  
USBH_MTP_OBJECT_PROP_VIDEO_BITRATE,  
USBH_MTP_OBJECT_PROP_FRAMES_PER_THOUSAND_SECONDS,  
USBH_MTP_OBJECT_PROP_KEYFRAME_DISTANCE,  
USBH_MTP_OBJECT_PROP_BUFFER_SIZE,  
USBH_MTP_OBJECT_PROP_ENCODING_QUALITY,  
USBH_MTP_OBJECT_PROP_ENCODING_PROFILE,  
USBH_MTP_OBJECT_PROP_DISPLAY_NAME,  
USBH_MTP_OBJECT_PROP_BODY_TEXT,  
USBH_MTP_OBJECT_PROP_SUBJECT,  
USBH_MTP_OBJECT_PROP_PRIORITY,  
USBH_MTP_OBJECT_PROP_GIVEN_NAME,  
USBH_MTP_OBJECT_PROP_MIDDLE_NAMES,  
USBH_MTP_OBJECT_PROP_FAMILY_NAME,  
USBH_MTP_OBJECT_PROP_PREFIX,  
USBH_MTP_OBJECT_PROP_SUFFIX,  
USBH_MTP_OBJECT_PROP_PHONETIC_GIVEN_NAME,  
USBH_MTP_OBJECT_PROP_PHONETIC_FAMILY_NAME,  
USBH_MTP_OBJECT_PROP_EMAIL_PRIMARY,  
USBH_MTP_OBJECT_PROP_EMAIL_PERSONAL_1,  
USBH_MTP_OBJECT_PROP_EMAIL_PERSONAL_2,  
USBH_MTP_OBJECT_PROP_EMAIL_BUSINESS_1,  
USBH_MTP_OBJECT_PROP_EMAIL_BUSINESS_2,  
USBH_MTP_OBJECT_PROP_EMAIL_OTHERS,  
USBH_MTP_OBJECT_PROP_PHONE_NUMBER_PRIMARY,  
USBH_MTP_OBJECT_PROP_PHONE_NUMBER_PERSONAL,  
USBH_MTP_OBJECT_PROP_PHONE_NUMBER_PERSONAL_2,  
USBH_MTP_OBJECT_PROP_PHONE_NUMBER_BUSINESS,  
USBH_MTP_OBJECT_PROP_PHONE_NUMBER_BUSINESS_2,  
USBH_MTP_OBJECT_PROP_PHONE_NUMBER_MOBILE,  
USBH_MTP_OBJECT_PROP_PHONE_NUMBER_MOBILE_2,  
USBH_MTP_OBJECT_PROP_FAX_NUMBER_PRIMARY,  
USBH_MTP_OBJECT_PROP_FAX_NUMBER_PERSONAL,  
USBH_MTP_OBJECT_PROP_FAX_NUMBER_BUSINESS,  
USBH_MTP_OBJECT_PROP_PAGER_NUMBER,  
USBH_MTP_OBJECT_PROP_PHONE_NUMBER_OTHERS,  
USBH_MTP_OBJECT_PROP_PRIMARY_WEB_ADDRESS,  
USBH_MTP_OBJECT_PROP_PERSONAL_WEB_ADDRESS,  
USBH_MTP_OBJECT_PROP_BUSINESS_WEB_ADDRESS,  
USBH_MTP_OBJECT_PROP_INSTANT_MESSENGER_ADDRESS,  
USBH_MTP_OBJECT_PROP_INSTANT_MESSENGER_ADDRESS_2,  
USBH_MTP_OBJECT_PROP_INSTANT_MESSENGER_ADDRESS_3,  
USBH_MTP_OBJECT_PROP_POSTAL_ADDRESS_PERSONAL_FULL,  
USBH_MTP_OBJECT_PROP_POSTAL_ADDRESS_PERSONAL_LINE_1,  
USBH_MTP_OBJECT_PROP_POSTAL_ADDRESS_PERSONAL_LINE_2,  
USBH_MTP_OBJECT_PROP_POSTAL_ADDRESS_PERSONAL_CITY,  
USBH_MTP_OBJECT_PROP_POSTAL_ADDRESS_PERSONAL_REGION,  
USBH_MTP_OBJECT_PROP_POSTAL_ADDRESS_PERSONAL_POSTAL_CODE,  
USBH_MTP_OBJECT_PROP_POSTAL_ADDRESS_PERSONAL_COUNTRY,  
USBH_MTP_OBJECT_PROP_POSTAL_ADDRESS_BUSINESS_FULL,  
USBH_MTP_OBJECT_PROP_POSTAL_ADDRESS_BUSINESS_LINE_1,
```

```
USBH_MTP_OBJECT_PROP_POSTAL_ADDRESS_BUSINESS_LINE_2,  
USBH_MTP_OBJECT_PROP_POSTAL_ADDRESS_BUSINESS_CITY,  
USBH_MTP_OBJECT_PROP_POSTAL_ADDRESS_BUSINESS_REGION,  
USBH_MTP_OBJECT_PROP_POSTAL_ADDRESS_BUSINESS_POSTAL_CODE,  
USBH_MTP_OBJECT_PROP_POSTAL_ADDRESS_BUSINESS_COUNTRY,  
USBH_MTP_OBJECT_PROP_POSTAL_ADDRESS_OTHER_FULL,  
USBH_MTP_OBJECT_PROP_POSTAL_ADDRESS_OTHER_LINE_1,  
USBH_MTP_OBJECT_PROP_POSTAL_ADDRESS_OTHER_LINE_2,  
USBH_MTP_OBJECT_PROP_POSTAL_ADDRESS_OTHER_CITY,  
USBH_MTP_OBJECT_PROP_POSTAL_ADDRESS_OTHER_REGION,  
USBH_MTP_OBJECT_PROP_POSTAL_ADDRESS_OTHER_POSTAL_CODE,  
USBH_MTP_OBJECT_PROP_POSTAL_ADDRESS_OTHER_COUNTRY,  
USBH_MTP_OBJECT_PROP_ORGANIZATION_NAME,  
USBH_MTP_OBJECT_PROP_PHONETIC_ORGANIZATION_NAME,  
USBH_MTP_OBJECT_PROP_ROLE,  
USBH_MTP_OBJECT_PROP_BIRTHDATE,  
USBH_MTP_OBJECT_PROP_MESSAGE_TO,  
USBH_MTP_OBJECT_PROP_MESSAGE_CC,  
USBH_MTP_OBJECT_PROP_MESSAGE_BCC,  
USBH_MTP_OBJECT_PROP_MESSAGE_READ,  
USBH_MTP_OBJECT_PROP_MESSAGE_RECEIVED_TIME,  
USBH_MTP_OBJECT_PROP_MESSAGE_SENDER,  
USBH_MTP_OBJECT_PROP_ACTIVITY_BEGIN_TIME,  
USBH_MTP_OBJECT_PROP_ACTIVITY_END_TIME,  
USBH_MTP_OBJECT_PROP_ACTIVITY_LOCATION,  
USBH_MTP_OBJECT_PROP_ACTIVITY_REQUIRED_ATTENDEES,  
USBH_MTP_OBJECT_PROP_ACTIVITY_OPTIONAL_ATTENDEES,  
USBH_MTP_OBJECT_PROP_ACTIVITY_RESOURCES,  
USBH_MTP_OBJECT_PROP_ACTIVITY_ACCEPTED,  
USBH_MTP_OBJECT_PROP_OWNER,  
USBH_MTP_OBJECT_PROP_EDITOR,  
USBH_MTP_OBJECT_PROP_WEBMASTER,  
USBH_MTP_OBJECT_PROP_URL_SOURCE,  
USBH_MTP_OBJECT_PROP_URL_DESTINATION,  
USBH_MTP_OBJECT_PROP_TIME_BOOKMARK,  
USBH_MTP_OBJECT_PROP_OBJECT_BOOKMARK,  
USBH_MTP_OBJECT_PROP_BYTE_BOOKMARK,  
USBH_MTP_OBJECT_PROP_LAST_BUILD_DATE,  
USBH_MTP_OBJECT_PROP_TIME_TO_LIVE,  
USBH_MTP_OBJECT_PROP_MEDIA_GUID  
} USBH_MTP_OBJECT_PROPERTIES;
```


9.5.3 USBH_MTP_RESPONSE_CODES

Description

Possible response codes reported by the device upon completion of an operation.

Type definition

```
typedef enum {  
    USBH_MTP_RESPONSE_UNDEFINED,  
    USBH_MTP_RESPONSE_OK,  
    USBH_MTP_RESPONSE_GENERAL_ERROR,  
    USBH_MTP_RESPONSE_PARAMETER_NOT_SUPPORTED,  
    USBH_MTP_RESPONSE_INVALID_STORAGE_ID,  
    USBH_MTP_RESPONSE_INVALID_OBJECT_HANDLE,  
    USBH_MTP_RESPONSE_DEVICEPROP_NOT_SUPPORTED,  
    USBH_MTP_RESPONSE_STORE_FULL,  
    USBH_MTP_RESPONSE_STORE_NOT_AVAILABLE,  
    USBH_MTP_RESPONSE_SPECIFICATION_BY_FORMAT_NOT_SUPPORTED,  
    USBH_MTP_RESPONSE_NO_VALID_OBJECT_INFO,  
    USBH_MTP_RESPONSE_DEVICE_BUSY,  
    USBH_MTP_RESPONSE_INVALID_PARENT_OBJECT,  
    USBH_MTP_RESPONSE_INVALID_PARAMETER,  
    USBH_MTP_RESPONSE_SESSION_ALREADY_OPEN,  
    USBH_MTP_RESPONSE_TRANSACTION_CANCELLED,  
    USBH_MTP_RESPONSE_INVALID_OBJECT_PROP_CODE,  
    USBH_MTP_RESPONSE_SPECIFICATION_BY_GROUP_UNSUPPORTED,  
    USBH_MTP_RESPONSE_OBJECT_PROP_NOT_SUPPORTED  
} USBH_MTP_RESPONSE_CODES;
```

9.5.4 USBH_MTP_OBJECT_FORMAT

Description

Identifiers describing the format type of a given object.

Type definition

```
typedef enum {
    USBH_MTP_OBJECT_FORMAT_UNDEFINED,
    USBH_MTP_OBJECT_FORMAT_ASSOCIATION,
    USBH_MTP_OBJECT_FORMAT_SCRIPT,
    USBH_MTP_OBJECT_FORMAT_EXECUTABLE,
    USBH_MTP_OBJECT_FORMAT_TEXT,
    USBH_MTP_OBJECT_FORMAT_HTML,
    USBH_MTP_OBJECT_FORMAT_DPOF,
    USBH_MTP_OBJECT_FORMAT_AIFF,
    USBH_MTP_OBJECT_FORMAT_WAV,
    USBH_MTP_OBJECT_FORMAT_MP3,
    USBH_MTP_OBJECT_FORMAT_AVI,
    USBH_MTP_OBJECT_FORMAT_MPEG,
    USBH_MTP_OBJECT_FORMAT_ASF,
    USBH_MTP_OBJECT_FORMAT_DEFINED,
    USBH_MTP_OBJECT_FORMAT_EXIF_JPEG,
    USBH_MTP_OBJECT_FORMAT_TIFF_EP,
    USBH_MTP_OBJECT_FORMAT_FLASHPIX,
    USBH_MTP_OBJECT_FORMAT_BMP,
    USBH_MTP_OBJECT_FORMAT_CIFF,
    USBH_MTP_OBJECT_FORMAT_UNDEFINED2,
    USBH_MTP_OBJECT_FORMAT_GIF,
    USBH_MTP_OBJECT_FORMAT_JFIF,
    USBH_MTP_OBJECT_FORMAT_CD,
    USBH_MTP_OBJECT_FORMAT_PICT,
    USBH_MTP_OBJECT_FORMAT_PNG,
    USBH_MTP_OBJECT_FORMAT_UNDEFINED3,
    USBH_MTP_OBJECT_FORMAT_TIFF,
    USBH_MTP_OBJECT_FORMAT_TIFF_IT,
    USBH_MTP_OBJECT_FORMAT_JP2,
    USBH_MTP_OBJECT_FORMAT_JPX,
    USBH_MTP_OBJECT_FORMAT_UNDEFINED_FIRMWARE,
    USBH_MTP_OBJECT_FORMAT_WINDOWS_IMAGE_FORMAT,
    USBH_MTP_OBJECT_FORMAT_UNDEFINED_AUDIO,
    USBH_MTP_OBJECT_FORMAT_WMA,
    USBH_MTP_OBJECT_FORMAT_OGG,
    USBH_MTP_OBJECT_FORMAT_AAC,
    USBH_MTP_OBJECT_FORMAT_AUDIBLE,
    USBH_MTP_OBJECT_FORMAT_FLAC,
    USBH_MTP_OBJECT_FORMAT_UNDEFINED_VIDEO,
    USBH_MTP_OBJECT_FORMAT_WMV,
    USBH_MTP_OBJECT_FORMAT_MP4_CONTAINER,
    USBH_MTP_OBJECT_FORMAT_MP2,
    USBH_MTP_OBJECT_FORMAT_3GP_CONTAINER,
    USBH_MTP_OBJECT_FORMAT_ABSTRACT_MULTIMEDIA_ALBUM,
    USBH_MTP_OBJECT_FORMAT_ABSTRACT_IMAGE_ALBUM,
    USBH_MTP_OBJECT_FORMAT_ABSTRACT_AUDIO_ALBUM,
    USBH_MTP_OBJECT_FORMAT_ABSTRACT_VIDEO_ALBUM,
    USBH_MTP_OBJECT_FORMAT_ABSTRACT_AUDIO_VIDEO_PLAYLIST,
    USBH_MTP_OBJECT_FORMAT_ABSTRACT_CONTACT_GROUP,
    USBH_MTP_OBJECT_FORMAT_ABSTRACT_MESSAGE_FOLDER,
    USBH_MTP_OBJECT_FORMAT_ABSTRACT_CHAPTERED_PRODUCTION,
    USBH_MTP_OBJECT_FORMAT_ABSTRACT_AUDIO_PLAYLIST,
    USBH_MTP_OBJECT_FORMAT_ABSTRACT_VIDEO_PLAYLIST,
    USBH_MTP_OBJECT_FORMAT_ABSTRACT_MEDIACAST,
    USBH_MTP_OBJECT_FORMAT_WPL_PLAYLIST,
    USBH_MTP_OBJECT_FORMAT_M3U_PLAYLIST,
    USBH_MTP_OBJECT_FORMAT_MPL_PLAYLIST,
    USBH_MTP_OBJECT_FORMAT_ASX_PLAYLIST,
}
```

```
USBH_MTP_OBJECT_FORMAT_PLS_PLAYLIST,  
USBH_MTP_OBJECT_FORMAT_UNDEFINED_DOCUMENT,  
USBH_MTP_OBJECT_FORMAT_ABSTRACT_DOCUMENT,  
USBH_MTP_OBJECT_FORMAT_XML_DOCUMENT,  
USBH_MTP_OBJECT_FORMAT_MICROSOFT_WORD_DOCUMENT,  
USBH_MTP_OBJECT_FORMAT_MHT_COMPILED_HTML_DOCUMENT,  
USBH_MTP_OBJECT_FORMAT_MICROSOFT_EXCEL_SPREADSHEET,  
USBH_MTP_OBJECT_FORMAT_MICROSOFT_POWERPOINT_PRESENTATION,  
USBH_MTP_OBJECT_FORMAT_UNDEFINED_MESSAGE,  
USBH_MTP_OBJECT_FORMAT_ABSTRACT_MESSAGE,  
USBH_MTP_OBJECT_FORMAT_UNDEFINED_CONTACT,  
USBH_MTP_OBJECT_FORMAT_ABSTRACT_CONTACT,  
USBH_MTP_OBJECT_FORMAT_VCARD_2  
} USBH_MTP_OBJECT_FORMAT;
```

9.5.5 USBH_MTP_EVENT_CODES

Description

Events are described by a 16-bit code.

Type definition

```
typedef enum {  
    USBH_MTP_EVENT_UNDEFINED,  
    USBH_MTP_EVENT_CANCEL_TRANSACTION,  
    USBH_MTP_EVENT_OBJECT_ADDED,  
    USBH_MTP_EVENT_OBJECT_REMOVED,  
    USBH_MTP_EVENT_STORE_ADDED,  
    USBH_MTP_EVENT_STORE_REMOVED,  
    USBH_MTP_EVENT_DEVICE_PROP_CHANGED,  
    USBH_MTP_EVENT_OBJECT_INFO_CHANGED,  
    USBH_MTP_EVENT_DEVICE_INFO_CHANGED,  
    USBH_MTP_EVENT_REQUEST_OBJECT_TRANSFER,  
    USBH_MTP_EVENT_STORE_FULL,  
    USBH_MTP_EVENT_DEVICE_RESET,  
    USBH_MTP_EVENT_STORAGE_INFO_CHANGED,  
    USBH_MTP_EVENT_CAPTURE_COMPLETE,  
    USBH_MTP_EVENT_UNREPORTED_STATUS,  
    USBH_MTP_EVENT_OBJECT_PROP_CHANGED,  
    USBH_MTP_EVENT_OBJECT_PROP_DESC_CHANGED,  
    USBH_MTP_EVENT_OBJECT_REFERENCES_CHANGED  
} USBH_MTP_EVENT_CODES;
```

Chapter 10

CDC Device Driver (Add-On)

This chapter describes the optional emUSB-Host add-on “CDC device driver”. It allows communication with a CDC USB device.



10.1 Introduction

The CDC driver software component of emUSB-Host allows communication with CDC devices. The Communication Device Class (CDC) is an abstract USB class protocol defined by the USB Implementers Forum. The protocol allows emulation of serial communication via USB.

This chapter provides an explanation of the functions available to application developers via the CDC driver software. All the functions and data types of this add-on are prefixed with 'USBH_CDC_'.

10.1.1 Overview

A CDC device connected to the emUSB-Host is automatically configured and added to an internal list. If the CDC driver has been registered, it is notified via a callback when a CDC device has been added or removed. The driver then can notify the application program, when a callback function has been registered via `USBH_CDC_AddNotification()`. In order to communicate with such a device, the application has to call the `USBH_CDC_Open()`, passing the device index. CDC devices are identified by an index. The first connected device gets assigned the index 0, the second index 1, and so on.

10.1.2 Features

The following features are provided:

- Compatibility with different CDC devices.
- Ability to send and receive data.
- Ability to set various parameters, such as baudrate, number of stop bits, parity.
- Handling of multiple CDC devices at the same time.
- Notifications about CDC connection status.
- Ability to query the CDC line and modem status.

10.1.3 Example code

An example application which uses the API is provided in the `USBH_CDC_Start.c` file. This example displays information about the CDC device in the I/O terminal of the debugger. In addition the application then starts a simple echo server, sending back the received data.

10.2 API Functions

This chapter describes the emUSB-Host CDC driver API functions. These functions are defined in the header file `USBH_CDC.h`.

Function	Description
<code>USBH_CDC_Init()</code>	Initializes and registers the CDC device module with emUSB-Host.
<code>USBH_CDC_Exit()</code>	Unregisters and de-initializes the CDC device module from emUSB-Host.
<code>USBH_CDC_AddNotification()</code>	Adds a callback in order to be notified when a device is added or removed.
<code>USBH_CDC_RemoveNotification()</code>	Removes a callback added via <code>USBH_CDC_AddNotification</code> .
<code>USBH_CDC_RegisterNotification()</code>	This function is deprecated, please use function <code>USBH_CDC_AddNotification</code> ! Sets a callback in order to be notified when a device is added or removed.
<code>USBH_CDC_ConfigureDefaultTimeout()</code>	Sets the default read and write time-out that shall be used when a new device is connected.
<code>USBH_CDC_Open()</code>	Opens a device given by an index.
<code>USBH_CDC_Close()</code>	Closes a handle to an opened device.
<code>USBH_CDC_AllowShortRead()</code>	Enables or disables short read mode.
<code>USBH_CDC_GetDeviceInfo()</code>	Retrieves information about the CDC device.
<code>USBH_CDC_SetTimeouts()</code>	Sets up the timeouts for read and write operations.
<code>USBH_CDC_Read()</code>	Reads from the CDC device.
<code>USBH_CDC_Write()</code>	Writes data to the CDC device.
<code>USBH_CDC_GetMaxTransferSize()</code>	Return the maximum transfer sizes allowed for the <code>USBH_CDC_*Async</code> functions.
<code>USBH_CDC_ReadAsync()</code>	Triggers a read transfer to the CDC device.
<code>USBH_CDC_WriteAsync()</code>	Triggers a write transfer to the CDC device.
<code>USBH_CDC_CancelRead()</code>	Cancels a running read transfer.
<code>USBH_CDC_CancelWrite()</code>	Cancels a running write transfer.
<code>USBH_CDC_SetCommParas()</code>	Setups the serial communication with the given characteristics.
<code>USBH_CDC_SetDtr()</code>	Sets the Data Terminal Ready (DTR) control signal.
<code>USBH_CDC_ClrDtr()</code>	Clears the Data Terminal Ready (DTR) control signal.
<code>USBH_CDC_SetRts()</code>	Sets the Request To Send (RTS) control signal.
<code>USBH_CDC_ClrRts()</code>	Clears the Request To Send (RTS) control signal.
<code>USBH_CDC_GetQueueStatus()</code>	Gets the number of bytes in the receive queue.

Function	Description
<code>USBH_CDC_SetBreak()</code>	Sets the BREAK condition for the device for a limited time.
<code>USBH_CDC_SetBreakOn()</code>	Sets the BREAK condition for the device to "on".
<code>USBH_CDC_SetBreakOff()</code>	Resets the BREAK condition for the device.
<code>USBH_CDC_GetSerialState()</code>	Gets the modem status and line status from the device.
<code>USBH_CDC_SetOnSerialStateChange()</code>	Sets a callback which informs the user about serial state changes.
<code>USBH_CDC_SetOnIntStateChange()</code>	Sets the callback to retrieve data that are received on the interrupt endpoint.
<code>USBH_CDC_GetSerialNumber()</code>	Get the serial number of a CDC device.
<code>USBH_CDC_AddDevice()</code>	Register a device with a non-standard interface layout as a CDC device.
<code>USBH_CDC_RemoveDevice()</code>	Removes a non-standard CDC device which was added by <code>USBH_CDC_AddDevice()</code> .
<code>USBH_CDC_SetConfigFlags()</code>	Sets configuration flags for the CDC module.
<code>USBH_CDC_SuspendResume()</code>	Prepares a CDC device for suspend (stops the interrupt endpoint) or re-starts the interrupt endpoint functionality after a resume.
<code>USBH_CDC_GetInterfaceHandle()</code>	Return the handle to the (open) USB interface.

10.2.1 USBH_CDC_Init()

Description

Initializes and registers the CDC device module with emUSB-Host.

Prototype

```
U8 USBH_CDC_Init(void);
```

Return value

- 1 Success or module already initialized.
- 0 Could not register CDC device module.

Additional information

This function can be called multiple times, but only the first call initializes the module. Any further calls only increase the initialization counter. This is useful for cases where the module is initialized from different places which do not interact with each other, To de-initialize the module `USBH_CDC_Exit` has to be called the same number of times as this function was called.

10.2.2 USBH_CDC_Exit()

Description

Unregisters and de-initializes the CDC device module from emUSB-Host.

Prototype

```
void USBH_CDC_Exit(void);
```

Additional information

Before this function is called any notifications added via `USBH_CDC_AddNotification()` must be removed via `USBH_CDC_RemoveNotification()`. Has to be called the same number of times `USBH_CDC_Init` was called in order to de-initialize the module. This function will release resources that were used by this device driver. It has to be called if the application is closed. This has to be called before `USBH_Exit()` is called. No more functions of this module may be called after calling `USBH_CDC_Exit()`. The only exception is `USBH_CDC_Init()`, which would in turn re-init the module and allow further calls.

10.2.3 USBH_CDC_AddNotification()

Description

Adds a callback in order to be notified when a device is added or removed.

Prototype

```
USBH_STATUS USBH_CDC_AddNotification(USBH_NOTIFICATION_HOOK * pHook,
                                     USBH_NOTIFICATION_FUNC * pfNotification,
                                     void * pContext);
```

Parameters

Parameter	Description
<code>pHook</code>	Pointer to a user provided <code>USBH_NOTIFICATION_HOOK</code> variable.
<code>pfNotification</code>	Pointer to a function the stack should call when a device is connected or disconnected.
<code>pContext</code>	Pointer to a user context that is passed to the callback function.

Return value

`USBH_STATUS_SUCCESS` on success or error code on failure.

Example

```
static USBH_NOTIFICATION_HOOK _Hook;

/*****
 *
 *      _cbOnAddRemoveDevice
 *
 * Function description
 *      Callback, called when a device is added or removed.
 *      Call in the context of the USBH_Task.
 *      The functionality in this routine should not block
 */
static void _cbOnAddRemoveDevice(void * pContext, U8 DevIndex, USBH_DEVICE_EVENT Event) {
    (void)pContext;
    switch (Event) {
        case USBH_DEVICE_EVENT_ADD:
            USBH_Logf_Application("**** Device added\n");
            _DevIndex = DevIndex;
            _DevIsReady = 1;
            break;
        case USBH_DEVICE_EVENT_REMOVE:
            USBH_Logf_Application("**** Device removed\n");
            _DevIsReady = 0;
            _DevIndex = -1;
            break;
        default:; // Should never happen
    }
}

<...>
USBH_CDC_Init();
USBH_CDC_AddNotification(&_Hook, _cbOnAddRemoveDevice, NULL);
<...>
```

10.2.4 USBH_CDC_RemoveNotification()

Description

Removes a callback added via USBH_CDC_AddNotification.

Prototype

```
USBH_STATUS USBH_CDC_RemoveNotification(const USBH_NOTIFICATION_HOOK * pHook);
```

Parameters

Parameter	Description
<code>pHook</code>	Pointer to a user provided USBH_NOTIFICATION_HOOK variable.

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

10.2.5 USBH_CDC_RegisterNotification()

Description

This function is deprecated, please use function `USBH_CDC_AddNotification`! Sets a callback in order to be notified when a device is added or removed.

Prototype

```
void USBH_CDC_RegisterNotification(USBH_NOTIFICATION_FUNC * pfNotification,  
                                  void * pContext);
```

Parameters

Parameter	Description
<code>pfNotification</code>	Pointer to a function the stack should call when a device is connected or disconnected.
<code>pContext</code>	Pointer to a user context that is passed to the callback function.

Additional information

This function is deprecated, please use function `USBH_CDC_AddNotification`.

10.2.6 USBH_CDC_ConfigureDefaultTimeout()

Description

Sets the default read and write time-out that shall be used when a new device is connected.

Prototype

```
void USBH_CDC_ConfigureDefaultTimeout(U32 ReadTimeout,  
                                       U32 WriteTimeout);
```

Parameters

Parameter	Description
ReadTimeout	Default read timeout given in ms.
WriteTimeout	Default write timeout given in ms.

10.2.7 USBH_CDC_Open()

Description

Opens a device given by an index.

Prototype

```
USBH_CDC_HANDLE USBH_CDC_Open(unsigned Index);
```

Parameters

Parameter	Description
Index	Index of the device that shall be opened. In general this means: the first connected device is 0, second device is 1 etc.

Return value

= USBH_CDC_INVALID_HANDLE Device not available or removed.
≠ USBH_CDC_INVALID_HANDLE Handle to a CDC device

Additional information

The index of a new connected device is provided to the callback function registered with USBH_CDC_AddNotification().

10.2.8 USBH_CDC_Close()

Description

Closes a handle to an opened device.

Prototype

```
USBH_STATUS USBH_CDC_Close(USBH_CDC_HANDLE hDevice);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to an open device returned by <code>USBH_CDC_Open()</code> .

Return value

`USBH_STATUS_SUCCESS` on success or error code on failure.

10.2.9 USBH_CDC_AllowShortRead()

Description

Enables or disables short read mode. If enabled, the function `USBH_CDC_Read()` returns as soon as a short packet was read from the device. This allows the application to read data where the number of bytes to read is undefined.

Prototype

```
USBH_STATUS USBH_CDC_AllowShortRead(USBH_CDC_HANDLE hDevice,  
                                     U8               AllowShortRead);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to an open device returned by <code>USBH_CDC_Open()</code> .
<code>AllowShortRead</code>	Define whether short read mode shall be used or not. • 1 - Allow short read. • 0 - Short read mode disabled.

Return value

`USBH_STATUS_SUCCESS` on success or error code on failure.

10.2.10 USBH_CDC_GetDeviceInfo()

Description

Retrieves information about the CDC device.

Prototype

```
USBH_STATUS USBH_CDC_GetDeviceInfo(USBH_CDC_HANDLE      hDevice,  
                                   USBH_CDC_DEVICE_INFO * pDevInfo);
```

Parameters

Parameter	Description
hDevice	Handle to an open device returned by USBH_CDC_Open().
pDevInfo	Pointer to a USBH_CDC_DEVICE_INFO structure that receives the information.

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

10.2.11 USBH_CDC_SetTimeouts()

Description

Sets up the timeouts for read and write operations.

Prototype

```
USBH_STATUS USBH_CDC_SetTimeouts(USBH_CDC_HANDLE hDevice,  
                                  U32              ReadTimeout,  
                                  U32              WriteTimeout);
```

Parameters

Parameter	Description
hDevice	Handle to an open device returned by <code>USBH_CDC_Open()</code> .
ReadTimeout	Read timeout given in ms.
WriteTimeout	Write timeout given in ms.

Return value

`USBH_STATUS_SUCCESS` on success or error code on failure.

10.2.12 USBH_CDC_Read()

Description

Reads from the CDC device. The behavior depends on the Short Read mode (see `USBH_CDC_AllowShortRead()`): If Short Read mode is enabled, the function tries to read up to `NumBytes`, but will stop reading and returns if a short packet was received (a packet that is smaller than the maximum packet size of the endpoint). If Short Read mode is disabled, the function tries to reads exactly `NumBytes` of data, independent of the sizes of the received packets. This function will also return when the timeout expires, whatever comes first. is not specified via `USBH_CDC_SetTimeouts()` the default timeout (`USBH_CDC_DEFAULT_TIMEOUT`) is used.

The USB stack can only read complete packets from the USB device. If the size of a received packet exceeds `NumBytes` then all data that does not fit into the callers buffer (`pData`) is stored in an internal buffer and will be returned by the next call to `USBH_CDC_Read()`. See also `USBH_CDC_GetQueueStatus()`.

To read a null packet, set `pData` = NULL and `NumBytes` = 0. For this, the internal buffer must be empty.

Prototype

```
USBH_STATUS USBH_CDC_Read(USBH_CDC_HANDLE hDevice,  
                           U8              * pData,  
                           U32              NumBytes,  
                           U32              * pNumBytesRead);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to an open device returned by <code>USBH_CDC_Open()</code> .
<code>pData</code>	Pointer to a buffer to store the read data.
<code>NumBytes</code>	Number of bytes to be read from the device.
<code>pNumBytesRead</code>	Pointer to a variable which receives the number of bytes read from the device. Can be NULL.

Return value

`USBH_STATUS_SUCCESS` on success or error code on failure.

Additional information

If the function returns an error code (including `USBH_STATUS_TIMEOUT`) it already may have read part of the data. The number of bytes read successfully is always stored in the variable pointed to by `pNumBytesRead`.

10.2.13 USBH_CDC_Write()

Description

Writes data to the CDC device. The function blocks until all data has been written or until the timeout has been reached. If a timeout is not specified via USBH_CDC_SetTimeouts() the default timeout (USBH_CDC_DEFAULT_TIMEOUT) is used.

Prototype

```
USBH_STATUS USBH_CDC_Write(      USBH_CDC_HANDLE  hDevice,
                                const U8          * pData,
                                U32               NumBytes,
                                U32               * pNumBytesWritten);
```

Parameters

Parameter	Description
hDevice	Handle to an open device returned by USBH_CDC_Open().
pData	Pointer to data to be sent.
NumBytes	Number of bytes to send.
pNumBytesWritten	Pointer to a variable which receives the number of bytes written to the device. Can be NULL.

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

Additional information

If the function returns an error code (including USBH_STATUS_TIMEOUT) it already may have written part of the data. The number of bytes written successfully is always stored in the variable pointed to by pNumBytesWritten.

10.2.14 USBH_CDC_GetMaxTransferSize()

Description

Return the maximum transfer sizes allowed for the USBH_CDC_*Async functions.

Prototype

```
USBH_STATUS USBH_CDC_GetMaxTransferSize(USBH_CDC_HANDLE hDevice,  
                                         U32             * pMaxOutTransferSize,  
                                         U32             * pMaxInTransferSize);
```

Parameters

Parameter	Description
hDevice	Handle to an open device returned by USBH_CDC_Open().
pMaxOutTransferSize	Pointer to a variable which will receive the maximum transfer size for the Bulk OUT endpoint (for USBH_CDC_ReadAsync()).
pMaxInTransferSize	Pointer to a variable which will receive the maximum transfer size for the Bulk IN endpoint (for USBH_CDC_WriteAsync()).

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

Additional information

Using this function is only necessary with the USBH_CDC_*Async functions, other functions handle the limits internally. These limits exist because certain USB controllers have hardware limitations. Some USB controllers (OHCI, EHCI, ...) do not have these limitations, therefore 0xFFFFFFFF will be returned.

10.2.15 USBH_CDC_ReadAsync()

Description

Triggers a read transfer to the CDC device. The result of the transfer is received through the user callback. This function will return immediately while the read transfer is done asynchronously. The read operation terminates either, if 'BuffSize' bytes have been read or if a short packet was received from the device.

Prototype

```
USBH_STATUS USBH_CDC_ReadAsync(USBH_CDC_HANDLE hDevice,
                                void * pBuffer,
                                U32 BufferSize,
                                USBH_CDC_ON_COMPLETE_FUNC * pfOnComplete,
                                USBH_CDC_RW_CONTEXT * pRWContext);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to an open device returned by <code>USBH_CDC_Open()</code> .
<code>pBuffer</code>	Pointer to the buffer that receives the data from the device.
<code>BufferSize</code>	Size of the buffer in bytes. Must be a multiple of of the maximum packet size of the USB device. Use <code>USBH_CDC_GetMaxTransferSize()</code> to get the maximum allowed size.
<code>pfOnComplete</code>	Pointer to a user function of type <code>USBH_CDC_ON_COMPLETE_FUNC</code> which will be called after the transfer has been completed.
<code>pRWContext</code>	Pointer to a <code>USBH_CDC_RW_CONTEXT</code> structure which will be filled with data after the transfer has been completed and passed as a parameter to the <code>pfOnComplete</code> function. The member 'pUserContext' may be set before calling <code>USBH_CDC_ReadAsync()</code> . Other members do not need to be initialized and are set by the function <code>USBH_CDC_ReadAsync()</code> . The memory used for this structure must be valid, until the transaction is completed.

Return value

= `USBH_STATUS_PENDING` Success, the data transfer is queued, the user callback will be called after the transfer is finished.
 ≠ `USBH_STATUS_PENDING` An error occurred, the transfer is not started and user callback will not be called.

Additional information

This function performs an unbuffered read operation (in contrast to `USBH_CDC_Read()`), so care should be taken if intermixing calls to `USBH_CDC_ReadAsync()` and `USBH_CDC_Read()`.

Example

```
static USBH_CDC_RW_CONTEXT _ReadWriteContext;

<...>

/*****
 *
 *      _OnReadComplete
 */
static void _OnReadComplete(USBH_CDC_RW_CONTEXT * pRWContext) {
```

```
if (pRWContext->Status == USBH_STATUS_SUCCESS) {
    printf("Successfully read %u bytes \n",
        (unsigned int)pRWContext->NumBytesTransferred);
} else {
    printf("ReadAsync callback returned %s \n",
        USBH_GetStatusStr(pRWContext->Status));
    // Error handling
}
<...>
}

<...>

Status = USBH_CDC_ReadAsync(_hDevice,
                            _acBuffer,
                            NumBytes,
                            _OnReadComplete,
                            &_ReadWriteContext);
if (Status != USBH_STATUS_PENDING) {
    // Error handling.
}
<...>
```


10.2.16 USBH_CDC_WriteAsync()

Description

Triggers a write transfer to the CDC device. The result of the transfer is received through the user callback. This function will return immediately while the write transfer is done asynchronously.

Prototype

```
USBH_STATUS USBH_CDC_WriteAsync(USBH_CDC_HANDLE hDevice,
                                void * pBuffer,
                                U32 BufferSize,
                                USBH_CDC_ON_COMPLETE_FUNC * pfOnComplete,
                                USBH_CDC_RW_CONTEXT * pRWContext);
```

Parameters

Parameter	Description
hDevice	Handle to an open device returned by USBH_CDC_Open() .
pBuffer	Pointer to a buffer which holds the data.
BufferSize	Number of bytes to write. Use USBH_CDC_GetMaxTransferSize() to get the maximum allowed size.
pfOnComplete	Pointer to a user function of type USBH_CDC_ON_COMPLETE_FUNC which will be called after the transfer has been completed.
pRWContext	Pointer to a USBH_CDC_RW_CONTEXT structure which will be filled with data after the transfer has been completed and passed as a parameter to the pfOnComplete function. The member 'pUserContext' may be set before calling USBH_CDC_WriteAsync() . Other members do not need to be initialized and are set by the function USBH_CDC_WriteAsync() . The memory used for this structure must be valid, until the transaction is completed.

Return value

= USBH_STATUS_PENDING	Success, the data transfer is queued, the user callback will be called after the transfer is finished.
≠ USBH_STATUS_PENDING	An error occurred, the transfer is not started and user callback will not be called.

Example

```
static USBH_CDC_RW_CONTEXT _ReadWriteContext;

<...>

/*****
 *
 *      _OnWriteComplete
 */
static void _OnWriteComplete(USBH_CDC_RW_CONTEXT * pRWContext) {
    if (pRWContext->Status == USBH_STATUS_SUCCESS) {
        printf("Successfully written data to the device \n");
    } else {
        printf("WriteAsync callback returned %s \n",
            USBH_GetStatusStr(pRWContext->Status));
        // Error handling
    }
}
```

```
<...>
}

<...>

Status = USBH_CDC_WriteAsync(_hDevice,
                             _acBuffer,
                             NumBytes,
                             _OnWriteComplete,
                             &_ReadWriteContext);
if (Status != USBH_STATUS_PENDING) {
    // Error handling.
}
<...>
```

10.2.17 USBH_CDC_CancelRead()

Description

Cancels a running read transfer.

Prototype

```
USBH_STATUS USBH_CDC_CancelRead(USBH_CDC_HANDLE hDevice);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to an open device returned by <code>USBH_CDC_Open()</code> .

Return value

`USBH_STATUS_SUCCESS` on success or error code on failure.

Additional information

This function can be used to cancel a transfer which was initiated by `USBH_CDC_ReadAsync` or `USBH_CDC_Read`. In the later case this function has to be called from a different task.

10.2.18 USBH_CDC_CancelWrite()

Description

Cancels a running write transfer.

Prototype

```
USBH_STATUS USBH_CDC_CancelWrite(USBH_CDC_HANDLE hDevice);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to an open device returned by <code>USBH_CDC_Open()</code> .

Return value

`USBH_STATUS_SUCCESS` on success or error code on failure.

Additional information

This function can be used to cancel a transfer which was initiated by `USBH_CDC_WriteAsync` or `USBH_CDC_Write`. In the later case this function has to be called from a different task.

10.2.19 USBH_CDC_SetCommParas()

Description

Setups the serial communication with the given characteristics.

Prototype

```
USBH_STATUS USBH_CDC_SetCommParas(USBH_CDC_HANDLE hDevice,
                                   U32              Baudrate,
                                   U8                DataBits,
                                   U8                StopBits,
                                   U8                Parity);
```

Parameters

Parameter	Description
hDevice	Handle to an open device returned by USBH_CDC_Open().
Baudrate	Transfer rate.
DataBits	Number of bits per word. Must be between USBH_CDC_BITS_5 and USBH_CDC_BITS_8.
StopBits	Number of stop bits. Must be USBH_CDC_STOP_BITS_1 or USBH_CDC_STOP_BITS_2.
Parity	Parity - must be must be one of the following values: • USBH_CDC_PARITY_NONE • USBH_CDC_PARITY_ODD • USBH_CDC_PARITY_EVEN • USBH_CDC_PARITY_MARK • USBH_CDC_PARITY_SPACE

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

10.2.20 USBH_CDC_SetDtr()

Description

Sets the Data Terminal Ready (DTR) control signal.

Prototype

```
USBH_STATUS USBH_CDC_SetDtr(USBH_CDC_HANDLE hDevice);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to an open device returned by <code>USBH_CDC_Open()</code> .

Return value

`USBH_STATUS_SUCCESS` on success or error code on failure.

10.2.21 USBH_CDC_ClrDtr()

Description

Clears the Data Terminal Ready (DTR) control signal.

Prototype

```
USBH_STATUS USBH_CDC_ClrDtr(USBH_CDC_HANDLE hDevice);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to an open device returned by <code>USBH_CDC_Open()</code> .

Return value

`USBH_STATUS_SUCCESS` on success or error code on failure.

10.2.22 USBH_CDC_SetRts()

Description

Sets the Request To Send (RTS) control signal.

Prototype

```
USBH_STATUS USBH_CDC_SetRts(USBH_CDC_HANDLE hDevice);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to an open device returned by <code>USBH_CDC_Open()</code> .

Return value

`USBH_STATUS_SUCCESS` on success or error code on failure.

10.2.23 USBH_CDC_ClrRts()

Description

Clears the Request To Send (RTS) control signal.

Prototype

```
USBH_STATUS USBH_CDC_ClrRts(USBH_CDC_HANDLE hDevice);
```

Parameters

Parameter	Description
hDevice	Handle to an open device returned by <code>USBH_CDC_Open()</code> .

Return value

`USBH_STATUS_SUCCESS` on success or error code on failure.

10.2.24 USBH_CDC_GetQueueStatus()

Description

Gets the number of bytes in the receive queue.

The USB stack can only read complete packets from the USB device. If the size of a received packet exceeds the number of bytes requested with `USBH_CDC_Read()`, then all data that is not returned by `USBH_CDC_Read()` is stored in an internal buffer.

The number of bytes returned by `USBH_CDC_GetQueueStatus()` can be read using `USBH_CDC_Read()` out of the buffer without a USB transaction to the USB device being executed.

Prototype

```
USBH_STATUS USBH_CDC_GetQueueStatus(USBH_CDC_HANDLE hDevice,
                                     U32              * pRxBytes);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to an open device returned by <code>USBH_CDC_Open()</code> .
<code>pRxBytes</code>	Pointer to a variable which receives the number of bytes in the receive queue.

Return value

`USBH_STATUS_SUCCESS` on success or error code on failure.

Example

```
//
// Read only ONE byte to trigger the read transfer.
// This means that the remaining bytes are in the internal packet buffer!
//
USBH_CDC_Read(hDevice, acData, 1, &NumBytes);
if (NumBytes) {
    //
    // We do not know how big the packet was which we received from the device,
    // since we only read 1 byte from the packet.
    // Therefore we still might have some data in the internal buffer!
    // Using USBH_CDC_GetQueueStatus we can check how many bytes are still in the
    // internal buffer (if any) and read those as well.
    //
    if (USBH_CDC_GetQueueStatus(hDevice, &RxBytes) == USBH_STATUS_SUCCESS) {
        //
        // Read the remaining bytes.
        //
        if (RxBytes > 0) {
            USBH_CDC_Read(hDevice, &acData[1], RxBytes, &NumBytes);
        }
    }
}
```

10.2.25 USBH_CDC_SetBreak()

Description

Sets the BREAK condition for the device for a limited time.

Prototype

```
USBH_STATUS USBH_CDC_SetBreak(USBH_CDC_HANDLE hDevice,  
                               U16              Duration);
```

Parameters

Parameter	Description
hDevice	Handle to an open device returned by <code>USBH_CDC_Open()</code> .
Duration	Duration of the break condition in ms.

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

10.2.26 USBH_CDC_SetBreakOn()

Description

Sets the BREAK condition for the device to "on".

Prototype

```
USBH_STATUS USBH_CDC_SetBreakOn(USBH_CDC_HANDLE hDevice);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to an open device returned by <code>USBH_CDC_Open()</code> .

Return value

`USBH_STATUS_SUCCESS` on success or error code on failure.

10.2.27 USBH_CDC_SetBreakOff()

Description

Resets the BREAK condition for the device.

Prototype

```
USBH_STATUS USBH_CDC_SetBreakOff(USBH_CDC_HANDLE hDevice);
```

Parameters

Parameter	Description
hDevice	Handle to an open device returned by USBH_CDC_Open().

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

10.2.28 USBH_CDC_GetSerialState()

Description

Gets the modem status and line status from the device.

Prototype

```
USBH_STATUS USBH_CDC_GetSerialState(USBH_CDC_HANDLE hDevice,  
                                     USBH_CDC_SERIALSTATE * pSerialState);
```

Parameters

Parameter	Description
hDevice	Handle to an open device returned by <code>USBH_CDC_Open()</code> .
pSerialState	Pointer to a structure of type <code>USBH_CDC_SERIALSTATE</code> which receives the serial status from the device.

Return value

`USBH_STATUS_SUCCESS` on success or error code on failure.

Additional information

The least significant byte of the [pSerialState](#) value holds the modem status. The line status is held in the second least significant byte of the [pSerialState](#) value. The status is bit-mapped as follows:

- Data Carrier Detect (DCD) = 0x01
- Data Set Ready (DSR) = 0x02
- Break Interrupt (BI) = 0x04
- Ring Indicator (RI) = 0x08
- Framing Error (FE) = 0x10
- Parity Error (PE) = 0x20
- Overrun Error (OE) = 0x40

10.2.29 USBH_CDC_SetOnSerialStateChange()

Description

Sets a callback which informs the user about serial state changes.

Prototype

```
USBH_STATUS USBH_CDC_SetOnSerialStateChange
(USBH_CDC_HANDLE hDevice,
 USBH_CDC_SERIAL_STATE_CALLBACK * pfOnSerialStateChange);
```

Parameters

Parameter	Description
hDevice	Handle to an open device returned by USBH_CDC_Open().
pfOnSerialStateChange	Pointer to the user callback. Can be NULL (to remove the callback).

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

Additional information

The callback is called in the context of the ISR task. The callback should not block.

10.2.30 USBH_CDC_SetOnIntStateChange()

Description

Sets the callback to retrieve data that are received on the interrupt endpoint.

Prototype

```
USBH_STATUS USBH_CDC_SetOnIntStateChange
(
    USBH_CDC_HANDLE          hDevice,
    USBH_CDC_INT_STATE_CALLBACK * pfOnIntState,
    void                      * pUserContext);
```

Parameters

Parameter	Description
hDevice	Handle to an open device returned by USBH_CDC_Open().
pfOnIntState	Pointer to the callback that shall retrieve the data.
pUserContext	Pointer to the user context.

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

10.2.31 USBH_CDC_GetSerialNumber()

Description

Get the serial number of a CDC device. The serial number is in UNICODE format, not zero terminated.

Prototype

```
USBH_STATUS USBH_CDC_GetSerialNumber(USBH_CDC_HANDLE hDevice,
                                     U32             BuffSize,
                                     U8               * pSerialNumber,
                                     U32             * pSerialNumberSize);
```

Parameters

Parameter	Description
hDevice	Handle to an open device returned by USBH_CDC_Open().
BuffSize	Pointer to a buffer which holds the data.
pSerialNumber	Size of the buffer in bytes.
pSerialNumberSize	Pointer to a user function which will be called.

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

10.2.32 USBH_CDC_AddDevice()

Description

Register a device with a non-standard interface layout as a CDC device. This function should not be used for CDC compliant devices! After registering the device the application will receive ADD and REMOVE notifications to the user callback which was set by USBH_CDC_AddNotification().

Prototype

```
USBH_STATUS USBH_CDC_AddDevice(USBH_INTERFACE_ID ControlInterfaceID,  
                               USBH_INTERFACE_ID DataInterfaceId,  
                               unsigned           Flags);
```

Parameters

Parameter	Description
ControlInterfaceID	Numeric index of the CDC ACM interface.
DataInterfaceId	Numeric index of the CDC Data interface.
Flags	Reserved for future use. Should be zero.

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

Additional information

The numeric interface IDs can be retrieved by setting up a PnP notification via USBH_RegisterPnPNotification(). Please note that the PnP notification callback will be triggered for each interface, but you only have to add the device once. Alternatively you can simply set the IDs if you know the interface layout.

10.2.33 USBH_CDC_RemoveDevice()

Description

Removes a non-standard CDC device which was added by `USBH_CDC_AddDevice()`.

Prototype

```
USBH_STATUS USBH_CDC_RemoveDevice(USBH_INTERFACE_ID ControlInterfaceID,  
                                   USBH_INTERFACE_ID DataInterfaceId);
```

Parameters

Parameter	Description
ControlInterfaceID	Numeric index of the CDC ACM interface.
DataInterfaceId	Numeric index of the CDC Data interface.

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

10.2.34 USBH_CDC_SetConfigFlags()

Description

Sets configuration flags for the CDC module.

Prototype

```
void USBH_CDC_SetConfigFlags(U32 Flags);
```

Parameters

Parameter	Description
Flags	A bitwise OR-combination of flags that shall be set for each device. At the moment the following are available: • USBH_CDC_IGNORE_INT_EP: This flag prevents the interrupt endpoint of the CDC interface from being polled by the CDC module. The interrupt endpoint is normally used in the CDC protocol to communicate the changes of serial states, using this flag essentially prevents the callbacks set via <code>USBH_CDC_SetOnIntStateChange()</code> and <code>USBH_CDC_SetOnSerialStateChange()</code> from ever executing. • USBH_CDC_DISABLE_INTERFACE_CHECK: According to the CDC specification CDC devices must contain two interfaces, the first being the control interface, containing an interrupt IN endpoint, the second being a data interface containing a bulk IN and a bulk OUT endpoint. Some manufacturers sometimes decide to put all 3 endpoints into one interface, despite the device otherwise being compatible to the CDC specification. This flag allows such devices to be added to the CDC module.

10.2.35 USBH_CDC_SuspendResume()

Description

Prepares a CDC device for suspend (stops the interrupt endpoint) or re-starts the interrupt endpoint functionality after a resume.

Prototype

```
USBH_STATUS USBH_CDC_SuspendResume(USBH_CDC_HANDLE hDevice,  
                                     U8               State);
```

Parameters

Parameter	Description
hDevice	Handle to an open device returned by <code>USBH_CDC_Open()</code> .
State	0 - Prepare for suspend. 1 - Return from resume.

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

Additional information

The application must make sure that no transactions are running when setting a device into suspend mode. This function is used in combination with `USBH_SetRootPortPower()`. To suspend: Call this function before `USBH_SetRootPortPower(x, y, USBH_SUSPEND)` with [State](#) = 0. To resume: Call this function after `USBH_SetRootPortPower(x, y, USBH_NORMAL_POWER)` with [State](#) = 1.

10.2.36 USBH_CDC_GetInterfaceHandle()

Description

Return the handle to the (open) USB interface. Can be used to call USBH core functions like USBH_GetStringDescriptor().

Prototype

```
USBH_INTERFACE_HANDLE USBH_CDC_GetInterfaceHandle(USBH_CDC_HANDLE hDevice);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to an open device returned by USBH_CDC_Open().

Return value

Handle to an open interface.

10.3 Data structures

This chapter describes the emUSB-Host CDC driver data structures.

Structure	Description
USBH_CDC_DEVICE_INFO	Structure containing information about a CDC device.
USBH_CDC_SERIALSTATE	Structure describing the serial state of CDC device.
USBH_CDC_RW_CONTEXT	Contains information about a completed, asynchronous transfers.

10.3.1 USBH_CDC_DEVICE_INFO

Description

Structure containing information about a CDC device.

Type definition

```
typedef struct {
    U16      VendorId;
    U16      ProductId;
    USBH_SPEED Speed;
    U8       ControlInterfaceNo;
    U8       DataInterfaceNo;
    U16      MaxPacketSize;
    U16      ControlClass;
    U16      ControlSubClass;
    U16      ControlProtocol;
    U16      DataClass;
    U16      DataSubClass;
    U16      DataProtocol;
    USBH_INTERFACE_ID ControlInterfaceID;
    USBH_INTERFACE_ID DataInterfaceID;
} USBH_CDC_DEVICE_INFO;
```

Structure members

Member	Description
VendorId	The Vendor ID of the device.
ProductId	The Product ID of the device.
Speed	The USB speed of the device, see USBH_SPEED .
ControlInterfaceNo	Interface index of the ACM Control interface (from USB descriptor).
DataInterfaceNo	Interface index of the ACM Data interface (from USB descriptor).
MaxPacketSize	Maximum packet size of the device, usually 64 in full-speed and 512 in high-speed.
ControlClass	The Class value field of the control interface
ControlSubClass	The SubClass value field of the control interface
ControlProtocol	The Protocol value field of the control interface
DataClass	The Class value field of the data interface
DataSubClass	The SubClass value field of the data interface
DataProtocol	The Protocol value field of the data interface
ControlInterfaceID	ID of the ACM control interface.
DataInterfaceID	ID of the ACM data interface.

10.3.2 USBH_CDC_SERIALSTATE

Description

Structure describing the serial state of CDC device. All members can have a value of 0 (= false/off) or 1 (= true/on).

Type definition

```
typedef struct {  
    U8  bRxCARRIER;  
    U8  bTxCARRIER;  
    U8  bBREAK;  
    U8  bRINGSignal;  
    U8  bFRAMING;  
    U8  bPARITY;  
    U8  bOVERRUN;  
} USBH_CDC_SERIALSTATE;
```

Structure members

Member	Description
bRxCARRIER	State of receiver carrier detection mechanism of device. This signal corresponds to V.24 signal 109 and RS-232 signal DCD.
bTxCARRIER	State of transmission carrier. This signal corresponds to V.24 signal 106 and RS-232 signal DSR.
bBREAK	State of break detection mechanism of the device.
bRINGSignal	State of ring signal detection of the device.
bFRAMING	A framing error has occurred.
bPARITY	A parity error has occurred.
bOVERRUN	Received data has been discarded due to overrun in the device.

10.3.3 USBH_CDC_RW_CONTEXT

Description

Contains information about a completed, asynchronous transfers. Is passed to the USBH_CDC_ON_COMPLETE_FUNC user callback when using asynchronous write and read. When this structure is passed to USBH_CDC_ReadAsync() or USBH_CDC_WriteAsync() its member need not to be initialized.

Type definition

```
typedef struct {  
    void * pUserContext;  
    USBH_STATUS Status;  
    U32 NumBytesTransferred;  
    void * pUserBuffer;  
    U32 UserBufferSize;  
} USBH_CDC_RW_CONTEXT;
```

Structure members

Member	Description
pUserContext	Pointer to a user context. Can be arbitrarily used by the application.
Status	Result status of the asynchronous transfer.
NumBytesTransferred	Number of bytes transferred.
pUserBuffer	Pointer to the buffer provided to USBH_CDC_ReadAsync() or USBH_CDC_WriteAsync().
UserBufferSize	Size of the buffer as provided to USBH_CDC_ReadAsync() or USBH_CDC_WriteAsync().

10.4 Type definitions

This chapter describes the types defined in the header file `USBH_CDC.h`.

Type	Description
USBH_CDC_ON_COMPLETE_FUNC	Function called on completion of an asynchronous transfer.
USBH_CDC_SERIAL_STATE_CALLBACK	Function called on a reception of a CDC ACM serial state change.

10.4.1 USBH_CDC_ON_COMPLETE_FUNC

Description

Function called on completion of an asynchronous transfer. Used by the functions USBH_CDC_ReadAsync() and USBH_CDC_WriteAsync().

Type definition

```
typedef void USBH_CDC_ON_COMPLETE_FUNC(USBH_CDC_RW_CONTEXT * pRWContext);
```

Parameters

Parameter	Description
<code>pRWContext</code>	Pointer to a USBH_CDC_RW_CONTEXT structure.

10.4.2 USBH_CDC_SERIAL_STATE_CALLBACK

Description

Function called on a reception of a CDC ACM serial state change. Used by the function `USBH_CDC_SetOnSerialStateChange()`.

Type definition

```
typedef void USBH_CDC_SERIAL_STATE_CALLBACK(USBH_CDC_HANDLE hDevice,  
                                             USBH_CDC_SERIALSTATE * pSerialState);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to an open device returned by <code>USBH_CDC_Open()</code> .
<code>pSerialState</code>	Pointer to a structure of type <code>USBH_CDC_SERIALSTATE</code> showing the serial status from the device.

Chapter 11

FT232 Device Driver (Add-On)

This chapter describes the optional emUSB-Host add-on “FT232 device driver”. It allows communication with an FTDI FT232 USB device, typically serving as USB to RS232 converter.



11.1 Introduction

The FT232 driver software component of emUSB-Host allows the communication with FTDI FT232 devices. It implements the FT232 protocol specified by FTDI which is a vendor specific protocol. The protocol allows emulation of serial communication via USB. This chapter provides an explanation of the functions available to application developers via the FT232 driver software. All the functions and data types of this add-on are prefixed with the "USBH_FT232_" text.

11.1.1 Features

The following features are provided:

- Compatibility with different FT232 devices.
- Ability to send and receive data.
- Ability to set various parameters, such as baudrate, number of stop bits, parity.
- Handling of multiple FT232 devices at the same time.
- Notifications about FT232 connection status.
- Ability to query the FT232 line and modem status.

11.1.2 Example code

An example application which uses the API is provided in the `USBH_FT232_Start.c` file. This example displays information about the FT232 device in the I/O terminal of the debugger. In addition the application then starts a simple echo server, sending back the received data.

11.1.3 Compatibility

The following devices work with the current FT232 driver:

- FT8U232AM
- FT232B
- FT232R
- FT2232D

11.1.4 Further reading

For more information about the FTDI FT232 devices, please take a look at the hardware manual and D2XX Programmer's Guide manual (Document Reference No.: FT_000071) available from www.ftdichip.com.

11.2 API Functions

This chapter describes the emUSB-Host FT232 driver API functions.

Function	Description
USBH_FT232_Init()	Initializes and registers the FT232 device driver with emUSB-Host.
USBH_FT232_Exit()	Unregisters and de-initializes the FT232 device driver from emUSB-Host.
USBH_FT232_AddNotification()	Adds a callback in order to be notified when a device is added or removed.
USBH_FT232_RemoveNotification()	Removes a callback added via USBH_FT232_AddNotification() .
USBH_FT232_RegisterNotification()	This function is deprecated, please use function USBH_FT232_AddNotification() ! Sets a callback in order to be notified when a device is added or removed.
USBH_FT232_AddCustomDeviceMask()	This function allows the FT232 module to receive notifications about devices which do not present themselves with FTDI's vendor ID (0x0403).
USBH_FT232_ConfigureDefaultTimeout()	Sets the default read and write timeout that shall be used when a new device is connected.
USBH_FT232_Open()	Opens a device given by an index.
USBH_FT232_Close()	Closes a handle to an opened device.
USBH_FT232_GetDeviceInfo()	Retrieves the information about the FT232 device.
USBH_FT232_ResetDevice()	Resets the FT232 device.
USBH_FT232_SetTimeouts()	Sets up the timeouts the host waits until the data transfer will be aborted, for a specific FT232 device.
USBH_FT232_ConfigurePollDelay()	Configures the delay when polling a FT232 device, which has no data to send.
USBH_FT232_Read()	Reads data from the FT232 device.
USBH_FT232_Write()	Writes data to the FT232 device.
USBH_FT232-AllowShortRead()	The configuration function allows to let the read function to return as soon as data are available.
USBH_FT232_SetBaudRate()	Sets the baud rate for the opened device.
USBH_FT232_SetDataCharacteristics()	Setups the serial communication with the given characteristics.
USBH_FT232_SetFlowControl()	This function sets the flow control for the device.
USBH_FT232_SetDtr()	Sets the Data Terminal Ready (DTR) control signal.
USBH_FT232_ClrDtr()	Clears the Data Terminal Ready (DTR) control signal.
USBH_FT232_SetRts()	Sets the Request To Send (RTS) control signal.
USBH_FT232_ClrRts()	Clears the Request To Send (RTS) control signal.

Function	Description
USBH_FT232_GetModemStatus()	Gets the modem status and line status from the device.
USBH_FT232_SetChars()	Sets the special characters for the device.
USBH_FT232_Purge()	Purges receive and transmit buffers in the device.
USBH_FT232_GetQueueStatus()	Gets the number of bytes in the receive queue.
USBH_FT232_SetBreakOn()	Sets the BREAK condition for the device.
USBH_FT232_SetBreakOff()	Resets the BREAK condition for the device.
USBH_FT232_SetLatencyTimer()	The latency timer controls the timeout for the FTDI device to transfer data from the FT232 interface to the USB interface.
USBH_FT232_GetLatencyTimer()	Get the current value of the latency timer.
USBH_FT232_SetBitMode()	Enables different chip modes.
USBH_FT232_GetBitMode()	Returns the current values on the data bus pins.
USBH_FT232_GetInterfaceHandle()	Return the handle to the (open) USB interface.

11.2.1 USBH_FT232_Init()

Description

Initializes and registers the FT232 device driver with emUSB-Host.

Prototype

```
U8 USBH_FT232_Init(void);
```

Return value

- | | |
|---|---|
| 1 | Success. |
| 0 | Could not register FT232 device driver. |

11.2.2 USBH_FT232_Exit()

Description

Unregisters and de-initializes the FT232 device driver from emUSB-Host.

Prototype

```
void USBH_FT232_Exit(void);
```

Additional information

Before this function is called any notifications added via `USBH_FT232_AddNotification()` must be removed via `USBH_FT232_RemoveNotification()`. This function will release resources that were used by this device driver. It has to be called if the application is closed. This has to be called before `USBH_Exit()` is called. No more functions of this module may be called after calling `USBH_FT232_Exit()`. The only exception is `USBH_FT232_Init()`, which would in turn reinitialize the module and allows further calls.

11.2.3 USBH_FT232_AddNotification()

Description

Adds a callback in order to be notified when a device is added or removed.

Prototype

```
USBH_STATUS USBH_FT232_AddNotification(USBH_NOTIFICATION_HOOK * pHook,
                                       USBH_NOTIFICATION_FUNC * pfNotification,
                                       void * pContext);
```

Parameters

Parameter	Description
<code>pHook</code>	Pointer to a user provided <code>USBH_NOTIFICATION_HOOK</code> variable.
<code>pfNotification</code>	Pointer to a function the stack should call when a device is connected or disconnected.
<code>pContext</code>	Pointer to a user context that is passed to the callback function.

Return value

`USBH_STATUS_SUCCESS` on success or error code on failure.

Example

```
static USBH_NOTIFICATION_HOOK _Hook;

/*****
 *
 *      _cbOnAddRemoveDevice
 *
 * Function description
 *   Callback, called when a device is added or removed.
 *   Call in the context of the USBH_Task.
 *   The functionality in this routine should not block
 */
static void _cbOnAddRemoveDevice(void * pContext, U8 DevIndex, USBH_DEVICE_EVENT Event) {
    (void)pContext;
    switch (Event) {
        case USBH_DEVICE_EVENT_ADD:
            USBH_Logf_Application("**** Device added\n");
            _DevIndex = DevIndex;
            _DevIsReady = 1;
            break;
        case USBH_DEVICE_EVENT_REMOVE:
            USBH_Logf_Application("**** Device removed\n");
            _DevIsReady = 0;
            _DevIndex = -1;
            break;
        default:; // Should never happen
    }
}

<...>
USBH_FT232_Init();
USBH_FT232_AddNotification(&_Hook, _cbOnAddRemoveDevice, NULL);
<...>
```

11.2.4 USBH_FT232_RemoveNotification()

Description

Removes a callback added via USBH_FT232_AddNotification.

Prototype

```
USBH_STATUS USBH_FT232_RemoveNotification(const USBH_NOTIFICATION_HOOK * pHook);
```

Parameters

Parameter	Description
<code>pHook</code>	Pointer to a user provided USBH_NOTIFICATION_HOOK variable.

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

11.2.5 USBH_FT232_RegisterNotification()

Description

This function is deprecated, please use function USBH_FT232_AddNotification! Sets a callback in order to be notified when a device is added or removed.

Prototype

```
void USBH_FT232_RegisterNotification(USBH_NOTIFICATION_FUNC * pfNotification,  
                                     void * pContext);
```

Parameters

Parameter	Description
<code>pfNotification</code>	Pointer to a function the stack should call when a device is connected or disconnected.
<code>pContext</code>	Pointer to a user context that is passed to the callback function.

Additional information

This function is deprecated, please use function USBH_FT232_AddNotification.

11.2.6 USBH_FT232_AddCustomDeviceMask()

Description

This function allows the FT232 module to receive notifications about devices which do not present themselves with FTDI’s vendor ID (0x0403).

Prototype

```
USBH_STATUS USBH_FT232_AddCustomDeviceMask(const U16 * pVendorIds,
                                             const U16 * pProductIds,
                                             U16 NumIds);
```

Parameters

Parameter	Description
pVendorIds	Array of vendor IDs.
pProductIds	Array of product IDs.
NumIds	Number of elements in both arrays, each index in both arrays is used as a pair to create a filter.

Return value

USBH_STATUS_SUCCESS	Success.
USBH_STATUS_ERROR	Notification could not be registered.

11.2.7 USBH_FT232_ConfigureDefaultTimeout()

Description

Sets the default read and write timeout that shall be used when a new device is connected.

Prototype

```
void USBH_FT232_ConfigureDefaultTimeout(U32 ReadTimeout,  
                                         U32 WriteTimeout);
```

Parameters

Parameter	Description
ReadTimeout	Default read timeout given in ms.
WriteTimeout	Default write timeout given in ms.

Additional information

The function shall be called after `USBH_FT232_Init()` has been called, otherwise the behavior is undefined.

11.2.8 USBH_FT232_Open()

Description

Opens a device given by an index.

Prototype

```
USBH_FT232_HANDLE USBH_FT232_Open(unsigned Index);
```

Parameters

Parameter	Description
Index	Index of the device that shall be opened. In general this means: the first connected device is 0, second device is 1 etc.

Return value

≠ USBH_FT232_INVALID_HANDLE
= USBH_FT232_INVALID_HANDLE

Handle to the device.
Device could not be opened (removed or not available).

11.2.9 USBH_FT232_Close()

Description

Closes a handle to an opened device.

Prototype

```
USBH_STATUS USBH_FT232_Close(USBH_FT232_HANDLE hDevice);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to a opened device.

Return value

= USBH_STATUS_SUCCESS	Successful.
≠ USBH_STATUS_SUCCESS	An error occurred.

11.2.10 USBH_FT232_GetDeviceInfo()

Description

Retrieves the information about the FT232 device.

Prototype

```
USBH_STATUS USBH_FT232_GetDeviceInfo(USBH_FT232_HANDLE    hDevice,  
                                     USBH_FT232_DEVICE_INFO * pDevInfo);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device.
pDevInfo	out Pointer to a USBH_FT232_DEVICE_INFO structure to store information related to the device.

Return value

= USBH_STATUS_SUCCESS	Successful.
≠ USBH_STATUS_SUCCESS	An error occurred.

11.2.11 USBH_FT232_ResetDevice()

Description

Resets the FT232 device

Prototype

```
USBH_STATUS USBH_FT232_ResetDevice(USBH_FT232_HANDLE hDevice);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to the opened device.

Return value

= USBH_STATUS_SUCCESS	Successful.
≠ USBH_STATUS_SUCCESS	An error occurred.

11.2.12 USBH_FT232_SetTimeouts()

Description

Sets up the timeouts the host waits until the data transfer will be aborted, for a specific FT232 device.

Prototype

```
USBH_STATUS USBH_FT232_SetTimeouts(USBH_FT232_HANDLE hDevice,  
                                   U32                ReadTimeout,  
                                   U32                WriteTimeout);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device.
ReadTimeout	Read time-out given in ms.
WriteTimeout	Write time-out given in ms.

Return value

- = USBH_STATUS_SUCCESS Successful.
- ≠ USBH_STATUS_SUCCESS An error occurred.

11.2.13 USBH_FT232_ConfigurePollDelay()

Description

Configures the delay when polling a FT232 device, which has no data to send. The default is 15ms (recommended by FTDI Chip).

FT232 devices have a minimum delay between two USB transfers in idle state (no data). Setting the polling delay below this value might not work as expected.

Prototype

```
USBH_STATUS USBH_FT232_ConfigurePollDelay(USBH_FT232_HANDLE hDevice,
                                         unsigned PollDelay);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device.
PollDelay	New poll delay in ms. Maximum is 255ms.

Return value

- = USBH_STATUS_SUCCESS Successful.
- ≠ USBH_STATUS_SUCCESS An error occurred.

11.2.14 USBH_FT232_Read()

Description

Reads data from the FT232 device.

Prototype

```
USBH_STATUS USBH_FT232_Read(USBH_FT232_HANDLE hDevice,  
                             U8 * pData,  
                             U32 NumBytes,  
                             U32 * pNumBytesRead);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device.
pData	Pointer to a buffer to store the read data.
NumBytes	Number of bytes to be read from the device.
pNumBytesRead	out Pointer to a variable which receives the number of bytes read from the device.

Return value

= USBH_STATUS_SUCCESS Successful.
≠ USBH_STATUS_SUCCESS An error occurred.

Additional information

USBH_FT232_Read() always returns the number of bytes read in [pNumBytesRead](#). This function does not return until [NumBytes](#) bytes have been read into the buffer unless short read mode is enabled. This allows USBH_FT232_Read() to return when either data have been read from the queue or as soon as some data have been read from the device. The number of bytes in the receive queue can be determined by calling USBH_FT232_GetQueueStatus(), and passed to USBH_FT232_Read() as [NumBytes](#) so that the function reads the data and returns immediately. When a read timeout value has been specified in a previous call to USBH_FT232_SetTimeouts(), USBH_FT232_Read() returns when the timer expires or [NumBytes](#) have been read, whichever occurs first. If the timeout occurs, USBH_FT232_Read() reads available data into the buffer and returns USBH_STATUS_TIMEOUT. An application should use the function return value and [pNumBytesRead](#) when processing the buffer. If the return value is USBH_STATUS_SUCCESS, and [pNumBytesRead](#) is equal to [NumBytes](#) then USBH_FT232_Read has completed normally. If the return value is USBH_STATUS_TIMEOUT, [pNumBytesRead](#) may be less or even 0, in any case, [pData](#) will be filled with [pNumBytesRead](#). Any other return value suggests an error in the parameters of the function, or a fatal error like a USB disconnect.

11.2.15 USBH_FT232_Write()

Description

Writes data to the FT232 device.

Prototype

```
USBH_STATUS USBH_FT232_Write(      USBH_FT232_HANDLE  hDevice,
                                   const U8              * pData,
                                   U32                    NumBytes,
                                   U32                    * pNumBytesWritten);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to the opened device.
<code>pData</code>	<code>in</code> Pointer to data to be sent.
<code>NumBytes</code>	Number of bytes to write to the device.
<code>pNumBytesWritten</code>	<code>out</code> Pointer to a variable which receives the number of bytes written to the device.

Return value

- = USBH_STATUS_SUCCESS

Successful.
- ≠ USBH_STATUS_SUCCESS

An error occurred.

11.2.16 USBH_FT232-AllowShortRead()

Description

The configuration function allows to let the read function to return as soon as data are available.

Prototype

```
USBH_STATUS USBH_FT232-AllowShortRead(USBH_FT232_HANDLE hDevice,  
                                       U8 AllowShortRead);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device.
AllowShortRead	Define whether short read mode shall be used or not. 1 - Allow short read. 0 - Short read mode disabled.

Return value

= USBH_STATUS_SUCCESS Successful.
≠ USBH_STATUS_SUCCESS An error occurred.

Additional information

USBH_FT232-AllowShortRead() sets the USBH_FT232_Read() into a special mode - short read mode. When this mode is enabled, the function returns as soon as any data has been read from the device. This allows the application to read data where the number of bytes to read is undefined. To disable this mode, [AllowShortRead](#) should be set to 0.

11.2.17 USBH_FT232_SetBaudRate()

Description

Sets the baud rate for the opened device.

Prototype

```
USBH_STATUS USBH_FT232_SetBaudRate(USBH_FT232_HANDLE hDevice,  
                                   U32 BaudRate);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device.
BaudRate	Baudrate to set. The upper limit is either 3,000,000 or 12,000,000 baud depending on the type of FT232 device.

Return value

- = USBH_STATUS_SUCCESS Successful.
- ≠ USBH_STATUS_SUCCESS An error occurred.

11.2.18 USBH_FT232_SetDataCharacteristics()

Description

Setups the serial communication with the given characteristics.

Prototype

```
USBH_STATUS USBH_FT232_SetDataCharacteristics(USBH_FT232_HANDLE hDevice,  
                                              U8 Length,  
                                              U8 StopBits,  
                                              U8 Parity);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device.
Length	Number of bits per word. Must be either USBH_FT232_BITS_8 or USBH_FT232_BITS_7.
StopBits	Number of stop bits. Must be USBH_FT232_STOP_BITS_1 or USBH_FT232_STOP_BITS_2.
Parity	Parity - must be one of the following values: USBH_FT232_PARITY_NONE USBH_FT232_PARITY_ODD USBH_FT232_PARITY_EVEN USBH_FT232_PARITY_MARK USBH_FT232_PARITY_SPACE

Return value

- = USBH_STATUS_SUCCESS

Successful.
- ≠ USBH_STATUS_SUCCESS

An error occurred.

11.2.19 USBH_FT232_SetFlowControl()

Description

This function sets the flow control for the device.

Prototype

```
USBH_STATUS USBH_FT232_SetFlowControl(USBH_FT232_HANDLE hDevice,  
                                       U16                FlowControl,  
                                       U8                 XonChar,  
                                       U8                 XoffChar);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device.
FlowControl	Must be one of the following values: USBH_FT232_FLOW_NONE USBH_FT232_FLOW_RTS_CTS USBH_FT232_FLOW_DTR_DSR USBH_FT232_FLOW_XON_XOFF
XonChar	Character used to signal Xon. Only used if flow control is FT_FLOW_XON_XOFF.
XoffChar	Character used to signal Xoff. Only used if flow control is FT_FLOW_XON_XOFF.

Return value

- = USBH_STATUS_SUCCESS Successful.
- ≠ USBH_STATUS_SUCCESS An error occurred.

11.2.20 USBH_FT232_SetDtr()

Description

Sets the Data Terminal Ready (DTR) control signal.

Prototype

```
USBH_STATUS USBH_FT232_SetDtr(USBH_FT232_HANDLE hDevice);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to the opened device.

Return value

= USBH_STATUS_SUCCESS	Successful.
≠ USBH_STATUS_SUCCESS	An error occurred.

11.2.21 USBH_FT232_ClrDtr()

Description

Clears the Data Terminal Ready (DTR) control signal.

Prototype

```
USBH_STATUS USBH_FT232_ClrDtr(USBH_FT232_HANDLE hDevice);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to the opened device.

Return value

= USBH_STATUS_SUCCESS	Successful.
≠ USBH_STATUS_SUCCESS	An error occurred.

11.2.22 USBH_FT232_SetRts()

Description

Sets the Request To Send (RTS) control signal.

Prototype

```
USBH_STATUS USBH_FT232_SetRts(USBH_FT232_HANDLE hDevice);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to the opened device.

Return value

= USBH_STATUS_SUCCESS	Successful.
≠ USBH_STATUS_SUCCESS	An error occurred.

11.2.23 USBH_FT232_ClrRts()

Description

Clears the Request To Send (RTS) control signal.

Prototype

```
USBH_STATUS USBH_FT232_ClrRts(USBH_FT232_HANDLE hDevice);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to the opened device.

Return value

= USBH_STATUS_SUCCESS	Successful.
≠ USBH_STATUS_SUCCESS	An error occurred.

11.2.24 USBH_FT232_GetModemStatus()

Description

Gets the modem status and line status from the device.

Prototype

```
USBH_STATUS USBH_FT232_GetModemStatus(USBH_FT232_HANDLE hDevice,  
                                       U32 * pModemStatus);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device.
pModemStatus	Pointer to a variable of type U32 which receives the modem status and line status from the device.

Return value

= USBH_STATUS_SUCCESS Successful.
≠ USBH_STATUS_SUCCESS An error occurred.

Additional information

The least significant byte of the [pModemStatus](#) value holds the modem status. The line status is held in the second least significant byte of the [pModemStatus](#) value. The modem status is bit-mapped as follows:

- Clear To Send (CTS) = 0x10
- Data Set Ready (DSR) = 0x20
- Ring Indicator (RI) = 0x40
- Data Carrier Detect (DCD) = 0x80

The line status is bit-mapped as follows:

- Overrun Error (OE) = 0x02
- Parity Error (PE) = 0x04
- Framing Error (FE) = 0x08
- Break Interrupt (BI) = 0x10
- TxHolding register empty = 0x20
- TxEmpty = 0x40

11.2.25 USBH_FT232_SetChars()

Description

Sets the special characters for the device.

Prototype

```
USBH_STATUS USBH_FT232_SetChars(USBH_FT232_HANDLE hDevice,
                                U8                 EventChar,
                                U8                 EventCharEnabled,
                                U8                 ErrorChar,
                                U8                 ErrorCharEnabled);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device.
EventChar	Eventc character.
EventCharEnabled	0, if event character disabled, non-zero otherwise.
ErrorChar	Error character.
ErrorCharEnabled	0, if error character disabled, non-zero otherwise.

Return value

= USBH_STATUS_SUCCESS Successful.
≠ USBH_STATUS_SUCCESS An error occurred.

Additional information

This function allows to insert special characters in the data stream to represent events triggering or errors occurring.

11.2.26 USBH_FT232_Purge()

Description

Purges receive and transmit buffers in the device.

Prototype

```
USBH_STATUS USBH_FT232_Purge(USBH_FT232_HANDLE hDevice,  
                             U32 Mask);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device.
Mask	Combination of USBH_FT232_PURGE_RX and USBH_FT232_FT_PURGE_TX.

Return value

= USBH_STATUS_SUCCESS	Successful.
≠ USBH_STATUS_SUCCESS	An error occurred.

11.2.27 USBH_FT232_GetQueueStatus()

Description

Gets the number of bytes in the receive queue.

Prototype

```
USBH_STATUS USBH_FT232_GetQueueStatus(USBH_FT232_HANDLE hDevice,  
                                       U32 * pRxBytes);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device.
pRxBytes	Pointer to a variable of type U32 which receives the number of bytes in the receive queue.

Return value

= USBH_STATUS_SUCCESS	Successful.
≠ USBH_STATUS_SUCCESS	An error occurred.

11.2.28 USBH_FT232_SetBreakOn()

Description

Sets the BREAK condition for the device.

Prototype

```
USBH_STATUS USBH_FT232_SetBreakOn(USBH_FT232_HANDLE hDevice);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to the opened device.

Return value

= USBH_STATUS_SUCCESS	Successful.
≠ USBH_STATUS_SUCCESS	An error occurred.

11.2.29 USBH_FT232_SetBreakOff()

Description

Resets the BREAK condition for the device.

Prototype

```
USBH_STATUS USBH_FT232_SetBreakOff(USBH_FT232_HANDLE hDevice);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to the opened device.

Return value

= USBH_STATUS_SUCCESS	Successful.
≠ USBH_STATUS_SUCCESS	An error occurred.

11.2.30 USBH_FT232_SetLatencyTimer()

Description

The latency timer controls the timeout for the FTDI device to transfer data from the FT232 interface to the USB interface. The FTDI device transfers data from the FT232 to the USB interface when it receives 62 bytes over FT232 (one full packet with 2 status bytes) or when the latency timeout elapses.

Prototype

```
USBH_STATUS USBH_FT232_SetLatencyTimer(USBH_FT232_HANDLE hDevice,  
                                       U8                Latency);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device.
Latency	Required value, in milliseconds, of latency timer. Valid range is 2 - 255.

Return value

= USBH_STATUS_SUCCESS Successful.
≠ USBH_STATUS_SUCCESS An error occurred.

Additional information

In the FT8U232AM and FT8U245AM devices, the receive buffer timeout that is used to flush remaining data from the receive buffer was fixed at 16 ms. Therefore this function cannot be used with these devices. In all other FTDI devices, this timeout is programmable and can be set at 1 ms intervals between 2ms and 255 ms. This allows the device to be better optimized for protocols requiring faster response times from short data packets.

11.2.31 USBH_FT232_GetLatencyTimer()

Description

Get the current value of the latency timer.

Prototype

```
USBH_STATUS USBH_FT232_GetLatencyTimer(USBH_FT232_HANDLE hDevice,  
                                         U8 * pLatency);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device.
pLatency	Pointer to a value which receives the device latency setting.

Return value

= USBH_STATUS_SUCCESS Successful.
≠ USBH_STATUS_SUCCESS An error occurred.

Additional information

Please refer to `USBH_FT232_SetLatencyTimer()` for more information about the latency timer.

11.2.32 USBH_FT232_SetBitMode()

Description

Enables different chip modes.

Prototype

```
USBH_STATUS USBH_FT232_SetBitMode(USBH_FT232_HANDLE hDevice,
                                   U8 Mask,
                                   U8 Enable);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device.
Mask	Required value for bit mode mask. This sets up which bits are inputs and outputs. A bit value of 0 sets the corresponding pin to an input. A bit value of 1 sets the corresponding pin to an output. In the case of CBUS Bit Bang, the upper nibble of this value controls which pins are inputs and outputs, while the lower nibble controls which of the outputs are high and low.
Enable	Mode value. Can be one of the following values: • 0x00 = Reset • 0x01 = Asynchronous Bit Bang • 0x02 = MPSSE (FT2232, FT2232H, FT4232H and FT232H devices only) • 0x04 = Synchronous Bit Bang (FT232R, FT245R, FT2232, FT2232H, FT4232H and FT232H devices only) • 0x08 = MCU Host Bus Emulation Mode (FT2232, FT2232H, FT4232H and FT232H devices only) • 0x10 = Fast Opto-Isolated Serial Mode (FT2232, FT2232H, FT4232H and FT232H devices only) • 0x20 = CBUS Bit Bang Mode (FT232R and FT232H devices only) • 0x40 = Single Channel Synchronous 245 FIFO Mode (FT2232H and FT232H devices only).

Return value

= USBH_STATUS_SUCCESS Successful.
 ≠ USBH_STATUS_SUCCESS An error occurred.

Additional information

For further information please refer to the HW-reference manuals and application note on the FTDI website.

11.2.33 USBH_FT232_GetBitMode()

Description

Returns the current values on the data bus pins. This function does NOT return the configured mode.

Prototype

```
USBH_STATUS USBH_FT232_GetBitMode(USBH_FT232_HANDLE hDevice,  
                                   U8 * pMode);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device.
pMode	Pointer to a U8 variable to store the current value.

Return value

= USBH_STATUS_SUCCESS	Successful.
≠ USBH_STATUS_SUCCESS	An error occurred.

11.2.34 USBH_FT232_GetInterfaceHandle()

Description

Return the handle to the (open) USB interface. Can be used to call USBH core functions like USBH_GetStringDescriptor().

Prototype

```
USBH_INTERFACE_HANDLE USBH_FT232_GetInterfaceHandle(USBH_FT232_HANDLE hDevice);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to an open device returned by USBH_FT232_Open().

Return value

Handle to an open interface.

11.3 Data structures

This chapter describes the emUSB-Host FT232 driver data structures.

Function	Description
USBH_FT232_DEVICE_INFO	Contains information about an FT232 device.

11.3.1 USBH_FT232_DEVICE_INFO

Description

Contains information about an FT232 device.

Type definition

```
typedef struct {  
    U16      VendorId;  
    U16      ProductId;  
    U16      bcdDevice;  
    USBH_SPEED Speed;  
    U16      MaxPacketSize;  
} USBH_FT232_DEVICE_INFO;
```

Structure members

Member	Description
VendorId	USB Vendor Id.
ProductId	USB Product Id.
bcdDevice	The BCD coded device version.
Speed	Operation speed of the device. See USBH_SPEED.
MaxPacketSize	Maximum size of a single packet in bytes.

Chapter 12

FT260 Device Driver (Add-On)

This chapter describes the optional emUSB-Host add-on “FT260 device driver”. It allows communication with an FTDI FT260 USB device, typically serving as USB to I²C converter.



12.1 Introduction

The FT260 driver software component of emUSB-Host allows the communication with FTDI FT260 devices. It implements an HID based protocol specified by FTDI. The protocol allows use the FT260 as I²C master to communicate with different I²C slave devices. UART communication is also available. This chapter provides an explanation of the functions available to application developers via the FT260 driver software. All the functions and data types of this add-on are prefixed with the "USBH_FT260_" text.

12.1.1 Features

The following features are provided:

- Compatibility with different FT260 devices.
- Ability to use I²C or UART operation.
- Ability to set up speed for the I²C bus.
- Ability to set various parameters, such as baudrate, number of stop bits, parity for UART.
- Handling of multiple FT260 devices at the same time.
- Ability to query the FT260 UART line and modem status.

12.1.2 Example code

An example application which uses the API is provided in the `USBH_FT260_Start.c` file. This example displays the dump of the EEPROM that is available on device address 0x50 on the UMFT260EV1A board.

12.1.3 Compatibility

The following devices work with the current FT260 driver:

- FT260Q
- FT260S

12.1.4 Further reading

For more information about the FTDI FT260 devices, please take a look at the hardware manual and AN_394 User Guide for FT260 (Document Reference No.: FT_001279) available from www.ftdichip.com.

12.2 API Functions

This chapter describes the emUSB-Host FT260 driver API functions.

Function	Description
USBH_FT260_Init()	Initialize the FT260 module and register the module within the HID class.
USBH_FT260_AddSupportItem()	Adds a VID/PID pair to the supported list.
USBH_FT260_RemoveSupportItem()	Removes a VID/PID pair to the supported list.
USBH_FT260_AddNotification()	Adds a callback to be notified when a device or interface is added or removed.
USBH_FT260_RemoveNotification()	Removes a callback added via USBH_HID_FT260_AddNotification .
USBH_FT260_Open()	Opens a device given by an index.
USBH_FT260_Close()	Closes a handle to opened FT260 device/interface.
USBH_FT260_SetClock()	Sets the system clock of the chip.
USBH_FT260_SetWakeupInterrupt()	Enables or disables the wakeup interrupt feature of the device/interface.
USBH_FT260_SetInterruptTriggerType()	Sets the parameter for the interrupt trigger.
USBH_FT260_SelectGpio2Function()	Sets the GPIO[2] function.
USBH_FT260_SelectGpioAFunction()	Sets the GPIO[A] function.
USBH_FT260_SelectGpioGFunction()	Sets the GPIO[G] function.
USBH_FT260_SetSuspendOutPolarity()	Sets polarity of the Suspend out pin.
USBH_FT260_GetChipVersion()	Gets the version of the FTDI chip.
USBH_FT260_EnableI2CPin()	Enables or disables the I2C pins of the dedicated device or interface.
USBH_FT260_SetUartToGPIOPin()	Sets UART pins to generic GPIO pins.
USBH_FT260_EnableDcdRiPin()	Enable the DCD and RI UART pins.
USBH_FT260_I2CMaster_Init()	Initialize the I2C function of the device/interface and setups the I2C bus speed.
USBH_FT260_I2CMaster_Read()	Reads data from a given I2C slave device.
USBH_FT260_I2CMaster_Write()	Writes data to a given I2C slave device.
USBH_FT260_I2CMaster_ReadReg()	Reads a register address from a specific I2C device address.
USBH_FT260_I2CMaster_WriteReg()	Writes to a register address from a specific I2C device address.
USBH_FT260_I2CMaster_GetStatus()	Gets the current I2C status.
USBH_FT260_I2CMaster_Reset()	Performs a reset of the I2C internal controller.
USBH_FT260_UART_Init()	Initialize the UART module of the device/interface.
USBH_FT260_UART_SetBaudRate()	Sets the UART baud rate.
USBH_FT260_UART_SetFlowControl()	Sets the UART flow control.
USBH_FT260_UART_SetDataCharacteristics()	Setups the UART based on the given data characteristics.
USBH_FT260_UART_SetBreak()	Sets UART break state.

Function	Description
USBH_FT260_UART_SetXonXoffChar()	Sets XON XOFF character for XON/XOFF flow control.
USBH_FT260_UART_GetConfig()	Retrieves the device's or interfaces's UART configuration.
USBH_FT260_UART_Read()	Reads data from the UART.
USBH_FT260_UART_Write()	Writes data to the UART.
USBH_FT260_UART_Reset()	Resets the UART.
USBH_FT260_UART_GetDcdRiStatus()	Get the UART DCD RI status.
USBH_FT260_UART_EnableRiWakeup()	Enables or disables the UART RI wake-up.
USBH_FT260_UART_SetRiWakeupConfig()	Setups the UART RI wake-up type.
USBH_FT260_GetInterruptFlag()	Interrupt is transmitted by UART interface.
USBH_FT260_CleanInterruptFlag()	Cleans the interrupt flag.
USBH_FT260_GPIO_Set()	Sets the GPIO pin status based on the USBH_FT260_GPIO_REPORT structure.
USBH_FT260_GPIO_Get()	Gets the GPIO pin status.
USBH_FT260_GPIO_SetDir()	Sets the direction for a specific GPIO pin.
USBH_FT260_GPIO_Read()	Retrieves the pin state for a specified GPIO pin.
USBH_FT260_GPIO_Write()	Sets the pin state for a specified GPIO pin.
USBH_FT260_GPIO_Set_OD()	Set GPIO open drain.
USBH_FT260_GPIO_Reset_OD()	Resets the open drain state for all pins.

12.2.1 USBH_FT260_Init()

Description

Initialize the FT260 module and register the module within the HID class.

Prototype

```
void USBH_FT260_Init(void);
```

12.2.2 USBH_FT260_AddSupportItem()

Description

Adds a VID/PID pair to the supported list.

Prototype

```
void USBH_FT260_AddSupportItem(USBH_FT260_VID_PID_PAIR * pItem);
```

Parameters

Parameter	Description
<code>pItem</code>	Pointer to the item to be added.

12.2.3 USBH_FT260_RemoveSupportItem()

Description

Removes a VID/PID pair to the supported list.

Prototype

```
void USBH_FT260_RemoveSupportItem(USBH_FT260_VID_PID_PAIR * pItem);
```

Parameters

Parameter	Description
<code>pItem</code>	Pointer to the item to be removed.

12.2.4 USBH_FT260_AddNotification()

Description

Adds a callback to be notified when a device or interface is added or removed.

Prototype

```
USBH_STATUS USBH_FT260_AddNotification(USBH_NOTIFICATION_HOOK * pHook,
                                       USBH_NOTIFICATION_FUNC * pfNotification,
                                       void * pContext);
```

Parameters

Parameter	Description
<code>pHook</code>	Pointer to a <code>USBH_NOTIFICATION_HOOK</code> structure.
<code>pfNotification</code>	Pointer to the function/callback.
<code>pContext</code>	Pointer to a user-defined context.

Return value

= `USBH_STATUS_SUCCESS` Operation was successful.
 ≠ `USBH_STATUS_SUCCESS` Operation was not successful.

Example

```
static USBH_NOTIFICATION_HOOK _Hook;

/*****
 *
 *      _cbOnAddRemoveDevice
 *
 * Function description
 * Callback, called when a device is added or removed.
 * Call in the context of the USBH_Task.
 * The functionality in this routine should not block
 */
static void _cbOnAddRemoveDevice(void * pContext, U8 DevIndex, USBH_DEVICE_EVENT Event) {
    (void)pContext;
    switch (Event) {
        case USBH_DEVICE_EVENT_ADD:
            USBH_Logf_Application("**** Device added\n");
            _DevIndex = DevIndex;
            _DevIsReady = 1;
            break;
        case USBH_DEVICE_EVENT_REMOVE:
            USBH_Logf_Application("**** Device removed\n");
            _DevIsReady = 0;
            _DevIndex = -1;
            break;
        default:; // Should never happen
    }
}

<...>
USBH_FT260_Init();
USBH_FT260_AddNotification(&_Hook, _cbOnAddRemoveDevice, NULL);
<...>
```

12.2.5 USBH_FT260_RemoveNotification()

Description

Removes a callback added via USBH_HID_FT260_AddNotification.

Prototype

```
USBH_STATUS USBH_FT260_RemoveNotification(const USBH_NOTIFICATION_HOOK * pHook);
```

Parameters

Parameter	Description
<code>pHook</code>	Pointer to the hook structure to be removed.

Return value

= USBH_STATUS_SUCCESS	Operation was successful.
≠ USBH_STATUS_SUCCESS	Operation was not successful.

12.2.6 USBH_FT260_Open()

Description

Opens a device given by an index.

Prototype

```
USBH_HID_HANDLE USBH_FT260_Open(unsigned DevIndex);
```

Parameters

Parameter	Description
<code>DevIndex</code>	Zero-based index of the device/interface.

Return value

= USBH_HID_INVALID_HANDLE Could not open the device.
≠ USBH_HID_INVALID_HANDLE A valid HID handle to the device/interface.

Additional information

Since the handle is identical to the HID module thus any USBH_HID_* function can be used.

12.2.7 USBH_FT260_Close()

Description

Closes a handle to opened FT260 device/interface.

Prototype

```
USBH_STATUS USBH_FT260_Close(USBH_HID_HANDLE hDevice);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to the opened device/interface.

Return value

= USBH_STATUS_SUCCESS	Operation was successful.
≠ USBH_STATUS_SUCCESS	Operation was not successful.

12.2.8 USBH_FT260_SetClock()

Description

Sets the system clock of the chip.

Prototype

```
USBH_STATUS USBH_FT260_SetClock(USBH_HID_HANDLE      hDevice,  
                                USBH_FT260_CLOCK_RATE ClkRate);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device/interface.
ClkRate	The following clock rate values are available: USBH_FT260_SYS_CLK_12M USBH_FT260_SYS_CLK_24M USBH_FT260_SYS_CLK_48M

Return value

- = USBH_STATUS_SUCCESS

≠ USBH_STATUS_SUCCESS
- Operation was successful.

Operation was not successful.

12.2.9 USBH_FT260_SetWakeupInterrupt()

Description

Enables or disables the wakeup interrupt feature of the device/interface.

Prototype

```
USBH_STATUS USBH_FT260_SetWakeupInterrupt(USBH_HID_HANDLE hDevice,  
                                           unsigned int    Enable);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to the opened device/interface.
<code>Enable</code>	Value to be set: 1 : Enables wake-up support. 0 : Disables wake-up support.

Return value

= USBH_STATUS_SUCCESS	Operation was successful.
≠ USBH_STATUS_SUCCESS	Operation was not successful.

12.2.10 USBH_FT260_SetInterruptTriggerType()

Description

Sets the parameter for the interrupt trigger.

Prototype

```
USBH_STATUS USBH_FT260_SetInterruptTriggerType
(USBH_HID_HANDLE hDevice,
 USBH_FT260_INTERRUPT_TRIGGER_TYPE Type,
 USBH_FT260_INTERRUPT_LEVEL_TIME_DELAY Delay);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device/interface.
Type	For Type the following values are valid: USBH_FT260_INTR_RISING_EDGE USBH_FT260_INTR_LEVEL_HIGH USBH_FT260_INTR_FALLING_EDGE USBH_FT260_INTR_LEVEL_LOW
Delay	Delay can be one of the following values: USBH_FT260_INTR_DELY_1MS USBH_FT260_INTR_DELY_5MS USBH_FT260_INTR_DELY_30MS

Return value

- = USBH_STATUS_SUCCESS

Operation was successful.
- ≠ USBH_STATUS_SUCCESS

Operation was not successful.

12.2.11 USBH_FT260_SelectGpio2Function()

Description

Sets the GPIO[2] function.

Prototype

```
USBH_STATUS USBH_FT260_SelectGpio2Function(USBH_HID_HANDLE hDevice,  
                                           USBH_FT260_GPIO2_PIN Gpio2Function);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device/interface.
Gpio2Function	For GPIO[2] the following values can be used: USBH_FT260_GPIO2_GPIO USBH_FT260_GPIO2_SUSP0UT USBH_FT260_GPIO2_PWREN USBH_FT260_GPIO2_TX_LED

Return value

- = USBH_STATUS_SUCCESS Operation was successful.
- ≠ USBH_STATUS_SUCCESS Operation was not successful.

12.2.12 USBH_FT260_SelectGpioAFunction()

Description

Sets the GPIO[A] function.

Prototype

```
USBH_STATUS USBH_FT260_SelectGpioAFunction(USBH_HID_HANDLE hDevice,  
                                           USBH_FT260_GPIOA_PIN GpioAFunction);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device/interface.
GpioAFunction	For GPIO[A] the following values can be used: USBH_FT260_GPIOA_GPIO USBH_FT260_GPIOA_TX_ACTIVE USBH_FT260_GPIOA_TX_LED

Return value

- = USBH_STATUS_SUCCESS Operation was successful.
- ≠ USBH_STATUS_SUCCESS Operation was not successful.

12.2.13 USBH_FT260_SelectGpioGFunction()

Description

Sets the GPIO[G] function.

Prototype

```
USBH_STATUS USBH_FT260_SelectGpioGFunction(USBH_HID_HANDLE    hDevice,  
                                           USBH_FT260_GPIOG_PIN GpioGFunction);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device/interface.
GpioGFunction	For GPIO[G] the following values can be used: USBH_FT260_GPIOG_GPIO USBH_FT260_GPIOG_PWREN USBH_FT260_GPIOG_RX_LED USBH_FT260_GPIOG_BCD_DET

Return value

= USBH_STATUS_SUCCESS	Operation was successful.
≠ USBH_STATUS_SUCCESS	Operation was not successful.

12.2.14 USBH_FT260_SetSuspendOutPolarity()

Description

Sets polarity of the Suspend out pin.

Prototype

```
USBH_STATUS USBH_FT260_SetSuspendOutPolarity
(USBH_HID_HANDLE hDevice,
 USBH_FT260_SUSPEND_OUT_POLARITY Polarity);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device/interface.
Polarity	Polarity can be one of the following values: USBH_FT260_SUSPEND_OUT_LEVEL_HIGH USBH_FT260_SUSPEND_OUT_LEVEL_LOW

Return value

- = USBH_STATUS_SUCCESS

Operation was successful.
- ≠ USBH_STATUS_SUCCESS

Operation was not successful.

12.2.15 USBH_FT260_GetChipVersion()

Description

Gets the version of the FTDI chip.

Prototype

```
USBH_STATUS USBH_FT260_GetChipVersion(USBH_HID_HANDLE hDevice,  
                                       U32             * pChipVersion);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device/interface.
pChipVersion	Pointer to a U32 variable to store the chip version.

Return value

= USBH_STATUS_SUCCESS	Operation was successful.
≠ USBH_STATUS_SUCCESS	Operation was not successful.

12.2.16 USBH_FT260_EnableI2CPin()

Description

Enables or disables the I2C pins of the dedicated device or interface.

Prototype

```
USBH_STATUS USBH_FT260_EnableI2CPin(USBH_HID_HANDLE hDevice,  
                                     unsigned int    Enable);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to the opened device/interface.
<code>Enable</code>	Value to be set: 1 : Enables the I2C pins. 0 : Disables the I2C pins.

Return value

= USBH_STATUS_SUCCESS	Operation was successful.
≠ USBH_STATUS_SUCCESS	Operation was not successful.

12.2.17 USBH_FT260_SetUartToGPIOPin()

Description

Sets UART pins to generic GPIO pins.

Prototype

```
USBH_STATUS USBH_FT260_SetUartToGPIOPin(USBH_HID_HANDLE hDevice);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to the opened device/interface.

Return value

= USBH_STATUS_SUCCESS	Operation was successful.
≠ USBH_STATUS_SUCCESS	Operation was not successful.

12.2.18 USBH_FT260_EnableDcdRiPin()

Description

Enable the DCD and RI UART pins.

Prototype

```
USBH_STATUS USBH_FT260_EnableDcdRiPin(USBH_HID_HANDLE hDevice,  
                                       unsigned int    Enable);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device/interface.
Enable	Value to be set: 1 : Enables the I2C pins. 0 : Disables the I2C pins.

Return value

- = USBH_STATUS_SUCCESS

≠ USBH_STATUS_SUCCESS
- Operation was successful.

Operation was not successful.

12.2.19 USBH_FT260_I2CMaster_Init()

Description

Initialize the I2C function of the device/interface and setups the I2C bus speed.

Prototype

```
USBH_STATUS USBH_FT260_I2CMaster_Init(USBH_HID_HANDLE hDevice,  
                                       U16              kbps);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device/interface.
kbps	I2C bus speed given in kbps .

Return value

= USBH_STATUS_SUCCESS	Operation was successful.
≠ USBH_STATUS_SUCCESS	Operation was not successful.

12.2.20 USBH_FT260_I2CMaster_Read()

Description

Reads data from a given I2C slave device.

Prototype

```
USBH_STATUS USBH_FT260_I2CMaster_Read(USBH_HID_HANDLE    hDevice,
                                       U8                  DeviceAddress,
                                       USBH_FT260_I2C_FLAG I2cFlag,
                                       void                * pBuffer,
                                       U32                  NumBytesToRead,
                                       U32                  * pNumBytesReturned);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device/interface.
DeviceAddress	Device address.
I2cFlag	FT260 specific I2C transfer flags.
pBuffer	Pointer to a buffer to be store the read data.
NumBytesToRead	Number of bytes to be read.
pNumBytesReturned	Pointer to a U32 variable storing the Number of bytes read.

Return value

- = USBH_STATUS_SUCCESS Operation was successful.
- ≠ USBH_STATUS_SUCCESS Operation was not successful.

12.2.21 USBH_FT260_I2CMaster_Write()

Description

Writes data to a given I2C slave device.

Prototype

```
USBH_STATUS USBH_FT260_I2CMaster_Write(USBH_HID_HANDLE    hDevice,
                                         U8                DeviceAddress,
                                         USBH_FT260_I2C_FLAG I2cFlag,
                                         void              * pBuffer,
                                         U32                NumBytesToWrite,
                                         U32                * pNumBytesWritten);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device/interface.
DeviceAddress	I2C slave address.
I2cFlag	FT260 specific I2C transfer flags.
pBuffer	Pointer to a buffer containing the register data to be written.
NumBytesToWrite	Number of bytes to write.
pNumBytesWritten	The number bytes that have been written.

Return value

- = USBH_STATUS_SUCCESS

Operation was successful.
- ≠ USBH_STATUS_SUCCESS

Operation was not successful.

12.2.22 USBH_FT260_I2CMaster_ReadReg()

Description

Reads a register address from a specific I2C device address.

Prototype

```
USBH_STATUS USBH_FT260_I2CMaster_ReadReg(USBH_HID_HANDLE    hDevice,
                                           U8                 DeviceAddress,
                                           USBH_FT260_I2C_FLAG I2cFlag,
                                           void*              * pRegAddr,
                                           U32                 RegAddrSize,
                                           void*              * pBuffer,
                                           U32                 NumBytesToRead,
                                           U32                 * pNumBytesReturned);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device/interface.
DeviceAddress	Device address.
I2cFlag	FT260 specific I2C transfer flags.
pRegAddr	Pointer to variable containing the register number to be read.
RegAddrSize	Size of the register variable.
pBuffer	Pointer to a buffer containing to store to the register.
NumBytesToRead	Number of bytes to read.
pNumBytesReturned	The number bytes that have been read.

Return value

- = USBH_STATUS_SUCCESS Operation was successful.
- ≠ USBH_STATUS_SUCCESS Operation was not successful.

12.2.23 USBH_FT260_I2CMaster_WriteReg()

Description

Writes to a register address from a specific I2C device address.

Prototype

```
USBH_STATUS USBH_FT260_I2CMaster_WriteReg(USBH_HID_HANDLE    hDevice,
                                           U8                  DeviceAddress,
                                           USBH_FT260_I2C_FLAG I2cFlag,
                                           void                * pRegAddr,
                                           U32                  RegAddrSize,
                                           void                * pBuffer,
                                           U32                  NumBytesToWrite,
                                           U32                  * pNumBytesWritten);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device/interface.
DeviceAddress	I2C slave address.
I2cFlag	FT260 specific I2C transfer flags.
pRegAddr	Pointer to variable containing the register number to be written.
RegAddrSize	Size of the register variable.
pBuffer	Pointer to a buffer containing the register data to be written.
NumBytesToWrite	Number of bytes to write.
pNumBytesWritten	The number bytes that have been written.

Return value

= USBH_STATUS_SUCCESS Operation was successful.
≠ USBH_STATUS_SUCCESS Operation was not successful.

12.2.24 USBH_FT260_I2CMaster_GetStatus()

Description

Gets the current I2C status.

Prototype

```
USBH_STATUS USBH_FT260_I2CMaster_GetStatus(USBH_HID_HANDLE hDevice,  
                                             U8 * pStatus);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device/interface.
pStatus	Pointer to a U8 Variable to store the I2C status: : The status can be an or combination of the following: USBH_FT260_I2CM_STATUS_CONTROLLER_BUSY - Controller busy: all other status bits invalid USBH_FT260_I2CM_STATUS_ERROR_CONDITION - Error condition USBH_FT260_I2CM_STATUS_SLAVE_NACK - Slave address was not acknowledged during last operation USBH_FT260_I2CM_STATUS_DATA_NACK - Data not acknowledged during last operation USBH_FT260_I2CM_STATUS_ARBITRATION_LOST - Arbitration lost during last operation USBH_FT260_I2CM_STATUS_CONTROLLER_IDLE - Controller idle USBH_FT260_I2CM_STATUS_BUS_BUSY - Bus busy

Return value

- = USBH_STATUS_SUCCESS

≠ USBH_STATUS_SUCCESS
- Operation was successful.

Operation was not successful.

12.2.25 USBH_FT260_I2CMaster_Reset()

Description

Performs a reset of the I2C internal controller.

Prototype

```
USBH_STATUS USBH_FT260_I2CMaster_Reset(USBH_HID_HANDLE hDevice);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to the opened device/interface.

Return value

= USBH_STATUS_SUCCESS	Operation was successful.
≠ USBH_STATUS_SUCCESS	Operation was not successful.

12.2.26 USBH_FT260_UART_Init()

Description

Initialize the UART module of the device/interface.

Prototype

```
USBH_STATUS USBH_FT260_UART_Init(USBH_HID_HANDLE hDevice);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to the opened device/interface.

Return value

= USBH_STATUS_SUCCESS	Operation was successful.
≠ USBH_STATUS_SUCCESS	Operation was not successful.

12.2.27 USBH_FT260_UART_SetBaudRate()

Description

Sets the UART baud rate.

Prototype

```
USBH_STATUS USBH_FT260_UART_SetBaudRate(USBH_HID_HANDLE hDevice,  
                                         U32             BaudRate);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device/interface.
BaudRate	Specifies the desired UART baud rate.

Return value

= USBH_STATUS_SUCCESS	Operation was successful.
≠ USBH_STATUS_SUCCESS	Operation was not successful.

12.2.28 USBH_FT260_UART_SetFlowControl()

Description

Sets the UART flow control.

Prototype

```
USBH_STATUS USBH_FT260_UART_SetFlowControl(USBH_HID_HANDLE hDevice,  
                                             USBH_FT260_UART_MODE FlowControl);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device/interface.
FlowControl	The following flow controls are available: USBH_FT260_UART_OFF - Disables the UART module. USBH_FT260_UART_RTS_CTS_MODE - Use hardware flow control RTS, CTS mode. USBH_FT260_UART_DTR_DSR_MODE - Use hardware flow control DTR, DSR mode. USBH_FT260_UART_XON_XOFF_MODE, - Use software flow control mode. USBH_FT260_UART_NO_FLOW_CTRL_MODE - Do not use any flow control mode.

Return value

- = USBH_STATUS_SUCCESS

≠ USBH_STATUS_SUCCESS
- Operation was successful.

Operation was not successful.

12.2.29 USBH_FT260_UART_SetDataCharacteristics()

Description

Setups the UART based on the given data characteristics.

Prototype

```
USBH_STATUS USBH_FT260_UART_SetDataCharacteristics(USBH_HID_HANDLE    hDevice,  
                                                    USBH_FT260_DATA_BIT DataBits,  
                                                    USBH_FT260_STOP_BIT StopBits,  
                                                    USBH_FT260_PARITY   Parity);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device/interface.
DataBits	The following data bits are available: USBH_FT260_DATA_BIT_7 - Use 7 bit mode. USBH_FT260_DATA_BIT_8 - use 8 bit mode.
StopBits	The desired stop bits can be the following: USBH_FT260_STOP_BITS_1 - Use one stop bit. USBH_FT260_STOP_BITS_2 - Use two stop bits.
Parity	The following parity options are available: USBH_FT260_PARITY_NONE - Use no parity. USBH_FT260_PARITY_ODD, - Use odd parity. USBH_FT260_PARITY_EVEN - Use even parity. USBH_FT260_PARITY_MARK - Use mark parity. USBH_FT260_PARITY_SPACE - Use space parity.

Return value

= USBH_STATUS_SUCCESS	Operation was successful.
≠ USBH_STATUS_SUCCESS	Operation was not successful.

12.2.30 USBH_FT260_UART_SetBreak()

Description

Sets UART break state.

Prototype

```
USBH_STATUS USBH_FT260_UART_SetBreak(USBH_HID_HANDLE hDevice,  
                                     unsigned          OnOff);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to the opened device/interface.
<code>OnOff</code>	1 - Sets the break. 0 - Unsets the break.

Return value

= USBH_STATUS_SUCCESS	Operation was successful.
≠ USBH_STATUS_SUCCESS	Operation was not successful.

12.2.31 USBH_FT260_UART_SetXonXoffChar()

Description

Sets XON XOFF character for XON/XOFF flow control.

Prototype

```
USBH_STATUS USBH_FT260_UART_SetXonXoffChar(USBH_HID_HANDLE hDevice,  
                                             U8 Xon,  
                                             U8 Xoff);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device/interface.
Xon	XON character to be used.
Xoff	XOFF character to be used.

Return value

- = USBH_STATUS_SUCCESS

Operation was successful.
- ≠ USBH_STATUS_SUCCESS

Operation was not successful.

12.2.32 USBH_FT260_UART_GetConfig()

Description

Retrieves the device's or interfaces's UART configuration.

Prototype

```
USBH_STATUS USBH_FT260_UART_GetConfig(USBH_HID_HANDLE hDevice,  
                                       USBH_FT260_UART_CONFIG * pUartConfig);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to the opened device/interface.
<code>pUartConfig</code>	Pointer to a <code>USBH_FT260_UART_CONFIG</code> structure to store the UART configuration.

Return value

= <code>USBH_STATUS_SUCCESS</code>	Operation was successful.
≠ <code>USBH_STATUS_SUCCESS</code>	Operation was not successful.

12.2.33 USBH_FT260_UART_Read()

Description

Reads data from the UART.

Prototype

```
USBH_STATUS USBH_FT260_UART_Read(USBH_HID_HANDLE hDevice,  
                                void * pBuffer,  
                                U32 NumBytesToRead,  
                                U32 * pNumBytesReturned);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device/interface.
pBuffer	POinter to a buffer to store the read data.
NumBytesToRead	Size of the buffer.
pNumBytesReturned	Pointer to U32 variable to store the number of bytes returned.

Return value

- = USBH_STATUS_SUCCESS
- Operation was successful.
- ≠ USBH_STATUS_SUCCESS
- Operation was not successful.

12.2.34 USBH_FT260_UART_Write()

Description

Writes data to the UART.

Prototype

```
USBH_STATUS USBH_FT260_UART_Write(USBH_HID_HANDLE hDevice,
                                   void * pBuffer,
                                   U32 NumBytesToWrite,
                                   U32 * pNumBytesWritten);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device/interface.
pBuffer	Pointer to a buffer containing the data to write.
NumBytesToWrite	Number of bytes to write.
pNumBytesWritten	Pointer to a U32 variable to store the number of bytes written.

Return value

- = USBH_STATUS_SUCCESS

≠ USBH_STATUS_SUCCESS
- Operation was successful.

Operation was not successful.

12.2.35 USBH_FT260_UART_Reset()

Description

Resets the UART.

Prototype

```
USBH_STATUS USBH_FT260_UART_Reset(USBH_HID_HANDLE hDevice);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to the opened device/interface.

Return value

= USBH_STATUS_SUCCESS	Operation was successful.
≠ USBH_STATUS_SUCCESS	Operation was not successful.

12.2.36 USBH_FT260_UART_GetDcdRiStatus()

Description

Get the UART DCD RI status.

Prototype

```
USBH_STATUS USBH_FT260_UART_GetDcdRiStatus(USBH_HID_HANDLE hDevice,  
                                             U8 * pVal);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to the opened device/interface.
<code>pVal</code>	Pointer to a U8 variable to store the DCD RI status: bit[0] - DCD. bit[1] - RI.

Return value

= USBH_STATUS_SUCCESS	Operation was successful.
≠ USBH_STATUS_SUCCESS	Operation was not successful.

12.2.37 USBH_FT260_UART_EnableRiWakeup()

Description

Enables or disables the UART RI wake-up.

Prototype

```
USBH_STATUS USBH_FT260_UART_EnableRiWakeup(USBH_HID_HANDLE hDevice,  
                                             unsigned int Enable);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device/interface.
Enable	Value to be set: 1 : Enables UART RI wake-up. 0 : Disables UART RI wake-up.

Return value

- = USBH_STATUS_SUCCESS

Operation was successful.
- ≠ USBH_STATUS_SUCCESS

Operation was not successful.

12.2.38 USBH_FT260_UART_SetRiWakeupConfig()

Description

Setups the UART RI wake-up type.

Prototype

```
USBH_STATUS USBH_FT260_UART_SetRiWakeupConfig(USBH_HID_HANDLE hDevice,  
                                               USBH_FT260_RI_WAKEUP_TYPE Type);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device/interface.
Type	The type can be one of the following: USBH_FT260_RI_WAKEUP_RISING_EDGE USBH_FT260_RI_WAKEUP_FALLING_EDGE

Return value

- = USBH_STATUS_SUCCESS

Operation was successful.
- ≠ USBH_STATUS_SUCCESS

Operation was not successful.

12.2.39 USBH_FT260_GetInterruptFlag()

Description

Interrupt is transmitted by UART interface.

Prototype

```
USBH_STATUS USBH_FT260_GetInterruptFlag(USBH_HID_HANDLE hDevice,  
                                         unsigned int * pIntFlag);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to the opened device/interface.
<code>pIntFlag</code>	Pointer to a unsigned variable to store the value.

Return value

= USBH_STATUS_SUCCESS	Operation was successful.
≠ USBH_STATUS_SUCCESS	Operation was not successful.

12.2.40 USBH_FT260_CleanInterruptFlag()

Description

Cleans the interrupt flag.

Prototype

```
USBH_STATUS USBH_FT260_CleanInterruptFlag(USBH_HID_HANDLE hDevice,  
                                           unsigned int * pIntFlag);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to the opened device/interface.
<code>pIntFlag</code>	Pointer to a unsigned int variable for storing the interrupt flag value.

Return value

= USBH_STATUS_SUCCESS	Operation was successful.
≠ USBH_STATUS_SUCCESS	Operation was not successful.

12.2.41 USBH_FT260_GPIO_Set()

Description

Sets the GPIO pin status based on the USBH_FT260_GPIO_REPORT structure.

Prototype

```
USBH_STATUS USBH_FT260_GPIO_Set(      USBH_HID_HANDLE      hDevice,  
                                     const USBH_FT260_GPIO_REPORT * pReport);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to the opened device/interface.
<code>pReport</code>	Pointer to USBH_FT260_GPIO_REPORT variable.

Return value

= USBH_STATUS_SUCCESS	Operation was successful.
≠ USBH_STATUS_SUCCESS	Operation was not successful.

12.2.42 USBH_FT260_GPIO_Get()

Description

Gets the GPIO pin status.

Prototype

```
USBH_STATUS USBH_FT260_GPIO_Get(USBH_HID_HANDLE hDevice,  
                                USBH_FT260_GPIO_REPORT * pReport);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device/interface.
pReport	Pointer to USBH_FT260_GPIO_REPORT variable.

Return value

= USBH_STATUS_SUCCESS	Operation was successful.
≠ USBH_STATUS_SUCCESS	Operation was not successful.

12.2.43 USBH_FT260_GPIO_SetDir()

Description

Sets the direction for a specific GPIO pin.

Prototype

```
USBH_STATUS USBH_FT260_GPIO_SetDir(USBH_HID_HANDLE hDevice,  
                                   U16               PinNum,  
                                   U8                Dir);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device/interface.
PinNum	GPIO pin to set.
Dir	Value to be set: 1 : Sets to output mode. 0 : Sets to input mode.

Return value

- = USBH_STATUS_SUCCESS

≠ USBH_STATUS_SUCCESS
- Operation was successful.

Operation was not successful.

12.2.44 USBH_FT260_GPIO_Read()

Description

Retrieves the pin state for a specified GPIO pin.

Prototype

```
USBH_STATUS USBH_FT260_GPIO_Read(USBH_HID_HANDLE hDevice,
                                  U16              PinNum,
                                  U8                * pVal);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device/interface.
PinNum	GPIO pin to get.
pVal	Pointer to U8 variable to store the pin state.

Return value

- = USBH_STATUS_SUCCESS
- Operation was successful.
- ≠ USBH_STATUS_SUCCESS
- Operation was not successful.

12.2.45 USBH_FT260_GPIO_Write()

Description

Sets the pin state for a specified GPIO pin.

Prototype

```
USBH_STATUS USBH_FT260_GPIO_Write(USBH_HID_HANDLE hDevice,  
                                   U16               PinNum,  
                                   U8                Val);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device/interface.
PinNum	GPIO pin to set.
Val	Value to set.

Return value

- = USBH_STATUS_SUCCESS Operation was successful.
- ≠ USBH_STATUS_SUCCESS Operation was not successful.

12.2.46 USBH_FT260_GPIO_Set_OD()

Description

Set GPIO open drain.

Prototype

```
USBH_STATUS USBH_FT260_GPIO_Set_OD(USBH_HID_HANDLE hDevice,  
                                     U8               Pins);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device/interface.
Pins	A GPIO pin bit mask where each pin represents a bit. Mask[0..5] - GPIO[0..5].

Return value

- = USBH_STATUS_SUCCESS Operation was successful.
- ≠ USBH_STATUS_SUCCESS Operation was not successful.

12.2.47 USBH_FT260_GPIO_Reset_OD()

Description

Resets the open drain state for all pins.

Prototype

```
USBH_STATUS USBH_FT260_GPIO_Reset_OD(USBH_HID_HANDLE hDevice);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to the opened device/interface.

Return value

= USBH_STATUS_SUCCESS	Operation was successful.
≠ USBH_STATUS_SUCCESS	Operation was not successful.

12.3 Data structures

This chapter describes the emUSB-Host FT260 driver data structures.

Function	Description
USBH_FT260_VID_PID_PAIR	This structure is used to add a FT260 device that has a different Vendor and Product Id stored in EEPROM.
USBH_FT260_UART_CONFIG	This structure used to retrieve the current UART status of the chip.
USBH_FT260_GPIO_REPORT	This structure is used to retrieve or to set the status of the various GPIO pins of the FT260 chip.

12.3.1 USBH_FT260_VID_PID_PAIR

Description

This structure is used to add a FT260 device that has a different Vendor and Product Id stored in EEPROM. Only the [VendorId](#) and [ProductId](#) members needs to be filled. Anything else is handled by the FT260 module. Note: Do not modify these values after the `USBH_FT260_AddSupportedItem()` was called.

Type definition

```
typedef struct {  
    USBH_DLIST ListEntry;  
    U16 VendorId;  
    U16 ProductId;  
    U32 Magic;  
} USBH_FT260_VID_PID_PAIR;
```

Structure members

Member	Description
ListEntry	Internal use
VendorId	Identifier for the vendor
ProductId	Identifier for the product
Magic	Internal use

12.3.2 USBH_FT260_UART_CONFIG

Description

This structure used to retrieve the current UART status of the chip

Type definition

```
typedef struct {
    U8    FlowCtrl;
    U32   BaudRate;
    U8    DataBit;
    U8    Parity;
    U8    StopBit;
    U8    Breaking;
} USBH_FT260_UART_CONFIG;
```

Structure members

Member	Description
FlowCtrl	Specifies the set flow mode control. The following modes are available: 0x00 : OFF, and switch UART pins to GPIO 0x01 : RTS_CTS mode(GPIOB = > RTSN, GPIOE = > CTSN) 0x02 : DTR_DSR mode(GPIOF = > DTRN, GPIOH = > DSRN) 0x03 : XON_XOFF(software flow control) 0x04 : No flow control mode
BaudRate	UART baud rate The FT260 supports baud rate range from 1200 to 12M.
DataBit	The current data bit mode: 0x07: 7 data bits 0x08: 8 data bits
Parity	The parity that is used by the UART module: 0x00: No parity 0x01: Odd parity. This means that the parity bit is set to either '1' or '0' so that an odd number of 1's are sent. 0x02: Even parity. This means that the parity bit is set to either '1' or '0' so that an even number of 1's are sent. 0x03: Mark parity. This simply means that the parity bit is always High. 0x04: Space parity. This simply means that the parity bit is always Low.
StopBit	The number of stop bits that are used: 0x00: one stop bit 0x02: two stop bits
Breaking	When active the TXD line goes into "spacing" state which causes a break in the receiving UART. 0x00 : no break 0x02 : break

12.3.3 USBH_FT260_GPIO_REPORT

Description

This structure is used to retrieve or to set the status of the various GPIO pins of the FT260 chip. Please refer to the data sheet of the FT260 to check the GPIO capability.

Type definition

```
typedef struct {  
    U8  Value;  
    U8  Dir;  
    U8  GpioN_Value;  
    U8  GpioN_Dir;  
} USBH_FT260_GPIO_REPORT;
```

Structure members

Member	Description
Value	GPIO[0..5] Pin state
Dir	GPIO[0..5] Pin direction whereas: 1 : Output 0 : Input
GpioN_Value	GPIO[A..H] Pin state
GpioN_Dir	GPIO[A..H] Pin direction whereas: 1 : Output 0 : Input

Chapter 13

BULK Device Driver (Add-On)

This chapter describes the optional emUSB-Host add-on “BULK device driver”. It allows communication with a vendor specific USB devices.



13.1 Introduction

The BULK driver software component of emUSB-Host allows communication with vendor specific devices using an arbitrary number of bulk or interrupt endpoints.

This chapter provides an explanation of the functions available to application developers via the BULK driver software. All the functions and data types of this add-on are prefixed with 'USBH_BULK_'.

13.1.1 Overview

A BULK device connected to the emUSB-Host is automatically configured and added to an internal list. If the BULK driver has been registered, it is notified via a callback when a BULK device has been added or removed. The driver then can notify the application program, when a callback function has been registered via `USBH_BULK_RegisterNotification()`. In order to communicate with such a device, the application has to call the `USBH_BULK_Open()`, passing the device index. BULK devices are identified by an index. The first connected device gets assigned the index 0, the second index 1, and so on.

13.1.2 Features

The following features are provided:

- Ability to send and receive data.
- Handling of multiple BULK devices at the same time.
- Notifications about BULK connection status.
- Handling for an arbitrary number of endpoints.

13.1.3 Example code

An example application which uses the API is provided in the `USBH_BULK_Start.c` file. This example demonstrates simple communication between the host and a Bulk device. To run this sample a device programmed with the emUSB-Device sample `USB_BULK_Test.c` is required. The sample demonstrates how to extract the endpoint addresses, which are required by the emUSB-Host BULK API. The sample will send and receive data starting with 1 byte, after each successful echo the number of bytes is increased, up to 1024.

13.2 API Functions

This chapter describes the emUSB-Host BULK driver API functions. These functions are defined in the header file `USBH_BULK.h`.

Function	Description
<code>USBH_BULK_Init()</code>	Initializes and registers the BULK device module with emUSB-Host.
<code>USBH_BULK_Exit()</code>	Unregisters and de-initializes the BULK device module from emUSB-Host.
<code>USBH_BULK_RegisterNotification()</code>	(Deprecated) Sets a callback in order to be notified when a device is added or removed.
<code>USBH_BULK_AddNotification()</code>	Adds a callback in order to be notified when a device is added or removed.
<code>USBH_BULK_RemoveNotification()</code>	Removes a callback registered through <code>USBH_BULK_AddNotification</code> .
<code>USBH_BULK_Open()</code>	Opens a device interface given by an index.
<code>USBH_BULK_Close()</code>	Closes a handle to an opened device.
<code>USBH_BULK-AllowShortRead()</code>	Enables or disables short read mode.
<code>USBH_BULK_GetDeviceInfo()</code>	Retrieves information about the BULK device.
<code>USBH_BULK_GetEndpointInfo()</code>	Retrieves information about an endpoint of a BULK device.
<code>USBH_BULK_Read()</code>	Reads from the BULK device.
<code>USBH_BULK_Receive()</code>	Reads one packet from the device.
<code>USBH_BULK_Write()</code>	Writes data to the BULK device.
<code>USBH_BULK_GetMaxTransferSize()</code>	Return the maximum transfer size allowed for the <code>USBH_BULK_*Async</code> functions.
<code>USBH_BULK_ReadAsync()</code>	Triggers a read transfer to the BULK device.
<code>USBH_BULK_WriteAsync()</code>	Triggers a write transfer to the BULK device.
<code>USBH_BULK_Cancel()</code>	Cancels a running transfer.
<code>USBH_BULK_GetNumBytesInBuffer()</code>	Gets the number of bytes in the receive buffer.
<code>USBH_BULK_SetupRequest()</code>	Sends a specific request (class vendor etc) to the device.
<code>USBH_BULK_IsoDataCtrl()</code>	Acknowledge ISO data received from an IN EP or provide data for OUT EPs.
<code>USBH_BULK_GetInterfaceHandle()</code>	Return the handle to the (open) USB interface.

13.2.1 USBH_BULK_Init()

Description

Initializes and registers the BULK device module with emUSB-Host.

Prototype

```
USBH_STATUS USBH_BULK_Init(const USBH_INTERFACE_MASK * pInterfaceMask);
```

Parameters

Parameter	Description
<code>pInterfaceMask</code>	Deprecated parameter. Please use <code>USBH_BULK_AddNotification</code> to add new interfaces masks. To be backward compatible the mask added through this parameter will be automatically added when <code>USBH_BULK_RegisterNotification</code> is called.

Return value

`USBH_STATUS_SUCCESS` Success or module already initialized.

Additional information

This function can be called multiple times, but only the first call initializes the module. Any further calls only increase the initialization counter. This is useful for cases where the module is initialized from different places which do not interact with each other, To de-initialize the module `USBH_BULK_Exit` has to be called the same number of times as this function was called.

13.2.2 USBH_BULK_Exit()

Description

Unregisters and de-initializes the BULK device module from emUSB-Host.

Prototype

```
void USBH_BULK_Exit(void);
```

Additional information

Before this function is called any notifications added via `USBH_BULK_AddNotification()` must be removed via `USBH_BULK_RemoveNotification()`. Has to be called the same number of times `USBH_BULK_Init` was called in order to de-initialize the module. This function will release resources that were used by this device driver. It has to be called if the application is closed. This has to be called before `USBH_Exit()` is called. No more functions of this module may be called after calling `USBH_BULK_Exit()`. The only exception is `USBH_BULK_Init()`, which would in turn re-init the module and allow further calls.

13.2.3 USBH_BULK_RegisterNotification()

Description

(Deprecated) Sets a callback in order to be notified when a device is added or removed.

Prototype

```
void USBH_BULK_RegisterNotification(USBH_NOTIFICATION_FUNC * pfNotification,  
                                   void * pContext);
```

Parameters

Parameter	Description
<code>pfNotification</code>	Pointer to a function the stack should call when a device is connected or disconnected.
<code>pContext</code>	Pointer to a user context that is passed to the callback function.

Additional information

This function is deprecated, please use function `USBH_BULK_AddNotification()`.

13.2.4 USBH_BULK_RemoveNotification()

Description

Removes a callback registered through USBH_BULK_AddNotification.

Prototype

```
USBH_STATUS USBH_BULK_RemoveNotification(const USBH_NOTIFICATION_HOOK * pHook);
```

Parameters

Parameter	Description
<code>pHook</code>	Pointer to a user provided USBH_NOTIFICATION_HOOK variable.

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

13.2.5 USBH_BULK_AddNotification()

Description

Adds a callback in order to be notified when a device is added or removed.

Prototype

```
USBH_STATUS USBH_BULK_AddNotification
(
    USBH_NOTIFICATION_HOOK * pHook,
    USBH_NOTIFICATION_FUNC * pfNotification,
    void * pContext,
    const USBH_INTERFACE_MASK * pInterfaceMask);
```

Parameters

Parameter	Description
<code>pHook</code>	Pointer to a user provided <code>USBH_NOTIFICATION_HOOK</code> variable.
<code>pfNotification</code>	Pointer to a function the stack should call when a device is connected or disconnected.
<code>pContext</code>	Pointer to a user context that is passed to the callback function.
<code>pInterfaceMask</code>	Pointer to a structure of type <code>USBH_INTERFACE_MASK</code> . NULL means that all interfaces will be forwarded to the callback.

Return value

`USBH_STATUS_SUCCESS` on success or error code on failure.

Example

```
static USBH_NOTIFICATION_HOOK _Hook;

/*****
 *
 *      _cbOnAddRemoveDevice
 *
 *      Function description
 *      Callback, called when a device is added or removed.
 *      Call in the context of the USBH_Task.
 *      The functionality in this routine should not block
 */
static void _cbOnAddRemoveDevice(void * pContext, U8 DevIndex, USBH_DEVICE_EVENT Event) {
    (void)pContext;
    switch (Event) {
        case USBH_DEVICE_EVENT_ADD:
            USBH_Logf_Application("**** Device added\n");
            _DevIndex = DevIndex;
            _DevIsReady = 1;
            break;
        case USBH_DEVICE_EVENT_REMOVE:
            USBH_Logf_Application("**** Device removed\n");
            _DevIsReady = 0;
            _DevIndex = -1;
            break;
        default:; // Should never happen
    }
}

<...>
USBH_BULK_Init();
USBH_BULK_AddNotification(&_Hook, _cbOnAddRemoveDevice, NULL);
```

<...>

13.2.6 USBH_BULK_RegisterNotification()

Description

(Deprecated) Sets a callback in order to be notified when a device is added or removed.

Prototype

```
void USBH_BULK_RegisterNotification(USBH_NOTIFICATION_FUNC * pfNotification,  
                                   void * pContext);
```

Parameters

Parameter	Description
<code>pfNotification</code>	Pointer to a function the stack should call when a device is connected or disconnected.
<code>pContext</code>	Pointer to a user context that is passed to the callback function.

Additional information

This function is deprecated, please use function `USBH_BULK_AddNotification()`.

13.2.7 USBH_BULK_Open()

Description

Opens a device interface given by an index.

Prototype

```
USBH_BULK_HANDLE USBH_BULK_Open(unsigned Index);
```

Parameters

Parameter	Description
Index	Index of the interface that shall be opened. In general this means: the first connected interface is 0, second interface is 1 etc.

Return value

= USBH_BULK_INVALID_HANDLE Device not available or removed.
≠ USBH_BULK_INVALID_HANDLE Handle to a BULK device

Additional information

The index of a new connected device is provided to the callback function registered with USBH_BULK_AddNotification().

13.2.8 USBH_BULK_Close()

Description

Closes a handle to an opened device.

Prototype

```
USBH_STATUS USBH_BULK_Close(USBH_BULK_HANDLE hDevice);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to an open device returned by <code>USBH_BULK_Open()</code> .

Return value

`USBH_STATUS_SUCCESS` on success or error code on failure.

13.2.9 USBH_BULK_AllowShortRead()

Description

Enables or disables short read mode. If enabled, the function `USBH_BULK_Read()` returns as soon as a short packet was read from the device. This allows the application to read data where the number of bytes to read is undefined.

Prototype

```
USBH_STATUS USBH_BULK_AllowShortRead(USBH_BULK_HANDLE hDevice,  
                                     U8                AllowShortRead);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to an open device returned by <code>USBH_BULK_Open()</code> .
<code>AllowShortRead</code>	Define whether short read mode shall be used or not. • 1 - Allow short read. • 0 - Short read mode disabled.

Return value

`USBH_STATUS_SUCCESS` on success or error code on failure.

13.2.10 USBH_BULK_GetDeviceInfo()

Description

Retrieves information about the BULK device.

Prototype

```
USBH_STATUS USBH_BULK_GetDeviceInfo(USBH_BULK_HANDLE    hDevice,  
                                     USBH_BULK_DEVICE_INFO * pDevInfo);
```

Parameters

Parameter	Description
hDevice	Handle to an open device returned by <code>USBH_BULK_Open()</code> .
pDevInfo	Pointer to a <code>USBH_BULK_DEVICE_INFO</code> structure that receives the information.

Return value

`USBH_STATUS_SUCCESS` on success or error code on failure.

13.2.11 USBH_BULK_GetEndpointInfo()

Description

Retrieves information about an endpoint of a BULK device.

Prototype

```
USBH_STATUS USBH_BULK_GetEndpointInfo(USBH_BULK_HANDLE    hDevice,
                                       unsigned            EPIndex,
                                       USBH_BULK_EP_INFO *  pEPInfo);
```

Parameters

Parameter	Description
hDevice	Handle to an open device returned by USBH_BULK_Open().
EPIndex	Index of the EP (0 ... DevInfo.NumEPs-1)
pEPInfo	Pointer to a USBH_BULK_EP_INFO structure that receives the information.

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

13.2.12 USBH_BULK_Read()

Description

Reads from the BULK device. The behavior depends on the Short Read mode (see `USBH_BULK_AllowShortRead()`): If Short Read mode is enabled, the function tries to read up to `NumBytes`, but will stop reading and returns if a short packet was received (a packet that is smaller than the maximum packet size of the endpoint). If Short Read mode is disabled, the function tries to reads exactly `NumBytes` of data, independent of the sizes of the received packets. This function will also return when the timeout expires, whatever comes first.

The USB stack can only read complete packets from the USB device. If the size of a received packet exceeds `NumBytes` then all data that does not fit into the callers buffer (`pData`) is stored in an internal buffer and will be returned by the next call to `USBH_BULK_Read()`. See also `USBH_BULK_GetNumBytesInBuffer()`.

To read a null packet, set `pData` = NULL and `NumBytes` = 0. For this, the internal buffer must be empty.

Prototype

```
USBH_STATUS USBH_BULK_Read(USBH_BULK_HANDLE hDevice,
                           U8               EPAddr,
                           U8               * pData,
                           U32               NumBytes,
                           U32               * pNumBytesRead,
                           U32               Timeout);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to an open device returned by <code>USBH_BULK_Open()</code> .
<code>EPAddr</code>	Endpoint address (can be retrieved via <code>USBH_BULK_GetEndpointInfo()</code>). Must be an IN endpoint.
<code>pData</code>	Pointer to a buffer to store the read data.
<code>NumBytes</code>	Number of bytes to be read from the device.
<code>pNumBytesRead</code>	Pointer to a variable which receives the number of bytes read from the device. Can be NULL.
<code>Timeout</code>	<code>Timeout</code> in ms. 0 means infinite timeout.

Return value

`USBH_STATUS_SUCCESS` on success or error code on failure.

Additional information

If the function returns an error code (including `USBH_STATUS_TIMEOUT`) it already may have read part of the data. The number of bytes read successfully is always stored in the variable pointed to by `pNumBytesRead`.

13.2.13 USBH_BULK_Receive()

Description

Reads one packet from the device. The size of the buffer provided by the caller must be at least the maximum packet size of the endpoint referenced. The maximum packet size of the endpoint can be retrieved using USBH_BULK_GetEndpointInfo().

Prototype

```
USBH_STATUS USBH_BULK_Receive(USBH_BULK_HANDLE hDevice,
                               U8               EPAddr,
                               U8               * pData,
                               U32             * pNumBytesRead,
                               U32             Timeout);
```

Parameters

Parameter	Description
hDevice	Handle to an open device returned by USBH_BULK_Open().
EPAddr	Endpoint address (can be retrieved via USBH_BULK_GetEndpointInfo()). Must be an IN endpoint.
pData	Pointer to a buffer to store the data read.
pNumBytesRead	Pointer to a variable which receives the number of bytes read from the device.
Timeout	Timeout in ms. 0 means infinite timeout.

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

Additional information

This function does not access the buffer used by the function USBH_BULK_Read(). Data contained in this buffer are not returned by USBH_BULK_Receive(). Intermixing calls to USBH_BULK_Read() and USBH_BULK_Receive() for the same endpoint should be avoided or used with care.

13.2.14 USBH_BULK_Write()

Description

Writes data to the BULK device. The function blocks until all data has been written or until the timeout has been reached.

Prototype

```
USBH_STATUS USBH_BULK_Write(      USBH_BULK_HANDLE  hDevice,
                                  U8                    EPAddr,
                                  const U8               * pData,
                                  U32                    NumBytes,
                                  U32                    * pNumBytesWritten,
                                  U32                    Timeout);
```

Parameters

Parameter	Description
hDevice	Handle to an open device returned by USBH_BULK_Open().
EPAddr	Endpoint address (can be retrieved via USBH_BULK_GetEndpointInfo()). Must be an OUT endpoint.
pData	Pointer to data to be sent.
NumBytes	Number of bytes to send.
pNumBytesWritten	Pointer to a variable which receives the number of bytes written to the device. Can be NULL.
Timeout	Timeout in ms. 0 means infinite timeout.

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

Additional information

If the function returns an error code (including USBH_STATUS_TIMEOUT) it already may have written part of the data. The number of bytes written successfully is always stored in the variable pointed to by pNumBytesWritten.

13.2.15 USBH_BULK_GetMaxTransferSize()

Description

Return the maximum transfer size allowed for the USBH_BULK_*Async functions.

Prototype

```
USBH_STATUS USBH_BULK_GetMaxTransferSize(USBH_BULK_HANDLE hDevice,  
                                           U8 EPAddr,  
                                           U32 * pMaxTransferSize);
```

Parameters

Parameter	Description
hDevice	Handle to an open device returned by USBH_BULK_Open().
EPAddr	Endpoint address (can be retrieved via USBH_BULK_GetEndpointInfo()).
pMaxTransferSize	Pointer to a variable which will receive the maximum transfer size for the specified endpoint.

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

Additional information

Using this function is only necessary with the USBH_BULK_*Async functions, other functions handle the limits internally. These limits exist because certain USB controllers have hardware limitations. Some USB controllers (OHCI, EHCI, ...) do not have these limitations, therefore 0xFFFFFFFF will be returned.

13.2.16 USBH_BULK_ReadAsync()

Description

Triggers a read transfer to the BULK device. The result of the transfer is received through the user callback. This function will return immediately while the read transfer is done asynchronously.

Prototype

```
USBH_STATUS USBH_BULK_ReadAsync(USBH_BULK_HANDLE hDevice,
                                U8 EPAddr,
                                void * pBuffer,
                                U32 BufferSize,
                                USBH_BULK_ON_COMPLETE_FUNC * pfOnComplete,
                                USBH_BULK_RW_CONTEXT * pRWContext);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to an open device returned by <code>USBH_BULK_Open()</code> .
<code>EPAddr</code>	Endpoint address. Must be an IN endpoint.
<code>pBuffer</code>	Pointer to the buffer that receives the data from the device. Ignored for ISO transfers.
<code>BufferSize</code>	Size of the buffer in bytes. Must be a multiple of the maximum packet size of the USB device. Use <code>USBH_BULK_GetMaxTransferSize()</code> to get the maximum allowed size. Ignored for ISO transfers.
<code>pfOnComplete</code>	Pointer to a user function of type <code>USBH_BULK_ON_COMPLETE_FUNC</code> which will be called after the transfer has been completed.
<code>pRWContext</code>	Pointer to a <code>USBH_BULK_RW_CONTEXT</code> structure which will be filled with data after the transfer has been completed and passed as a parameter to the <code>pfOnComplete</code> function. The member 'pUserContext' may be set before calling <code>USBH_BULK_ReadAsync()</code> . Other members do not need to be initialized and are set by the function <code>USBH_BULK_ReadAsync()</code> . The memory used for this structure must be valid, until the transaction is completed.

Return value

<code>= USBH_STATUS_PENDING</code>	Success, the data transfer is queued, the user callback will be called after the transfer is finished.
<code>≠ USBH_STATUS_PENDING</code>	An error occurred, the transfer is not started and user callback will not be called.

Example

```
static USBH_BULK_RW_CONTEXT _ReadWriteContext;

<...>

/*****
 *
 *      _OnReadComplete
 */
static void _OnReadComplete(USBH_BULK_RW_CONTEXT * pRWContext) {
    if (pRWContext->Status == USBH_STATUS_SUCCESS) {
        printf("Successfully read %u bytes \n",
              (unsigned int)pRWContext->NumBytesTransferred);
    }
}
```



```
    } else {  
        printf("ReadAsync callback returned %s \n",  
              USBH_GetStatusStr(pRWContext->Status));  
        // Error handling  
    }  
    <...>  
}  
  
<...>  
  
Status = USBH_BULK_ReadAsync(_hDevice,  
                             EPIAddr,  
                             _acBuffer,  
                             NumBytes,  
                             _OnReadComplete,  
                             &_ReadWriteContext);  
  
if (Status != USBH_STATUS_PENDING) {  
    // Error handling.  
}  
  
<...>
```

13.2.17 USBH_BULK_WriteAsync()

Description

Triggers a write transfer to the BULK device. The result of the transfer is received through the user callback. This function will return immediately while the write transfer is done asynchronously.

Prototype

```
USBH_STATUS USBH_BULK_WriteAsync(USBH_BULK_HANDLE hDevice,
                                U8 EPAddr,
                                void * pBuffer,
                                U32 BufferSize,
                                USBH_BULK_ON_COMPLETE_FUNC * pfOnComplete,
                                USBH_BULK_RW_CONTEXT * pRWContext);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to an open device returned by <code>USBH_BULK_Open()</code> .
<code>EPAddr</code>	Endpoint address. Must be an OUT endpoint.
<code>pBuffer</code>	Pointer to a buffer which holds the data. Ignored for ISO transfers.
<code>BufferSize</code>	Number of bytes to write. Use <code>USBH_BULK_GetMaxTransferSize()</code> to get the maximum allowed size. Ignored for ISO transfers.
<code>pfOnComplete</code>	Pointer to a user function of type <code>USBH_BULK_ON_COMPLETE_FUNC</code> which will be called after the transfer has been completed.
<code>pRWContext</code>	Pointer to a <code>USBH_BULK_RW_CONTEXT</code> structure which will be filled with data after the transfer has been completed and passed as a parameter to the <code>pfOnComplete</code> function. The member 'pUserContext' may be set before calling <code>USBH_BULK_WriteAsync()</code> . Other members do not need to be initialized and are set by the function <code>USBH_BULK_WriteAsync()</code> . The memory used for this structure must be valid, until the transaction is completed.

Return value

= `USBH_STATUS_PENDING` Success, the data transfer is queued, the user callback will be called after the transfer is finished.

≠ `USBH_STATUS_PENDING` An error occurred, the transfer is not started and user callback will not be called.

Example

```
static USBH_BULK_RW_CONTEXT _ReadWriteContext;

<...>

/*****
 *
 *      _OnWriteComplete
 */
static void _OnWriteComplete(USBH_BULK_RW_CONTEXT * pRWContext) {
    if (pRWContext->Status == USBH_STATUS_SUCCESS) {
        printf("Successfully written data to the device \n");
    } else {
        printf("WriteAsync callback returned %s \n",
```

```
        USBH_GetStatusStr(pRWContext->Status));
    // Error handling
    }
    <...>
}

<...>

Status = USBH_BULK_WriteAsync(_hDevice,
                              EPPAddr,
                              _acBuffer,
                              NumBytes,
                              _OnWriteComplete,
                              &_ReadWriteContext);
if (Status != USBH_STATUS_PENDING) {
    // Error handling.
}
<...>
```

13.2.18 USBH_BULK_Cancel()

Description

Cancels a running transfer.

Prototype

```
USBH_STATUS USBH_BULK_Cancel(USBH_BULK_HANDLE hDevice,  
                             U8 EPAddr);
```

Parameters

Parameter	Description
hDevice	Handle to an open device returned by <code>USBH_BULK_Open()</code> .
EPAddr	Endpoint address.

Return value

`USBH_STATUS_SUCCESS` on success or error code on failure.

Additional information

This function can be used to cancel a transfer which was initiated by `USBH_BULK_ReadAsync()/USBH_BULK_WriteAsync()` or `USBH_BULK_Read()/USBH_BULK_Write()`. In the later case this function has to be called from a different task.

13.2.19 USBH_BULK_GetNumBytesInBuffer()

Description

Gets the number of bytes in the receive buffer.

The USB stack can only read complete packets from the USB device. If the size of a received packet exceeds the number of bytes requested with `USBH_BULK_Read()`, then all data that is not returned by `USBH_BULK_Read()` is stored in an internal buffer.

The number of bytes returned by `USBH_BULK_GetNumBytesInBuffer()` can be read using `USBH_BULK_Read()` out of the buffer without a USB transaction to the USB device being executed.

Prototype

```
USBH_STATUS USBH_BULK_GetNumBytesInBuffer(USBH_BULK_HANDLE hDevice,
                                           U8               EPAddr,
                                           U32               * pRxBytes);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to an open device returned by <code>USBH_BULK_Open()</code> .
<code>EPAddr</code>	Endpoint address.
<code>pRxBytes</code>	Pointer to a variable which receives the number of bytes in the receive buffer.

Return value

`USBH_STATUS_SUCCESS` on success or error code on failure.

Example

```
//
// Read only ONE byte to trigger the read transfer.
// This means that the remaining bytes are in the internal packet buffer!
//
USBH_BULK_Read(hDevice, EPAddr, acData, 1, &NumBytes, Timeout);
if (NumBytes) {
    //
    // We do not know how big the packet was which we received from the device,
    // since we only read 1 byte from the packet.
    // Therefore we still might have some data in the internal buffer!
    // Using USBH_BULK_GetNumBytesInBuffer we can check how many bytes are still in the
    // internal buffer (if any) and read those as well.
    //
    if (USBH_BULK_GetNumBytesInBuffer(hDevice, EPAddr, &RxBytes) == USBH_STATUS_SUCCESS) {
        //
        // Read the remaining bytes.
        //
        if (RxBytes > 0) {
            USBH_BULK_Read(hDevice, EPAddr, &acData[1], RxBytes, &NumBytes, Timeout);
        }
    }
}
```

13.2.20 USBH_BULK_SetupRequest()

Description

Sends a specific request (class vendor etc) to the device.

Prototype

```
USBH_STATUS USBH_BULK_SetupRequest(USBH_BULK_HANDLE hDevice,
                                     U8               RequestType,
                                     U8               Request,
                                     U16              wValue,
                                     U16              wIndex,
                                     void             * pData,
                                     U32              * pNumBytesData,
                                     U32              Timeout);
```

Parameters

Parameter	Description
hDevice	Handle to an open device returned by USBH_BULK_Open().
RequestType	This parameter is a bitmap containing the following values: • bit 7 transfer direction: • 0 = OUT (Host to Device) • 1 = IN (Device to Host) • bits 6..5 request type: • 0 = Standard • 1 = Class • 2 = Vendor • 3 = Reserved • bits 4..0 recipient: • 0 = Device • 1 = Interface • 2 = Endpoint • 3 = Other
Request	Request code in the setup request.
wValue	wValue in the setup request.
wIndex	wIndex in the setup request.
pData	Additional data for the setup request.
pNumBytesData	• in Number of bytes to be received/sent in pData. • out Number of bytes processed.
Timeout	Timeout in ms. 0 means infinite timeout.

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

Additional information

wLength which is normally part of the setup packet will be determined given by the pNumBytes and pData. In case no pBuffer is given, wLength will be 0.

13.2.21 USBH_BULK_IsoDataCtrl()

Description

Acknowledge ISO data received from an IN EP or provide data for OUT EPs.

On order to start ISO OUT transfers after calling USBH_BULK_WriteAsync(), initially the output packet queue must be filled. For that purpose this function must be called repeatedly until is does not return USBH_STATUS_NEED_MORE_DATA any more.

Prototype

```
USBH_STATUS USBH_BULK_IsoDataCtrl(USBH_BULK_HANDLE    hDevice,
                                   U8                   EPAddr,
                                   USBH_ISO_DATA_CTRL * pIsoData);
```

Parameters

Parameter	Description
hDevice	Handle to an open device returned by USBH_BULK_Open().
EPAddr	Endpoint address.
pIsoData	ISO data structure.

Return value

USBH_STATUS_SUCCESS or USBH_STATUS_NEED_MORE_DATA on success or error code on failure.

13.2.22 USBH_BULK_GetInterfaceHandle()

Description

Return the handle to the (open) USB interface. Can be used to call USBH core functions like USBH_GetStringDescriptor().

Prototype

```
USBH_INTERFACE_HANDLE USBH_BULK_GetInterfaceHandle(USBH_BULK_HANDLE hDevice);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to an open device returned by USBH_BULK_Open().

Return value

Handle to an open interface.

13.3 Data structures

This chapter describes the emUSB-Host BULK driver data structures.

Structure	Description
USBH_BULK_DEVICE_INFO	Structure containing information about a BULK device.
USBH_BULK_EP_INFO	Structure containing information about an endpoint.
USBH_BULK_RW_CONTEXT	Contains information about a completed, asynchronous transfers.
USBH_ISO_DATA_CTRL	This data structure is used to provide or acknowledge ISO data.

13.3.1 USBH_BULK_DEVICE_INFO

Description

Structure containing information about a BULK device.

Type definition

```
typedef struct {
    U16      VendorId;
    U16      ProductId;
    U8       Class;
    U8       SubClass;
    U8       Protocol;
    U8       AlternateSetting;
    USBH_SPEED Speed;
    U8       InterfaceNo;
    U8       NumEPs;
    USBH_BULK_EP_INFO EndpointInfo[];
    USBH_DEVICE_ID DeviceId;
    USBH_INTERFACE_ID InterfaceID;
} USBH_BULK_DEVICE_INFO;
```

Structure members

Member	Description
VendorId	The Vendor ID of the device.
ProductId	The Product ID of the device.
Class	The interface class.
SubClass	The interface sub class.
Protocol	The interface protocol.
AlternateSetting	The current alternate setting
Speed	The USB speed of the device, see USBH_SPEED.
InterfaceNo	Index of the interface (from USB descriptor).
NumEPs	Number of endpoints.
EndpointInfo	Obsolete. See USBH_BULK_GetEndpointInfo().
DeviceId	The unique device Id. This Id is assigned if the USB device was successfully enumerated. It is valid until the device is removed from the host. If the device is reconnected a different device Id is assigned.
InterfaceID	Interface ID of the device.

13.3.2 USBH_BULK_EP_INFO

Description

Structure containing information about an endpoint.

Type definition

```
typedef struct {  
    U8    Addr;  
    U8    Type;  
    U8    Direction;  
    U16   MaxPacketSize;  
} USBH_BULK_EP_INFO;
```

Structure members

Member	Description
Addr	Endpoint Address.
Type	Endpoint Type (see USB_EP_TYPE_... macros).
Direction	Endpoint direction (see USB_..._DIRECTION macros).
MaxPacketSize	Maximum packet size for the endpoint.

13.3.3 USBH_BULK_RW_CONTEXT

Description

Contains information about a completed, asynchronous transfers. Is passed to the USBH_BULK_ON_COMPLETE_FUNC user callback when using asynchronous write and read. When this structure is passed to USBH_BULK_ReadAsync() or USBH_BULK_WriteAsync() its member need not to be initialized.

Type definition

```
typedef struct {
    void * pUserContext;
    USBH_STATUS Status;
    I8 Terminated;
    U32 NumBytesTransferred;
    void * pUserBuffer;
    U32 UserBufferSize;
} USBH_BULK_RW_CONTEXT;
```

Structure members

Member	Description
pUserContext	Pointer to a user context. Can be arbitrarily used by the application.
Status	Result status of the asynchronous transfer.
Terminated	1: Operation is terminated. 0: More data may be transfered and callback function may be called again (ISO transfers only).
NumBytesTransferred	Number of bytes transferred.
pUserBuffer	For BULK and INT transfers: Pointer to the buffer provided to USBH_BULK_ReadAsync() or USBH_BULK_WriteAsync(). For ISO IN transfers: Pointer to to data read.
UserBufferSize	For BULK and INT transfers: Size of the buffer as provided to USBH_BULK_ReadAsync() or USBH_BULK_WriteAsync(). Not used For ISO transfers.

13.3.4 USBH_ISO_DATA_CTRL

Description

This data structure is used to provide or acknowledge ISO data. Used with function `USBH_IsoDataCtrl()`.

Type definition

```
typedef struct {
    U32      Length;
    const U8 * pData;
    U32      Length2;
    const U8 * pData2;
    const U8 * pBuffer;
} USBH_ISO_DATA_CTRL;
```

Structure members

Member	Description
Length	in Length of the first data part to be transferred via ISO OUT EP in bytes. The ISO packet send has size 'Length' + 'Length2'.
pData	in Pointer to the first data part to be transferred via ISO OUT EP. The ISO packet send is constructed by concatenating both data parts 'pData' and 'pData2'.
Length2	in Length of the second data part to be transferred via ISO OUT EP in bytes (optional).
pData2	in Pointer to the second data part to be transferred via ISO OUT EP.
pBuffer	out Buffer used by the driver.

13.4 Type definitions

This chapter describes the types defined in the header file `USBH_BULK.h`.

Type	Description
USBH_BULK_ON_COMPLETE_FUNC	Function called on completion of an asynchronous transfer.

13.4.1 USBH_BULK_ON_COMPLETE_FUNC

Description

Function called on completion of an asynchronous transfer. Used by the functions USBH_BULK_ReadAsync() and USBH_BULK_WriteAsync().

Type definition

```
typedef void USBH_BULK_ON_COMPLETE_FUNC(USBH_BULK_RW_CONTEXT * pRWContext);
```

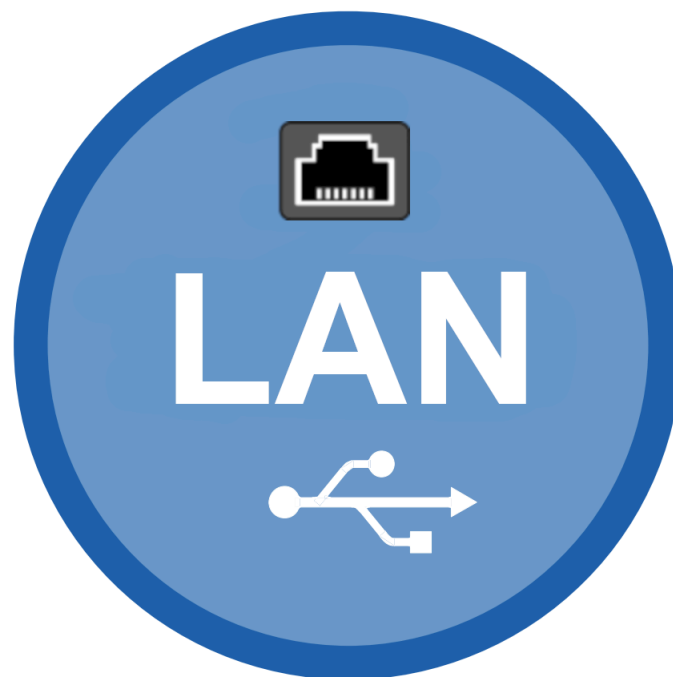
Parameters

Parameter	Description
<code>pRWContext</code>	Pointer to a USBH_BULK_RW_CONTEXT structure.

Chapter 14

LAN component (Add-On)

This chapter describes the optional emUSB-Host add-on “LAN”. It allows interfacing Ethernet-over-USB adapters with emNet.



14.1 Introduction

The LAN software component of emUSB-Host allows communication with Ethernet-over-USB adapters. These devices usually implement the CDC-ECM, RNDIS protocol or a proprietary protocol from the company ASIX. All above protocols allow the transfer of Ethernet packets over USB. emUSB-Host LAN provides a seamless interface with emNet irrespective of the underlying USB protocol thereby allowing devices without Ethernet connectors to connect with a network.

This chapter provides an explanation of the LAN software component functions available to application developers. All the functions and data types of this add-on are prefixed with 'USBH_LAN_'.

14.1.1 Overview

emNet adds Ethernet interfaces for as many Ethernet-over-USB adapters as are expected to be used with the product. The interfaces are initially "down". emUSB-Host LAN accommodates multiple underlying classes to support different adapters. Each registered LAN driver notifies the main LAN module when a device matching the LAN driver's supported class (ASIX, RNDIS or CDC-ECM) has enumerated. The LAN module in turn notifies the IP stack that an interface is "up" and communication begins. emUSB-Host LAN is supported with version 3.30 of emNet and higher.

14.1.2 Features

The following features are provided:

- Compatibility with different Ethernet-over-USB adapters.
- Integration with emNet

14.1.3 Example code

Any emNet example can be used.

14.2 IP_Config_USBH_LAN.c in detail

The emNet configuration file IP_Config_USBH_LAN.c is a sample configuration for using emUSB-Host LAN as an interface with emNet.

The function IP_X_Config is the main configuration function of the emNet stack. In this sample the NUM_INSTANCES define (4 by default) is used to determine how many interfaces are registered. Each emNet interface corresponds to one Ethernet-over-USB adapter on the emUSB-Host LAN side. This means that in the default configuration 4 adapters can be used simultaneously (e.g. 4x CDC-ECM adapter or 2x ASIX, 1x CDC-ECM and 1x RNDIS or any other combination of the supported protocols).

```
int aIFaceId[NUM_INSTANCES];
<...>
IP_ConfigMaxIFaces(NUM_INSTANCES);
for (i = 0; i < NUM_INSTANCES; i++) {
    aIFaceId[i] = IP_AddEtherInterface(&IP_Driver_USBH);
    mtu = 1500;
    IP_SetMTU(aIFaceId[i], mtu);
    IP_DHCP_Activate(aIFaceId[i], "USBH_LAN", NULL, NULL);
<...>
}
```

The sample configuration initializes emUSB-Host and the emUSB-Host LAN component in the same function. This is for convenience only, you can initialize emUSB-Host anywhere inside your application. The initialization starts the stack and the LAN module allowing the Ethernet-over-USB adapters to enumerate.

```
<...>
USBH_Init();
OS_CREATETASK(&_TCBMain, "USBH_Task", USBH_Task, TASK_PRIO_USBH_MAIN, _StackMain);
OS_CREATETASK(&_TCBIsr, "USBH_isr", USBH_ISRTask, TASK_PRIO_USBH_ISR, _StackIsr);
USBH_LAN_Init();
USBH_LAN_RegisterDriver(&USBH_LAN_DRIVER_ASIX);
USBH_LAN_RegisterDriver(&USBH_LAN_DRIVER_ECM);
USBH_LAN_RegisterDriver(&USBH_LAN_DRIVER_RNDIS);
<...>
```

14.3 API Functions

This chapter describes the emUSB-Host LAN API functions. These functions are defined in the header file `USBH_LAN.h`.

Function	Description
<code>USBH_LAN_Init()</code>	Initializes and registers the LAN component with emUSB-Host.
<code>USBH_LAN_RegisterDriver()</code>	Registers a device specific driver (CDC-ECM, ASIX, RNDIS) with the LAN component.
<code>USBH_LAN_Exit()</code>	De-initializes the LAN component.
<code>USBH_LAN_SetOnAddRemove()</code>	Sets a callback function which is called when LAN devices are added or removed.

14.3.1 USBH_LAN_Init()

Description

Initializes and registers the LAN component with emUSB-Host.

Prototype

```
USBH_STATUS USBH_LAN_Init(void);
```

Return value

= USBH_STATUS_SUCCESS	Success
≠ USBH_STATUS_SUCCESS	Could not initialize LAN component.

14.3.2 USBH_LAN_RegisterDriver()

Description

Registers a device specific driver (CDC-ECM, ASIX, RNDIS) with the LAN component.

Prototype

```
USBH_STATUS USBH_LAN_RegisterDriver(const USBH_LAN_DRIVER * pDriver);
```

Parameters

Parameter	Description
<code>pDriver</code>	Pointer to an LAN driver structure of type <code>USBH_LAN_DRIVER</code> . Currently the following drivers are available: • <code>USBH_LAN_DRIVER_ASIX</code> • <code>USBH_LAN_DRIVER_ECM</code> • <code>USBH_LAN_DRIVER_RNDIS</code>

Return value

= <code>USBH_STATUS_SUCCESS</code>	Success
≠ <code>USBH_STATUS_SUCCESS</code>	Could not register LAN driver.

14.3.3 USBH_LAN_Exit()

Description

De-initializes the LAN component.

Prototype

```
void USBH_LAN_Exit(void);
```

14.3.4 USBH_LAN_SetOnAddRemove()

Description

Sets a callback function which is called when LAN devices are added or removed. Must be called after `USBH_LAN_Init()`.

Prototype

```
void USBH_LAN_SetOnAddRemove(USBH_LAN_ADD_REMOVE_CALLBACK * pfOnAddRemove);
```

Parameters

Parameter	Description
<code>pfOnAddRemove</code>	Pointer to a callback function of type <code>USBH_LAN_ADD_REMOVE_CALLBACK</code> .

14.4 Function Types

This chapter describes the emUSB-Host LAN API function types.

Type	Description
USBH_LAN_ADD_REMOVE_CALLBACK	Callback function called when LAN devices (CDC-ECM, RNDIS, ASIX) are added or removed.

14.4.1 USBH_LAN_ADD_REMOVE_CALLBACK

Description

Callback function called when LAN devices (CDC-ECM, RNDIS, ASIX) are added or removed. This function must not block.

Type definition

```
typedef void USBH_LAN_ADD_REMOVE_CALLBACK(U8 DevIndex,  
USBH_DEVICE_EVENT Event);
```

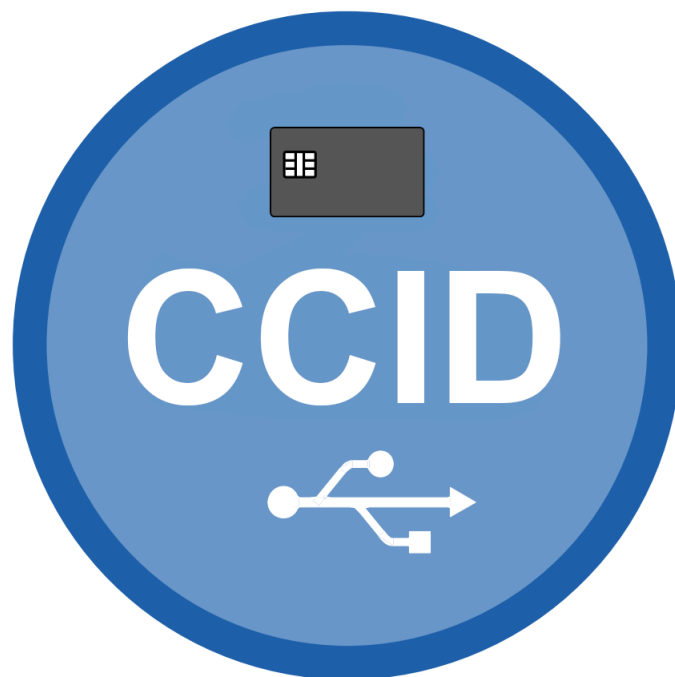
Parameters

Parameter	Description
DevIndex	Zero based index of the device that was added or removed. First device has index 0, second one has index 1, etc
Event	Enum USBH_DEVICE_EVENT which gives information about the event that occurred.

Chapter 15

CCID Device Driver (Add-On)

This chapter describes the optional emUSB-Host add-on “CCID device driver”. It allows communication with a smart card reader.



15.1 Introduction

The CCID driver software component of emUSB-Host allows communication with CCID compatible smart card readers. The Communication Device Class (CCID) is an abstract USB class protocol defined by the USB Implementers Forum.

This chapter provides an explanation of the functions available to application developers via the CCID driver software. All the functions and data types of this add-on are prefixed with 'USBH_CCID_'.

15.1.1 Overview

A CCID device connected to the emUSB-Host is automatically configured and added to an internal list. If the CCID driver has been registered, it is notified via a callback when a CCID device has been added or removed. The driver then can notify the application program, when a callback function has been registered via `USBH_CCID_AddNotification()`. In order to communicate with such a device, the application has to call the `USBH_CCID_Open()`, passing the device index. CCID devices are identified by an index. The first connected device gets assigned the index 0, the second index 1, and so on.

15.1.2 Features

The following features are provided:

- Compatibility with different CCID devices.
- Handling of multiple CCID devices at the same time.
- Handling of CCID devices with multiple card slots.
- Ability to send commands to a smart card and receive the response.
- Ability to query the slot status.
- Notifications about insertion and removal of smart cards.

15.1.3 Example code

An example application which uses the API is provided in the `USBH_CCID_Start.c` file. This example waits for a smart card to be inserted and displays the serial number of the smart card in the I/O terminal of the debugger (if it supports a serial number).

15.2 API Functions

This chapter describes the emUSB-Host CCID driver API functions. These functions are defined in the header file `USBH_CCID.h`.

Function	Description
<code>USBH_CCID_Init()</code>	Initializes and registers the CCID device module with emUSB-Host.
<code>USBH_CCID_Exit()</code>	Unregisters and de-initializes the CCID device module from emUSB-Host.
<code>USBH_CCID_AddNotification()</code>	Adds a callback in order to be notified when a device is added or removed.
<code>USBH_CCID_RemoveNotification()</code>	Removes a callback added via <code>USBH_CCID_AddNotification</code> .
<code>USBH_CCID_Open()</code>	Opens a device given by an index.
<code>USBH_CCID_Close()</code>	Closes a handle to an opened device.
<code>USBH_CCID_SetTimeout()</code>	Sets up the timeout for communication with the device.
<code>USBH_CCID_SetOnSlotChange()</code>	Sets the callback to retrieve slot change events from the interrupt endpoint.
<code>USBH_CCID_GetDeviceInfo()</code>	Retrieves information about the CCID device.
<code>USBH_CCID_GetSerialNumber()</code>	Get the serial number of a CCID device.
<code>USBH_CCID_GetNumSlots()</code>	Retrieves the number of card slots of the CCID device.
<code>USBH_CCID_GetCSDesc()</code>	Retrieves the specific CCID descriptor from the interface.
<code>USBH_CCID_Cmd()</code>	Sends a command to the CCID device and receives the response.
<code>USBH_CCID_PowerOn()</code>	Power on a slot and receive ATR.
<code>USBH_CCID_PowerOnATR()</code>	Power on a slot and receive ATR.
<code>USBH_CCID_PowerOff()</code>	Power off a slot.
<code>USBH_CCID_GetSlotStatus()</code>	Get status of a card slot.
<code>USBH_CCID_APDU()</code>	Send APDU to smart card and read answer.
<code>USBH_CCID_GetResponse()</code>	Get response data of a previous APDU command.
<code>USBH_CCID_GetParameters()</code>	Get slot parameters.
<code>USBH_CCID_ResetParameters()</code>	Reset slot parameters to their default.
<code>USBH_CCID_SetParameters()</code>	Set slot parameters.
<code>USBH_CCID_Abort()</code>	Stop any current transfer at the slot and return to a state where the slot is ready to accept a new command.
<code>USBH_CCID_GetInterfaceHandle()</code>	Return the handle to the (open) USB interface.

15.2.1 USBH_CCID_Init()

Description

Initializes and registers the CCID device module with emUSB-Host.

Prototype

```
USBH_STATUS USBH_CCID_Init(void);
```

Return value

= USBH_STATUS_SUCCESS	Success or module already initialized.
≠ USBH_STATUS_SUCCESS	An error occurred.

15.2.2 USBH_CCID_Exit()

Description

Unregisters and de-initializes the CCID device module from emUSB-Host.

Prototype

```
void USBH_CCID_Exit(void);
```

Additional information

Has to be called the same number of times `USBH_CCID_Init` was called in order to de-initialize the module. This function will release resources that were used by this device driver. It has to be called if the application is closed. This has to be called before `USBH_Exit()` is called. No more functions of this module may be called after calling `USBH_CCID_Exit()`. The only exception is `USBH_CCID_Init()`, which would in turn re-init the module and allow further calls.

15.2.3 USBH_CCID_AddNotification()

Description

Adds a callback in order to be notified when a device is added or removed.

Prototype

```
USBH_STATUS USBH_CCID_AddNotification(USBH_NOTIFICATION_HOOK * pHook,  
                                     USBH_NOTIFICATION_FUNC * pfNotification,  
                                     void * pContext);
```

Parameters

Parameter	Description
<code>pHook</code>	Pointer to a user provided USBH_NOTIFICATION_HOOK variable.
<code>pfNotification</code>	Pointer to a function the stack should call when a device is connected or disconnected.
<code>pContext</code>	Pointer to a user context that is passed to the callback function.

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

15.2.4 USBH_CCID_RemoveNotification()

Description

Removes a callback added via USBH_CCID_AddNotification.

Prototype

```
USBH_STATUS USBH_CCID_RemoveNotification(const USBH_NOTIFICATION_HOOK * pHook);
```

Parameters

Parameter	Description
<code>pHook</code>	Pointer to a user provided USBH_NOTIFICATION_HOOK variable.

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

15.2.5 USBH_CCID_Open()

Description

Opens a device given by an index.

Prototype

```
USBH_CCID_HANDLE USBH_CCID_Open(unsigned Index);
```

Parameters

Parameter	Description
Index	Index of the device that shall be opened.

Return value

= USBH_CCID_INVALID_HANDLE Device not available or removed.
≠ USBH_CCID_INVALID_HANDLE Handle to a CCID device

Additional information

The index of a new connected device is provided to the callback function registered with USBH_CCID_AddNotification().

15.2.6 USBH_CCID_Close()

Description

Closes a handle to an opened device.

Prototype

```
void USBH_CCID_Close(USBH_CCID_HANDLE hDevice);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to an open device returned by <code>USBH_CCID_Open()</code> .

15.2.7 USBH_CCID_SetTimeout()

Description

Sets up the timeout for communication with the device.

Prototype

```
void USBH_CCID_SetTimeout(USBH_CCID_HANDLE hDevice,  
                          U32                Timeout);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to an open device returned by <code>USBH_CCID_Open()</code> .
<code>Timeout</code>	Read timeout given in ms.

15.2.8 USBH_CCID_SetOnSlotChange()

Description

Sets the callback to retrieve slot change events from the interrupt endpoint. If the device does not have an interrupt endpoint, this function returns an error code.

Prototype

```
USBH_STATUS USBH_CCID_SetOnSlotChange
(USBH_CCID_HANDLE hDevice,
 USBH_CCID_SLOT_CHANGE_CALLBACK * pfOnSlotChange,
 void * pUserContext);
```

Parameters

Parameter	Description
hDevice	Handle to an open device returned by USBH_CCID_Open().
pfOnSlotChange	Pointer to the callback that shall be called on the event.
pUserContext	Pointer to the user context.

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

15.2.9 USBH_CCID_GetDeviceInfo()

Description

Retrieves information about the CCID device.

Prototype

```
USBH_STATUS USBH_CCID_GetDeviceInfo(USBH_CCID_HANDLE    hDevice,  
                                     USBH_CCID_DEVICE_INFO * pDevInfo);
```

Parameters

Parameter	Description
hDevice	Handle to an open device returned by <code>USBH_CCID_Open()</code> .
pDevInfo	Pointer to a <code>USBH_CCID_DEVICE_INFO</code> structure that receives the information.

Return value

`USBH_STATUS_SUCCESS` on success or error code on failure.

15.2.10 USBH_CCID_GetSerialNumber()

Description

Get the serial number of a CCID device. The serial number is in UNICODE format, not zero terminated.

Prototype

```
USBH_STATUS USBH_CCID_GetSerialNumber(USBH_CCID_HANDLE hDevice,
                                         U32             BuffSize,
                                         U8              * pSerialNumber,
                                         U32             * pSerialNumberSize);
```

Parameters

Parameter	Description
hDevice	Handle to an open device returned by USBH_CCID_Open().
BuffSize	Pointer to a buffer which holds the data.
pSerialNumber	Size of the buffer in bytes.
pSerialNumberSize	Pointer to a user function which will be called.

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

15.2.11 USBH_CCID_GetNumSlots()

Description

Retrieves the number of card slots of the CCID device.

Prototype

```
USBH_STATUS USBH_CCID_GetNumSlots(USBH_CCID_HANDLE hDevice,  
                                   unsigned          * pNumSlots);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to an open device returned by <code>USBH_CCID_Open()</code> .
<code>pNumSlots</code>	Receives the number of slots.

15.2.12 USBH_CCID_GetCSDesc()

Description

Retrieves the specific CCID descriptor from the interface.

Prototype

```
USBH_STATUS USBH_CCID_GetCSDesc(USBH_CCID_HANDLE hDevice,  
                                U8 * pData,  
                                unsigned * pNumBytesData);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to an open device returned by <code>USBH_CCID_Open()</code> .
<code>pData</code>	Pointer to a buffer where the descriptor will be stored.
<code>pNumBytesData</code>	• <code>in</code> Size of the buffer. • <code>out</code> Upon successful completion this variable will contain the number of bytes copied, which is either the size of the descriptor or the size of the buffer if the descriptor was longer than the given buffer.

Return value

`USBH_STATUS_SUCCESS` on success or error code on failure.

15.2.13 USBH_CCID_Cmd()

Description

Sends a command to the CCID device and receives the response.

Prototype

```
USBH_STATUS USBH_CCID_Cmd(USBH_CCID_HANDLE    hDevice,  
                           USBH_CCID_CMD_PARA * pCmd);
```

Parameters

Parameter	Description
hDevice	Handle to an open device returned by <code>USBH_CCID_Open()</code> .
pCmd	Pointer to structure with command parameters.

Return value

`USBH_STATUS_SUCCESS` on success or error code on failure.

15.2.14 USBH_CCID_PowerOn()

Description

Power on a slot and receive ATR.

Prototype

```
USBH_STATUS USBH_CCID_PowerOn(USBH_CCID_HANDLE hDevice,
                                U8 Slot,
                                U32 BuffSize,
                                U8 * pATR);
```

Parameters

Parameter	Description
hDevice	Handle to an open device returned by USBH_CCID_Open().
Slot	Slot index (counting from 0).
BuffSize	Size of buffer pointed to by pATR (may be 0 to discard the ATR).
pATR	Pointer to buffer that receives the ATR.

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

15.2.15 USBH_CCID_PowerOnATR()

Description

Power on a slot and receive ATR.

Prototype

```
USBH_STATUS USBH_CCID_PowerOnATR(USBH_CCID_HANDLE hDevice,
                                   U8 Slot,
                                   U32 * pRespLen,
                                   U8 * pATR);
```

Parameters

Parameter	Description
hDevice	Handle to an open device returned by USBH_CCID_Open().
Slot	Slot index (counting from 0).
pRespLen	- in Size of buffer for the ATR (may be 0 to discard the ATR). - out Length of the ATR received from the smart card.
pATR	Pointer to buffer that receives the ATR.

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

15.2.16 USBH_CCID_PowerOff()

Description

Power off a slot.

Prototype

```
USBH_STATUS USBH_CCID_PowerOff(USBH_CCID_HANDLE hDevice,  
                                U8                Slot);
```

Parameters

Parameter	Description
hDevice	Handle to an open device returned by <code>USBH_CCID_Open()</code> .
Slot	Slot index (counting from 0).

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

15.2.17 USBH_CCID_GetSlotStatus()

Description

Get status of a card slot.

Prototype

```
USBH_STATUS USBH_CCID_GetSlotStatus(USBH_CCID_HANDLE hDevice,
                                     U8 Slot,
                                     U16 * pSlotStatus);
```

Parameters

Parameter	Description
hDevice	Handle to an open device returned by USBH_CCID_Open().
Slot	Slot index (counting from 0).
pSlotStatus	Pointer to variable that receives the slot status: • 0 - smart card present and ready • 1 - smart card present but inactive • 2 - no smart card present • other - device error

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

15.2.18 USBH_CCID_APDU()

Description

Send APDU to smart card and read answer.

Prototype

```
USBH_STATUS USBH_CCID_APDU(      USBH_CCID_HANDLE hDevice,
                                U8 Slot,
                                U32 CmdLen,
                                const U8 * pCmd,
                                U32 * pRespLen,
                                U8 * pResp);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to an open device returned by <code>USBH_CCID_Open()</code> .
<code>Slot</code>	<code>Slot</code> index (counting from 0).
<code>CmdLen</code>	Len of APDU data in bytes.
<code>pCmd</code>	Pointer to APDU data.
<code>pRespLen</code>	<code>in</code> Size of buffer for response data (may be 0 to discard the response data). <code>out</code> Length of response data received from the smart card.
<code>pResp</code>	Pointer to the buffer that received the response data. Response data is stored by the function only, when it returns either <code>USBH_STATUS_SUCCESS</code> or <code>USBH_STATUS_DEVICE_ERROR</code> .

Return value

`USBH_STATUS_SUCCESS` on success or error code on failure.

Additional information

The response data of the card is truncated to a maximum of `*pRespLen` bytes. If `*pRespLen` = 0 when calling the function, then all response data are discarded.

15.2.19 USBH_CCID_GetResponse()

Description

Get response data of a previous APDU command. Is needed, after the the card has returned status code (SW1 SW2) = 61xx.

Prototype

```
USBH_STATUS USBH_CCID_GetResponse(USBH_CCID_HANDLE hDevice,  
                                   U8 Slot,  
                                   U8 CLA,  
                                   U8 Le,  
                                   U32 * pRespLen,  
                                   U8 * pResp);
```

Parameters

Parameter	Description
hDevice	Handle to an open device returned by USBH_CCID_Open() .
Slot	Slot index (counting from 0).
CLA	Value of Class bytes (usually 0).
Le	Value of Le byte (xx from status code 61xx).
pRespLen	in Size of buffer for response data. out Length of response data received from the smart card.
pResp	Pointer to the buffer that received the response data. Response data is stored by the function only, when it returns either USBH_STATUS_SUCCESS or USBH_STATUS_DEVICE_ERROR .

Return value

[USBH_STATUS_SUCCESS](#) on success or error code on failure.

Additional information

The response data of the card is truncated to a maximum of [*pRespLen](#) bytes. If [*pRespLen](#) = 0 when calling the function, then all response data are discarded.

15.2.20 USBH_CCID_GetParameters()

Description

Get slot parameters.

Prototype

```
USBH_STATUS USBH_CCID_GetParameters(USBH_CCID_HANDLE hDevice,
                                     U8 Slot,
                                     U32 * pParamLen,
                                     U8 * pParams,
                                     U8 * pProt);
```

Parameters

Parameter	Description
hDevice	Handle to an open device returned by USBH_CCID_Open().
Slot	Slot index (counting from 0).
pParamLen	- in Size of buffer for response data. - out Length of parameter block received from the device.
pParams	Pointer to the buffer that receives the parameter block.
pProt	out Receives the protocol type: 0: T=0, 1: T=1.

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

15.2.21 USBH_CCID_ResetParameters()

Description

Reset slot parameters to their default.

Prototype

```
USBH_STATUS USBH_CCID_ResetParameters(USBH_CCID_HANDLE hDevice,
                                         U8 Slot,
                                         U32 * pParamLen,
                                         U8 * pParams,
                                         U8 * pProt);
```

Parameters

Parameter	Description
hDevice	Handle to an open device returned by USBH_CCID_Open().
Slot	Slot index (counting from 0).
pParamLen	- in Size of buffer for response data (may be 0). - out Length of parameter block received from the device.
pParams	Pointer to the buffer that receives the parameter block.
pProt	out Receives the protocol type: 0: T=0, 1: T=1.

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

15.2.22 USBH_CCID_SetParameters()

Description

Set slot parameters.

Prototype

```
USBH_STATUS USBH_CCID_SetParameters(          USBH_CCID_HANDLE hDevice,
                                              U8 Slot,
                                              U32 ParamLen,
                                              const U8 * pParams,
                                              U8 Prot);
```

Parameters

Parameter	Description
hDevice	Handle to an open device returned by USBH_CCID_Open().
Slot	Slot index (counting from 0).
ParamLen	Length of parameter block pointed to by pParams.
pParams	Pointer to the parameter block.
Prot	Protocol type: 0: T=0, 1: T=1.

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

15.2.23 USBH_CCID_Abort()

Description

Stop any current transfer at the slot and return to a state where the slot is ready to accept a new command. This function should also be called after serious errors from any of the USB_CCID...() functions, to restore the synchronization between the host and the card reader.

Prototype

```
USBH_STATUS USBH_CCID_Abort(USBH_CCID_HANDLE hDevice,  
                             U8 Slot);
```

Parameters

Parameter	Description
hDevice	Handle to an open device returned by USBH_CCID_Open().
Slot	Slot index (counting from 0).

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

15.2.24 USBH_CCID_GetInterfaceHandle()

Description

Return the handle to the (open) USB interface. Can be used to call USBH core functions like USBH_GetStringDescriptor().

Prototype

```
USBH_INTERFACE_HANDLE USBH_CCID_GetInterfaceHandle(USBH_CCID_HANDLE hDevice);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to an open device returned by USBH_CCID_Open().

Return value

Handle to an open interface.

15.3 Data structures

This chapter describes the emUSB-Host CCID driver data structures.

Structure	Description
USBH_CCID_DEVICE_INFO	Structure containing information about a CCID device.
USBH_CCID_CMD_PARA	Structure describing all parameters to issue a command to the CCID device.

15.3.1 USBH_CCID_DEVICE_INFO

Description

Structure containing information about a CCID device.

Type definition

```
typedef struct {
    U16      VendorId;
    U16      ProductId;
    USBH_SPEED Speed;
    U8       InterfaceNo;
    U8       Class;
    U8       SubClass;
    U8       Protocol;
    U8       ICCProtocols;
    U8       ExchangeLevel;
    USBH_INTERFACE_ID InterfaceID;
} USBH_CCID_DEVICE_INFO;
```

Structure members

Member	Description
VendorId	The Vendor ID of the device.
ProductId	The Product ID of the device.
Speed	The USB speed of the device, see USBH_SPEED.
InterfaceNo	Interface index (from USB descriptor).
Class	The Class value field of the interface
SubClass	The SubClass value field of the interface
Protocol	The Protocol value field of the interface
ICCProtocols	Supported ICC protocols (bit field), see USBH_CCID_PROTOCOL_... macros
ExchangeLevel	Host exchange level, see USBH_CCID_EXLVL_... macros
InterfaceID	ID of the interface.

15.3.2 USBH_CCID_CMD_PARA

Description

Structure describing all parameters to issue a command to the CCID device.

Type definition

```
typedef struct {
    U8      CmdType;
    U8      RespType;
    U8      Slot;
    U8      Param[];
    U32     LenData;
    const U8 * pData;
    U32     BuffSize;
    U8      * pAnswer;
    U32     LenAnswer;
    U32     LenAnswerOrig;
    U8      bStatus;
    U8      bError;
    U8      bInfo;
} USBH_CCID_CMD_PARA;
```

Structure members

Member	Description
CmdType	in CCID message type.
RespType	in Response message type.
Slot	in Slot index (counting from 0).
Param	in Message specific parameters.
LenData	in Length of command data in bytes.
pData	in Pointer to the command data.
BuffSize	in Size of buffer pointed to by pAnswer .
pAnswer	in Pointer to buffer that receives the answer data.
LenAnswer	out Length of answer data stored into pAnswer . May be truncated to BuffSize .
LenAnswerOrig	out Length of answer data received from the CCID. May be greater than BuffSize .
bStatus	out Status code returned by the CCID device.
bError	out Error code returned by the CCID device.
bInfo	out Additional info returned by the CCID device.

15.4 Type definitions

This chapter describes the types defined in the header file `USBH_CCID.h`.

Type	Description
USBH_CCID_SLOT_CHANGE_CALLBACK	Function called on a reception of a slot change event.

15.4.1 USBH_CCID_SLOT_CHANGE_CALLBACK

Description

Function called on a reception of a slot change event. Used by the function `USBH_CCID_SetOnSlotChange()`.

Type definition

```
typedef void USBH_CCID_SLOT_CHANGE_CALLBACK(void * pContext,  
                                             U32      SlotState);
```

Parameters

Parameter	Description
<code>pContext</code>	Pointer to the user-provided user context.
<code>SlotState</code>	Bit mask of slot states. Bit n indicates state of slot n (n = 0 ... NumSlots-1). If a smart card is present in the slot, the respective bit has a value of 1.

Chapter 16

MIDI Device Driver (Add-On)

This chapter describes the optional emUSB-Host add-on “MIDI device driver”. It allows communication with MIDI devices over USB.



16.1 Introduction

The MIDI driver software component of emUSB-Host allows communication with MIDI-compatible devices. The MIDI Device Class (MIDI) is an abstract USB class protocol defined by the USB Implementers Forum.

This chapter provides an explanation of the functions available to application developers via the MIDI driver software. All the functions and data types of this add-on are prefixed with 'USBH_MIDI_'.

16.1.1 Overview

A MIDI device connected to the emUSB-Host is automatically configured and added to an internal list. If the MIDI driver has been registered, it is notified via a callback when a MIDI device has been added or removed. The driver then can notify the application program, when a callback function has been registered via `USBH_MIDI_AddNotification()`. In order to communicate with such a device, the application must call `USBH_MIDI_Open()`, passing the device index. MIDI devices are identified by an index: the first connected device is assigned the index 0, the second index 1, and so on.

16.1.2 Features

The following features are provided:

- Compatibility with different MIDI devices.
- Handling of multiple MIDI devices at the same time (e.g. drum machine and synthesizer).
- Handling of MIDI devices with multiple cables (e.g. USB to MIDI converters).
- Ability to send MIDI commands to a device and receive MIDI commands from a device.
- Notifications about insertion and removal of MIDI devices.

16.1.3 Example code

Example applications that demonstrate the capabilities of the USB host controlling MIDI devices and processing events from MIDI devices.

The following examples are provided in the Application directory.

File name	Description
USBH_MIDI_Start.c	Runs on all boards. Displays USB MIDI events received from any virtual cable from any USB MIDI device.
USBH_MIDI_GUI_Keyboard.c	Configured to run on an STM32F746G-Discovery board. Displays a virtual piano keyboard on the LCD and reflects MIDI keyboard state (through Note On and Note Off MIDI events) on the virtual keyboard. Pressing the keys on the virtual keyboard sends Note On and Note Off events to all attached MIDI devices, so playing a connected synthesizer from the virtual display is possible.
USBH_MIDI_HID_Keyboard.c	Will run on any board that supports two USB devices simultaneously (e.g. single root hub plus external hub, or a microcontroller with two root hubs). Plugging in a MIDI device and a standard HID-compliant USB keyboard enables control of the MIDI device from a readily-available accessory. Each HID Key Down and Key Up event is translated to an appropriate MIDI Note On and Note Off event. The USB keyboard can change the MIDI send channel to 1 through 12 using F1 to F12.
USBH_MIDI_HID_ScrollMessage.c	Will run on any board that supports two USB devices. Scrolls a message across a Novation Launchpad Mini MK2 under

File name	Description
	control of a Griffin Powermate, a HID device that is a rotary encoder.
USBH_MIDI_GUI_PlayMIDI.C	Configured to run on an STM32F746G-Discovery board. This example demonstrates integrating the emUSB-Host MIDI class driver with the SEGGER emLib-MIDI library in order to play Standard MIDI Files.
USBH_MIDI_Sequencer.c	Runs on all boards. With a Novation Launchpad Mini MK2 you can turn your embedded host into a pattern sequencer or groovebox. Works especially well with the emPower USB Host board.
USBH_MIDI_Message.c	Runs on all boards. Uses the Novation Launchpad Mini MK2 as a display to scroll a message. Works especially well with the emPower USB Host board.
USBH_MIDI_Reversi.c	Runs on all boards. This is a non-musical application of a MIDI controller. With a Novation Launchpad Mini MK2 you can play Reversi against the computer. Works especially well with the emPower USB Host board.

16.2 API Functions

This section describes the emUSB-Host MIDI driver API functions. These functions are defined in the header file `USBH_MIDI.h`.

Function	Description
<code>USBH_MIDI_Init()</code>	Initializes and registers the MIDI device driver with emUSB-Host.
<code>USBH_MIDI_Exit()</code>	Unregisters and deinitializes the MIDI device driver from emUSB-Host.
<code>USBH_MIDI_AddNotification()</code>	Add notification callback.
<code>USBH_MIDI_RemoveNotification()</code>	Remove notification callback.
<code>USBH_MIDI_ConfigureDefaultTimeout()</code>	Sets the default read and write timeout that shall be used when a new device is connected.
<code>USBH_MIDI_Open()</code>	Opens a device given by an index.
<code>USBH_MIDI_Close()</code>	Closes a handle to an opened device.
<code>USBH_MIDI_SetTimeouts()</code>	Sets up the timeouts the host waits until the data transfer will be aborted for a specific MIDI device.
<code>USBH_MIDI_SetBuffer()</code>	Set device buffer.
<code>USBH_MIDI_GetDeviceInfo()</code>	Retrieves the information about the MIDI device.
<code>USBH_MIDI_RdData()</code>	Read USB-MIDI data.
<code>USBH_MIDI_RdEvent()</code>	Read MIDI event.
<code>USBH_MIDI_WrData()</code>	Write USB-MIDI data.
<code>USBH_MIDI_WrEvent()</code>	Write MIDI event.
<code>USBH_MIDI_GetQueueStatus()</code>	Gets the number of bytes in the receive queue.

16.2.1 USBH_MIDI_Init()

Description

Initializes and registers the MIDI device driver with emUSB-Host.

Prototype

```
U8 USBH_MIDI_Init(void);
```

Return value

- | | |
|---|--|
| 1 | Success. |
| 0 | Could not register MIDI device driver. |

16.2.2 USBH_MIDI_Exit()

Description

Unregisters and deinitializes the MIDI device driver from emUSB-Host.

Prototype

```
void USBH_MIDI_Exit(void);
```

Additional information

Before this function is called any notifications added by `USBH_MIDI_AddNotification()` must be removed using `USBH_MIDI_RemoveNotification()`.

This function will release resources that were used by this device driver. It must be called if the application is closed and must to be called before `USBH_Exit()`. No more functions of this module may be called after calling `USBH_MIDI_Exit()`. The only exception is `USBH_MIDI_Init()`, which would in turn reinitialize the module and allows further calls.

16.2.3 USBH_MIDI_AddNotification()

Description

Add notification callback.

Prototype

```
USBH_STATUS USBH_MIDI_AddNotification(USBH_NOTIFICATION_HOOK * pHook,
                                     USBH_NOTIFICATION_FUNC * pfNotification,
                                     void * pContext);
```

Parameters

Parameter	Description
pHook	Pointer to user-provided USBH_NOTIFICATION_HOOK variable.
pfNotification	Pointer to function to be called when a device is connected or disconnected.
pContext	Pointer to user context that is passed to the callback function.

Return value

- = USBH_STATUS_SUCCESS

≠ USBH_STATUS_SUCCESS
- Success.

Error indication.

16.2.4 USBH_MIDI_RemoveNotification()

Description

Remove notification callback.

Prototype

```
USBH_STATUS USBH_MIDI_RemoveNotification(const USBH_NOTIFICATION_HOOK * pHook);
```

Parameters

Parameter	Description
<code>pHook</code>	Pointer to a user-provided USBH_NOTIFICATION_HOOK variable.

Return value

= USBH_STATUS_SUCCESS	Success.
≠ USBH_STATUS_SUCCESS	Error indication.

16.2.5 USBH_MIDI_ConfigureDefaultTimeout()

Description

Sets the default read and write timeout that shall be used when a new device is connected.

Prototype

```
void USBH_MIDI_ConfigureDefaultTimeout(U32 RdTimeout,  
                                       U32 WrTimeout);
```

Parameters

Parameter	Description
RdTimeout	Default read timeout, milliseconds.
WrTimeout	Default write timeout, milliseconds.

Additional information

The function shall be called after `USBH_MIDI_Init()` has been called, otherwise the behavior is undefined.

16.2.6 USBH_MIDI_Open()

Description

Opens a device given by an index.

Prototype

```
USBH_MIDI_HANDLE USBH_MIDI_Open(unsigned Index);
```

Parameters

Parameter	Description
Index	Index of the device that shall be opened. In general this means: the first connected device is 0, second device is 1 etc.

Return value

≠ USBH_MIDI_INVALID_HANDLE
= USBH_MIDI_INVALID_HANDLE

Handle to the device.
Device could not be opened (removed or not available).

16.2.7 USBH_MIDI_Close()

Description

Closes a handle to an opened device.

Prototype

```
USBH_STATUS USBH_MIDI_Close(USBH_MIDI_HANDLE hDevice);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to a opened device.

Return value

= USBH_STATUS_SUCCESS	Success.
≠ USBH_STATUS_SUCCESS	Error indication.

16.2.8 USBH_MIDI_SetBuffer()

Description

Set device buffer.

Prototype

```
USBH_STATUS USBH_MIDI_SetBuffer(USBH_MIDI_HANDLE hDevice,  
                                U8 * pBuffer,  
                                unsigned BufferLen);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to the opened device.
<code>pBuffer</code>	Pointer to buffer memory to use (minimum four bytes).
<code>BufferLen</code>	Size of buffer memory in bytes (minimum four bytes, and must also be a multiple of four).

Return value

= USBH_STATUS_SUCCESS Success.
≠ USBH_STATUS_SUCCESS Error indication.

Additional information

It is not necessary to set a buffer for a device, although doing so will improve throughput. By default, all MIDI events written by `USBH_MIDI_WrEvent()` will be sent immediately as a four-byte USB transaction.

The buffer should ideally be a multiple of the MIDI event size, which is four bytes, and of the USB endpoint's maximum packet size. Usually a buffer size of 64 bytes is recommended, it will hold 16 MIDI events and nicely matches the typical bulk endpoint size of 64 bytes.

16.2.9 USBH_MIDI_SetTimeouts()

Description

Sets up the timeouts the host waits until the data transfer will be aborted for a specific MIDI device.

Prototype

```
USBH_STATUS USBH_MIDI_SetTimeouts(USBH_MIDI_HANDLE hDevice,  
                                   U32               ReadTimeout,  
                                   U32               WriteTimeout);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device.
ReadTimeout	Read time-out given in ms.
WriteTimeout	Write time-out given in ms.

Return value

= USBH_STATUS_SUCCESS	Success.
≠ USBH_STATUS_SUCCESS	Error indication.

16.2.10 USBH_MIDI_GetDeviceInfo()

Description

Retrieves the information about the MIDI device.

Prototype

```
USBH_STATUS USBH_MIDI_GetDeviceInfo(USBH_MIDI_HANDLE    hDevice,  
                                   USBH_MIDI_DEVICE_INFO * pDevInfo);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device.
pDevInfo	Pointer to a USBH_MIDI_DEVICE_INFO structure that receives information related to the device.

Return value

= USBH_STATUS_SUCCESS	Success.
≠ USBH_STATUS_SUCCESS	Error indication.

16.2.11 USBH_MIDI_RdData()

Description

Read USB-MIDI data.

Prototype

```
USBH_STATUS USBH_MIDI_RdData(USBH_MIDI_HANDLE hDevice,  
                             U8 * pData,  
                             U32 DataLen,  
                             U32 * pRdDataLen);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device.
pData	Pointer to object that receives the USB-MIDI data.
DataLen	Octet length of the receiving object.
pRdDataLen	Pointer to object that receives the number of octets read into the receiving object. Can be NULL.

Return value

= USBH_STATUS_SUCCESS	Successful.
≠ USBH_STATUS_SUCCESS	An error or timeout occurred.

16.2.12 USBH_MIDI_RdEvent()

Description

Read MIDI event.

Prototype

```
USBH_STATUS USBH_MIDI_RdEvent(USBH_MIDI_HANDLE hDevice,  
                               U32 * pEvent);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device.
pEvent	Pointer to object that receives the MIDI event.

Return value

= USBH_STATUS_SUCCESS Successful.
≠ USBH_STATUS_SUCCESS An error or timeout occurred.

Additional information

This function always reads exactly zero or one events. If there is an error or a timeout, that status is returned by the function. If an event is correctly received, the event is written to the object pointed to by [pEvent](#) and the function returns zero.

The USB-MIDI event that is sent as CC SS XX YY in transmission order is returned as a 32-bit value where CC is encoded in bits 31...24, SS in bits 23...16, XX in bits 15...8, and YY in bits 7...0.

16.2.13 USBH_MIDI_WrData()

Description

Write USB-MIDI data.

Prototype

```
USBH_STATUS USBH_MIDI_WrData(      USBH_MIDI_HANDLE hDevice,  
                                   const U8          * pData,  
                                   U32                DataLen,  
                                   U32                * pWrDataLen);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device.
pData	Pointer to object to write.
DataLen	Octet length of the object to write. Must be a multiple of four.
pWrDataLen	Pointer to the object that receives the number of bytes written. Can be null.

Return value

= USBH_STATUS_SUCCESS Success.
≠ USBH_STATUS_SUCCESS Error indication.

Additional information

By default the MIDI data is written to the device immediately and is not buffered. If a buffer has been set using `USBH_MIDI_SetBuffer()`, the data may be buffered internally. Any buffered data can be sent to the device using `USBH_MIDI_Send()`.

16.2.14 USBH_MIDI_WrEvent()

Description

Write MIDI event.

Prototype

```
USBH_STATUS USBH_MIDI_WrEvent(USBH_MIDI_HANDLE hDevice,  
                               U32               Event);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device.
Event	MIDI event encoded as a 32-bit unsigned.

Return value

= USBH_STATUS_SUCCESS Success.
≠ USBH_STATUS_SUCCESS Error indication.

Additional information

By default the MIDI event is written to the device immediately and is not buffered. If a buffer has been set using `USBH_MIDI_SetBuffer()`, the event is written to the buffer and, when the buffer is full, all buffered MIDI events are sent. Any buffered data can be sent to the device using `USBH_MIDI_Send()`.

The USB-MIDI event that is sent as CC SS XX YY in transmission order is encoded in [Event](#) as a 32-bit value where CC is encoded in bits 31...24, SS in bits 23...16, XX in bits 15...8, and YY in bits 7...0.

16.2.15 USBH_MIDI_Send()

Description

Send any buffered data to device.

Prototype

```
USBH_STATUS USBH_MIDI_Send(USBH_MIDI_HANDLE hDevice);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to the opened device.

Return value

= USBH_STATUS_SUCCESS	Success.
≠ USBH_STATUS_SUCCESS	Error indication.

16.2.16 USBH_MIDI_GetQueueStatus()

Description

Gets the number of bytes in the receive queue.

Prototype

```
USBH_STATUS USBH_MIDI_GetQueueStatus(USBH_MIDI_HANDLE hDevice,  
                                     U32               * pRxBytes);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device.
pRxBytes	Pointer to a variable of type U32 which receives the number of bytes in the receive queue.

Return value

= USBH_STATUS_SUCCESS Success.
≠ USBH_STATUS_SUCCESS Error indication.

16.3 Data structures

This section describes the emUSB-Host MIDI driver data structures.

Structure	Description
USBH_MIDI_DEVICE_INFO	Structure containing information about a MIDI device.

16.3.1 USBH_MIDI_DEVICE_INFO

Description

Structure containing information about a MIDI device.

Type definition

```
typedef struct {  
    U16          VendorId;  
    U16          ProductId;  
    unsigned     DevIndex;  
    U16          bcdDevice;  
    USBH_SPEED   Speed;  
    USBH_INTERFACE_ID InterfaceId;  
    unsigned     NumInCables;  
    unsigned     NumOutCables;  
} USBH_MIDI_DEVICE_INFO;
```

Structure members

Member	Description
VendorId	Vendor ID of the device.
ProductId	Product ID of the device.
DevIndex	Device index.
bcdDevice	BCD-coded device version.
Speed	USB speed of the device, see USBH_SPEED .
InterfaceId	USBH interface ID.
NumInCables	Number of MIDI IN cables
NumOutCables	Number of MIDI OUT cables

Chapter 17

AUDIO Device Driver (Add-On)

This chapter describes the optional emUSB-Host add-on “AUDIO device driver”. It allows communication with USB audio devices.

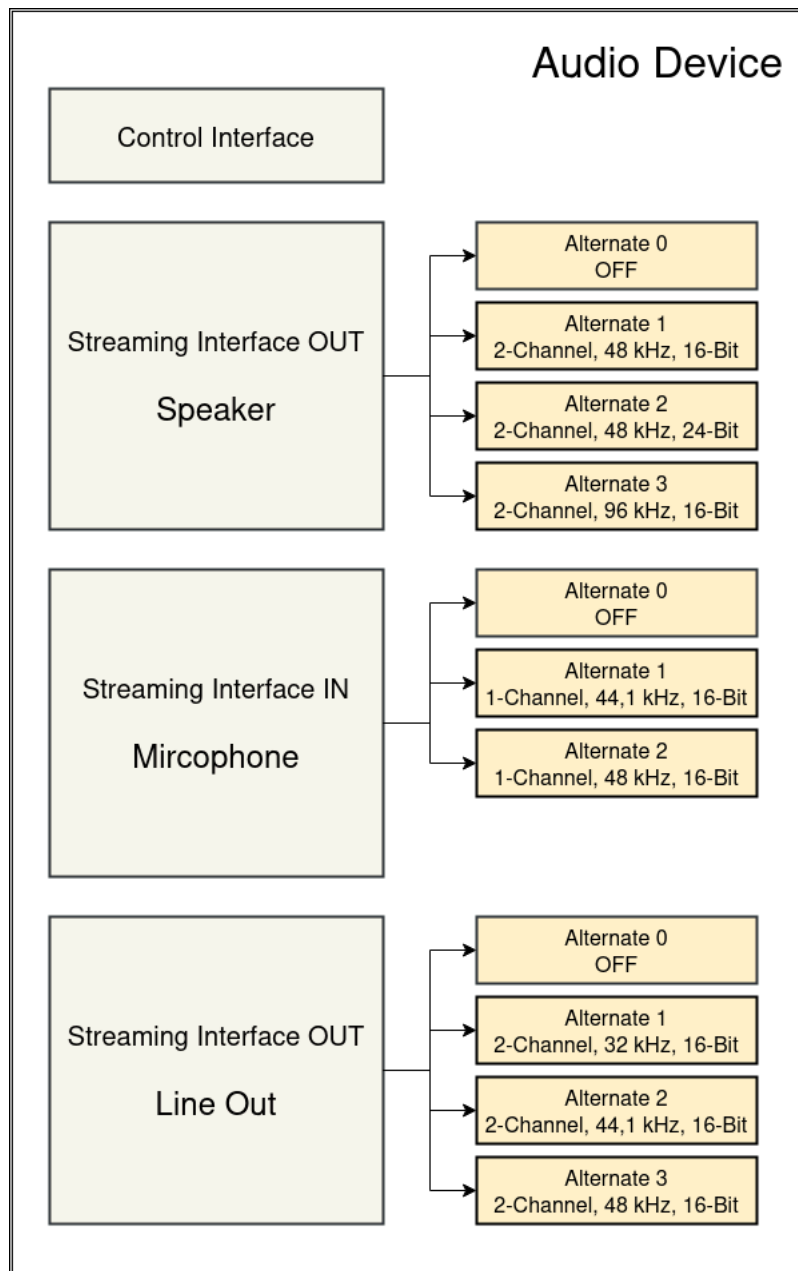


17.1 Overview

The AUDIO driver software component of emUSB-Host allows communication with USB Audio V1 compatible devices like speakers or microphones.

Audio devices contains multiple USB interfaces: One control interface and one or more audio streaming interfaces. The control interface is used to manage audio controls like volume, mute, etc. A streaming interface is used to transfer audio data into and out of audio functions like speakers, microphones or a Line In/Out connector.

Each streaming interface has multiple alternate settings that can be selected by the application. Alternate setting number 0 is used for the 'off' state of the streaming interface. Other alternate settings may be selected to use different audio stream configurations like number of audio channels (mono, stereo, ...), sample frequencies or sample resolution.



The emUSB-Host audio class manages all alternate setting of all interfaces in a single list, see `USBH_AUDIO_GetAltInfo()`.

17.1.1 Using an audio device

This section provides an explanation of the functions available to application developers via the AUDIO driver software. All the functions and data types of this add-on are prefixed with 'USBH_AUDIO_'.

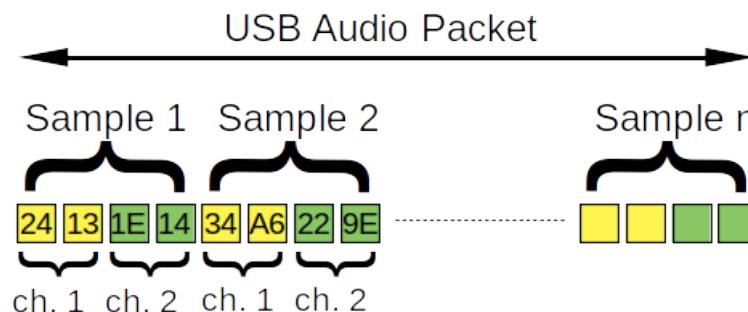
An audio device connected to the emUSB-Host is automatically configured and added to an internal list. If the AUDIO driver has been registered, it is notified via a callback when a audio device has been added or removed. The driver then can notify the application program, when a callback function has been registered via `USBH_AUDIO_AddNotification()`. In order to communicate with such a device, the application has to call the `USBH_AUDIO_Open()`, passing the interface index.

With the handle returned by `USBH_AUDIO_Open()` different functional units of the control interface can be accessed:

- Feature units (to set volume, muting, etc.)
- Selector units
- Mixer units

In order to transfer audio data, a suitable streaming interface and alternate setting of the device has to be found. For that purpose all existing streaming configurations can be explored with the function `USBH_AUDIO_GetAltInfo()` or a matching streaming configurations may be searched using `USBH_AUDIO_FindStreamConfiguration()`. If an appropriate configuration was found, the index of that configuration can be provided to `USBH_AUDIO_OpenOutputStream()` or `USBH_AUDIO_OpenInputStream()` to initialize an audio data transfer.

Audio data transfer is then done using the Read/Write API function. When audio data is transferred in PCM encoding, it consists of multiple audio samples. If for example the device uses 2 channels (stereo) and 16 bit data per channel the the data stream looks like:



Note

In order to use the AUDIO class driver, support for isochronous transfers must be enabled in the USB stack. Make sure that there is a preprocessor define in the file `USBH_Conf.h`:

```
#define USBH_SUPPORT_ISO_TRANSFER 1
```

17.1.2 Example code

There are two example application which uses the API. One to access an audio output device like a speaker, that shows how to output sound, see the file `USBH_AUDIO_Speaker.c`. Another sample application demonstrates access to an audio input device like a microphone, see the file `USBH_AUDIO_Microphone.c`.

17.2 API Functions

This chapter describes the emUSB-Host AUDIO driver API functions. These functions are defined in the header file `USBH_AUDIO.h`.

Function	Description
<code>USBH_AUDIO_Init()</code>	Initializes and registers the AUDIO device module with emUSB-Host.
<code>USBH_AUDIO_Exit()</code>	Unregisters and de-initializes the AUDIO device module from emUSB-Host.
<code>USBH_AUDIO_AddNotification()</code>	Adds a callback in order to be notified when a device is added or removed.
<code>USBH_AUDIO_RemoveNotification()</code>	Removes a callback added via <code>USBH_AUDIO_AddNotification</code> .
<code>USBH_AUDIO_GetIndex()</code>	Return an index that can be used for call to <code>USBH_AUDIO_Open()</code> for a given interface ID.
<code>USBH_AUDIO_Open()</code>	Opens an audio interface given by an index.
<code>USBH_AUDIO_Close()</code>	Closes a handle to an opened interface.
<code>USBH_AUDIO_GetInterfaceInfo()</code>	Retrieves information about the AUDIO control interface.
<code>USBH_AUDIO_GetInterfaceHandle()</code>	Return the handle to the (open) USB interface.
<code>USBH_AUDIO_GetDescriptorList()</code>	Retrieves a list of all descriptors for the AUDIO interface.
<code>USBH_AUDIO_FreeDescriptorList()</code>	Releases memory allocated by <code>USBH_AUDIO_GetDescriptorList</code> .
<code>USBH_AUDIO_GetAltInfo()</code>	Retrieves information about an audio streaming configuration.
<code>USBH_AUDIO_FindStreamConfiguration()</code>	Find a streaming configuration (alternate setting) that matches pMask.
<code>USBH_AUDIO_GetFeatureUnitInfo()</code>	Retrieves information about a feature unit.
<code>USBH_AUDIO_GetSelectorUnitInfo()</code>	Retrieves information about a selector unit.
<code>USBH_AUDIO_GetMixerUnitInfo()</code>	Retrieves information about a mixer unit.
<code>USBH_AUDIO_GetSampleFrequency()</code>	Get the current sample frequency for an audio streaming interface.
<code>USBH_AUDIO_SetSampleFrequency()</code>	Set the sample frequency for an audio streaming interface.
<code>USBH_AUDIO_GetVolume()</code>	Get the volume setting of a feature unit.
<code>USBH_AUDIO_SetVolume()</code>	Set the volume of a feature unit.
<code>USBH_AUDIO_GetMute()</code>	Get the mute setting of a feature unit.
<code>USBH_AUDIO_SetMute()</code>	Enable or disable muting of a feature unit.
<code>USBH_AUDIO_GetFeatureUnitControl()</code>	Get a control value of a feature unit.
<code>USBH_AUDIO_SetFeatureUnitControl()</code>	Set a control value of a feature unit.
<code>USBH_AUDIO_GetMixerUnitControl()</code>	Get a control value of a mixer unit.
<code>USBH_AUDIO_SetMixerUnitControl()</code>	Set a control value of a mixer unit.
<code>USBH_AUDIO_GetSelectorUnitControl()</code>	Get a control value of a selector unit.
<code>USBH_AUDIO_SetSelectorUnitControl()</code>	Set a control value of a selector unit.

Function	Description
USBH_AUDIO_GetOutPacketSize()	The function returns the packet size in bytes of an ISO packet to be send to the host.
USBH_AUDIO_OpenOutputStream()	Prepare an interface for audio OUT data.
USBH_AUDIO_OpenInStream()	Prepare an interface for audio IN data.
USBH_AUDIO_CloseStream()	Close an audio data stream.
USBH_AUDIO_Write()	Write data to an audio streaming endpoint.
USBH_AUDIO_GetNumQueueEntries()	Get number of data entries in the write queue, see USBH_AUDIO_Write() .
USBH_AUDIO_GetFreeQueueEntries()	Get number of free slots in the write queue, see USBH_AUDIO_Write() .
USBH_AUDIO_CheckBufferIdle()	Check if a buffer used by a preceding call to USBH_AUDIO_Write() can be reused for the next transfer.
USBH_AUDIO_Receive()	Receive a data packet from an audio streaming endpoint.
USBH_AUDIO_Ack()	Acknowledge a data packet that was received using USBH_AUDIO_Receive() .
USBH_AUDIO_Read()	Read data from an audio streaming endpoint.
USBH_AUDIO_GetInterfaceHandle()	Return the handle to the (open) USB interface.

17.2.1 USBH_AUDIO_Init()

Description

Initializes and registers the AUDIO device module with emUSB-Host.

Prototype

```
USBH_STATUS USBH_AUDIO_Init(void);
```

Return value

USBH_STATUS_SUCCESS Success or module already initialized.

17.2.2 USBH_AUDIO_Exit()

Description

Unregisters and de-initializes the AUDIO device module from emUSB-Host.

Prototype

```
void USBH_AUDIO_Exit(void);
```

Additional information

Has to be called the same number of times `USBH_AUDIO_Init` was called in order to de-initialize the module. This function will release resources that were used by this device driver. It has to be called if the application is closed. This has to be called before `USBH_Exit()` is called. No more functions of this module may be called after calling `USBH_AUDIO_Exit()`. The only exception is `USBH_AUDIO_Init()`, which would in turn re-init the module and allow further calls.

17.2.3 USBH_AUDIO_AddNotification()

Description

Adds a callback in order to be notified when a device is added or removed.

Prototype

```
USBH_STATUS USBH_AUDIO_AddNotification(USBH_NOTIFICATION_HOOK * pHook,  
                                       USBH_NOTIFICATION_FUNC * pfNotification,  
                                       void * pContext);
```

Parameters

Parameter	Description
pHook	Pointer to a user provided USBH_NOTIFICATION_HOOK variable.
pfNotification	Pointer to a function the stack should call when a device is connected or disconnected.
pContext	Pointer to a user context that is passed to the callback function.

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

17.2.4 USBH_AUDIO_RemoveNotification()

Description

Removes a callback added via USBH_AUDIO_AddNotification.

Prototype

```
USBH_STATUS USBH_AUDIO_RemoveNotification(const USBH_NOTIFICATION_HOOK * pHook);
```

Parameters

Parameter	Description
<code>pHook</code>	Pointer to a user provided USBH_NOTIFICATION_HOOK variable.

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

17.2.5 USBH_AUDIO_GetIndex()

Description

Return an index that can be used for call to USBH_AUDIO_Open() for a given interface ID.

Prototype

```
int USBH_AUDIO_GetIndex(USBH_INTERFACE_ID InterfaceID);
```

Parameters

Parameter	Description
InterfaceID	Id of the interface.

Return value

≥ 0 Index of the AUDIO interface.
< 0 InterfaceID not found.

17.2.6 USBH_AUDIO_Open()

Description

Opens an audio interface given by an index.

Prototype

```
USBH_STATUS USBH_AUDIO_Open(unsigned          Index,  
                             USBH_AUDIO_HANDLE * pHandle);
```

Parameters

Parameter	Description
Index	Index of the interface that shall be opened. The index of a new connected device is provided to the callback function registered with USBH_AUDIO_AddNotification().
pHandle	out Handle to an AUDIO interface on success.

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

17.2.7 USBH_AUDIO_Close()

Description

Closes a handle to an opened interface.

Prototype

```
void USBH_AUDIO_Close(USBH_AUDIO_HANDLE hDevice);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to an open interface returned by <code>USBH_AUDIO_Open()</code> .

17.2.8 USBH_AUDIO_GetInterfaceInfo()

Description

Retrieves information about the AUDIO control interface.

Prototype

```
USBH_STATUS USBH_AUDIO_GetInterfaceInfo(USBH_AUDIO_HANDLE hDevice,  
                                         USBH_AUDIO_INTERFACE_INFO * pDevInfo);
```

Parameters

Parameter	Description
hDevice	Handle to an open interface returned by <code>USBH_AUDIO_Open()</code> .
pDevInfo	Pointer to a <code>USBH_AUDIO_INTERFACE_INFO</code> structure that receives the information.

Return value

`USBH_STATUS_SUCCESS` on success or error code on failure.

17.2.9 USBH_AUDIO_GetInterfaceHandle()

Description

Return the handle to the (open) USB interface. Can be used to call USBH core functions like USBH_GetStringDescriptor().

Prototype

```
USBH_INTERFACE_HANDLE USBH_AUDIO_GetInterfaceHandle(USBH_AUDIO_HANDLE hDevice);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to an open interface returned by USBH_AUDIO_Open().

Return value

Handle to an open interface.

17.2.10 USBH_AUDIO_GetDescriptorList()

Description

Retrieves a list of all descriptors for the AUDIO interface. The memory for the list returned is allocated by this function and must be freed later using `USBH_AUDIO_FreeDescriptorList()`.

Prototype

```
unsigned USBH_AUDIO_GetDescriptorList(USBH_AUDIO_HANDLE    hDevice,  
                                     USBH_AUDIO_DESCRIPTOR ** ppDescList);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to an open interface returned by <code>USBH_AUDIO_Open()</code> .
<code>ppDescList</code>	Returns a pointer to an array of <code>USBH_AUDIO_DESCRIPTOR</code> structures.

Return value

Number of descriptors returned in the list. 0 on error.

17.2.11 USBH_AUDIO_FreeDescriptorList()

Description

Releases memory allocated by USBH_AUDIO_GetDescriptorList.

Prototype

```
void USBH_AUDIO_FreeDescriptorList(USBH_AUDIO_HANDLE    hDevice,  
                                   USBH_AUDIO_DESCRIPTOR * pDescList);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to an open interface returned by USBH_AUDIO_Open().
<code>pDescList</code>	Pointer to an array of USBH_AUDIO_DESCRIPTOR structures, returned by USBH_AUDIO_GetDescriptorList().

17.2.12 USBH_AUDIO_GetAltInfo()

Description

Retrieves information about an audio streaming configuration.

Prototype

```
USBH_AUDIO_ALT_INFO *USBH_AUDIO_GetAltInfo(USBH_AUDIO_HANDLE hDevice,  
                                             unsigned n);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to an open interface returned by <code>USBH_AUDIO_Open()</code> .
<code>n</code>	Return the <code>n</code> -th streaming configuration of the audio device.

Return value

Pointer to a streaming configuration (read-only).

Additional information

To get information about all streaming configurations, the function should be called for `n=0,1,2,...` until it returns NULL.

17.2.13 USBH_AUDIO_FindStreamConfiguration()

Description

Find a streaming configuration (alternate setting) that matches `pMask`. The comparison with the mask is true if each mandatory member matches the stream configuration and each optional member that is marked as valid by a flag in the mask member matches.

Prototype

```
int USBH_AUDIO_FindStreamConfiguration(    USBH_AUDIO_HANDLE    hDevice,
                                           unsigned            n,
                                           const USBH_AUDIO_STREAM_MASK * pMask);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to an open interface returned by <code>USBH_AUDIO_Open()</code> .
<code>n</code>	Start searching at the <code>n</code> -th streaming configuration of the audio device. May be set to a value > 0 for repeated calls of this function to find multiple matches.
<code>pMask</code>	Pointer to a structure of type <code>USBH_AUDIO_STREAM_MASK</code> , that specifies the audio stream selection pattern.

Return value

Index of a matching streaming configuration of the audio device. A negative value means that nothing was found.

17.2.14 USBH_AUDIO_GetFeatureUnitInfo()

Description

Retrieves information about a feature unit.

Prototype

```
USBH_STATUS USBH_AUDIO_GetFeatureUnitInfo(USBH_AUDIO_HANDLE hDevice,
                                           unsigned n,
                                           USBH_AUDIO_FU_INFO * pFUInfo);
```

Parameters

Parameter	Description
hDevice	Handle to an open interface returned by USBH_AUDIO_Open().
n	Get information about the n-th feature unit. Units are counted starting with 0.
pFUInfo	Pointer to a USBH_AUDIO_FU_INFO structure, which is filled by the function.

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

Additional information

To get information of all feature units, the function should be called with n = 0,1,2,... until the function returns USBH_STATUS_NOT_FOUND.

17.2.15 USBH_AUDIO_GetSelectorUnitInfo()

Description

Retrieves information about a selector unit.

Prototype

```
USBH_STATUS USBH_AUDIO_GetSelectorUnitInfo(USBH_AUDIO_HANDLE hDevice,  
                                           unsigned n,  
                                           USBH_AUDIO_SU_INFO * pSUInfo);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to an open interface returned by <code>USBH_AUDIO_Open()</code> .
<code>n</code>	Get information about the <code>n</code> -th selector unit. Units are counted starting with 0.
<code>pSUInfo</code>	Pointer to a <code>USBH_AUDIO_SU_INFO</code> structure, which is filled by the function.

Return value

`USBH_STATUS_SUCCESS` on success or error code on failure.

Additional information

To get information of all selector units, the function should be called with `n = 0,1,2,...` until the function returns `USBH_STATUS_NOT_FOUND`.

17.2.16 USBH_AUDIO_GetMixerUnitInfo()

Description

Retrieves information about a mixer unit.

Prototype

```
USBH_STATUS USBH_AUDIO_GetMixerUnitInfo(USBH_AUDIO_HANDLE    hDevice,
                                         unsigned             n,
                                         USBH_AUDIO_MU_INFO *  pMUInfo);
```

Parameters

Parameter	Description
hDevice	Handle to an open interface returned by USBH_AUDIO_Open().
n	Get information about the n-th mixer unit. Units are counted starting with 0.
pMUInfo	Pointer to a USBH_AUDIO_MU_INFO structure, which is filled by the function.

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

Additional information

To get information of all mixer units, the function should be called with n = 0,1,2,... until the function returns USBH_STATUS_NOT_FOUND.

17.2.17 USBH_AUDIO_GetSampleFrequency()

Description

Get the current sample frequency for an audio streaming interface. This function can be used for audio 1.0 devices only.

Prototype

```
USBH_STATUS USBH_AUDIO_GetSampleFrequency(USBH_AUDIO_STREAM hStream,  
                                           U32 * pSampleFrequency);
```

Parameters

Parameter	Description
hStream	Handle to an open audio stream returned by <code>USBH_AUDIO_OpenOutputStream()</code> or <code>USBH_AUDIO_OpenInStream()</code> .
pSampleFrequency	out Sample frequency currently set by the device.

Return value

`USBH_STATUS_SUCCESS` on success or error code on failure.

17.2.18 USBH_AUDIO_SetSampleFrequency()

Description

Set the sample frequency for an audio streaming interface. This function can be used for audio 1.0 devices only.

Prototype

```
USBH_STATUS USBH_AUDIO_SetSampleFrequency(USBH_AUDIO_STREAM hStream,
                                           U32 * pSampleFrequency);
```

Parameters

Parameter	Description
<code>hStream</code>	Handle to an open audio stream returned by <code>USBH_AUDIO_OpenOutputStream()</code> or <code>USBH_AUDIO_OpenInStream()</code> .
<code>pSampleFrequency</code>	<code>in</code> Sample frequency to set. <code>out</code> Sample frequency actually set by the device.

Return value

`USBH_STATUS_SUCCESS` on success or error code on failure.

17.2.19 USBH_AUDIO_GetVolume()

Description

Get the volume setting of a feature unit. This function can be used for audio 1.0 devices only.

Prototype

```
USBH_STATUS USBH_AUDIO_GetVolume(USBH_AUDIO_HANDLE    hDevice,
                                  U8                    Unit,
                                  U8                    Channel,
                                  USBH_AUDIO_VOLUME_INFO * pVolumeInfo);
```

Parameters

Parameter	Description
hDevice	Handle to an open interface returned by USBH_AUDIO_Open().
Unit	ID of the feature unit.
Channel	Control channel number.
pVolumeInfo	out Volume setting.

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

17.2.20 USBH_AUDIO_SetVolume()

Description

Set the volume of a feature unit. This function can be used for audio 1.0 devices only.

Prototype

```
USBH_STATUS USBH_AUDIO_SetVolume(USBH_AUDIO_HANDLE hDevice,
                                   U8 Unit,
                                   U8 Channel,
                                   I16 Volume);
```

Parameters

Parameter	Description
hDevice	Handle to an open interface returned by <code>USBH_AUDIO_Open()</code> .
Unit	ID of the feature unit.
Channel	Control channel number.
Volume	Volume value (in decibel * 256).

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

17.2.21 USBH_AUDIO_GetMute()

Description

Get the mute setting of a feature unit. This function can be used for audio 1.0 devices only.

Prototype

```
USBH_STATUS USBH_AUDIO_GetMute(USBH_AUDIO_HANDLE hDevice,
                                U8 Unit,
                                U8 Channel,
                                U8 * pMute);
```

Parameters

Parameter	Description
hDevice	Handle to an open interface returned by USBH_AUDIO_Open().
Unit	ID of the feature unit.
Channel	Control channel number.
pMute	out Mute setting (0 = audible, 1 = muted).

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

17.2.22 USBH_AUDIO_SetMute()

Description

Enable or disable muting of a feature unit. This function can be used for audio 1.0 devices only.

Prototype

```
USBH_STATUS USBH_AUDIO_SetMute(USBH_AUDIO_HANDLE hDevice,
                                U8 Unit,
                                U8 Channel,
                                U8 Mute);
```

Parameters

Parameter	Description
hDevice	Handle to an open interface returned by <code>USBH_AUDIO_Open()</code> .
Unit	ID of the feature unit.
Channel	Control channel number.
Mute	Mute value (0 = audible, 1 = muted).

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

17.2.23 USBH_AUDIO_GetFeatureUnitControl()

Description

Get a control value of a feature unit. This function can be used for audio 1.0 devices only.

Prototype

```
USBH_STATUS USBH_AUDIO_GetFeatureUnitControl(USBH_AUDIO_HANDLE hDevice,
                                             U8 RequestType,
                                             U8 Unit,
                                             U16 Selector,
                                             U16 Channel,
                                             U32 * pDataLen,
                                             U8 * pBuff);
```

Parameters

Parameter	Description
hDevice	Handle to an open interface returned by <code>USBH_AUDIO_Open()</code> .
RequestType	Control unit request types. Use one of the <code>USBH_AUDIO_REQUEST_GET_...</code> macros.
Unit	ID of the feature unit.
Selector	Control selector, see <code>USBH_AUDIO_SELECTOR_FU_...</code> macros.
Channel	Control channel number.
pDataLen	in Size of the data buffer provided by <code>pData</code> (in bytes). out Length of the data returned by the audio device.
pBuff	Pointer to the buffer where the control data is stored. Details of the data returned depending on the given selector are specified in the document "Universal Serial Bus Device Class Definition for Audio Devices".

Return value

`USBH_STATUS_SUCCESS` on success or error code on failure.

17.2.24 USBH_AUDIO_SetFeatureUnitControl()

Description

Set a control value of a feature unit. This function can be used for audio 1.0 devices only.

Prototype

```
USBH_STATUS USBH_AUDIO_SetFeatureUnitControl(    USBH_AUDIO_HANDLE  hDevice,
                                                  U8              Unit,
                                                  U16           Selector,
                                                  U16           Channel,
                                                  U32           DataLen,
                                                  const U8        * pData);
```

Parameters

Parameter	Description
hDevice	Handle to an open interface returned by USBH_AUDIO_Open().
Unit	ID of the feature unit.
Selector	Control selector, see USBH_AUDIO_SELECTOR_FU_... macros.
Channel	Control channel number.
DataLen	Length of the data to be set.
pData	Pointer to the data. Details of the data required depending on the given selector are specified in the document "Universal Serial Bus Device Class Definition for Audio Devices".

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

17.2.25 USBH_AUDIO_GetMixerUnitControl()

Description

Get a control value of a mixer unit. This function can be used for audio 1.0 devices only.

Prototype

```
USBH_STATUS USBH_AUDIO_GetMixerUnitControl(USBH_AUDIO_HANDLE hDevice,
                                             U8 RequestType,
                                             U8 Unit,
                                             U8 InChannel,
                                             U8 OutChannel,
                                             I16 * pValue);
```

Parameters

Parameter	Description
hDevice	Handle to an open interface returned by USBH_AUDIO_Open().
RequestType	Control unit request types. Use one of the USBH_AUDIO_REQUEST_GET_... macros.
Unit	ID of the feature unit.
InChannel	Input channel of the mixer (must be ≥ 1).
OutChannel	Output channel of the mixer (must be ≥ 1).
pValue	out Mixer control value in decibel * 256.

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

17.2.26 USBH_AUDIO_SetMixerUnitControl()

Description

Set a control value of a mixer unit. This function can be used for audio 1.0 devices only.

Prototype

```
USBH_STATUS USBH_AUDIO_SetMixerUnitControl(USBH_AUDIO_HANDLE hDevice,
                                             U8 Unit,
                                             U8 InChannel,
                                             U8 OutChannel,
                                             I16 Value);
```

Parameters

Parameter	Description
hDevice	Handle to an open interface returned by USBH_AUDIO_Open().
Unit	ID of the feature unit.
InChannel	Input channel of the mixer (must be ≥ 1).
OutChannel	Output channel of the mixer (must be ≥ 1).
Value	Mixer control value in decibel * 256.

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

17.2.27 USBH_AUDIO_GetSelectorUnitControl()

Description

Get a control value of a selector unit. This function can be used for audio 1.0 devices only.

Prototype

```
USBH_STATUS USBH_AUDIO_GetSelectorUnitControl(USBH_AUDIO_HANDLE hDevice,
                                              U8 RequestType,
                                              U8 Unit,
                                              U8 * pValue);
```

Parameters

Parameter	Description
hDevice	Handle to an open interface returned by USBH_AUDIO_Open().
RequestType	Control unit request types. Use one of the USBH_AUDIO_REQUEST_GET_... macros.
Unit	ID of the feature unit.
pValue	out Selector control value.

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

17.2.28 USBH_AUDIO_SetSelectorUnitControl()

Description

Set a control value of a selector unit. This function can be used for audio 1.0 devices only.

Prototype

```
USBH_STATUS USBH_AUDIO_SetSelectorUnitControl(USBH_AUDIO_HANDLE hDevice,
                                              U8 Unit,
                                              U8 Value);
```

Parameters

Parameter	Description
hDevice	Handle to an open interface returned by USBH_AUDIO_Open().
Unit	ID of the feature unit.
Value	Selector control value.

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

17.2.29 USBH_AUDIO_GetOutPacketSize()

Description

The function returns the packet size in bytes of an ISO packet to be send to the host.

Prototype

```
unsigned USBH_AUDIO_GetOutPacketSize(USBH_AUDIO_STREAM hStream,  
                                     int Max);
```

Parameters

Parameter	Description
<code>hStream</code>	Handle to an open audio output stream returned by <code>USBH_AUDIO_OpenOutputStream()</code> .
<code>Max</code>	Select minimum or maximum packet size. • 0 - Return the minimum size of an ISO packet. • 1 - Return the maximum size of an ISO packet.

Return value

ISO packet size in bytes. 0 on error.

17.2.30 USBH_AUDIO_OpenOutputStream()

Description

Prepare an interface for audio OUT data. If successful, successive calls to USBH_AUDIO_write() should be used to send audio data.

Prototype

```
USBH_STATUS USBH_AUDIO_OpenOutputStream(USBH_AUDIO_HANDLE hDevice,
                                         unsigned          StreamConfIdx,
                                         U32                SampleFrequency,
                                         USBH_AUDIO_STREAM * pHandle);
```

Parameters

Parameter	Description
hDevice	Handle to an open interface returned by USBH_AUDIO_Open().
StreamConfIdx	Index of a streaming configuration (alternate setting) to be used. Can be retrieved using USBH_AUDIO_GetAltInfo() or USBH_AUDIO_FindStreamConfiguration().
SampleFrequency	Sample frequency in Hz to be used.
pHandle	out Handle to a audio output stream.

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

17.2.31 USBH_AUDIO_OpenInStream()

Description

Prepare an interface for audio IN data. If successful, audio data will be read from the device and can be retrieved using USBH_AUDIO_Read().

Prototype

```
USBH_STATUS USBH_AUDIO_OpenInStream(USBH_AUDIO_HANDLE hDevice,
                                     unsigned          StreamConfIdx,
                                     USBH_AUDIO_STREAM * pHandle);
```

Parameters

Parameter	Description
hDevice	Handle to an open interface returned by USBH_AUDIO_Open().
StreamConfIdx	Index of a streaming configuration (alternate setting) to be used. Can be retrieved using USBH_AUDIO_GetAltInfo() or USBH_AUDIO_FindStreamConfiguration().
pHandle	out Handle to a audio input stream.

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

17.2.32 USBH_AUDIO_CloseStream()

Description

Close an audio data stream. Ongoing transfers are aborted. The handle will become invalid and must not be used further.

Prototype

```
void USBH_AUDIO_CloseStream(USBH_AUDIO_STREAM hStream);
```

Parameters

Parameter	Description
<code>hStream</code>	Handle to an open audio stream returned by <code>USBH_AUDIO_OpenOutputStream()</code> or <code>USBH_AUDIO_OpenInputStream()</code> .

17.2.33 USBH_AUDIO_Write()

Description

Write data to an audio streaming endpoint. The write request data is put into a queue and will be send executed asynchronously. The function returns immediately. If there are no space in the queue, it returns status USBH_STATUS_BUSY.

Prototype

```
USBH_STATUS USBH_AUDIO_Write(      USBH_AUDIO_STREAM  hStream,
                                   U32                  Len,
                                   const U8              * pData);
```

Parameters

Parameter	Description
hStream	Handle to an open audio stream returned by USBH_AUDIO_OpenOutputStream().
Len	Size of the data to be send. Must be at least the size of one ISO packet for the endpoint used (≥ USBH_AUDIO_GetOutPacketSize(Handle, 0)).
pData	Pointer to the data. The data must remain valid until the transaction is complete.

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

17.2.34 USBH_AUDIO_GetNumQueueEntries()

Description

Get number of data entries in the write queue, see USBH_AUDIO_Write().

Prototype

```
unsigned USBH_AUDIO_GetNumQueueEntries(USBH_AUDIO_STREAM hStream);
```

Parameters

Parameter	Description
<code>hStream</code>	Handle to an open audio stream returned by USBH_AUDIO_OpenOutputStream().

Return value

Number of data entries in the write queue.

17.2.35 USBH_AUDIO_GetFreeQueueEntries()

Description

Get number of free slots in the write queue, see USBH_AUDIO_Write().

Prototype

```
unsigned USBH_AUDIO_GetFreeQueueEntries(USBH_AUDIO_STREAM hStream);
```

Parameters

Parameter	Description
<code>hStream</code>	Handle to an open audio stream returned by USBH_AUDIO_OpenOutputStream().

Return value

Number of free slots in the write queue.

17.2.36 USBH_AUDIO_CheckBufferIdle()

Description

Check if a buffer used by a preceding call to `USBH_AUDIO_Write()` can be reused for the next transfer. If there is no entry in the write queue for the given buffer, the function returns immediately with status `USBH_STATUS_SUCCESS`. This also applies, if there was no preceding call to `USBH_AUDIO_Write()` with the given buffer. The functions returns `USBH_STATUS_NEED_MORE_DATA`, if the buffer can't be flushed because there is not enough data in the queue to build a full ISO data packet.

Prototype

```
USBH_STATUS USBH_AUDIO_CheckBufferIdle(      USBH_AUDIO_STREAM  hStream,
                                             const U8          * pBuff,
                                             int              bWait);
```

Parameters

Parameter	Description
<code>hStream</code>	Handle to an open audio stream returned by <code>USBH_AUDIO_OpenOutStream()</code> .
<code>pBuff</code>	Pointer to a data buffer that was used in a preceding call to <code>USBH_AUDIO_Write()</code> .
<code>bWait</code>	Defines behavior of the function: • 0 - Function returns immediately. • 1 - Function blocks until the buffer gets idle or an error occurs.

Return value

<code>USBH_STATUS_SUCCESS</code>	Buffer idle and can be used for new transfers.
<code>USBH_STATUS_BUSY</code>	Buffer is busy, transfer not completed. Only if <code>bWait</code> = 0.
other	Error code of failure.

17.2.37 USBH_AUDIO_Receive()

Description

Receive a data packet from an audio streaming endpoint. On success the function returns a pointer to an internal buffer that contains the data packet. The data must be acknowledged within one millisecond after it was received using the function `USBH_AUDIO_Ack()`. This will enable the driver to reuse the buffer to receive another packet.

Prototype

```
USBH_STATUS USBH_AUDIO_Receive(      USBH_AUDIO_STREAM  hStream,
                                     U32                  * pLen,
                                     const U8              ** ppData,
                                     int                    Timeout);
```

Parameters

Parameter	Description
<code>hStream</code>	Handle to an open input stream returned by <code>USBH_AUDIO_OpenInStream()</code> .
<code>pLen</code>	out Size of the data returned by the function.
<code>ppData</code>	out Pointer to the data returned by the function.
<code>Timeout</code>	<code>Timeout</code> given in milliseconds. A zero value results in an infinite timeout. If <code>Timeout</code> is -1, the function never blocks.

Return value

<code>USBH_STATUS_SUCCESS</code>	A data packet is returned by the function.
<code>USBH_STATUS_TIMEOUT</code>	No data is available.
other	Error code of failure.

17.2.38 USBH_AUDIO_Ack()

Description

Acknowledge a data packet that was received using `USBH_AUDIO_Receive()`. This will enable the driver to reuse the buffer provided by `USBH_AUDIO_Receive()` to receive another packet.

Prototype

```
USBH_STATUS USBH_AUDIO_Ack(USBH_AUDIO_STREAM hStream);
```

Parameters

Parameter	Description
<code>hStream</code>	Handle to an open input stream returned by <code>USBH_AUDIO_OpenInStream()</code> .

Return value

`USBH_STATUS_SUCCESS` on success or error code on failure.

17.2.39 USBH_AUDIO_Read()

Description

Read data from an audio streaming endpoint. This function will either return if all data have been read from the device or when the timeout is expired. The function may have read some data and stored into the buffer even if it returns an error.

The USB stack can only read complete packets from the USB device. If the size of a received packet exceeds the requested length to be read then all data that does not fit into the callers buffer (`pBuff`) is stored in an internal buffer and will be returned by the next call to `USBH_AUDIO_Read()`.

Prototype

```
USBH_STATUS USBH_AUDIO_Read(USBH_AUDIO_STREAM hStream,
                             U32                * pLen,
                             U8                  * pBuff,
                             int                Timeout);
```

Parameters

Parameter	Description
<code>hStream</code>	Handle to an open input stream returned by <code>USBH_AUDIO_OpenInStream()</code> .
<code>pLen</code>	<code>in</code> Number of bytes to be read. <code>out</code> Number of bytes actually read by the function.
<code>pBuff</code>	Pointer to the buffer to store the data.
<code>Timeout</code>	<code>Timeout</code> given in milliseconds. A zero value results in an infinite timeout. If <code>Timeout</code> is -1, the function never blocks.

Return value

`USBH_STATUS_SUCCESS` on success or error code on failure.

17.2.40 USBH_AUDIO_GetInterfaceHandle()

Description

Return the handle to the (open) USB interface. Can be used to call USBH core functions like USBH_GetStringDescriptor().

Prototype

```
USBH_INTERFACE_HANDLE USBH_AUDIO_GetInterfaceHandle(USBH_AUDIO_HANDLE hDevice);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to an open interface returned by USBH_AUDIO_Open().

Return value

Handle to an open interface.

17.3 Data structures

This chapter describes the emUSB-Host AUDIO driver data structures.

Structure	Description
USBH_AUDIO_INTERFACE_INFO	Structure containing information about an audio interface.
USBH_AUDIO_DESCRIPTOR	Structure containing a USB descriptor.
USBH_AUDIO_TERMINAL_INFO	Structure containing information about an audio output terminal unit.
USBH_AUDIO_ALT_INFO	Structure containing information about one audio streaming configuration.
USBH_AUDIO_STREAM_MASK	Is used as an input parameter for the function <code>USBH_AUDIO_FindStreamConfiguration()</code> .
USBH_AUDIO_FU_INFO	Structure containing information about an audio feature unit.
USBH_AUDIO_SU_INFO	Structure containing information about an audio selector unit.
USBH_AUDIO_MU_INFO	Structure containing information about an audio mixer unit.
USBH_AUDIO_VOLUME_INFO	Structure containing information about the volume setting of a feature unit.

17.3.1 USBH_AUDIO_INTERFACE_INFO

Description

Structure containing information about an audio interface.

Type definition

```
typedef struct {  
    U16      VendorId;  
    U16      ProductId;  
    USBH_SPEED  Speed;  
    U8      InterfaceNo;  
    U8      Class;  
    U8      SubClass;  
    U8      Protocol;  
    USBH_INTERFACE_ID  InterfaceID;  
    USBH_DEVICE_ID     DeviceId;  
} USBH_AUDIO_INTERFACE_INFO;
```

Structure members

Member	Description
VendorId	The Vendor ID of the device.
ProductId	The Product ID of the device.
Speed	The USB speed of the device, see USBH_SPEED .
InterfaceNo	Interface index (from USB descriptor).
Class	The Class value field of the interface.
SubClass	The SubClass value field of the interface.
Protocol	The Protocol value field of the interface.
InterfaceID	ID of the interface.
DeviceId	The unique device Id. This Id is assigned if the USB device was successfully enumerated. It is valid until the device is removed from the host. If the device is reconnected a different device Id is assigned.

17.3.2 USBH_AUDIO_DESCRIPTOR

Description

Structure containing a USB descriptor

Type definition

```
typedef struct {  
    U8        AlternateSetting;  
    U8        Type;  
    U8        SubType;  
    const U8 * pDesc;  
} USBH_AUDIO_DESCRIPTOR;
```

Structure members

Member	Description
AlternateSetting	Descriptor belongs to this alternate setting.
Type	The Type value field of the descriptor.
SubType	The SubType value field of the descriptor.
pDesc	Pointer to the descriptor data.

17.3.3 USBH_AUDIO_TERMINAL_INFO

Description

Structure containing information about an audio output terminal unit.

Type definition

```
typedef struct {  
    U8    UnitID;  
    U8    SourceID;  
    U16   TerminalType;  
    U8    NrChannels;  
    U8    StringIndex;  
    U16   wChannelConfig;  
} USBH_AUDIO_TERMINAL_INFO;
```

Structure members

Member	Description
UnitID	AUDIO unit ID.
SourceID	For input terminals: Value 0. For output terminals: ID of the unit connected to the input.
TerminalType	USB Audio Terminal Type.
NrChannels	Number of channels (input terminal only).
StringIndex	Index of the string descriptor for this terminal.
wChannelConfig	Describes the spatial location of the logical channels (input terminal only).

17.3.4 USBH_AUDIO_ALT_INFO

Description

Structure containing information about one audio streaming configuration.

Type definition

```
typedef struct {  
    U8    InterfaceNo;  
    U8    AlternateSetting;  
    U8    EPAddr;  
    U8    TerminalLink;  
    U16   MaxPacketSize;  
    U16   FormatTag;  
    U8    bmAttributes;  
    U8    FormatType;  
} USBH_AUDIO_ALT_INFO;
```

Structure members

Member	Description
InterfaceNo	Index of the steaming interface (0-based).
AlternateSetting	Alternate setting number.
EPAddr	Endpoint address.
TerminalLink	ID of the terminal unit, this endpoint is connected to.
MaxPacketSize	Maximum packet size of the endpoint.
FormatTag	Audio format tag.
bmAttributes	Attributes (SamplingFrequency, Pitch, MaxPacketsOnly).
FormatType	Type of format descriptor.

17.3.5 USBH_AUDIO_STREAM_MASK

Description

Is used as an input parameter for the function `USBH_AUDIO_FindStreamConfiguration()`. The comparison with the mask is true if each mandatory member matches the stream configuration and each optional member that is marked as valid by a flag in the mask member matches.

Type definition

```
typedef struct {  
    U16  Mask;  
    U8   Direction;  
    U8   NrChannels;  
    U8   BitResolution;  
    U32  SampleFrequency;  
} USBH_AUDIO_STREAM_MASK;
```

Structure members

Member	Description
Mask	This member contains the information which of the optional fields are valid. It is an OR combination of the following flags: - USBH_AUDIO_STREAM_MASK_BIT_RESOLUTION <code>BitResolution</code> is used for comparison. - USBH_AUDIO_STREAM_MASK_FREQUENCY <code>SampleFrequency</code> is used for comparison.
Direction	Specifies a direction (mandatory). It is one of the following values: - USB_IN_DIRECTION - USB_OUT_DIRECTION
NrChannels	Number of audio channels (mandatory).
BitResolution	Number of bits per audio subframe (optional).
SampleFrequency	Sampling frequency in Hz (optional).

17.3.6 USBH_AUDIO_FU_INFO

Description

Structure containing information about an audio feature unit.

Type definition

```
typedef struct {  
    U8    UnitID;  
    U8    SourceID;  
    U8    StringIndex;  
    U8    NumControlChannels;  
    U16   bmControls[];  
} USBH_AUDIO_FU_INFO;
```

Structure members

Member	Description
UnitID	AUDIO unit ID.
SourceID	ID of the unit connected to the input.
StringIndex	Index of the string descriptor for this unit.
NumControlChannels	Number of entries in Controls[].
bmControls	Bit mask containing supported controls.

17.3.7 USBH_AUDIO_SU_INFO

Description

Structure containing information about an audio selector unit.

Type definition

```
typedef struct {  
    U8  UnitID;  
    U8  StringIndex;  
    U8  NumInputs;  
    U8  SourceID[];  
} USBH_AUDIO_SU_INFO;
```

Structure members

Member	Description
UnitID	AUDIO unit ID.
StringIndex	Index of the string descriptor for this unit.
NumInputs	Number of inputs of the selector.
SourceID	ID of the unit connected to the input.

17.3.8 USBH_AUDIO_MU_INFO

Description

Structure containing information about an audio mixer unit.

Type definition

```
typedef struct {  
    U8    UnitID;  
    U8    StringIndex;  
    U8    NumInputs;  
    U8    SourceID[];  
    U8    NumOutChannels;  
    U16   wChannelConfig;  
    U8    LenControls;  
    U8    bmControls[];  
} USBH_AUDIO_MU_INFO;
```

Structure members

Member	Description
UnitID	AUDIO unit ID.
StringIndex	Index of the string descriptor for this unit.
NumInputs	Number of inputs of the selector.
SourceID	ID of the unit connected to the input.
NumOutChannels	Number of output channels.
wChannelConfig	Describes the spatial location of the logical channels.
LenControls	Size of the bmControls field in bytes.
bmControls	Bit map indicating which mixing Controls are programmable. Contains ' NumInputs * NumOutChannels ' bits.

17.3.9 USBH_AUDIO_VOLUME_INFO

Description

Structure containing information about the volume setting of a feature unit.

Type definition

```
typedef struct {  
    I16  Cur;  
    I16  Min;  
    I16  Max;  
    I16  Res;  
} USBH_AUDIO_VOLUME_INFO;
```

Structure members

Member	Description
Cur	Current volume value (in decibel * 256).
Min	Minimum volume value (in decibel * 256).
Max	Maximum volume value (in decibel * 256).
Res	Resolution of volume (in decibel * 256).

Chapter 18

CP210X Device Driver (Add-On)

This chapter describes the optional emUSB-Host add-on “CP210X device driver”. It allows communication with CP210x USB devices, typically serving as USB to UART bridges.



18.1 Introduction

The CP210X driver software component of emUSB-Host allows the communication with CP210x devices. It implements the CP210x protocol specified by Silicon Labs which is a vendor specific protocol. The protocol allows emulation of serial communication via USB. This chapter provides an explanation of the functions available to application developers via the CP210X driver software. All the functions and data types of this add-on are prefixed with the "USBH_CP210X_" text.

18.1.1 Features

The following features are provided:

- Compatibility with different CP210X devices.
- Ability to send and receive data.
- Ability to set various parameters, such as baudrate, number of stop bits, parity.
- Handling of multiple CP210X devices at the same time.
- Notifications about CP210X connection status.
- Ability to query the CP210X line and modem status.

18.1.2 Example code

An example application which uses the API is provided in the `USBH_CP210X_Start.c` file. This example displays information about the CP210X device in the I/O terminal of the debugger. In addition the application then starts a simple echo server, sending back the received data.

18.1.3 Compatibility

The following devices have been tested with the current CP210X driver:

- CP2102
- CP2103
- CP2104

18.1.4 Further reading

For more information about the CP210X devices, please take a look at the hardware manuals and CP210x Virtual COM Port Interface (Document Reference No.: AN571) available from <https://www.silabs.com/products/interface/usb-bridges>

18.2 API Functions

This chapter describes the emUSB-Host CP210X driver API functions.

Function	Description
USBH_CP210X_Init()	Initializes and registers the CP210X device driver with emUSB-Host.
USBH_CP210X_Exit()	Unregisters and de-initializes the CP210X device driver from emUSB-Host.
USBH_CP210X_AddNotification()	Adds a callback in order to be notified when a device is added or removed.
USBH_CP210X_RemoveNotification()	Removes a callback added via USBH_CP210X_AddNotification() .
USBH_CP210X_Open()	Opens a device given by an index.
USBH_CP210X_Close()	Closes a handle to an opened device.
USBH_CP210X_GetDeviceInfo()	Retrieves the information about the CP210X device.
USBH_CP210X_AllowShortRead()	The configuration function allows to let the read function to return as soon as data are available.
USBH_CP210X_SetBaudRate()	Sets the baud rate for the opened device.
USBH_CP210X_Read()	Reads data from the CP210X device.
USBH_CP210X_Write()	Writes data to the CP210X device.
USBH_CP210X_SetDataCharacteristics()	Setups the serial communication with the given characteristics.
USBH_CP210X_SetModemHandshaking()	Sets the handshaking parameters.
USBH_CP210X_GetModemStatus()	Gets the modem status from the device.
USBH_CP210X_Purge()	Purges receive and transmit queues in the device.
USBH_CP210X_SetBreak()	Sets the BREAK condition for the device.
USBH_CP210X_GetInterfaceHandle()	Return the handle to the (open) USB interface.

18.2.1 USBH_CP210X_Init()

Description

Initializes and registers the CP210X device driver with emUSB-Host.

Prototype

```
USBH_STATUS USBH_CP210X_Init(void);
```

Return value

= USBH_STATUS_SUCCESS	Success or module already initialized.
≠ USBH_STATUS_SUCCESS	An error occurred.

18.2.2 USBH_CP210X_Exit()

Description

Unregisters and de-initializes the CP210X device driver from emUSB-Host.

Prototype

```
void USBH_CP210X_Exit(void);
```

Additional information

Before this function is called any notifications added via `USBH_CP210X_AddNotification()` must be removed via `USBH_CP210X_RemoveNotification()`. This function will release resources that were used by this device driver. It has to be called if the application is closed. This has to be called before `USBH_Exit()` is called. No more functions of this module may be called after calling `USBH_CP210X_Exit()`. The only exception is `USBH_CP210X_Init()`, which would in turn reinitialize the module and allows further calls.

18.2.3 USBH_CP210X_AddNotification()

Description

Adds a callback in order to be notified when a device is added or removed.

Prototype

```
USBH_STATUS USBH_CP210X_AddNotification(USBH_NOTIFICATION_HOOK * pHook,
                                         USBH_NOTIFICATION_FUNC * pfNotification,
                                         void * pContext);
```

Parameters

Parameter	Description
pHook	Pointer to a user provided USBH_NOTIFICATION_HOOK variable.
pfNotification	Pointer to a function the stack should call when a device is connected or disconnected.
pContext	Pointer to a user context that is passed to the callback function.

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

18.2.4 USBH_CP210X_RemoveNotification()

Description

Removes a callback added via USBH_CP210X_AddNotification.

Prototype

```
USBH_STATUS USBH_CP210X_RemoveNotification(const USBH_NOTIFICATION_HOOK * pHook);
```

Parameters

Parameter	Description
<code>pHook</code>	Pointer to a user provided USBH_NOTIFICATION_HOOK variable.

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

18.2.5 USBH_CP210X_Open()

Description

Opens a device given by an index.

Prototype

```
USBH_CP210X_HANDLE USBH_CP210X_Open(unsigned Index);
```

Parameters

Parameter	Description
Index	Index of the device that shall be opened. In general this means: the first connected device is 0, second device is 1 etc.

Return value

≠ USBH_CP210X_INVALID_HANDLE
= USBH_CP210X_INVALID_HANDLE

Handle to the device.
Device could not be opened (removed or not available).

18.2.6 USBH_CP210X_Close()

Description

Closes a handle to an opened device.

Prototype

```
void USBH_CP210X_Close(USBH_CP210X_HANDLE hDevice);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to a opened device returned by <code>USBH_CP210X_Open()</code> .

18.2.7 USBH_CP210X_GetDeviceInfo()

Description

Retrieves the information about the CP210X device.

Prototype

```
USBH_STATUS USBH_CP210X_GetDeviceInfo(USBH_CP210X_HANDLE hDevice,  
                                       USBH_CP210X_DEVICE_INFO * pDevInfo);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to the opened device.
<code>pDevInfo</code>	out Pointer to a USBH_CP210X_DEVICE_INFO structure to store information related to the device.

Return value

= USBH_STATUS_SUCCESS	Successful.
≠ USBH_STATUS_SUCCESS	An error occurred.

18.2.8 USBH_CP210X_AllowShortRead()

Description

The configuration function allows to let the read function to return as soon as data are available.

Prototype

```
USBH_STATUS USBH_CP210X_AllowShortRead(USBH_CP210X_HANDLE hDevice,  
                                         U8 AllowShortRead);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device.
AllowShortRead	Define whether short read mode shall be used or not. 1 - Allow short read. 0 - Short read mode disabled.

Return value

= USBH_STATUS_SUCCESS Successful.
≠ USBH_STATUS_SUCCESS An error occurred.

Additional information

USBH_CP210X_AllowShortRead() sets the USBH_CP210X_Read() into a special mode - short read mode. When this mode is enabled, the function returns as soon as any data has been read from the device. This allows the application to read data where the number of bytes to read is undefined. To disable this mode, [AllowShortRead](#) should be set to 0.

18.2.9 USBH_CP210X_SetBaudRate()

Description

Sets the baud rate for the opened device.

Prototype

```
USBH_STATUS USBH_CP210X_SetBaudRate(USBH_CP210X_HANDLE hDevice,  
                                     U32 BaudRate);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device.
BaudRate	Baudrate to set.

Return value

= USBH_STATUS_SUCCESS	Successful.
≠ USBH_STATUS_SUCCESS	An error occurred.

18.2.10 USBH_CP210X_Read()

Description

Reads data from the CP210X device.

Prototype

```
USBH_STATUS USBH_CP210X_Read(USBH_CP210X_HANDLE hDevice,  
                               U8 * pData,  
                               U32 NumBytes,  
                               U32 * pNumBytesRead,  
                               U32 Timeout);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device.
pData	Pointer to a buffer to store the read data.
NumBytes	Number of bytes to be read from the device.
pNumBytesRead	out Pointer to a variable which receives the number of bytes read from the device.
Timeout	Timeout given in milliseconds. A zero value results in an infinite timeout.

Return value

= USBH_STATUS_SUCCESS Successful.
≠ USBH_STATUS_SUCCESS An error occurred.

Additional information

USBH_CP210X_Read() always returns the number of bytes read in [pNumBytesRead](#). This function does not return until [NumBytes](#) bytes have been read into the buffer unless short read mode is enabled.

18.2.11 USBH_CP210X_Write()

Description

Writes data to the CP210X device.

Prototype

```
USBH_STATUS USBH_CP210X_Write(      USBH_CP210X_HANDLE hDevice,
                                   const U8 * pData,
                                   U32 NumBytes,
                                   U32 * pNumBytesWritten,
                                   U32 Timeout);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device.
pData	in Pointer to data to be sent.
NumBytes	Number of bytes to write to the device.
pNumBytesWritten	out Pointer to a variable which receives the number of bytes written to the device.
Timeout	Timeout given in milliseconds. A zero value results in an infinite timeout.

Return value

- = USBH_STATUS_SUCCESS

Successful.
- ≠ USBH_STATUS_SUCCESS

An error occurred.

18.2.12 USBH_CP210X_SetDataCharacteristics()

Description

Setups the serial communication with the given characteristics.

Prototype

```
USBH_STATUS USBH_CP210X_SetDataCharacteristics(USBH_CP210X_HANDLE hDevice,
                                                U8 Length,
                                                U8 StopBits,
                                                U8 Parity);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device.
Length	Number of bits per word. Must be one of the following values: USBH_CP210X_BITS_8 USBH_CP210X_BITS_7 USBH_CP210X_BITS_6 USBH_CP210X_BITS_5
StopBits	Number of stop bits. Must be one of the following values: USBH_CP210X_STOP_BITS_1 USBH_CP210X_STOP_BITS_1_5 USBH_CP210X_STOP_BITS_2
Parity	Parity - must be one of the following values: USBH_CP210X_PARITY_NONE USBH_CP210X_PARITY_ODD USBH_CP210X_PARITY_EVEN USBH_CP210X_PARITY_MARK USBH_CP210X_PARITY_SPACE

Return value

- = USBH_STATUS_SUCCESSSuccessful.
- ≠ USBH_STATUS_SUCCESSAn error occurred.

18.2.13 USBH_CP210X_SetModemHandshaking()

Description

Sets the handshaking parameters.

Prototype

```
USBH_STATUS USBH_CP210X_SetModemHandshaking(USBH_CP210X_HANDLE hDevice,  
                                              U8 Mask,  
                                              U8 DTR,  
                                              U8 RTS);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device.
Mask	b0 - DTR mask, if clear, DTR will not be changed. b1 - RTS mask, if clear, RTS will not be changed.
DTR	Data Terminal Ready (DTR) control signal state.
RTS	Request To Send (RTS) control signal state.

Return value

- = USBH_STATUS_SUCCESS Successful.
- ≠ USBH_STATUS_SUCCESS An error occurred.

18.2.14 USBH_CP210X_GetModemStatus()

Description

Gets the modem status from the device.

Prototype

```
USBH_STATUS USBH_CP210X_GetModemStatus(USBH_CP210X_HANDLE hDevice,  
                                         U8 * pModemStatus);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device.
pModemStatus	Pointer to a variable of type U8 which receives the modem status from the device.

Return value

= USBH_STATUS_SUCCESS Successful.
≠ USBH_STATUS_SUCCESS An error occurred.

Additional information

The modem control line status byte ([pModemStatus](#)) is defined as follows: bit 0: DTR state (as set by host or by handshaking logic in CP210x). bit 1: RTS state (as set by host or by handshaking logic in CP210x). bits 2-3: reserved. bit 4: CTS state (as set by end device). bit 5: DSR state (as set by end device). bit 6: RI state (as set by end device). bit 7: DCD state (as set by end device).

18.2.15 USBH_CP210X_Purge()

Description

Purges receive and transmit queues in the device.

Prototype

```
USBH_STATUS USBH_CP210X_Purge(USBH_CP210X_HANDLE hDevice,  
                               U8 Mask);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device.
Mask	Combination of USBH_CP210X_PURGE_RX and USBH_CP210X_FT_PURGE_TX.

Return value

= USBH_STATUS_SUCCESS	Successful.
≠ USBH_STATUS_SUCCESS	An error occurred.

18.2.16 USBH_CP210X_SetBreak()

Description

Sets the BREAK condition for the device.

Prototype

```
USBH_STATUS USBH_CP210X_SetBreak(USBH_CP210X_HANDLE hDevice,  
                                U8 OnOff);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device.
OnOff	1 - Set break condition 0 - Clear break condition

Return value

- = USBH_STATUS_SUCCESS Successful.
- ≠ USBH_STATUS_SUCCESS An error occurred.

18.2.17 USBH_CP210X_GetInterfaceHandle()

Description

Return the handle to the (open) USB interface. Can be used to call USBH core functions like USBH_GetStringDescriptor().

Prototype

```
USBH_INTERFACE_HANDLE USBH_CP210X_GetInterfaceHandle(USBH_CP210X_HANDLE hDevice);
```

Parameters

Parameter	Description
hDevice	Handle to an open device returned by USBH_CP210X_Open().

Return value

Handle to an open interface.

18.3 Data structures

This chapter describes the emUSB-Host CP210X driver data structures.

Function	Description
USBH_CP210X_DEVICE_INFO	Contains information about an CP210X device.

18.3.1 USBH_CP210X_DEVICE_INFO

Description

Contains information about an CP210X device.

Type definition

```
typedef struct {  
    U16      VendorId;  
    U16      ProductId;  
    USBH_SPEED Speed;  
    U8       InterfaceNo;  
    U8       Class;  
    U8       SubClass;  
    U8       Protocol;  
    USBH_INTERFACE_ID InterfaceID;  
} USBH_CP210X_DEVICE_INFO;
```

Structure members

Member	Description
VendorId	The Vendor ID of the device.
ProductId	The Product ID of the device.
Speed	The USB speed of the device, see USBH_SPEED .
InterfaceNo	Interface index (from USB descriptor).
Class	The Class value field of the interface
SubClass	The SubClass value field of the interface
Protocol	The Protocol value field of the interface
InterfaceID	ID of the interface.

Chapter 19

CH34X Device Driver

This chapter describes the optional emUSB-Host add-on “CH34X device driver”. It allows communication with an WCH CH34X USB device, that is typically serving as USB to RS232 converter, please note that other types are not supported.



19.1 Introduction

The CH34X driver software component of emUSB-Host allows the communication with FTDI CH34X devices. It implements the CH34X protocol specified by FTDI which is a vendor specific protocol. The protocol allows emulation of serial communication via USB. This chapter provides an explanation of the functions available to application developers via the CH34X driver software. All the functions and data types of this add-on are prefixed with the "USBH_CH34X_" text.

19.1.1 Features

The following features are provided:

- Compatibility with different CH34X devices.
- Ability to send and receive data.
- Ability to set various parameters, such as baudrate, number of stop bits, parity.
- Handling of multiple CH34X devices at the same time.
- Ability to query the CH34X line and modem status.

19.1.2 Example code

An example application which uses the API is provided in the `USBH_CH34X_Start.c` file. This example displays information about the CH34X device in the I/O terminal of the debugger. In addition the application then starts a simple echo server, sending back the received data.

19.1.3 Compatibility

The following devices work with the current CH34X driver:

- CH340B
- CH340C
- CH340E
- CH340G
- CH340K
- CH340N
- CH340T
- CH341A

19.1.4 Further reading

For more information about the WCH CH34X devices, please take a look at the wch website <https://www.wch-ic.com/products/productsCenter/mcuInterface?categoryId=1&tName=USB%20to%20UART> Other helpful site:

- https://github.com/WCHSoftGroup/tty_uart
- https://github.com/WCHSoftGroup/ch341ser_linux
- <https://github.com/nospam2000/ch341-baudrate-calculation>

19.2 API Functions

This chapter describes the emUSB-Host CH34X driver API functions.

Function	Description
USBH_CH34X_Init()	Initializes and registers the CH34X device driver with emUSB-Host.
USBH_CH34X_Exit()	Unregisters and de-initializes the CH34X device driver from emUSB-Host.
USBH_CH34X_AddCustomDeviceMask()	This function allows the CH34X module to receive notifications about devices which do not present themselves with WCH's vendor ID (0x1a86). ;
USBH_CH34X_AddNotification()	Adds a callback in order to be notified when a device is added or removed.
USBH_CH34X_RemoveNotification()	Removes a callback added via USBH_CH34X_AddNotification() .
USBH_CH34X_Open()	Opens a device given by an index.
USBH_CH34X_Close()	Closes a handle to an opened device.
USBH_CH34X_GetDeviceInfo()	Retrieves the information about the CH34X device.
USBH_CH34X_AllowShortRead()	The configuration function allows to let the read function to return as soon as data are available.
USBH_CH34X_SetBaudRate()	Sets the baud rate for the opened device.
USBH_CH34X_Read()	Reads data from the CH34X device.
USBH_CH34X_Write()	Writes data to the CH34X device.
USBH_CH34X_SetDataCharacteristics()	Setups the serial communication with the given characteristics.
USBH_CH34X_SetModemHandshaking()	Sets the handshaking parameters.
USBH_CH34X_GetModemStatus()	Gets the modem status from the device.
USBH_CH34X_Reset()	Re-initialize the device.
USBH_CH34X_SetBreak()	Sets the BREAK condition for the device.
USBH_CH34X_GetInterfaceHandle()	Return the handle to the (open) USB interface.

19.2.1 USBH_CH34X_Init()

Description

Initializes and registers the CH34X device driver with emUSB-Host.

Prototype

```
USBH_STATUS USBH_CH34X_Init(void);
```

Return value

= USBH_STATUS_SUCCESS	Success or module already initialized.
≠ USBH_STATUS_SUCCESS	An error occurred.

19.2.2 USBH_CH34X_Exit()

Description

Unregisters and de-initializes the CH34X device driver from emUSB-Host.

Prototype

```
void USBH_CH34X_Exit(void);
```

Additional information

Before this function is called any notifications added via `USBH_CH34X_AddNotification()` must be removed via `USBH_CH34X_RemoveNotification()`. This function will release resources that were used by this device driver. It has to be called if the application is closed. This has to be called before `USBH_Exit()` is called. No more functions of this module may be called after calling `USBH_CH34X_Exit()`. The only exception is `USBH_CH34X_Init()`, which would in turn reinitialize the module and allows further calls.

19.2.3 USBH_CH34X_AddCustomDeviceMask()

Description

This function allows the CH34X module to receive notifications about devices which do not present themselves with WCH’s vendor ID (0x1a86).

Prototype

```
USBH_STATUS USBH_CH34X_AddCustomDeviceMask(const U16 * pVendorIds,
                                             const U16 * pProductIds,
                                             U16 NumIds);
```

Parameters

Parameter	Description
pVendorIds	Array of vendor IDs.
pProductIds	Array of product IDs.
NumIds	Number of elements in both arrays, each index in both arrays is used as a pair to create a filter.

Return value

USBH_STATUS_SUCCESS	Success.
USBH_STATUS_ERROR	Notification could not be registered.

19.2.4 USBH_CH34X_AddNotification()

Description

Adds a callback in order to be notified when a device is added or removed.

Prototype

```
USBH_STATUS USBH_CH34X_AddNotification(USBH_NOTIFICATION_HOOK * pHook,
                                       USBH_NOTIFICATION_FUNC * pfNotification,
                                       void * pContext);
```

Parameters

Parameter	Description
<code>pHook</code>	Pointer to a user provided <code>USBH_NOTIFICATION_HOOK</code> variable.
<code>pfNotification</code>	Pointer to a function the stack should call when a device is connected or disconnected.
<code>pContext</code>	Pointer to a user context that is passed to the callback function.

Return value

`USBH_STATUS_SUCCESS` on success or error code on failure.

Example

```
static USBH_NOTIFICATION_HOOK _Hook;

/*****
 *
 *      _cbOnAddRemoveDevice
 *
 * Function description
 *   Callback, called when a device is added or removed.
 *   Call in the context of the USBH_Task.
 *   The functionality in this routine should not block
 */
static void _cbOnAddRemoveDevice(void * pContext, U8 DevIndex, USBH_DEVICE_EVENT Event) {
    (void)pContext;
    switch (Event) {
        case USBH_DEVICE_EVENT_ADD:
            USBH_Logf_Application("**** Device added\n");
            _DevIndex = DevIndex;
            _DevIsReady = 1;
            break;
        case USBH_DEVICE_EVENT_REMOVE:
            USBH_Logf_Application("**** Device removed\n");
            _DevIsReady = 0;
            _DevIndex = -1;
            break;
        default:; // Should never happen
    }
}

<...>
USBH_CH34X_Init();
USBH_CH34X_AddNotification(&_Hook, _cbOnAddRemoveDevice, NULL);
<...>
```

19.2.5 USBH_CH34X_RemoveNotification()

Description

Removes a callback added via USBH_CH34X_AddNotification.

Prototype

```
USBH_STATUS USBH_CH34X_RemoveNotification(const USBH_NOTIFICATION_HOOK * pHook);
```

Parameters

Parameter	Description
<code>pHook</code>	Pointer to a user provided USBH_NOTIFICATION_HOOK variable.

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

19.2.6 USBH_CH34X_Open()

Description

Opens a device given by an index.

Prototype

```
USBH_CH34X_HANDLE USBH_CH34X_Open(unsigned Index);
```

Parameters

Parameter	Description
Index	Index of the device that shall be opened. In general this means: the first connected device is 0, second device is 1 etc.

Return value

≠ USBH_CH34X_INVALID_HANDLE
= USBH_CH34X_INVALID_HANDLE

Handle to the device.
Device could not be opened (removed or not available).

19.2.7 USBH_CH34X_Close()

Description

Closes a handle to an opened device.

Prototype

```
void USBH_CH34X_Close(USBH_CH34X_HANDLE hDevice);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to a opened device returned by <code>USBH_CH34X_Open()</code> .

19.2.8 USBH_CH34X_GetDeviceInfo()

Description

Retrieves the information about the CH34X device.

Prototype

```
USBH_STATUS USBH_CH34X_GetDeviceInfo(USBH_CH34X_HANDLE    hDevice,  
                                     USBH_CH34X_DEVICE_INFO * pDevInfo);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to the opened device.
<code>pDevInfo</code>	out Pointer to a <code>USBH_CH34X_DEVICE_INFO</code> structure to store information related to the device.

Return value

= <code>USBH_STATUS_SUCCESS</code>	Successful.
≠ <code>USBH_STATUS_SUCCESS</code>	An error occurred.

19.2.9 USBH_CH34X_AllowShortRead()

Description

The configuration function allows to let the read function to return as soon as data are available.

Prototype

```
USBH_STATUS USBH_CH34X_AllowShortRead(USBH_CH34X_HANDLE hDevice,  
                                       U8 AllowShortRead);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device.
AllowShortRead	Define whether short read mode shall be used or not. 1 - Allow short read. 0 - Short read mode disabled.

Return value

= USBH_STATUS_SUCCESS Successful.
≠ USBH_STATUS_SUCCESS An error occurred.

Additional information

USBH_CH34X_AllowShortRead() sets the USBH_CH34X_Read() into a special mode - short read mode. When this mode is enabled, the function returns as soon as any data has been read from the device. This allows the application to read data where the number of bytes to read is undefined. To disable this mode, [AllowShortRead](#) should be set to 0.

19.2.10 USBH_CH34X_SetBaudRate()

Description

Sets the baud rate for the opened device.

Prototype

```
USBH_STATUS USBH_CH34X_SetBaudRate(USBH_CH34X_HANDLE hDevice,  
                                     U32 BaudRate);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device.
BaudRate	Baudrate to set.

Return value

= USBH_STATUS_SUCCESS	Successful.
≠ USBH_STATUS_SUCCESS	An error occurred.

19.2.11 USBH_CH34X_Read()

Description

Reads data from the CH34X device.

Prototype

```
USBH_STATUS USBH_CH34X_Read(USBH_CH34X_HANDLE hDevice,  
                             U8 * pData,  
                             U32 NumBytes,  
                             U32 * pNumBytesRead,  
                             U32 Timeout);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device.
pData	Pointer to a buffer to store the read data.
NumBytes	Number of bytes to be read from the device.
pNumBytesRead	out Pointer to a variable which receives the number of bytes read from the device.
Timeout	Timeout given in milliseconds. A zero value results in an infinite timeout.

Return value

= USBH_STATUS_SUCCESS Successful.
≠ USBH_STATUS_SUCCESS An error occurred.

Additional information

USBH_CH34X_Read() always returns the number of bytes read in [pNumBytesRead](#). This function does not return until [NumBytes](#) bytes have been read into the buffer unless short read mode is enabled.

19.2.12 USBH_CH34X_Write()

Description

Writes data to the CH34X device.

Prototype

```
USBH_STATUS USBH_CH34X_Write(      USBH_CH34X_HANDLE  hDevice,
                                   const U8              * pData,
                                   U32                    NumBytes,
                                   U32                    * pNumBytesWritten,
                                   U32                    Timeout);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device.
pData	in Pointer to data to be sent.
NumBytes	Number of bytes to write to the device.
pNumBytesWritten	out Pointer to a variable which receives the number of bytes written to the device.
Timeout	Timeout given in milliseconds. A zero value results in an infinite timeout.

Return value

- = USBH_STATUS_SUCCESS
- Successful.
- ≠ USBH_STATUS_SUCCESS
- An error occurred.

19.2.13 USBH_CH34X_SetDataCharacteristics()

Description

Setups the serial communication with the given characteristics.

Prototype

```
USBH_STATUS USBH_CH34X_SetDataCharacteristics(USBH_CH34X_HANDLE hDevice,
                                                U8 Length,
                                                U8 StopBits,
                                                U8 Parity);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device.
Length	Number of bits per word. Must be one of the following values: USBH_CH34X_BITS_8 USBH_CH34X_BITS_7 USBH_CH34X_BITS_6 USBH_CH34X_BITS_5
StopBits	Number of stop bits. Must be one of the following values: USBH_CH34X_STOP_BITS_1 USBH_CH34X_STOP_BITS_2
Parity	Parity - must be one of the following values: USBH_CH34X_PARITY_NONE USBH_CH34X_PARITY_ODD USBH_CH34X_PARITY_EVEN USBH_CH34X_PARITY_MARK USBH_CH34X_PARITY_SPACE

Return value

- = USBH_STATUS_SUCCESS Successful.
- ≠ USBH_STATUS_SUCCESS An error occurred.

19.2.14 USBH_CH34X_SetModemHandshaking()

Description

Sets the handshaking parameters.

Prototype

```
USBH_STATUS USBH_CH34X_SetModemHandshaking(USBH_CH34X_HANDLE hDevice,  
                                             U8 OnOff);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device.
OnOff	When = 0, disable flow control via RTS/CTS otherwise flow control thus RTS/CTS is enabled.

Return value

- = USBH_STATUS_SUCCESS Successful.
- ≠ USBH_STATUS_SUCCESS An error occurred.

19.2.15 USBH_CH34X_GetModemStatus()

Description

Gets the modem status from the device.

Prototype

```
USBH_STATUS USBH_CH34X_GetModemStatus(USBH_CH34X_HANDLE hDevice,  
                                       U8 * pModemStatus);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device.
pModemStatus	Pointer to a variable of type U8 which receives the modem status from the device.

Return value

= USBH_STATUS_SUCCESS Successful.
≠ USBH_STATUS_SUCCESS An error occurred.

Additional information

The modem control line status byte ([pModemStatus](#)) is defined as follows: bit 0: CTS state (as set by end device). bit 1: DSR state (as set by end device). bit 2: RI state (as set by end device). bit 3: DCD state (as set by end device).

19.2.16 USBH_CH34X_Reset()

Description

Re-initialize the device.

Prototype

```
USBH_STATUS USBH_CH34X_Reset(USBH_CH34X_HANDLE hDevice);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to the opened device.

Return value

= USBH_STATUS_SUCCESS	Successful.
≠ USBH_STATUS_SUCCESS	An error occurred.

19.2.17 USBH_CH34X_SetBreak()

Description

Sets the BREAK condition for the device.

Prototype

```
USBH_STATUS USBH_CH34X_SetBreak(USBH_CH34X_HANDLE hDevice,  
                                U8 OnOff);
```

Parameters

Parameter	Description
hDevice	Handle to the opened device.
OnOff	1 - Set break condition 0 - Clear break condition

Return value

= USBH_STATUS_SUCCESS	Successful.
≠ USBH_STATUS_SUCCESS	An error occurred.

19.2.18 USBH_CH34X_GetInterfaceHandle()

Description

Return the handle to the (open) USB interface. Can be used to call USBH core functions like USBH_GetStringDescriptor().

Prototype

```
USBH_INTERFACE_HANDLE USBH_CH34X_GetInterfaceHandle(USBH_CH34X_HANDLE hDevice);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to an open device returned by USBH_CH34X_Open().

Return value

Handle to an open interface.

19.3 Data structures

This chapter describes the emUSB-Host CH34X driver data structures.

Function	Description
USBH_CH34X_DEVICE_INFO	Contains information about an CH34X device.

19.3.1 USBH_CH34X_DEVICE_INFO

Description

Contains information about an CH34X device.

Type definition

```
typedef struct {  
    U16      VendorId;  
    U16      ProductId;  
    USBH_SPEED Speed;  
    U8      InterfaceNo;  
    U8      Class;  
    U8      SubClass;  
    U8      Protocol;  
    U8      Version;  
    USBH_INTERFACE_ID InterfaceID;  
} USBH_CH34X_DEVICE_INFO;
```

Structure members

Member	Description
VendorId	The Vendor ID of the device.
ProductId	The Product ID of the device.
Speed	The USB speed of the device, see USBH_SPEED .
InterfaceNo	Interface index (from USB descriptor).
Class	The Class value field of the interface
SubClass	The SubClass value field of the interface
Protocol	The Protocol value field of the interface
Version	Internal use.
InterfaceID	ID of the interface.

Chapter 20

Video Device Driver (Add-On)

This chapter describes the optional emUSB-Host add-on “Video device driver”. It allows communication with USB video devices.



20.1 Introduction

The Video driver software component of emUSB-Host allows communication with USB video V1.00 compatible devices like webcams.

This chapter provides an explanation of the functions available to application developers via the video driver software. All the functions and data types of this add-on are prefixed with 'USBH_VIDEO_'.

20.1.1 Overview

A video device connected to the emUSB-Host is automatically configured and added to an internal list. If the Video driver has been registered, it is notified via a callback when a video device has been added or removed. The driver then can notify the application program, when a callback function has been registered via `USBH_VIDEO_AddNotification()`. In order to communicate with such a device, the application has to call the `USBH_VIDEO_Open()`, passing the interface index. Video interfaces are identified by an index. The first connected interface gets assigned the index 0, the second index 1, and so on.

Video devices contains multiple USB interfaces: One control interface and one or more video streaming interfaces.

Via the control interface, different functional units can be accessed:

- Input/Output terminals
- Selector units
- Processing units
- Extension units

The streaming interface is used to transfer video data. It must be configured first to select the appropriate video parameters: format, resolution and frame rate interval. Video data transfer is then done using a callback which is registered via `USBH_VIDEO_OpenStream()` together with the video parameters.

Still image capture method 0 is supported.

Note

In order to use the Video class driver, support for isochronous transfers must be enabled in the USB stack. Make sure that there is a preprocessor define in the file `USBH_Conf.h`:

```
#define USBH_SUPPORT_ISO_TRANSFER 1
```

Note

Depnding on the used video device and the selected format and resolution it may be necessary for the USB driver to support high-bandwidth isochronous transfers.

20.1.2 Example code

The sample application `USBH_VIDEO_Start.c` shows how to retrieve information and capabilities from a video device and how to receive frame data.

20.2 API Functions

This chapter describes the emUSB-Host Video driver API functions. These functions are defined in the header file `USBH_VIDEO.h`.

Function	Description
<code>USBH_VIDEO_Init()</code>	Initializes and registers the VIDEO device module with emUSB-Host.
<code>USBH_VIDEO_Exit()</code>	Unregisters and de-initializes the VIDEO device module from emUSB-Host.
<code>USBH_VIDEO_AddNotification()</code>	Adds a callback in order to be notified when a device is added or removed.
<code>USBH_VIDEO_RemoveNotification()</code>	Removes a callback added via <code>USBH_VIDEO_AddNotification()</code> .
<code>USBH_VIDEO_Open()</code>	Opens a Video interface given by an index.
<code>USBH_VIDEO_Close()</code>	Closes a handle to an opened interface.
<code>USBH_VIDEO_GetIndex()</code>	Return an index that can be used for a call to <code>USBH_VIDEO_Open()</code> for a given interface ID.
<code>USBH_VIDEO_GetInterfaceInfo()</code>	Retrieves information about the VIDEO interface.
<code>USBH_VIDEO_GetTermUnitInfo()</code>	Retrieve information about a Unit or a Terminal from the opened device.
<code>USBH_VIDEO_TermUnitID2Index()</code>	Helper function to retrieve the index of a terminal or unit using the terminal or unit ID.
<code>USBH_VIDEO_GetInputHeader()</code>	Retrieves information about the video stream interface.
<code>USBH_VIDEO_GetFormatInfo()</code>	Retrieve information about a video Format descriptor.
<code>USBH_VIDEO_GetFrameInfo()</code>	Retrieve information about a Frame descriptor which belongs to a specific format.
<code>USBH_VIDEO_GetColorMatchingInfo()</code>	Retrieves the color matching descriptor from a given format index.
<code>USBH_VIDEO_OpenStream()</code>	Open a video stream.
<code>USBH_VIDEO_Ack()</code>	Acknowledge a data packet that was received via the read callback which was registered via <code>USBH_VIDEO_OpenStream()</code> .
<code>USBH_VIDEO_GetStreamState()</code>	Allows the user to check whether a stream is stopped or not.
<code>USBH_VIDEO_RestartStream()</code>	Restart a stopped stream.
<code>USBH_VIDEO_CloseStream()</code>	Closes a stream previously opened via <code>USBH_VIDEO_OpenStream()</code> .
<code>USBH_VIDEO_GetControlVal()</code>	Get a control value of a unit or terminal.
<code>USBH_VIDEO_SetControlVal()</code>	Set a control value of a unit or terminal.
<code>USBH_VIDEO_ReadStatus()</code>	Reads status messages from the video device.

20.2.1 USBH_VIDEO_Init()

Description

Initializes and registers the VIDEO device module with emUSB-Host.

Prototype

```
USBH_STATUS USBH_VIDEO_Init(void);
```

Return value

USBH_STATUS_SUCCESS Success or module already initialized.

20.2.2 USBH_VIDEO_Exit()

Description

Unregisters and de-initializes the VIDEO device module from emUSB-Host.

Prototype

```
void USBH_VIDEO_Exit(void);
```

Additional information

Has to be called the same number of times `USBH_VIDEO_Init` was called in order to de-initialize the module. This function will release resources that were used by this device driver. It has to be called if the application is closed. This has to be called before `USBH_Exit()` is called. No more functions of this module may be called after calling `USBH_VIDEO_Exit()`. The only exception is `USBH_VIDEO_Init()`, which would in turn re-init the module and allow further calls.

20.2.3 USBH_VIDEO_AddNotification()

Description

Adds a callback in order to be notified when a device is added or removed.

Prototype

```
USBH_STATUS USBH_VIDEO_AddNotification(USBH_NOTIFICATION_HOOK * pHook,  
                                       USBH_NOTIFICATION_FUNC * pfNotification,  
                                       void * pContext);
```

Parameters

Parameter	Description
pHook	Pointer to a user provided USBH_NOTIFICATION_HOOK variable.
pfNotification	Pointer to a function the stack should call when a device is connected or disconnected.
pContext	Pointer to a user context that is passed to the callback function.

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

20.2.4 USBH_VIDEO_RemoveNotification()

Description

Removes a callback added via USBH_VIDEO_AddNotification.

Prototype

```
USBH_STATUS USBH_VIDEO_RemoveNotification(const USBH_NOTIFICATION_HOOK * pHook);
```

Parameters

Parameter	Description
<code>pHook</code>	Pointer to a user provided USBH_NOTIFICATION_HOOK variable.

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

20.2.5 USBH_VIDEO_Open()

Description

Opens a Video interface given by an index.

Prototype

```
USBH_STATUS USBH_VIDEO_Open(unsigned Index,  
                             USBH_VIDEO_DEVICE_HANDLE * pHandle);
```

Parameters

Parameter	Description
Index	Index of the interface that shall be opened.
pHandle	out Handle to a VIDEO interface on success.

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

Additional information

The index of a new connected device is provided to the callback function registered with USBH_VIDEO_AddNotification().

20.2.6 USBH_VIDEO_Close()

Description

Closes a handle to an opened interface.

Prototype

```
void USBH_VIDEO_Close(USBH_VIDEO_DEVICE_HANDLE hDevice);
```

Parameters

Parameter	Description
<code>hDevice</code>	Handle to an open interface returned by <code>USBH_VIDEO_Open()</code> .

20.2.7 USBH_VIDEO_GetIndex()

Description

Return an index that can be used for a call to USBH_VIDEO_Open() for a given interface ID.

Prototype

```
int USBH_VIDEO_GetIndex(USBH_INTERFACE_ID InterfaceID);
```

Parameters

Parameter	Description
InterfaceID	Id of the interface.

Return value

≥ 0 Index of the VIDEO interface.
< 0 InterfaceID not found.

20.2.8 USBH_VIDEO_GetInterfaceInfo()

Description

Retrieves information about the VIDEO interface.

Prototype

```
USBH_STATUS USBH_VIDEO_GetInterfaceInfo(USBH_VIDEO_DEVICE_HANDLE hDevice,  
                                         USBH_VIDEO_INTERFACE_INFO * pDevInfo);
```

Parameters

Parameter	Description
hDevice	Handle to an open interface returned by <code>USBH_VIDEO_Open()</code> .
pDevInfo	Pointer to a <code>USBH_VIDEO_INTERFACE_INFO</code> structure that receives the information.

Return value

`USBH_STATUS_SUCCESS` on success or error code on failure.

20.2.9 USBH_VIDEO_GetTermUnitInfo()

Description

Retrieve information about a Unit or a Terminal from the opened device.

Prototype

```
USBH_STATUS USBH_VIDEO_GetTermUnitInfo(USBH_VIDEO_DEVICE_HANDLE    hDevice,  
                                       unsigned                    Index,  
                                       USBH_VIDEO_TERM_UNIT_INFO * pTermUnitInfo);
```

Parameters

Parameter	Description
hDevice	Handle to an open interface returned by <code>USBH_VIDEO_Open()</code> .
Index	Zero-based index of the unit or terminal.
pTermUnitInfo	out Pointer to a structure of type <code>USBH_VIDEO_TERM_UNIT_INFO</code> which will be filled with information about the terminal or unit if the function returns <code>USBH_STATUS_SUCCESS</code> .

Return value

`USBH_STATUS_SUCCESS` on success or error code on failure.

Additional information

For the [Index](#) parameter `USBH_VIDEO_GetInterfaceInfo()` can be used to retrieve the number of terminals/units the device has.

20.2.10 USBH_VIDEO_TermUnitID2Index()

Description

Helper function to retrieve the index of a terminal or unit using the terminal or unit ID.

Prototype

```
USBH_STATUS USBH_VIDEO_TermUnitID2Index(USBH_VIDEO_DEVICE_HANDLE hDevice,
                                         U8 TermUnitID,
                                         unsigned * pIndex);
```

Parameters

Parameter	Description
hDevice	Handle to an open interface returned by USBH_VIDEO_Open().
TermUnitID	Terminal or unit ID.
pIndex	out Pointer to an unsigned variable which will be filled with the correct terminal/unit index if the function returns USBH_STATUS_SUCCESS.

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

20.2.11 USBH_VIDEO_GetInputHeader()

Description

Retrieves information about the video stream interface.

Prototype

```
USBH_STATUS USBH_VIDEO_GetInputHeader
(USBH_VIDEO_DEVICE_HANDLE hDevice,
 USBH_VIDEO_INPUT_HEADER_INFO * pInputHeaderInfo);
```

Parameters

Parameter	Description
hDevice	Handle to an open interface returned by USBH_VIDEO_Open().
pInputHeaderInfo	out Pointer to a structure of type USBH_VIDEO_INPUT_HEADER_INFO which will be filled with information about the stream if the function returns USBH_STATUS_SUCCESS.

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

20.2.12 USBH_VIDEO_GetFormatInfo()

Description

Retrieve information about a video Format descriptor.

Prototype

```
USBH_STATUS USBH_VIDEO_GetFormatInfo(USBH_VIDEO_DEVICE_HANDLE hDevice,  
                                     unsigned FormatIdx,  
                                     USBH_VIDEO_FORMAT_INFO * pFormatInfo);
```

Parameters

Parameter	Description
hDevice	Handle to an open interface returned by <code>USBH_VIDEO_Open()</code> .
FormatIdx	Zero-based index of the format descriptor.
pFormatInfo	out Pointer to a structure of type <code>USBH_VIDEO_FORMAT_INFO</code> which will be filled with information about a format if the function returns <code>USBH_STATUS_SUCCESS</code> .

Return value

`USBH_STATUS_SUCCESS` on success or error code on failure.

Additional information

For the [FormatIdx](#) parameter `USBH_VIDEO_GetInputHeader()` can be used to retrieve the number of formats the device has. The [FormatIdx](#) parameter is not the same as the `bFormatIndex` value inside the video format descriptor.

20.2.13 USBH_VIDEO_GetFrameInfo()

Description

Retrieve information about a Frame descriptor which belongs to a specific format.

Prototype

```
USBH_STATUS USBH_VIDEO_GetFrameInfo(USBH_VIDEO_DEVICE_HANDLE hDevice,  
                                     unsigned FormatIdx,  
                                     unsigned FrameIdx,  
                                     USBH_VIDEO_FRAME_INFO * pFrameInfo);
```

Parameters

Parameter	Description
hDevice	Handle to an open interface returned by <code>USBH_VIDEO_Open()</code> .
FormatIdx	Zero-based index of the format descriptor.
FrameIdx	Zero-based index of the frame descriptor.
pFrameInfo	out Pointer to a structure of type <code>USBH_VIDEO_FRAME_INFO</code> which will be filled with information about a frame if the function returns <code>USBH_STATUS_SUCCESS</code> .

Return value

`USBH_STATUS_SUCCESS` on success or error code on failure.

Additional information

A format descriptor contains one or more frame descriptors. For the [FormatIdx](#) parameter `USBH_VIDEO_GetInputHeader()` can be used to retrieve the number of formats the device has. The [FormatIdx](#) parameter is not the same as the `bFormatIndex` value inside the video format descriptor. For the [FrameIdx](#) parameter `USBH_VIDEO_GetFormatInfo()` can be used to retrieve the number of frame descriptors a particular format has. The [FrameIdx](#) parameter is not the same as the `bFrameIndex` value inside the video format descriptor.

20.2.14 USBH_VIDEO_GetColorMatchingInfo()

Description

Retrieves the color matching descriptor from a given format index.

Prototype

```
USBH_STATUS USBH_VIDEO_GetColorMatchingInfo(USBH_VIDEO_DEVICE_HANDLE hDevice,
                                             unsigned FormatIdx,
                                             USBH_VIDEO_COLOR_INFO * pColorInfo);
```

Parameters

Parameter	Description
hDevice	Handle to an open device.
FormatIdx	Format index.
pColorInfo	out Pointer to a structure of type USBH_VIDEO_COLOR_INFO which will be filled with information about the color matching descriptor if the function returns USBH_STATUS_SUCCESS.

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

20.2.15 USBH_VIDEO_OpenStream()

Description

Open a video stream.

Prototype

```
USBH_STATUS USBH_VIDEO_OpenStream(USBH_VIDEO_DEVICE_HANDLE hDevice,  
                                   USBH_VIDEO_STREAM_CONFIG * pStreamInfo,  
                                   USBH_VIDEO_STREAM_HANDLE * pHandle);
```

Parameters

Parameter	Description
hDevice	Handle to an open device.
pStreamInfo	Pointer to a structure of type USBH_VIDEO_STREAM_CONFIG. The user must fill the structure with valid information for the chosen video setting.
pHandle	out Pointer to a value of type USBH_VIDEO_STREAM_HANDLE which will receive the valid open stream handle if the function returns USBH_STATUS_SUCCESS.

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

20.2.16 USBH_VIDEO_Ack()

Description

Acknowledge a data packet that was received via the read callback which was registered via `USBH_VIDEO_OpenStream()`. This will enable the driver to reuse the buffer to receive another packet.

Prototype

```
USBH_STATUS USBH_VIDEO_Ack(USBH_VIDEO_STREAM_HANDLE hStream);
```

Parameters

Parameter	Description
<code>hStream</code>	Handle to an open stream from <code>USBH_VIDEO_OpenStream()</code> .

Return value

`USBH_STATUS_SUCCESS` on success or error code on failure.

Additional information

This function must be called to acknowledge the read packet, otherwise the stack will run out of buffers and stop reception.

20.2.17 USBH_VIDEO_GetStreamState()

Description

Allows the user to check whether a stream is stopped or not.

Prototype

```
USBH_STATUS USBH_VIDEO_GetStreamState(USBH_VIDEO_STREAM_HANDLE hStream,  
I8 * pIsStopped);
```

Parameters

Parameter	Description
hStream	Handle to an open stream.
pIsStopped	out Pointer to a variable which will receive the stream state. 1 - Stopped. 0 - Running.

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

Additional information

This function should be called by the user when the USBH_VIDEO_RX_CALLBACK or USBH_VIDEO_PAYLOAD_CALLBACK from USBH_VIDEO_STREAM_CONFIG registered via USBH_VIDEO_OpenStream() returns an error code inside the Status parameter. This allows the application to check whether the stream experienced a transfer error and can be restarted via USBH_VIDEO_RestartStream().

20.2.18 USBH_VIDEO_RestartStream()

Description

Restart a stopped stream. The stream must have a valid handle and must be stopped.

Prototype

```
USBH_STATUS USBH_VIDEO_RestartStream(USBH_VIDEO_STREAM_HANDLE hStream);
```

Parameters

Parameter	Description
hStream	Handle to a stream.

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

Additional information

To check if a stream is stopped `USBH_VIDEO_GetStreamState()` can be used.

20.2.19 USBH_VIDEO_CloseStream()

Description

Closes a stream previously opened via `USBH_VIDEO_OpenStream()`.

Prototype

```
USBH_STATUS USBH_VIDEO_CloseStream(USBH_VIDEO_STREAM_HANDLE hStream);
```

Parameters

Parameter	Description
<code>hStream</code>	Handle to an open stream.

Return value

`USBH_STATUS_SUCCESS` on success or error code on failure.

20.2.20 USBH_VIDEO_GetControlVal()

Description

Get a control value of a unit or terminal.

Prototype

```
USBH_STATUS USBH_VIDEO_GetControlVal(          USBH_VIDEO_DEVICE_HANDLE hDevice,
                                                unsigned Unit,
                                                const USBH_VIDEO_GET_CONTROL * pGetControl);
```

Parameters

Parameter	Description
hDevice	Handle to an open interface returned by USBH_VIDEO_Open().
Unit	ID of the unit or terminal.
pGetControl	Pointer to a structure of type USBH_VIDEO_GET_CONTROL.

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

20.2.21 USBH_VIDEO_SetControlVal()

Description

Set a control value of a unit or terminal.

Prototype

```
USBH_STATUS USBH_VIDEO_SetControlVal(          USBH_VIDEO_DEVICE_HANDLE hDevice,
                                                unsigned Unit,
                                                const USBH_VIDEO_SET_CONTROL * pSetControl);
```

Parameters

Parameter	Description
hDevice	Handle to an open interface returned by USBH_VIDEO_Open().
Unit	ID of the unit or terminal.
pSetControl	Pointer to a structure of type USBH_VIDEO_SET_CONTROL.

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

20.2.22 USBH_VIDEO_ReadStatus()

Description

Reads status messages from the video device. This function can also be used to check for hardware trigger interrupts (when the user presses a button on the webcam).

Prototype

```
USBH_STATUS USBH_VIDEO_ReadStatus(USBH_VIDEO_DEVICE_HANDLE hDevice,  
                                   U8 * pData,  
                                   U32 NumBytes,  
                                   U32 * pNumBytesRead,  
                                   U32 Timeout);
```

Parameters

Parameter	Description
hDevice	Handle to an open device.
pData	Pointer to a buffer to store the read data.
NumBytes	Number of bytes to be read from the device.
pNumBytesRead	Pointer to a variable which receives the number of bytes read from the device. Can be NULL.
Timeout	Timeout in ms. 0 means infinite timeout.

Return value

USBH_STATUS_SUCCESS on success or error code on failure.

Additional information

The status packet format is documented inside the UVC 1.1 specification chapter "2.4.2.2 Status Interrupt Endpoint". This function reads from the video device's interrupt IN endpoint. This endpoint is optional and some video devices do not have it, in this case status requests are not possible. If the function returns an error code (including USBH_STATUS_TIMEOUT) it already may have read part of the data. The number of bytes read successfully is always stored in the variable pointed to by pNumBytesRead.

20.3 Data structures

This chapter describes the emUSB-Host Video driver data structures.

Structure	Description
USBH_VIDEO_INTERFACE_INFO	Structure containing information about a video interface.
USBH_VIDEO_TERM_UNIT_INFO	Structure containing information about a video terminal or a video unit.
USBH_VIDEO_INPUT_HEADER_INFO	Structure containing information about a video stream.
USBH_VIDEO_FORMAT_INFO	Structure containing information about a video format.
USBH_VIDEO_FRAME_INFO	Structure containing information about a video frame.
USBH_VIDEO_COLOR_INFO	Structure containing information about a video format's color matching.
USBH_VIDEO_PAYLOAD_HEADER	Structure containing the payload header.
USBH_VIDEO_STREAM_CONFIG	Structure containing configuration with which the emUSB-Host Video class should start reading frame data from the device.
USBH_VIDEO_GET_CONTROL	Structure used with "Get" requests.
USBH_VIDEO_SET_CONTROL	Structure used with "Set Cur" requests.

20.3.1 USBH_VIDEO_INTERFACE_INFO

Description

Structure containing information about a video interface.

Type definition

```
typedef struct {
    U16      VendorId;
    U16      ProductId;
    U8       InterfaceNo;
    U8       NumTermUnits;
    USBH_INTERFACE_ID  InterfaceID;
    USBH_DEVICE_ID     DeviceId;
    USBH_SPEED         Speed;
    U8                 Class;
    U8                 SubClass;
    U8                 Protocol;
} USBH_VIDEO_INTERFACE_INFO;
```

Structure members

Member	Description
VendorId	The Vendor ID of the device.
ProductId	The Product ID of the device.
InterfaceNo	Interface index (from USB descriptor).
NumTermUnits	Number of terminals and units reported within the corresponding video control interface.
InterfaceID	ID of the interface.
DeviceId	The unique device Id. This Id is assigned if the USB device was successfully enumerated. It is valid until the device is removed from the host. If the device is reconnected a different device Id is assigned.
Speed	The USB speed of the device, see USBH_SPEED .
Class	The Class value field of the interface.
SubClass	The SubClass value field of the interface.
Protocol	The Protocol value field of the interface.

20.3.2 USBH_VIDEO_TERM_UNIT_INFO

Description

Structure containing information about a video terminal or a video unit.

Type definition

```
typedef struct {  
    U8  Type;  
    U8  bTermUnitID;  
} USBH_VIDEO_TERM_UNIT_INFO;
```

Structure members

Member	Description
Type	Terminal or Unit type. One of the following values: • USBH_VIDEO_VC_INPUT_TERMINAL • USBH_VIDEO_VC_OUTPUT_TERMINAL • USBH_VIDEO_VC_SELECTOR_UNIT • USBH_VIDEO_VC_PROCESSING_UNIT • USBH_VIDEO_VC_EXTENSION_UNIT
bTermUnitID	Unique identifier (address space is shared between terminal and unit IDs).

20.3.3 USBH_VIDEO_INPUT_HEADER_INFO

Description

Structure containing information about a video stream.

Type definition

```
typedef struct {
    U8  bNumFormats;
    U8  bmInfo;
    U8  bTerminalLink;
    U8  bStillCaptureMethod;
    U8  bTriggerSupport;
    U8  bTriggerUsage;
    U8  bControlSize;
    U8  bmaControls[];
} USBH_VIDEO_INPUT_HEADER_INFO;
```

Structure members

Member	Description
bNumFormats	Number of video payload formats.
bmInfo	Indicates the capabilities of this VideoStreaming interface: • D0: Dynamic Format Change supported • D7..1: Reserved
bTerminalLink	The terminal ID of the Output Terminal to which the video endpoint of this interface is connected
bStillCaptureMethod	Method of still image capture supported: • 0: None • 1: Method 1 • 2: Method 2 • 3: Method 3
bTriggerSupport	Specifies if hardware triggering is supported through this interface • 0: Not supported • 1: Supported
bTriggerUsage	Specifies how the host software shall respond to a hardware trigger interrupt event from this interface. This is ignored if the bTriggerSupport field is zero. • 0: Initiate still image capture • 1: General purpose button event.
bControlSize	Size of each bmaControls(x) field, in bytes
bmaControls	Each byte in this array corresponds to one format (e.g. bmaControls [0] is for the first format) Each bit indicates support for the following capabilities: • D0: wKeyFrameRate • D1: wPFrameRate • D2: wCompQuality • D3: wCompWindowSize • D4: Generate Key Frame • D5: Update Frame Segment

20.3.4 USBH_VIDEO_FORMAT_INFO

Description

Structure containing information about a video format.

Type definition

```
typedef struct {
    U16  FormatType;
} USBH_VIDEO_FORMAT_INFO;
```

Structure members

Member	Description
FormatType	Format type, one of the following: - USBH_VIDEO_VS_FORMAT_UNCOMPRESSED - USBH_VIDEO_VS_FORMAT_MJPEG - USBH_VIDEO_VS_FORMAT_STREAM_BASED (H.264)

20.3.5 USBH_VIDEO_FRAME_INFO

Description

Structure containing information about a video frame. The structure is used for all supported formats (Uncompressed, MJPEG or H.264 Video Frame).

Type definition

```
typedef struct {
    U16  FrameType;
    U8   bFrameIndex;
    U8   bmCapabilities;
    U16  wWidth;
    U16  wHeight;
    U32  dwMinBitRate;
    U32  dwMaxBitRate;
    U32  dwBytesPerLine;
    U32  dwDefaultFrameInterval;
    U8   bFrameIntervalType;
} USBH_VIDEO_FRAME_INFO;
```

Structure members

Member	Description
FrameType	Frame type: • USBH_VIDEO_VS_FORMAT_UNCOMPRESSED • USBH_VIDEO_VS_FORMAT_MJPEG • USBH_VIDEO_VS_FORMAT_FRAME_BASED (H.264)
bFrameIndex	Frame index ID (this not the index used with <code>USBH_VIDEO_GetFrameInfo()</code>), this is the internal ID used by the USB device.
bmCapabilities	Capabilities bitmap: • D0: Still image supported • D1: Fixed frame-rate • D2-D7: Reserved.
wWidth	X - frame width
wHeight	Y - frame height.
dwMinBitRate	Minimum bit rate
dwMaxBitRate	Maximum bit rate.
dwBytesPerLine	(H.264 only) Line stride alignment.
dwDefaultFrameInterval	Default frame interval.
bFrameIntervalType	Frame interval type. A value of 0 indicates a continuous frame interval (very rarely used), a value > 0 indicates the number of discrete frame intervals supported (amount of valid entries inside the <code>u.dwFrameInterval</code> array).

20.3.6 USBH_VIDEO_COLOR_INFO

Description

Structure containing information about a video format's color matching.

Type definition

```
typedef struct {
    U8  bColorPrimaries;
    U8  bTransferCharacteristics;
    U8  bMatrixCoefficients;
} USBH_VIDEO_COLOR_INFO;
```

Structure members

Member	Description
<code>bColorPrimaries</code>	Defines color primaries: • 1: BT.709, sRGB (default) • 2: BT.470-2 (M) • 3: BT.470-2 (B, G) • 4: SMPTE 170M • 5: SMPTE 240M • 6-255: Reserved
<code>bTransferCharacteristics</code>	Opto-electronic transfer characteristic: • 0: Unspecified (unknown) • 1: BT.709 (default) • 2: BT.470-2 M • 3: BT.470-2 B, G
<code>bMatrixCoefficients</code>	Matrix used to compute luma and chroma values from the color primaries: • 0: Unspecified • 1: BT. 709 • 2: FCC • 3: BT.470-2 B, G • 4: SMPTE 170M (BT.601, default) • 5: SMPTE 240M • 6-255: Reserved

20.3.7 USBH_VIDEO_PAYLOAD_HEADER

Description

Structure containing the payload header. Users must check `bHeaderLength` and `bmHeaderInfo` to make sure all fields are valid.

Type definition

```
typedef struct {
    U8    bHeaderLength;
    U8    bmHeaderInfo;
    U16   SOFCounter;
    U32   dwPresentationTime;
    U32   SourceTimeClock;
} USBH_VIDEO_PAYLOAD_HEADER;
```

Structure members

Member	Description
<code>bHeaderLength</code>	Length of the header.
<code>bmHeaderInfo</code>	Bitfield which provides information about the availability and validity of additional fields: • D0: Frame ID • D1: End of Frame flag • D2: Presentation Time field valid • D3: Source Clock Reference field valid • D4: Reserved • D5: Still Image flag • D6: Error flag • D7: End of header flag
<code>SOFCounter</code>	1KHz SOF token counter
<code>dwPresentationTime</code>	Presentation Time Stamp
<code>SourceTimeClock</code>	Source Clock Reference

20.3.8 USBH_VIDEO_STREAM_CONFIG

Description

Structure containing configuration with which the emUSB-Host Video class should start reading frame data from the device.

Type definition

```
typedef struct {
    U8                Flags;
    U8                FormatIdx;
    U8                FrameIdx;
    U8                FrameIntervalIdx;
    U32               IntervalValue;
    USBH_VIDEO_RX_CALLBACK * pfDataCallback;
    void              * pUserDataContext;
    USBH_VIDEO_PAYLOAD_CALLBACK * pfPayloadCallback;
    void              * pUserPayloadContext;
} USBH_VIDEO_STREAM_CONFIG;
```

Structure members

Member	Description
Flags	Reserved
FormatIdx	Index of the format to use
FrameIdx	Index of the frame to use
FrameIntervalIdx	Index of the Frame interval to use (0 if the frame interval type is "continuous").
IntervalValue	(Only used for continuous frame interval!) Interval in 100ns units. The application must make sure the value is within the dwMinFrameInterval and dwMaxFrameInterval limits and that dwFrameIntervalStep is respected.
pfDataCallback	Function that is called when data is received from the device.
pUserDataContext	Optional pointer to user data which is passed to the pfDataCallback function.
pfPayloadCallback	Optional function that is called with every received payload.
pUserPayloadContext	Optional pointer to user data which is passed to the pfPayloadCallback function.

20.3.9 USBH_VIDEO_GET_CONTROL

Description

Structure used with “Get” requests.

Type definition

```
typedef struct {
    U8    bGetControlType;
    U16   Selector;
    U8    * pData;
    U32   * pDataLen;
} USBH_VIDEO_GET_CONTROL;
```

Structure members

Member	Description
bGetControlType	Should be one of the following types: • USBH_VIDEO_GET_CUR • USBH_VIDEO_GET_MIN • USBH_VIDEO_GET_MAX • USBH_VIDEO_GET_RES • USBH_VIDEO_GET_LEN • USBH_VIDEO_GET_INFO • USBH_VIDEO_GET_DEF
Selector	Control Selector , see USBH_VIDEO_CS_* macros.
pData	Pointer to a buffer where the device response will be saved into.
pDataLen	in Number of bytes to requests from the device. This value must match the requested control selector. See “USB Device Class Definition for Video Devices revision 1.1” for details. out Number of bytes received.

20.3.10 USBH_VIDEO_SET_CONTROL

Description

Structure used with “Set Cur” requests.

Type definition

```
typedef struct {  
    U16  Selector;  
    U8  * pData;  
    U32  DataLen;  
} USBH_VIDEO_SET_CONTROL;
```

Structure members

Member	Description
Selector	Control Selector , see USBH_VIDEO_CS_* macros.
pData	Pointer to the SET_CUR request data.
DataLen	Length of the request.

20.4 Function Types

This chapter describes the emUSB-Host Video API function types.

Type	Description
USBH_VIDEO_RX_CALLBACK	Definition of the callback which is registered via <code>USBH_VIDEO_OpenStream()</code> .
USBH_VIDEO_PAYLOAD_CALLBACK	Definition of the callback which is registered via <code>USBH_VIDEO_OpenStream()</code> .

20.4.1 USBH_VIDEO_RX_CALLBACK

Description

Definition of the callback which is registered via USBH_VIDEO_OpenStream(). This callback is called by the stack when new video data is received. To ensure good throughput this callback should block as little as possible.

After the data received has been handled the application must call USBH_VIDEO_Ack() to allow for the internal buffer to be re-used by the stack.

If the function reports an error with Status ≠ USBH_STATUS_SUCCESS, then there is no data: pBuf = NULL and NumBytes = 0 and USBH_VIDEO_Ack() must not be called.

Type definition

```
typedef void USBH_VIDEO_RX_CALLBACK(
    USBH_VIDEO_DEVICE_HANDLE hDevice,
    USBH_VIDEO_STREAM_HANDLE hStream,
    USBH_STATUS               Status,
    const U8                  * pData,
    unsigned                  NumBytes,
    U32                       Flags,
    void                      * pUserDataContext);
```

Parameters

Parameter	Description
hDevice	Device handle.
hStream	Stream handle.
Status	USBH_STATUS_SUCCESS on success or error code on failure.
pBuf	Pointer to the last filled buffer.
NumBytes	Number of bytes inside the buffer.
Flags	Bitfield containing flags related to the received data: USBH_UVC_END_OF_FRAME

20.4.2 USBH_VIDEO_PAYLOAD_CALLBACK

Description

Definition of the callback which is registered via `USBH_VIDEO_OpenStream()`. This function is called with each received payload. This function is only necessary if you are interested in the payload header fields (see `USBH_VIDEO_PAYLOAD_HEADER` structure). Payloads are normally received every 125 us in high-speed and every 1 ms in full-speed.

Type definition

```
typedef void USBH_VIDEO_PAYLOAD_CALLBACK(
    USBH_VIDEO_DEVICE_HANDLE hDevice,
    USBH_STATUS Status,
    const USBH_VIDEO_PAYLOAD_HEADER * pHeader,
    void * pUserPayloadContext);
```

Parameters

Parameter	Description
<code>hDevice</code>	Device handle.
<code>Status</code>	<code>USBH_STATUS_SUCCESS</code> on success or error code on failure.
<code>pHeader</code>	Pointer to the payload header.
<code>NumBytes</code>	Number of bytes inside the buffer.

Chapter 21

USB On-The-Go (Add-On)

This chapter describes the emUSB-Host add-on emUSB-OTG and how to use it. The emUSB-OTG is an optional extension of emUSB-Host.

21.1 Introduction

21.1.1 Overview

USB On-The-Go (OTG) allows two USB devices to communicate with each other. OTG introduces the dual-role device, meaning a device capable of functioning as either host or peripheral. USB OTG retains the standard USB host/peripheral model, in which a single host talks to USB peripherals. emUSB OTG offers a simple interface in order to detect the role of the USB OTG controller.

21.1.2 Features

The following features are provided:

- Detection of the USB role of the device.
- Virtually any USB OTG transceiver can be used.
- Simple interface to OTG-hardware.
- Seamless integration with emUSB-Host and emUSB-Device.

21.1.3 Example code

An example application which uses the API is provided in the `USB_OTG_Start.c` file of your shipment. This example starts the OTG stack and waits until a valid session is detected. As soon as a valid session is detected, the ID-pin state is checked to detect whether emUSB-Device or emUSB-Host shall then be initialized. For emUSB-Device a simple mouse sample is used. On emUSB-Host side an MSD-sample is used that detects USB memory stick and shows information about the detected stick.

Excerpt from the example code:

```

/*****
 *
 *      OTGTask
 *
 * Function description
 *   USB OTG handling task.
 *   It implements a basic function how to check which USB stack shall be called.
 *   It first checks whether the OTG chip has detected a valid session.
 *   If so, the next step will be to check the state of the ID-pin of the cable.
 *   If pin is 0 (grounded) -> a USB host cable is connected.
 *   If pin is 1 (floating) -> a USB device is plugged in.
 *
 */
void OTGTask(void);
void OTGTask(void) {
    int State;
    while (1) {
        //
        // Initialize OTG stack
        //
        USB_OTG_Init();
        //
        // Wait for a valid session
        //
        for (;;) {
            State = USB_OTG_GetSessionState();
            if (State != USB_OTG_ID_PIN_STATE_IS_INVALID) {
                break;
            }
            USB_OTG_OS_Delay(25);
            BSP_ToggleLED(0);
            USB_OTG_OS_Delay(25);
            BSP_ToggleLED(1);
        }
    }
}

```

```
    }  
    //  
    // Determine whether Device or Host stack shall be initialized and started.  
    //  
    USB_OTG_DeInit();  
    USB_OS_Delay(10);  
    if (State == USB_OTG_ID_PIN_STATE_IS_HOST) {  
        _ExecUSBHost();  
    } else {  
        _ExecUSBDevice();  
    }  
}  
}
```

21.1.4 OTG Driver

To use emUSB OTG, a driver matching the target hardware is required. The driver handles both the OTG controller as well as the OTG transceiver. The driver interface has been designed to take full advantage of hardware features such as session detection and session request protocol.

21.2 API Functions

This chapter describes the emUSB-OTG API functions.

Function	Description
USB_OTG_Init()	Initializes the core.
USB_OTG_DeInit()	De-initialize the complete OTG module.
USB_OTG_GetSessionState()	Returns whether the OTG transceiver has detected a valid session.
USB_OTG_GetIdPin()	Returns the current state of the USB OTG ID pin.
USB_OTG_SetIdPinFunc()	Sets a custom function to sense the ID pin state.
USB_OTG_AddDriver()	Adds a OTG driver to the OTG stack.
USB_OTG_X_Config()	User-provided function which configures the emUSB-OTG stack.

21.2.1 USB_OTG_Init()

Description

Initializes the core. It calls the driver initialization callback.

Prototype

```
void USB_OTG_Init(void);
```

21.2.2 USB_OTG_DeInit()

Description

De-initialize the complete OTG module.

Prototype

```
void USB_OTG_DeInit(void);
```


21.2.3 USB_OTG_GetSessionState()

Description

Returns whether the OTG transceiver has detected a valid session.

Prototype

```
int USB_OTG_GetSessionState(void);
```

Return value

USB_OTG_ID_PIN_STATE_IS_HOST	Host session detected.
USB_OTG_ID_PIN_STATE_IS_DEVICE	Device session detected.
USB_OTG_ID_PIN_STATE_IS_INVALID	No valid session.

21.2.4 USB_OTG_GetIdPin()

Description

Returns the current state of the USB OTG ID pin.

Prototype

```
int USB_OTG_GetIdPin(void);
```

Return value

0	ID pin is low
1	otherwise

21.2.5 USB_OTG_SetIdPinFunc()

Description

Sets a custom function to sense the ID pin state.

Prototype

```
void USB_OTG_SetIdPinFunc(USB_OTG_IDPIN_FUNC * pfGetIdPin);
```

Parameters

Parameter	Description
<code>pfGetIdPin</code>	Custom function.

21.2.6 USB_OTG_AddDriver()

Description

Adds a OTG driver to the OTG stack. This function is generally called in the function USB_OTG_X_Config().

Prototype

```
void USB_OTG_AddDriver(const USB_OTG_HW_DRIVER * pDriver);
```

Parameters

Parameter	Description
<code>pDriver</code>	Pointer to the driver structure.

21.2.7 USB_OTG_IDPIN_FUNC

Description

Returns the current state of the USB OTG ID pin.

Type definition

```
typedef int USB_OTG_IDPIN_FUNC(void);
```

Return value

0	ID pin is low
1	otherwise

21.2.7.1 USB_OTG_X_Config()

Description

User provided function which configures the USB OTG stack.

Prototype

```
void USB_OTG_X_Config(void);
```

Additional information

This function is called by the start-up code of the USB OTG stack from USB_OTG_Init(). This function should initialize all necessary clocks and pins required for the OTG operation of your controller.

Example

```
/* *****  
 *  
 *      USB_OTG_X_Config  
 */  
void USB_OTG_X_Config(void) {  
    _InitClocks();  
    _InitPins();  
    USB_OTG_AddDriver(&USB_OTG_Driver_ST_STM32F2xxFS);  
}
```

Chapter 22

Configuring emUSB-Host

This chapter explains how to configure emUSB-Host.

22.1 Runtime configuration

The configuration of emUSB-Host for a target hardware is done at runtime: The emUSB-Host stack calls a function named `USBH_X_Config`, that must be provided by the application. This function performs board specific hardware initialization like configuring I/O pins of the MCU, setting up PLL and clock divider necessary for USB and installing the interrupt service routine for USB.

In general many devices need to configure GPIO pins in order to use them with the USB host controller. In most cases the following pins are necessary:

- USB D+
- USB D-
- USB VBUS
- USB GND
- USB PowerOn
- USB OverCurrent

Please note that those pins need to be initialized within the `USBH_X_Config()` function before the host controller driver Add-function is called.

Additionally all runtime configuration of the USB stack is done in this function, for example:

- Assign memory to be used by the emUSB-Host stack.
- Select an appropriate driver for the USB host controller.
- Set driver specific parameters like base address of the controller of transfer buffer sizes.
- Set debug message output filter.
- Set a memory address translation routine (if a MMU is used).
- Enable HUB support.

Sample configurations for popular evaluation boards are supplied with the driver shipment. They can be found in files called `USBH_Config_<TargetName>.c` in the folders `BSP/<Board-Name>/Setup`. This files can be used as a template for a customized configuration.

22.1.1 Memory pools

To dynamically manage multiple USB devices that may be connected to emUSB-Host, it has to create data structures containing information about these devices. This include data structures for each device, interface and endpoints. Additionally data buffers and DMA descriptors may be required to actually perform data transfers to the device. emUSB-Host will take all memory required for this data structures and buffers from memory pools that must be provided by the application.

emUSB-Host uses up to two memory pools: The first is assigned using `USBH_AssignMemory()`, the second one with `USBH_AssignTransferMemory()`. The latter may be necessary to satisfy special requirements of the USB controller and driver to access memory uncached and perform DMA to these memory areas. Some drivers do not distinguish between these two memory areas and therefore only require one memory pool. For details, see *Host controller specifics* on page 692.

The size of the memory pool should be customized for the needs of the application. The optimal memory pool size for an application can be determined as follows:

- Start with a bigger memory pool configuration.
- Run the application in DEBUG mode using the maximum functionality (e.g. connect all devices that should run simultaneously and perform all needful data transfers).
- If warning messages regarding memory allocation appear in the debug output, enlarge the memory pool and restart the procedure.
- Call the function `USBH_MEM_GetMaxUsed()` to find out, how much memory of the pool was actually used by the USB stack.
- Resize the memory pool accordingly.

Estimated values for the memory usage can be found in *RAM usage* on page 786. Some drivers require additional memory for data buffers for each endpoint. The buffer size may be tuned using `USBH_ConfigTransferBufferSize()`.

22.1.2 USBH_X_Config()

Description

Initialize USB hardware and configure the USB-Host stack. This function is called by the startup code of the emUSB-Host stack from USBH_Init(). This is the place where a hardware driver can be added and configured.

Prototype

```
void USBH_X_Config(void);
```

Example

```
void USBH_X_Config(void) {
    //
    // Assigning memory should be the first thing
    //
    USBH_AssignMemory(&_aPool[0], ALLOC_SIZE);
    USBH_AssignTransferMemory(&_aTransferBufferPool[0], ALLOC_TRANSFER_SIZE);
    //
    // Allow external hubs
    //
    USBH_ConfigSupportExternalHubs(1);
    //
    // Wait 300ms for a new connected device
    //
    USBH_ConfigPowerOnGoodTime(300);
    //
    // Define log and warn filter
    //
    USBH_ConfigMsgFilter(USBH_WARN_FILTER_SET_ALL, 0, NULL);
    // Output all warnings.
    USBH_ConfigMsgFilter(USBH_LOG_FILTER_SET, sizeof(_LogCategories), _LogCategories);
    //
    // Initialize USB hardware
    //
    _InitUSBHw();
    //
    // Add EHCI driver
    //
    USBH_EHCI_EX_Add((void*)(USB_EHCI_BASE_ADDR + 0x100));
    //
    // Install interrupt service routine
    //
    BSP_USBH_InstallISR_Ex(USB0_IRQn, _ISR, USB_ISR_PRI0);
}
```

Configuration functions

Functions that may or must be used in USBH_X_Config are listed in the following table. Additional driver dependent functions exist for every USB host controller driver, see *Host controller specifics* on page 692.

Function	Description
USBH_AssignMemory()	Assigns a memory area that will be used by the memory management functions for allocating memory.
USBH_AssignTransferMemory()	Assigns a memory area for a heap that will be used for allocating DMA memory.
USBH_Config_SetV2PHandler()	Sets a virtual address to physical address translator.

Function	Description
USBH_ConfigPowerOnGoodTime()	Configures the power on time that the host waits after connecting a device before starting to communicate with the device.
USBH_ConfigSupportExternalHubs()	Enable support for external USB hubs.
USBH_ConfigTransferBufferSize()	Configures the size of a copy buffer that can be used if the USB controller has limited access to the system memory or the system is using cached (data) memory.
USBH_SetCacheConfig()	Configures cache related functionality that might be required by the stack for several purposes such as cache handling in drivers.
USBH_SetOnSetPortPower()	Sets a callback for the set-port-power driver function.
USBH_ConfigMsgFilter()	Sets a mask that defines which logging or warning message should be logged.

22.2 Compile-time configuration

emUSB-Host can be used without changing any of the compile-time switches. All compile-time configuration switches are preconfigured with valid values which match the requirements of most applications. All compile-time switches and their default values can be found in the file `USBH_ConfDefaults.h`.

To change the default configuration of emUSB-Host compile-time switches can be added to `USBH_Conf.h`. Don't change the `USBH_ConfDefaults.h` file for easy updates of emUSB-Host.

22.2.1 Compile-time switches for debugging

22.2.1.1 USBH_DEBUG

Description

emUSB-Host can be configured to display debug messages and warnings to locate an error or potential problems. This can be useful for debugging. In a release (production) build of a target system, they are typically not required and should be switches off.

To output the messages, emUSB-Host uses the logging routines contained in `USBH_ConfigIO.c` which can be customized.

`USBH_DEBUG` can be set to the following values:

- 0 - Used for release builds. Includes no debug options.
- 1 - Used in debug builds to include support for "panic" checks.
- 2 - Used in debug builds to include warning, log messages and "panic" checks. This significantly increases the code size.

Definition

```
#define USBH_DEBUG 0
```

22.2.1.2 USBH_LOG_BUFFER_SIZE

Description

Maximum size of a debug / warning message (in characters) that can be output. A buffer of this size is created on the stack when a message is output.

Definition

```
#define USBH_LOG_BUFFER_SIZE 200
```

22.2.2 Use of standard C-library functions

emUSB-Host calls some functions from the standard C-library. If the standard C-library should not be used, the following macros can be changed to call user defined functions instead:

```
#define USBH_MEMCPY    memcpy
#define USBH_MEMSET    memset
#define USBH_MEMCMP    memcmp
#define USBH_MEMMOVE   memmove
#define USBH_STRLLEN   strlen
#define USBH_STRCAT     strcat
#define USBH_STRCHR     strchr
#define USBH_STRNCOPY   strncpy
#define USBH_STRCMP     strcmp
```

22.2.3 General USB configuration

22.2.3.1 USBH_SUPPORT_ISO_TRANSFER

Description

Must be set to 1 if the USB stack shall support isochronous transfers (e.g for audio and video applications). If set to 0, all code that handles isochronous transfers is disabled, which may significantly reduce the code size of the USB stack.

Definition

```
#define USBH_SUPPORT_ISO_TRANSFER 0
```

22.2.3.2 USBH_MAX_NUM_HOST_CONTROLLERS

Description

Maximum number of host controllers the USB stack can handle.

Definition

```
#define USBH_MAX_NUM_HOST_CONTROLLERS 4u
```

22.2.3.3 USBH_SUPPORT_VIRTUALMEM

Description

If set to 1 the USB stack allows translation of virtual to physical memory addresses used for DMA operations (see `USBH_Config_SetV2PHandler()`). If the target system does not have a MMU, it can be set to 0.

Definition

```
#define USBH_SUPPORT_VIRTUALMEM 1
```

22.2.3.4 USBH_REO_FREE_MEM_LIST

Description

The USB stack uses a memory heap to allocate data structures for each connected USB device (see `USBH_AssignMemory()` and `USBH_AssignTransferMemory()`). If USB devices are frequently connected and disconnected this may lead to fragmentation of the heap memory. If this options is set, a reorganization of all free memory areas in the heap is performed after each disconnection of a USB device.

Definition

```
#define USBH_REO_FREE_MEM_LIST 0
```

22.2.3.5 USBH_USE_APP_MEM_PANIC

Description

The USB host stack calls the function "void `USBH_MEM_Panic(void)`", if memory allocation fails during initialization of the host stack (`USBH_Init()`). The stack contains a default implementation of the function `USBH_MEM_Panic()` which halts the system, indicating a fatal error. An application may implement its own `USBH_MEM_Panic()` function, when setting `USBH_USE_APP_MEM_PANIC` to '1'. After successful initialization using `USBH_Init()`, `USBH_MEM_Panic()` is never called.

Definition

```
#define USBH_USE_APP_MEM_PANIC    0
```

22.2.3.6 USBH_MAX_INTERFACES_IN_IAD

Description

Maximum number of interface IDs inside a USBH_IAD_INFO structure. See USBH_GetIADInfo().

Definition

```
#define USBH_MAX_INTERFACES_IN_IAD    5u
```

22.2.4 USB device enumeration behavior

22.2.4.1 USBH_WAIT_AFTER_RESET

Description

The host controller waits this time after reset of a root hub port, before the device descriptor is requested or the Set Address command is sent. Given in milliseconds.

Definition

```
#define USBH_WAIT_AFTER_RESET    180
```

22.2.4.2 USBH_HUB_WAIT_AFTER_RESET

Description

The host controller waits this time after reset of a external hub port, before the device descriptor is requested or the Set Address command is sent. Given in milliseconds.

Definition

```
#define USBH_HUB_WAIT_AFTER_RESET    180u
```

22.2.4.3 WAIT_AFTER_SETADDRESS

Description

The USB stack waits this time before the next command is sent after Set Address. The device must answer to SetAddress on USB address 0 with the handshake and then set the new address. This is a potential racing condition if this step is performed in the firmware. Give the device this time to set the new address. Given in milliseconds.

Definition

```
#define WAIT_AFTER_SETADDRESS    30u
```

22.2.4.4 USBH_RESET_RETRY_COUNTER

Description

If an error is encountered during USB reset, set address or enumeration the process is repeated [USBH_RESET_RETRY_COUNTER](#) times before the port is finally disabled.

Definition

```
#define USBH_RESET_RETRY_COUNTER    5u
```

22.2.4.5 USBH_DELAY_FOR_REENUM

Description

Describes the time in milliseconds before a USB reset is restarted, after the enumeration of the device (get descriptors, set configuration) has failed.

Definition

```
#define USBH_DELAY_FOR_REENUM 1000u
```

22.2.4.6 USBH_DELAY_BETWEEN_ENUMERATIONS

Description

On default, enumeration for multiple devices may be processed in parallel. Setting [USBH_DELAY_BETWEEN_ENUMERATIONS](#) > 0 will serialize all enumerations using a delay before a new enumeration is performed. The delay can be given in milliseconds.

Definition

```
#define USBH_DELAY_BETWEEN_ENUMERATIONS 0
```

22.2.4.7 USBH_DEFAULT_SETUP_TIMEOUT

Description

Default timeout (in milliseconds) for all setup requests during enumeration of a device. After this time a not completed setup request is terminated. Windows gives 2 seconds to answer to a setup request. Less than that some devices behave quite strange. So it should ≥ 2000 .

Definition

```
#define USBH_DEFAULT_SETUP_TIMEOUT 2000
```

22.2.5 URB handling

22.2.5.1 USBH_URB_QUEUE_SIZE

Description

If not 0, queue URBs, when the driver reports USBH_STATUS_NO_CHANNEL and retry them later. The value gives the maximum number of URBs that can be queued. Only used for BULK transfers.

Definition

```
#define USBH_URB_QUEUE_SIZE 0u
```

22.2.5.2 USBH_URB_QUEUE_RETRY_INTV

Description

URB queue retry interval in ms. Only used, if USBH_URB_QUEUE_SIZE $\neq 0$.

Definition

```
#define USBH_URB_QUEUE_RETRY_INTV 5u
```

22.2.5.3 USBH_SUPPORT_HUB_CLEAR_TT_BUFFER

Description

If set, a CLEAR_TT_BUFFER request is send to the HUB after an URB was aborted. Used only for USB devices that are using split transactions. Not supported by all drivers.

Definition

```
#define USBH_SUPPORT_HUB_CLEAR_TT_BUFFER 0
```

22.2.6 Mass storage class configuration

22.2.6.1 USBH_MSD_MAX_DEVICES

Description

Maximum number of mass storage devices the USB stack can handle simultaneously.

Definition

```
#define USBH_MSD_MAX_DEVICES 10u
```

22.2.6.2 USBH_MSD_MAX_SECTORS_AT_ONCE

Description

Maximum number of sectors to read with a single MSD read command. Certain sticks have a limitation where they can not read or write too many sectors in one command. For example: DTSE9 16GB read limit ~ 4096, DT Ultimate G2 16GB write limit ~ 1024. Windows uses max 128 sectors. Linux uses max 240 sectors.

Definition

```
#define USBH_MSD_MAX_SECTORS_AT_ONCE 256u
```

22.2.6.3 USBH_MSD_EP0_TIMEOUT

Description

Specifies the timeout in milliseconds to be used for control requests to a mass storage device, especially for 'GetMaxLun' and 'ClearFeatureHalt' commands.

Definition

```
#define USBH_MSD_EP0_TIMEOUT 5000u
```

22.2.6.4 USBH_MSD_CBW_WRITE_TIMEOUT

Description

Specifies the timeout in milliseconds for sending a command block to a mass storage device.

Definition

```
#define USBH_MSD_CBW_WRITE_TIMEOUT 3000
```

22.2.6.5 USBH_MSD_CSW_READ_TIMEOUT

Description

Specifies the timeout in milliseconds for reading a status block from a mass storage device. 10 seconds is compatible to Windows.

Definition

```
#define USBH_MSD_CSW_READ_TIMEOUT    10000
```

22.2.6.6 USBH_MSD_COMMAND_TIMEOUT**Description**

Specifies the timeout in milliseconds for reading answer data from a mass storage device when not reading sector data.

Definition

```
#define USBH_MSD_COMMAND_TIMEOUT    3000u
```

22.2.6.7 USBH_MSD_DATA_READ_TIMEOUT**Description**

Read timeout in milliseconds for the data phase when reading 'Length' bytes of sector data from a mass storage device.

Definition

```
#define USBH_MSD_DATA_READ_TIMEOUT(Length)    (10000u + ((Length) / 512u) * 10u)
```

22.2.6.8 USBH_MSD_DATA_WRITE_TIMEOUT**Description**

Write timeout in milliseconds for the data phase when writing 'Length' bytes of sector data to a mass storage device.

Definition

```
#define USBH_MSD_DATA_WRITE_TIMEOUT(Length)    (10000u + ((Length) / 512u) * 10u)
```

22.2.6.9 USBH_MSD_MAX_TEST_READY_RETRIES**Description**

Maximum number of retries executed for TestUnitReady / ReadCapacity commands on failure during enumeration of a mass storage device. Value must be < 255.

Definition

```
#define USBH_MSD_MAX_TEST_READY_RETRIES    200u
```

22.2.6.10 USBH_MSD_MAX_READY_WAIT_TIME**Description**

Maximum time (in milliseconds) to wait for a LUN to become ready after enumeration of a mass storage device, before the user notification callback is called.

Definition

```
#define USBH_MSD_MAX_READY_WAIT_TIME    20000
```


22.2.6.11 USBH_MSD_TEST_UNIT_READY_DELAY

Description

Minimum time (in milliseconds) between two TestUnitReady commands send to mass storage device.

Definition

```
#define USBH_MSD_TEST_UNIT_READY_DELAY    5000
```

22.2.7 HID class configuration

22.2.7.1 USBH_HID_MAX_REPORTS

Description

Maximum number of reports (with different report ID's) of one HID interface that can be handled.

Definition

```
#define USBH_HID_MAX_REPORTS    6u
```

22.2.7.2 USBH_HID_DISABLE_INTERFACE_PROTOCOL_CHECK

Description

Some HID devices, namely touch screens, report their interface protocol as "mouse" despite being actual touch screens (normally bInterfaceProtocol is "None" for such devices). If this happens with your touch screens you can enable this flag.

Definition

```
#define USBH_HID_DISABLE_INTERFACE_PROTOCOL_CHECK    0
```

22.2.7.3 USBH_HID_EP0_TIMEOUT

Description

Specifies the timeout in milliseconds to be used for control requests to a HID device, like 'GetReportDescriptor' command.

Definition

```
#define USBH_HID_EP0_TIMEOUT    1000
```

22.2.8 CDC-ACM class configuration

22.2.8.1 USBH_CDC_DISABLE_AUTO_DETECT

Description

Can be used to disable the automatic detection of CDC devices. In that case the user must use USBH_CDC_AddDevice/USBH_CDC_RemoveDevice for addition and removal of devices.

Definition

```
#define USBH_CDC_DISABLE_AUTO_DETECT    0
```

22.2.8.2 USBH_CDC_EP0_TIMEOUT

Description

Specifies the timeout in milliseconds to be used for control requests to a CDC device, like 'SetAlternateInterface', 'ClearFeatureHalt', 'SetLineCoding' and 'SetControlLineState' commands.

Definition

```
#define USBH_CDC_EP0_TIMEOUT    1000
```

22.2.9 BULK (Vendor) class configuration

22.2.9.1 USBH_BULK_EP0_TIMEOUT

Description

Specifies the timeout in milliseconds to be used for control requests to a vendor (BULK) device, especially for 'SetAlternateInterface' and 'ClearFeatureHalt' commands.

Definition

```
#define USBH_BULK_EP0_TIMEOUT    1000
```

22.2.9.2 USBH_BULK_MAX_NUM_EPS

Description

Maximum number of endpoints that can be handled for a BULK (Vendor) device.

Definition

```
#define USBH_BULK_MAX_NUM_EPS    5u
```

22.2.10 Printer class configuration

22.2.10.1 USBH_PRINTER_EP0_TIMEOUT

Description

Specifies the timeout in milliseconds to be used for control requests to a printer device, especially for 'GetPortStatus' and 'SendVendorRequest' commands.

Definition

```
#define USBH_PRINTER_EP0_TIMEOUT    1000
```

22.2.11 AUDIO class configuration

22.2.11.1 USBH_AUDIO_EP0_TIMEOUT

Description

Specifies the timeout in milliseconds to be used for control requests to an AUDIO device.

Definition

```
#define USBH_AUDIO_EP0_TIMEOUT    1000
```

22.2.12 FT232 class configuration

22.2.12.1 USBH_FT232_EP0_TIMEOUT

Description

Specifies the timeout in milliseconds to be used for control requests to a FT232 device, like 'ResetDevice', 'SetBaudRate', 'SetDtr', 'ClrDtr', 'SetRts', 'ClrRts', 'GetModemStatus', 'SetChars', 'Purge', 'SetBreakOn', 'SetBreakOff', 'SetLatencyTimer', 'GetLatencyTimer', 'SetBitMode', 'GetBitMode', 'SetDataCharacteristics' and 'SetFlowControl' commands.

Definition

```
#define USBH_FT232_EP0_TIMEOUT 1000
```

22.2.13 CP210X class configuration

22.2.13.1 USBH_CP210X_EP0_TIMEOUT

Description

Specifies the timeout in milliseconds to be used for control requests to a CP210X device, like 'SetBaudRate', 'SetDataCharacteristics', 'SetModemHandshaking', 'GetModemStatus' and 'Purge' commands.

Definition

```
#define USBH_CP210X_EP0_TIMEOUT 1000
```

22.3 Host controller specifics

For emUSB-Host different USB host controller drivers are provided. Normally, the drivers are ready and do not need to be configured at all. Some drivers may need to be configured in a special manner, due to some limitation of the controller.

This section lists the drivers which require special configuration and describes how to configure those drivers.

22.3.1 EHCI driver

Normally EHCI controllers only handle high-speed USB devices. Some EHCI controllers contain a transaction translator (TT) that enables them to handle full- and low-speed devices also. There are different Add-functions to configure the driver for host controllers with or without a TT.

Systems with cached memory

If the EHCI driver is installed on a system using cached (data) memory, the following requirements must be considered:

- A special region of RAM is necessary that can be accessed non-cached and non-buffered by the CPU. The USB host controller must also be able to access this area via DMA. If the system contains a MMU all memory can be used cached as usual. Additionally the MMU should be configured to mirror whole or part of the cached memory to another address range for uncached access.
- The non-cached RAM area must be provided to the USB stack using the function `USBH_AssignTransferMemory()`. The memory area must be cache clean before calling the function `USBH_AssignTransferMemory()`.
- If the physical address is not equal to the virtual address of the non-cached memory area (address translation by an MMU), a mapping function must be installed using `USBH_Config_SetV2PHandler()`. The translated addresses (physical addresses) are used for DMA by the host controller.
- Cache functions that may be set with `USBH_SetCacheConfig()` are **not** used by the driver.
- The function `USBH_EHCI_Config_UseTransferBuffer()` must be called, in order to tell the driver that application defined buffers can not be accessed directly via DMA.
- Transfer buffers are allocated for each endpoint with a default size of 16KB. This results in a maximum transfer speed, but requires a huge memory pool. In order to save memory, the size of the transfer buffers can be reduced to at least 512 bytes, see `USBH_ConfigTransferBufferSize()`.

Systems without cached memory

On systems without cache there is no need to provide a separate memory area with `USBH_AssignTransferMemory()`. A single memory heap is sufficient for the USB stack, see `USBH_AssignMemory()`.

22.3.1.1 EHCI driver specific configuration functions

Function	Description
<code>USBH_EHCI_Add()</code>	Adds a HS capable EHCI controller to the stack.
<code>USBH_EHCI_EX_Add()</code>	Adds a LS/FS/HS capable EHCI controller to the stack.
<code>USBH_RT1050_Add()</code>	Adds a HS capable EHCI controller to the stack.
<code>USBH_RT1170_Add()</code>	Adds a HS capable EHCI controller to the stack.
<code>USBH_IMX6U5_Add()</code>	Adds a HS capable EHCI controller to the stack.
<code>USBH_EHCI_Config_SetM2MEndianMode()</code>	Setups the internal EHCI memory to memory transfer endianness mode.
<code>USBH_EHCI_Config_UseTransferBuffer()</code>	Configures the driver to use temporary transfer buffers instead using the user buffer directly.

Function	Description
<code>USBH_EHCI_Config_IgnoreOverCurrent()</code>	Configures the driver to ignore the over current condition reported by the hardware.

22.3.1.2 USBH_EHCI_Add()

Description

Adds a HS capable EHCI controller to the stack.

Prototype

```
U32 USBH_EHCI_Add(void * pBase);
```

Parameters

Parameter	Description
<code>pBase</code>	Pointer to the base of the EHCI controllers register set.

Return value

Reference to the added host controller (0-based index).

22.3.1.3 USBH_EHCI_EX_Add()

Description

Adds a LS/FS/HS capable EHCI controller to the stack.

Prototype

```
U32 USBH_EHCI_EX_Add(void * pBase);
```

Parameters

Parameter	Description
<code>pBase</code>	Pointer to the base of the EHCI controllers register set.

Return value

Reference to the added host controller (0-based index).

22.3.1.4 USBH_RT1050_Add()

Description

Adds a HS capable EHCI controller to the stack. This EHCI initialization function is specific to the RT1050 series.

Prototype

```
U32 USBH_RT1050_Add(void * pBase);
```

Parameters

Parameter	Description
<code>pBase</code>	Pointer to the base of the EHCI controllers register set.

Return value

Reference to the added host controller (0-based index).

Additional information

Suspend and resume of USB devices is not supported for the RT1050 series EHCI controller. Due to a hardware issue devices connected to the roothub port must stay connected for at least 250ms before they are removed. Should a device be removed before that the USB controller will crash and only recover after a power cycle of the MCU.

For this controller the transfer memory (`USBH_AssignTransferMemory`) must be put into the internal DTCM RAM (0x20000000 ~ 0x2007FFFF).

22.3.1.5 USBH_RT1170_Add()

Description

Adds a HS capable EHCI controller to the stack. This EHCI initialization function is specific to the RT1170 series.

Prototype

```
U32 USBH_RT1170_Add(void * pBase);
```

Parameters

Parameter	Description
<code>pBase</code>	Pointer to the base of the EHCI controllers register set.

Return value

Reference to the added host controller (0-based index).

Additional information

Suspend and resume of USB devices is not supported for the RT1170 series EHCI controller. Due to a hardware issue devices connected to the roothub port must stay connected for at least 250ms before they are removed. Should a device be removed before that the USB controller will crash and only recover after a power cycle of the MCU.

For this controller the transfer memory (`USBH_AssignTransferMemory`) must be put into the internal DTCM RAM.

22.3.1.6 USBH_IMX6U5_Add()

Description

Adds a HS capable EHCI controller to the stack. This EHCI initialization function is specific to the IMX6U5 series.

Prototype

```
U32 USBH_IMX6U5_Add(void * pBase);
```

Parameters

Parameter	Description
<code>pBase</code>	Pointer to the base of the EHCI controllers register set.

Return value

Reference to the added host controller (0-based index).

Additional information

Suspend and resume of USB devices is not supported for the IMX6U5 series EHCI controller. Due to a hardware issue devices connected to the roothub port must stay connected for at least 250ms before they are removed. Should a device be removed before that the USB controller will crash and only recover after a power cycle of the MCU.

22.3.1.7 USBH_EHCI_Config_SetM2MEndianMode()

Description

Setups the internal EHCI memory to memory transfer endianness mode. This only has an effect on the DMA transfers. Both SFRs and DMA descriptors are still in the endianness defined by the MCU/EHCI controller manufacturer. In normal cases the SFRs and DMA descriptors are in CPU native endianness mode.

Prototype

```
void USBH_EHCI_Config_SetM2MEndianMode(U32 HCIndex,
                                         int Endian);
```

Parameters

Parameter	Description
HCIndex	Index of the host controller returned by USBH_EHCI_EX_Add().
Endian	- USBH_EHCI_M2M_ENDIAN_MODE_LITTLE - use little endian mode for memory-2-memory (DMA) transfers - USBH_EHCI_M2M_ENDIAN_MODE_BIG - use big endian mode for memory-2-memory (DMA) transfers

22.3.1.8 USBH_EHCI_Config_UseTransferBuffer()

Description

Configures the driver to use temporary transfer buffers instead using the user buffer directly. This is necessary if the MCU uses cache.

Prototype

```
void USBH_EHCI_Config_UseTransferBuffer(U32 HCIndex);
```

Parameters

Parameter	Description
HCIndex	Index of the host controller, returned by <code>USBH_EHCI_Add()</code> or <code>USBH_EHCI_EX_Add()</code> .

22.3.1.9 USBH_EHCI_Config_IgnoreOverCurrent()

Description

Configures the driver to ignore the over current condition reported by the hardware.

Prototype

```
void USBH_EHCI_Config_IgnoreOverCurrent(U32 HCIndex);
```

Parameters

Parameter	Description
HCIndex	Index of the host controller, returned by USBH_EHCI_Add() or USBH_EHCI_EX_Add().

22.3.2 Synopsys DWC2 driver

Using this driver there is no need to provide a separate memory area with `USBH_AssignTransferMemory()`. A single memory heap is sufficient for the USB stack, see `USBH_AssignMemory()`.

22.3.2.1 High-speed driver

The following must be considered if using the DWC2 high-speed driver. This is valid, even if the USB controller is used in full-speed mode (like on the STM32H7xx with internal PHY).

If the driver is installed on a system using cached (data) memory, cache functions for cleaning and invalidating cache lines must be provided and set with `USBH_SetCacheConfig()`.

On some MCUs the USB controller is not able to access all RAM areas the application uses. In this case a function can be installed that checks for memory valid for DMA access (see `USBH_STM32Fx_HS_SetCheckAddress()` functions below). If the function reports a memory region not valid for DMA, the driver uses a temporary transfer buffer to copy data to and from this area. The memory area provided with `USBH_AssignMemory()` must be DMA capable. If there is not enough DMA capable memory, the application may provide two different memory areas: A DMA capable area using `USBH_AssignTransferMemory()` and a second one with `USBH_AssignMemory()` which then will not be accessed via DMA.

22.3.2.2 Restrictions

Low-speed devices

On any STM32 MCUs using the internal USB FS PHY low-speed USB devices connected via an external hub are not recognized due to a hardware limitation. If connected in this way it may happen, that the host controller gets disturbed and blocked. In order to return to normal operation, a reset of the controller and the external hub may be necessary.

In any case try to avoid low-speed devices in such a constellation. The issue seems to be related to the internal PHY of the STM32 MCUs. It usually not occurs, if the high-speed USB controller is used in connection with an external (high-speed) PHY.

Split transactions

If the host controller operates in high-speed and a full or low-speed device is connected via an external hub, the driver uses split transactions to access the device. Split transactions will only work reliable, if

- The task running `USBH_ISRTask()` must have the highest priority in the system.
- The task running `USBH_Task()` must have the second highest priority.
- All other task must have a lower priority than `USBH_Task()`.
- Both USB tasks must not be delayed by any interrupt service routine for more than 500 μ s.

Additionally applies: Split transactions require usage of a 125 μ s interrupt, which increases system load.

Isochronous endpoints (full-speed controller)

Due to insufficient FIFO memory in the full-speed USB controller, isochronous endpoints (which are used in AUDIO devices for example) are supported with a maximum packet size of up to 384 bytes only.

Isochronous endpoints (high-speed controller)

Isochronous transactions (which are used in AUDIO devices for example) are not supported for full-speed devices connected via an external hub. Only transfer intervals of one packet per frame (1ms) for full-speed devices and one packet per microframe (125 μ s) for high-speed devices are supported. Due to insufficient FIFO memory, isochronous endpoints are

supported with a maximum packet size of up to 512 bytes only (for OUT endpoints) and 2000 bytes for IN endpoints (high-bandwidth).

Channel stuck

The USB controller hardware provides a number of channels that can be used to independently schedule packet transfers to USB devices via different endpoints.

Because of an issue of the USB controller hardware, sometimes a channel used for a packet transfer can't be re-initialized after it was used. Instead the channel remains in an undefined state and can't be used for new transfers. This mainly happens after interrupted transfers, when a device is disconnected in the middle of a packet transfer, but the exact cause it not clear.

The emUSB driver implements some recovery mechanism to get stuck channels working again, but this does not always work. If not, the channel is stuck and can't be used any more until a reset of the USB controller. The driver continues working using the remaining channels.

Because there is a limited number of channels, it may happen, that the driver runs out of channels and is not able to service all transfer requests of the USB stack any more or gets not operational at all.

Related warning messages of the driver:

- "Re-trigger channel halt"
- "Re-trigger channel disable"
- "channel x is dead!"

22.3.2.3 Synopsys driver specific configuration functions

Function	Description
USBH_STM32_Add()	Adds a Synopsys DWC2 full speed controller of a STM32F107 device to the stack.
USBH_STM32F2_FS_Add()	Adds a Synopsys DWC2 full speed controller of a STM32F2xx or STM32F4xx device to the stack.
USBH_STM32F2_HS_Add()	Adds a Synopsys DWC2 high speed controller of a STM32F2xx or STM32F4xx device to the stack.
USBH_STM32F2_HS_AddEx()	Adds a Synopsys DWC2 high speed controller of a STM32F2xx or STM32F4xx device to the stack.
USBH_STM32F2_HS_SetCheckAddress()	Installs a function that checks if an address can be used for DMA transfers.
USBH_STM32F7_FS_Add()	Adds a Synopsys DWC2 full speed controller of a STM32F7xx device to the stack.
USBH_STM32F7_HS_Add()	Adds a Synopsys DWC2 high speed controller of a STM32F7xx device to the stack.
USBH_STM32F7_HS_AddEx()	Adds a Synopsys DWC2 high speed controller of a STM32F7xx or STM32F7xx device to the stack.
USBH_STM32F7_HS_SetCheckAddress()	Installs a function that checks if an address can be used for DMA transfers.
USBH_STM32H7_HS_Add()	Adds a Synopsys DWC2 high speed controller of a STM32H7xx device to the stack.
USBH_STM32H7_HS_AddEx()	Adds a Synopsys DWC2 high speed controller of a STM32H7xx or STM32H7xx device to the stack.

Function	Description
<code>USBH_STM32H7_HS_SetCheckAddress()</code>	Installs a function that checks if an address can be used for DMA transfers.
<code>USBH_XMC4xxx_FS_Add()</code>	Adds a Synopsys DWC2 full speed controller of a XMC4xxx device to the stack.

22.3.2.4 USBH_STM32_Add()

Description

Adds a Synopsys DWC2 full speed controller of a STM32F107 device to the stack.

Prototype

```
U32 USBH_STM32_Add(void * pBase);
```

Parameters

Parameter	Description
pBase	Pointer to the base of the controllers register set.

Return value

Reference to the added host controller (0-based index).

22.3.2.5 USBH_STM32F2_FS_Add()

Description

Adds a Synopsys DWC2 full speed controller of a STM32F2xx or STM32F4xx device to the stack.

Prototype

```
U32 USBH_STM32F2_FS_Add(void * pBase);
```

Parameters

Parameter	Description
<code>pBase</code>	Pointer to the base of the controllers register set.

Return value

Reference to the added host controller (0-based index).

22.3.2.6 USBH_STM32F2_HS_Add()

Description

Adds a Synopsys DWC2 high speed controller of a STM32F2xx or STM32F4xx device to the stack.

Prototype

```
U32 USBH_STM32F2_HS_Add(void * pBase);
```

Parameters

Parameter	Description
<code>pBase</code>	Pointer to the base of the controllers register set.

Return value

Reference to the added host controller (0-based index).

22.3.2.7 USBH_STM32F2_HS_AddEx()

Description

Adds a Synopsys DWC2 high speed controller of a STM32F2xx or STM32F4xx device to the stack.

Prototype

```
U32 USBH_STM32F2_HS_AddEx(void * pBase,
                           U8     PhyType);
```

Parameters

Parameter	Description
pBase	Pointer to the base of the controllers register set.
PhyType	0 - use external PHY connected via ULPI interface. 1 - use internal full speed PHY.

Return value

Reference to the added host controller (0-based index).

22.3.2.8 USBH_STM32F2_HS_SetCheckAddress()

Description

Installs a function that checks if an address can be used for DMA transfers. Installed function must return 0, if DMA access is allowed for the given address, 1 otherwise.

Prototype

```
void USBH_STM32F2_HS_SetCheckAddress  
      (USBH_CHECK_ADDRESS_FUNC * pfCheckValidDMAAddress);
```

Parameters

Parameter	Description
pfCheckValidDMAAddress	Pointer to the function.

Additional information

If the function reports a memory region not valid for DMA, the driver uses a temporary transfer buffer to copy data to and from this area.

22.3.2.9 USBH_STM32F7_FS_Add()

Description

Adds a Synopsys DWC2 full speed controller of a STM32F7xx device to the stack.

Prototype

```
U32 USBH_STM32F7_FS_Add(void * pBase);
```

Parameters

Parameter	Description
<code>pBase</code>	Pointer to the base of the controllers register set.

Return value

Reference to the added host controller (0-based index).

22.3.2.10 USBH_STM32F7_HS_Add()

Description

Adds a Synopsys DWC2 high speed controller of a STM32F7xx device to the stack.

Prototype

```
U32 USBH_STM32F7_HS_Add(void * pBase);
```

Parameters

Parameter	Description
pBase	Pointer to the base of the controllers register set.

Return value

Reference to the added host controller (0-based index).

22.3.2.11 USBH_STM32F7_HS_AddEx()

Description

Adds a Synopsys DWC2 high speed controller of a STM32F7xx or STM32F7xx device to the stack.

Prototype

```
U32 USBH_STM32F7_HS_AddEx(void * pBase,
                           U8      PhyType);
```

Parameters

Parameter	Description
pBase	Pointer to the base of the controllers register set.
PhyType	0 - use external PHY connected via ULPI interface. 1 - use internal full-speed PHY.

Return value

Reference to the added host controller (0-based index).

22.3.2.12 USBH_STM32F7_HS_SetCheckAddress()

Description

Installs a function that checks if an address can be used for DMA transfers. Installed function must return 0, if DMA access is allowed for the given address, 1 otherwise.

Prototype

```
void USBH_STM32F7_HS_SetCheckAddress  
      (USBH_CHECK_ADDRESS_FUNC * pfCheckValidDMAAddress);
```

Parameters

Parameter	Description
pfCheckValidDMAAddress	Pointer to the function.

Additional information

If the function reports a memory region not valid for DMA, the driver uses a temporary transfer buffer to copy data to and from this area.

22.3.2.13 USBH_STM32H7_HS_Add()

Description

Adds a Synopsys DWC2 high speed controller of a STM32H7xx device to the stack.

Prototype

```
U32 USBH_STM32H7_HS_Add(void * pBase);
```

Parameters

Parameter	Description
pBase	Pointer to the base of the controllers register set.

Return value

Reference to the added host controller (0-based index).

22.3.2.14 USBH_STM32H7_HS_AddEx()

Description

Adds a Synopsys DWC2 high speed controller of a STM32H7xx or STM32H7xx device to the stack.

Prototype

```
U32 USBH_STM32H7_HS_AddEx(void * pBase,
                           U8      PhyType);
```

Parameters

Parameter	Description
pBase	Pointer to the base of the controllers register set.
PhyType	0 - use external PHY connected via ULPI interface. 1 - use internal full speed PHY.

Return value

Reference to the added host controller (0-based index).

22.3.2.15 USBH_STM32H7_HS_SetCheckAddress()

Description

Installs a function that checks if an address can be used for DMA transfers. Installed function must return 0, if DMA access is allowed for the given address, 1 otherwise.

Prototype

```
void USBH_STM32H7_HS_SetCheckAddress  
      (USBH_CHECK_ADDRESS_FUNC * pfCheckValidDMAAddress);
```

Parameters

Parameter	Description
pfCheckValidDMAAddress	Pointer to the function.

Additional information

If the function reports a memory region not valid for DMA, the driver uses a temporary transfer buffer to copy data to and from this area.

22.3.2.16 USBH_XMC4xxx_FS_Add()

Description

Adds a Synopsys DWC2 full speed controller of a XMC4xxx device to the stack.

Prototype

```
U32 USBH_XMC4xxx_FS_Add(void * pBase);
```

Parameters

Parameter	Description
pBase	Pointer to the base of the controllers register set.

Return value

Reference to the added host controller (0-based index).

22.3.3 PSOC Edge E84 driver

Using this driver there is no need to provide a separate memory area with `USBH_AssignTransferMemory()`. A single memory heap is sufficient for the USB stack, see `USBH_AssignMemory()`.

If the driver is used on a CPU using cached (data) memory, cache functions for cleaning and invalidating cache lines must be provided and set with `USBH_SetCacheConfig()`.

If the USB controller is not able to access all RAM areas the application uses, a function can be installed that checks for memory valid for DMA access (see `USBH_Infineon_PSOCE84_SetCheckAddress()` functions below). If the function reports a memory region not valid for DMA, the driver uses a temporary transfer buffer to copy data to and from this area. The memory are provided with `USBH_AssignMemory()` must be DMA capable. If there is not enough DMA capable memory, the application may provide two different memory areas: A DMA capable area using `USBH_AssignTransferMemory()` and a second one with `USBH_AssignMemory()` which then will not be accessed via DMA.

22.3.3.1 Restrictions

Split transactions

If the host controller operates in high-speed and a full or low-speed device is connected via an external hub, the driver uses split transactions to access the device. Split transactions will only work reliable, if

- The task running `USBH_ISRTask()` must have the highest priority in the system.
- The task running `USBH_Task()` must have the second highest priority.
- All other task must have a lower priority than `USBH_Task()`.
- Both USB tasks must not be delayed by any interrupt service routine for more than 500 μ s.

Additionally applies: Split transactions require usage of a 125 μ s interrupt, which increases system load.

Isochronous endpoints

Due to a restricted hardware configuration the size of high bandwidth isochronous IN packets is limited to a maximum of 2048 bytes (2 x 1024).

Isochronous transactions (which are used in AUDIO devices for example) are not supported for full-speed devices connected via an external hub. Only transfer intervals of one packet per frame (1ms) for full-speed devices and one packet per microframe (125 μ s) for high-speed devices are supported.

22.3.3.2 Synopsys driver specific configuration functions

Function	Description
<code>USBH_Infineon_PSOCE84_DMA_Add()</code>	Adds a PSOCE84 high speed controller to the stack.
<code>USBH_Infineon_PSOCE84_SetCheckAddress()</code>	Installs a function that checks if an address can be used for DMA transfers.

22.3.3.3 USBH_Infineon_PSOCE84_DMA_Add()

Description

Adds a PSOCE84 high speed controller to the stack.

Prototype

```
U32 USBH_Infineon_PSOCE84_DMA_Add(void * pBase);
```

Parameters

Parameter	Description
pBase	Pointer to the base of the controllers register set.

Return value

Reference to the added host controller (0-based index).

22.3.3.4 USBH_Infineon_PSOCE84_SetCheckAddress()

Description

Installs a function that checks if an address can be used for DMA transfers. Installed function must return 0, if DMA access is allowed for the given address, 1 otherwise.

Prototype

```
void USBH_Infineon_PSOCE84_SetCheckAddress
(USBH_CHECK_ADDRESS_FUNC * pfCheckValidDMAAddress);
```

Parameters

Parameter	Description
pfCheckValidDMAAddress	Pointer to the function.

Additional information

If the function reports a memory region not valid for DMA, the driver uses a temporary transfer buffer to copy data to and from this area.

22.3.4 OHCI driver

OHCI controllers handle full-speed and low-speed USB devices.

Systems with cached memory

If the OHCI driver is installed on a system using cached (data) memory, the following requirements must be considered:

- A special region of RAM is necessary that can be accessed non-cached and non-buffered by the CPU. The USB host controller must also be able to access this area via DMA.
- The non-cached RAM area must be provided to the USB stack using the function `USBH_AssignTransferMemory()`. The memory area must be cache clean before calling the function `USBH_AssignTransferMemory()`.
- If the physical address is not equal to the virtual address of the non-cached memory area (address translation by an MMU), a mapping function must be installed using `USBH_Config_SetV2PHandler()`. The translated addresses (physical address) are used for DMA by the host controller.
- Cache functions that may be set with `USBH_SetCacheConfig()` are **not** used by the driver.

Systems without cached memory

On systems without cache there is no need to provide a separate memory area with `USBH_AssignTransferMemory()`. A single memory heap is sufficient for the USB stack, see `USBH_AssignMemory()`.

On systems without cache and where the OHCI controller has access to the memory where application buffers are located `USBH_OHCI_ZC_Add()` (instead of `USBH_OHCI_Add()`) can be used to improve performance.

22.3.4.1 OHCI driver specific configuration functions

Function	Description
<code>USBH_OHCI_Add()</code>	Adds a full-speed capable OHCI controller to the stack.
<code>USBH_OHCI_ZC_Add()</code>	Adds a full-speed capable OHCI controller to the stack (zero-copy version).
<code>USBH_OHCI_ZC_SetCheckZeroCopyAddress()</code>	Installs a function that checks if an address can be used for zero copy transfers.
<code>USBH_OHCI_LPC546_Add()</code>	Adds a full-speed capable OHCI controller to the stack.

22.3.4.2 USBH_OHCI_Add()

Description

Adds a full-speed capable OHCI controller to the stack.

Prototype

```
U32 USBH_OHCI_Add(void * pBase);
```

Parameters

Parameter	Description
<code>pBase</code>	Pointer to the base of the OHCI controllers register set.

Return value

Reference to the added host controller (0-based index).

22.3.4.3 USBH_OHCI_ZC_Add()

Description

Adds a full-speed capable OHCI controller to the stack (zero-copy version).

Prototype

```
U32 USBH_OHCI_ZC_Add(void * pBase);
```

Parameters

Parameter	Description
<code>pBase</code>	Pointer to the base of the OHCI controllers register set.

Return value

Reference to the added host controller (0-based index).

Additional information

This version of the OHCI driver uses zero-copy where possible thereby increasing transfer speeds and decreasing memory consumption. The zero-copy version of the OHCI driver is currently NOT compatible with any MCUs which use cache.

Certain MCUs do not allow the OHCI controller to access some memory regions. For this purpose a callback should be registered via `USBH_OHCI_ZC_SetCheckZeroCopyAddress()`

22.3.4.4 USBH_OHCI_ZC_SetCheckZeroCopyAddress()

Description

Installs a function that checks if an address can be used for zero copy transfers. Installed function must return 0, if zero copy access is allowed for the given address, 1 otherwise.

Prototype

```
void USBH_OHCI_ZC_SetCheckZeroCopyAddress
      (USBH_CHECK_ADDRESS_FUNC * pfCheckValidZCAddress);
```

Parameters

Parameter	Description
pfCheckValidZCAddress	Pointer to the function.

Additional information

If the function reports a memory region not valid for zero copy, the driver uses a temporary transfer buffer to copy data to and from this area.

22.3.4.5 USBH_OHCI_LPC546_Add()

Description

Adds a full-speed capable OHCI controller to the stack. This add function, enables workarounds inside the OHCI driver for the LPC546xx series. One of these workarounds creates a limitation where an interval timeout of 1 can not be guaranteed for interrupt endpoints as each interrupt endpoint transfer completion has to be delayed by up to two frames.

Prototype

```
U32 USBH_OHCI_LPC546_Add(void * pBase);
```

Parameters

Parameter	Description
<code>pBase</code>	Pointer to the base of the OHCI controllers register set.

Return value

Reference to the added host controller (0-based index).

22.3.5 Kinetis USBOTG FS driver

KinetisFS controllers handle full-speed and low-speed USB devices.

Systems without cached memory

On systems without cache there is no need to provide a separate memory area with `USBH_AssignTransferMemory()`. A single memory heap is sufficient for the USB stack, see `USBH_AssignMemory()`.

Restrictions

- WFI instruction can not be used when the USBOTG module is used. This is a hardware issue, see errata e7166 for chip masks 4N96B, 1N96B, 3N96B.
- When the controller accesses the flash memory (e.g. when an application writes to a device from a const char array) it is necessary to allow the controller access to the flash area and to set the bus master priority to the highest level, otherwise read accesses to the flash may be stalled which results in the controller getting less data than expected and respective follow-up errors.
- This controller can only schedule a single transaction at a time. This means that hubs can not be used with this host controller. Furthermore composite devices will not work properly if the application communicates on multiple interfaces at once. Devices which contain an Interrupt IN endpoint which needs to be constantly polled by the host controller will also have issues as the Interrupt IN endpoint will hog the only communication pipe, a good sample for this is the CDC class which consists of Bulk IN, Bulk OUT and Interrupt IN endpoints, in this case the controller will be constantly busy polling the Interrupt IN endpoint and will not be able to communicate via the Bulk endpoints. To work around the Interrupt IN issue emUSB-Host provides the functions `USBH_CDC_SetConfigFlags()` and `USBH_HID_ConfigureAllowLEDUpdate()`.

22.3.5.1 KinetisFS driver specific configuration functions

Function	Description
<code>USBH_KINETIS_FS_Add()</code>	Adds a full-speed capable KinetisFS controller to the stack.

22.3.5.2 USBH_KINETIS_FS_Add()

Description

Adds a full-speed capable KinetisFS controller to the stack.

Prototype

```
U32 USBH_KINETIS_FS_Add(void * pBase);
```

Parameters

Parameter	Description
<code>pBase</code>	Pointer to the base of the KinetisFS controllers register set.

Return value

Reference to the added host controller (0-based index).

22.3.6 Renesas driver

Using this driver there is no need to provide a separate memory area with `USBH_AssignTransferMemory()`. A single memory heap is sufficient for the USB stack, see `USBH_AssignMemory()`.

22.3.6.1 Restrictions

Maximum number of devices

The full-speed version of the controller can only handle up to 5 devices at once. High-speed controller can handle up to 10 devices.

Data corruption

This following issue seems to be valid for RX and RZ devices but not for the Synergy platform:

It seems that the concurrent transfers are not working correctly with higher bandwidth and may result in data corruption: Data that are sent on one pipe are received on another pipe or are mixed with received data.

ISO support

ISO packets are limited to a maximum size of 256 bytes.

Due to a hardware limitation, full-speed device connected through a USB high-speed hub to the highspeed controller, cannot handle more than 188 bytes as a transfer size in isochronous mode.

Low-speed devices

RX62x and RX63x series contains a USB controller where low-speed device such as mice, keyboards are not recognized properly. Therefore for these device, low-speed support is disabled. You may see in debug builds, that the warning message that this is not possible.

22.3.6.2 Overcurrent detection

Overcurrent detection can be configured using the function `USBH_ConfigOvercurrentDetection()`. The hardware must be designed for overcurrent detection. The `OvercurrentDetectionMode` must match the hardware design according to the following table:

OvercurrentDetectionMode	Overcurrent condition
0x1	never
0x4	OVRCURB pin low
0x5	OVRCURB pin high
0x8	OVRCURA pin low
0xA	OVRCURA pin high

22.3.6.3 Renesas driver specific configuration functions

Function	Description
<code>USBH_RX11_Add()</code>	Adds a Renesas USB controller of a RX11x device to the stack.
<code>USBH_RX23_Add()</code>	Adds a Renesas USB controller of a RX23x device to the stack.
<code>USBH_RX62_Add()</code>	Adds a Renesas USB controller of a RX62x device to the stack.

Function	Description
USBH_RX63_Add()	Adds a Renesas USB controller of a RX63x device to the stack.
USBH_RX64_Add()	Adds a Renesas USB controller of a RX64x device to the stack.
USBH_RX65_Add()	Adds a Renesas USB controller of a RX65x device to the stack.
USBH_RX71_FS_Add()	Adds a Renesas FS USB controller of a RX71x device to the stack.
USBH_RX71_HS_Add()	Adds a Renesas HS USB controller of a RX71x device to the stack.
USBH_RZA1_Add()	Adds a Renesas USB controller of a RZA1 device to the stack.
USBH_Synergy_FS_Add()	Adds a Renesas FS USB controller of a Synergy device to the stack.
USBH_Synergy_HS_Add()	Adds a Renesas HS USB controller of a Synergy device to the stack.

22.3.6.4 USBH_RX11_Add()

Description

Adds a Renesas USB controller of a RX11x device to the stack.

Prototype

```
U32 USBH_RX11_Add(void * pBase);
```

Parameters

Parameter	Description
<code>pBase</code>	Pointer to the base of the controllers register set.

Return value

Reference to the added host controller (0-based index).

22.3.6.5 USBH_RX23_Add()

Description

Adds a Renesas USB controller of a RX23x device to the stack.

Prototype

```
U32 USBH_RX23_Add(void * pBase);
```

Parameters

Parameter	Description
<code>pBase</code>	Pointer to the base of the controllers register set.

Return value

Reference to the added host controller (0-based index).

22.3.6.6 USBH_RX62_Add()

Description

Adds a Renesas USB controller of a RX62x device to the stack.

Prototype

```
U32 USBH_RX62_Add(void * pBase);
```

Parameters

Parameter	Description
<code>pBase</code>	Pointer to the base of the controllers register set.

Return value

Reference to the added host controller (0-based index).

22.3.6.7 USBH_RX63_Add()

Description

Adds a Renesas USB controller of a RX63x device to the stack.

Prototype

```
U32 USBH_RX63_Add(void * pBase);
```

Parameters

Parameter	Description
<code>pBase</code>	Pointer to the base of the controllers register set.

Return value

Reference to the added host controller (0-based index).

22.3.6.8 USBH_RX64_Add()

Description

Adds a Renesas USB controller of a RX64x device to the stack.

Prototype

```
U32 USBH_RX64_Add(void * pBase);
```

Parameters

Parameter	Description
<code>pBase</code>	Pointer to the base of the controllers register set.

Return value

Reference to the added host controller (0-based index).

22.3.6.9 USBH_RX65_Add()

Description

Adds a Renesas USB controller of a RX65x device to the stack.

Prototype

```
U32 USBH_RX65_Add(void * pBase);
```

Parameters

Parameter	Description
<code>pBase</code>	Pointer to the base of the controllers register set.

Return value

Reference to the added host controller (0-based index).

22.3.6.10 USBH_RX71_FS_Add()

Description

Adds a Renesas FS USB controller of a RX71x device to the stack.

Prototype

```
U32 USBH_RX71_FS_Add(void * pBase);
```

Parameters

Parameter	Description
<code>pBase</code>	Pointer to the base of the controllers register set.

Return value

Reference to the added host controller (0-based index).

22.3.6.11 USBH_RX71_HS_Add()

Description

Adds a Renesas HS USB controller of a RX71x device to the stack.

Prototype

```
U32 USBH_RX71_HS_Add(void * pBase);
```

Parameters

Parameter	Description
<code>pBase</code>	Pointer to the base of the controllers register set.

Return value

Reference to the added host controller (0-based index).

22.3.6.12 USBH_RZA1_Add()

Description

Adds a Renesas USB controller of a RZA1 device to the stack.

Prototype

```
U32 USBH_RZA1_Add(void * pBase);
```

Parameters

Parameter	Description
<code>pBase</code>	Pointer to the base of the controllers register set.

Return value

Reference to the added host controller (0-based index).

22.3.6.13 USBH_Synergy_FS_Add()

Description

Adds a Renesas FS USB controller of a Synergy device to the stack.

Prototype

```
U32 USBH_Synergy_FS_Add(void * pBase);
```

Parameters

Parameter	Description
pBase	Pointer to the base of the controllers register set.

Return value

Reference to the added host controller (0-based index).

22.3.6.14 USBH_Synergy_HS_Add()

Description

Adds a Renesas HS USB controller of a Synergy device to the stack.

Prototype

```
U32 USBH_Synergy_HS_Add(void * pBase);
```

Parameters

Parameter	Description
pBase	Pointer to the base of the controllers register set.

Return value

Reference to the added host controller (0-based index).

22.3.7 ATSAMx7 driver

Using this driver there is no need to provide a separate memory area with `USBH_AssignTransferMemory()`. A single memory heap is sufficient for the USB stack, see `USBH_AssignMemory()`.

22.3.7.1 Restrictions

LS support

Low-speed devices connected directly work starting with chip revision "B".

FS support

Full-speed devices connected directly do not work.

HUB support

Although the USB controller of the ATSAMx7 MCUs support external HUBs, the application is very limited. Because USB pipes can not be dynamically allocated to devices, connecting and disconnecting devices arbitrarily to and from the HUB will result in blocked pipes very soon, and new connected devices will not be recognized any more.

Also split transactions are not supported, so low-speed and full-speed devices can not be used via a high-speed HUB.

Endpoint Address limitation

This host controller only supports endpoint addresses from 0 to 10, if a device uses endpoint addresses higher than that it will not function. This is a limitation of the controller and there is no workaround (see description of the "PEPNUM: Pipe Endpoint Number" field of the `USBHS_HSTPIPCFGx` register in the reference manual).

22.3.7.2 ATSAMx7 driver specific configuration functions

Function	Description
<code>USBH_SAMx7_Add()</code>	Adds a HS capable ATSAM USBHS controller to the stack.

22.3.7.3 USBH_SAMx7_Add()

Description

Adds a HS capable ATSAM USBHS controller to the stack.

Prototype

```
U32 USBH_SAMx7_Add(PTR_ADDR BaseAddr,  
                   PTR_ADDR USBRAMAddr);
```

Parameters

Parameter	Description
BaseAddr	Base address of the USBHS controllers register set.
USBARAMAddr	Base address of the USBHS FIFO RAM.

Return value

Reference to the added host controller (0-based index).

22.3.8 LPC54xxx/LPC55xxx High-Speed driver

This section applies to LPC54xxx and LPC55xxx devices.

Using this driver a single memory heap is sufficient for the USB stack, see `USBH_AssignMemory()`. There is no need to provide a separate memory area with `USBH_AssignTransferMemory()`. The driver uses the dedicated USB1 RAM of the LPC54xxx MCU for endpoint transfer buffers.

22.3.8.1 Restrictions

Low-speed devices

The high-speed USB controller of the LPC54xxx/LPC55xxx will not reliably enumerate low-speed devices directly connected to the USB port. Although a workaround is implemented in the driver, it may not work in all cases. Sometimes a power cycle is necessary to recover from this error.

ISO support

ISO packets are limited to a maximum size of 1023 bytes.

22.3.8.2 LPC54xxx/LPC55xxx driver specific configuration functions

Function	Description
<code>USBH_LPC54xxx_Add()</code>	Adds a LPC54xxx high speed controller to the stack.
<code>USBH_LPC55xxx_Add()</code>	Adds a LPC55xxx high speed controller to the stack.

22.3.8.3 USBH_LPC54xxx_Add()

Description

Adds a LPC54xxx high speed controller to the stack.

Prototype

```
U32 USBH_LPC54xxx_Add(PTR_ADDR BaseAddr,
                      PTR_ADDR USBRAMAddr,
                      unsigned USBRAMSize);
```

Parameters

Parameter	Description
BaseAddr	Base address of the USB controllers register set.
USBARAMAddr	Base address of the dedicated USB RAM.
USBRAMSize	Size of the USB RAM in bytes.

Return value

Reference to the added host controller (0-based index).

22.3.8.4 USBH_LPC55xxx_Add()

Description

Adds a LPC55xxx high speed controller to the stack.

Prototype

```
U32 USBH_LPC55xxx_Add(PTR_ADDR BaseAddr,  
                      PTR_ADDR USBRAMAddr,  
                      unsigned USBRAMSize);
```

Parameters

Parameter	Description
BaseAddr	Base address of the USB controllers register set.
USBARAMAddr	Base address of the dedicated USB RAM.
USBARAMSize	Size of the USB RAM in bytes.

Return value

Reference to the added host controller (0-based index).

Additional information

This function enables a workaround for LPC55xx chips without which devices enumerate only once.

22.3.9 Cypress PSoC 6 driver

Using this driver there is no need to provide a separate memory area with `USBH_AssignTransferMemory()`. A single memory heap is sufficient for the USB stack, see `USBH_AssignMemory()`.

The driver can use either DMA for packet transfers or not.

22.3.9.1 Restrictions

LS support

Low-speed devices can not be used via a HUB.

ISO support

ISO packets are limited to a maximum size of 256 bytes.

Controller stuck

If devices are connected via a hub and any device is disconnected while a packet is transferred from this device, then the USB controller of the PSoC 6 gets stuck and connections to all devices (including the hub) are lost. The controller is not able to process any more tasks and will become operational only after a physical disconnect and re-connect of the device at the root port (usually the hub).

Suspend / Resume

Host suspend and resume does not work, because the controller is not able to generate correct signaling on the USB bus.

22.3.9.2 PSoC 6 driver specific configuration functions

Function	Description
<code>USBH_Cypress_PSoC_Add()</code>	Adds a Cypress PSoC6 full-speed controller to the stack.
<code>USBH_Cypress_PSoC_DMA_Add()</code>	Adds a Cypress PSoC6 full-speed controller to the stack using DMA transfers.

22.3.9.3 USBH_Cypress_PSoC_Add()

Description

Adds a Cypress PSoC6 full-speed controller to the stack.

Prototype

```
U32 USBH_Cypress_PSoC_Add(void);
```

Return value

Reference to the added host controller (0-based index).

22.3.9.4 USBH_Cypress_PSoC_DMA_Add()

Description

Adds a Cypress PSoC6 full-speed controller to the stack using DMA transfers. The DMA controller must be enabled and the EP1 trigger must be muxed to the DMA channel before calling this function.

Prototype

```
U32 USBH_Cypress_PSoC_DMA_Add(const USBH_CYPRESS_PSoC_DMA_API * pDMA_API,
                              PTR_ADDR DMACHannelAddr,
                              unsigned Priority);
```

Parameters

Parameter	Description
pDMA_API	DMA function API.
DMACHannelAddr	Address of the registers of the DMA channel to be used.
Priority	DMA priority (0-3).

Return value

Reference to the added host controller (0-based index).

22.3.10 ST full-speed driver

This driver can be used for:

- STM32H573
- STM32H56x

Using this driver there is no need to provide a separate memory area with `USBH_AssignTransferMemory()`. A single memory heap is sufficient for the USB stack, see `USBH_AssignMemory()`.

22.3.10.1 ST full-speed driver specific configuration functions

Function	Description
<code>USBH_STM32H5xx_Add()</code>	Adds the STM32H5 NG USB full speed controller to the stack.

22.3.10.2 USBH_STM32H5xx_Add()

Description

Adds the STM32H5 NG USB full speed controller to the stack.

Prototype

```
U32 USBH_STM32H5xx_Add(void);
```

Return value

Reference to the added host controller (0-based index).

Chapter 23

Support

23.1 Contacting support

Before contacting support please make sure that you are using the latest version of the emUSB-Host package. Also please check the chapter *Configuring debugging output* on page 36 and run your application with enabled debug support.

If you are a registered emUSB-Host user there are different ways to contact the emUSB-Host support:

1. You can create a support ticket via email to ticket_emusb@segger.com*
2. You can create a support ticket at [{segger.com/ticket}](https://segger.com/ticket).

Please include the following information in the email or ticket:

- The emUSB-Host version.
- Your emUSB-Host license number.
- If you are unsure about the above information you can also use the name of the emUSB-Host zip file (which contains the above information).
- A detailed description of the problem
- The configuration files USBH_Config*.*
- Any error messages.

Please also take a few moments to help us improve our services by providing a short feedback once your support case has been solved.

23.1.1 Where can I find the license number?

The license number is part of the shipped zip file name.

For example emUSBH_BASE_DWC2_FS_V2.36.2_USBH-01234_95C24726_230614.zip where USBH-01234 is the license number. The license number is also part of every *.c- and *.h-file header. For example, if you open USB.h you should find the license number as with the example below:

```
*****
*
*      emUSB-Host version: V2.36.2
*
*****

-----
Licensing information
Licensor:          SEGGER Microcontroller GmbH
Licensed to:       Customer name
Licensed SEGGER software: emUSB-Host
License number:    USBH-01234
License model:     SSL
Licensed product:  -
Licensed platform: Cortex-M, GCC
Licensed number of seats: 1
-----

Support and Update Agreement (SUA)
SUA period:        2023-05-30 - 2023-11-30
Contact to extend SUA: sales@segger.com
-----

Purpose           : API of the USB host stack
```

*By sending us an email your (personal) data will automatically be processed. For further information please refer to our privacy policy which is available at <https://www.segger.com/legal/privacy-policy/>.

Chapter 24

Debugging

emUSB-Host comes with various debugging options. These include optional warning and log outputs, as well as other runtime options which perform checks at run time.

24.1 Message output

The debug builds of emUSB-Host include a fine-grained debug system which helps to analyze the correct implementation of the stack in your application. All modules of the emUSB-Host stack can output logging and warning messages via terminal I/O, if the specific message type identifier is added to the log and/or warn filter mask. This approach provides the opportunity to get and interpret only the logging and warning messages which are relevant for the part of the stack that you want to debug.

By default, all warning messages are activated in all emUSB-Host sample configuration files. All logging messages are disabled except for the messages from the initialization phase.

Note

It is not advised to enable all log messages as the large amount of output may affect timing.

24.2 API functions

Function	Description
General functions	
USBH_ConfigMsgFilter()	Sets a mask that defines which logging or warning message should be logged.
USBH_Log()	This function is called by the stack in debug builds with log output.
USBH_Warn()	This function is called by the stack in debug builds with log output.
USBH_Panic()	Is called if the stack encounters a fatal error.
USBH_Logf_Application()	Displays application log information.
USBH_Warnf_Application()	Displays application warning information.
USBH_sprintf_Application()	A simple sprintf replacement.
USBH_Puts()	Prints a string without any additional output (no timestamp or newlines).

24.2.1 USBH_ConfigMsgFilter()

Description

Sets a mask that defines which logging or warning message should be logged. Logging messages are only available in debug builds of emUSB-Host.

Prototype

```
void USBH_ConfigMsgFilter(      unsigned Mode,
                               unsigned NumCategories,
                               const U8 * pCategories);
```

Parameters

Parameter	Description
Mode	Mode to configure message filter: • USBH_LOG_FILTER_SET: Set message categories in log filter. • USBH_LOG_FILTER_SET_ALL: Enable all log messages (parameter pCategories is ignored). • USBH_LOG_FILTER_ADD: Add message categories to log filter. • USBH_LOG_FILTER_CLR: Clear message categories in log filter. • USBH_WARN_FILTER_SET: Set message categories in warning filter. • USBH_WARN_FILTER_SET_ALL: Enable all warning messages (parameter pCategories is ignored). • USBH_WARN_FILTER_ADD: Add message categories to warning filter. • USBH_WARN_FILTER_CLR: Clear message categories in warning filter.
NumCategories	Number of messages categories contained in the array pCategories.
pCategories	Pointer to array of NumCategories messages categories that should be configured.

Additional information

Should be called from USBH_X_Config(). By default, the log message category USBH_MCAT_INIT and all warning messages are enabled.

Please note that the more logging is enabled, the more the timing of the application is influenced. For available message types see the USBH_MCAT_... definitions in USBH.h.

Please note that enabling all log messages is not necessary, nor is it advised as it will influence the timing greatly.

Example

```
static const U8 _LogCategories[] = {
    USBH_MCAT_INIT,
    USBH_MCAT_APPLICATION,
    USBH_MCAT_DRIVER
};

USBH_ConfigMsgFilter(USBH_LOG_FILTER_SET, sizeof(_LogCategories), _LogCategories);
```

24.2.2 USBH_Log()

Description

This function is called by the stack in debug builds with log output. In a release build, this function is not be linked in.

Prototype

```
void USBH_Log(const char * s);
```

Parameters

Parameter	Description
s	Pointer to a string holding the log message.

24.2.3 USBH_Warn()

Description

This function is called by the stack in debug builds with log output. In a release build, this function is not be linked in.

Prototype

```
void USBH_Warn(const char * s);
```

Parameters

Parameter	Description
s	Pointer to a string holding the warning message.

24.2.4 USBH_Panic()

Description

Is called if the stack encounters a fatal error.

Prototype

```
void USBH_Panic(const char * s);
```

Parameters

Parameter	Description
s	Pointer to a string holding the error message.

Additional information

In a release build this function is not linked in. The default implementation of this function disables all interrupts to avoid further task switches, outputs the error string via terminal I/O and loops forever. When using an emulator, you should set a break-point at the beginning of this routine or simply stop the program after a failure.

24.2.5 USBH_Logf_Application()

Description

Displays application log information.

Prototype

```
void USBH_Logf_Application(const char * sFormat,  
                           ...);
```

Parameters

Parameter	Description
sFormat	Message string with optional format specifiers.

24.2.6 USBH_Warnf_Application()

Description

Displays application warning information.

Prototype

```
void USBH_Warnf_Application(const char * sFormat,  
                             ...);
```

Parameters

Parameter	Description
sFormat	Message string with optional format specifiers.

24.2.7 USBH_sprintf_Application()

Description

A simple sprintf replacement.

Prototype

```
void USBH_sprintf_Application(    char    * pBuffer,
                                unsigned  BufferSize,
                                const char * sFormat,
                                ...);
```

Parameters

Parameter	Description
pBuffer	Pointer to a user provided buffer.
BufferSize	Size of the buffer in bytes.
sFormat	Message string with optional format specifiers.

24.2.8 USBH_Puts()

Description

Prints a string without any additional output (no timestamp or newlines).

Prototype

```
void USBH_Puts(const char * s);
```

Parameters

Parameter	Description
<code>s</code>	Pointer to a string holding the warning message.

Chapter 25

OS integration

emUSB-Host is designed to be used in a multitasking environment. The interface to the operating system is encapsulated in a single file, the USB-Host/OS interface. This chapter provides descriptions of the functions required to fully support emUSB-Host in multitasking environments.

25.1 General information

emUSB-Host includes an OS abstraction layer which should make it possible to use an arbitrary operating system together with emUSB-Host. To adapt emUSB-Host to a new OS one only has to map the functions listed below in section OS layer API functions to the native OS functions. SEGGER took great care when designing this abstraction layer, to make it easy to understand and to adapt to different operating systems. The target RTOS should at least have the following features:

- Events (Create, Delete, Set, Signal, Wait-for, Wait-for-with-timeout, Reset)
- Recursive (= reentrant) Mutex (Create, Delete, Use, Unuse)
- Timebase with millisecond precision
- Critical sections

25.1.1 Operating system support supplied with this release

In the current version, abstraction layer for embOS is available. Abstraction layers for other operating systems are available upon request.

25.2 OS layer API functions

Function	Description
General functions	
<code>USBH_OS_Delay()</code>	Blocks the calling task for a given time.
<code>USBH_OS_DisableInterrupt()</code>	Enter a critical region for the USB stack: Increments interrupt disable count and disables interrupts.
<code>USBH_OS_EnableInterrupt()</code>	Leave a critical region for the USB stack: Decrements interrupt disable count and enable interrupts if counter reaches 0.
<code>USBH_OS_GetTime32()</code>	Return the current system time in ms.
<code>USBH_OS_Init()</code>	Initialize (create) all objects required for task synchronization.
<code>USBH_OS_DeInit()</code>	Deletes all objects required for task synchronization.
<code>USBH_OS_Lock()</code>	This function locks a mutex object, guarding sections of the stack code against other threads.
<code>USBH_OS_Unlock()</code>	Unlocks the mutex used by a previous call to <code>USBH_OS_Lock()</code> .
USBH_Task synchronization	
<code>USBH_OS_SignalNetEvent()</code>	Wakes the <code>USBH_MainTask()</code> if it is waiting for a event or timeout in the function <code>USBH_OS_WaitNetEvent()</code> .
<code>USBH_OS_WaitNetEvent()</code>	Blocks until the timeout expires or a USBH-event occurs.
USBH_ISRTask synchronization	
<code>USBH_OS_SignalISREx()</code>	Wakes the <code>USBH_ISRTask()</code> .
<code>USBH_OS_WaitISR()</code>	Blocks until <code>USBH_OS_SignalISR()</code> is called (from ISR).
Application task synchronization	
<code>USBH_OS_AllocEvent()</code>	Allocates and returns an event object.
<code>USBH_OS_FreeEvent()</code>	Releases an object event.
<code>USBH_OS_SetEvent()</code>	Sets the state of the specified event object to signalled.
<code>USBH_OS_ResetEvent()</code>	Sets the state of the specified event object to non-signalled.
<code>USBH_OS_WaitEvent()</code>	Wait for the specific event.
<code>USBH_OS_WaitEventTimed()</code>	Wait for the specific event within a given timeout.

25.2.1 USBH_OS_Delay()

Description

Blocks the calling task for a given time.

Prototype

```
void USBH_OS_Delay(unsigned ms);
```

Parameters

Parameter	Description
<code>ms</code>	Delay in milliseconds.

25.2.2 USBH_OS_DisableInterrupt()

Description

Enter a critical region for the USB stack: Increments interrupt disable count and disables interrupts.

Prototype

```
void USBH_OS_DisableInterrupt(void);
```

Additional information

The USB stack will perform nested calls to USBH_OS_DisableInterrupt() and USBH_OS_EnableInterrupt().

25.2.3 USBH_OS_EnableInterrupt()

Description

Leave a critical region for the USB stack: Decrements interrupt disable count and enable interrupts if counter reaches 0.

Prototype

```
void USBH_OS_EnableInterrupt(void);
```

Additional information

The USB stack will perform nested calls to USBH_OS_DisableInterrupt() and USBH_OS_EnableInterrupt().

25.2.4 USBH_OS_GetTime32()

Description

Return the current system time in ms. The value will wrap around after app. 49.7 days. This is taken into account by the stack.

Prototype

```
USBH_TIME USBH_OS_GetTime32(void);
```

Return value

Current system time.

25.2.5 USBH_OS_Init()

Description

Initialize (create) all objects required for task synchronization.

Prototype

```
void USBH_OS_Init(void);
```

25.2.6 USBH_OS_DeInit()

Description

Deletes all objects required for task synchronization.

Prototype

```
void USBH_OS_DeInit(void);
```

25.2.7 USBH_OS_Lock()

Description

This function locks a mutex object, guarding sections of the stack code against other threads. Mutexes are recursive.

Prototype

```
void USBH_OS_Lock(unsigned Idx);
```

Parameters

Parameter	Description
<code>Idx</code>	Index of the mutex to be locked (0 .. USBH_MUTEX_COUNT-1).

25.2.8 USBH_OS_Unlock()

Description

Unlocks the mutex used by a previous call to `USBH_OS_Lock()`. Mutexes are recursive.

Prototype

```
void USBH_OS_Unlock(unsigned Idx);
```

Parameters

Parameter	Description
<code>Idx</code>	Index of the mutex to be released (0 .. <code>USBH_MUTEX_COUNT-1</code>).

25.2.9 USBH_OS_SignalNetEvent()

Description

Wakes the USBH_MainTask() if it is waiting for a event or timeout in the function USBH_OS_WaitNetEvent().

Prototype

```
void USBH_OS_SignalNetEvent(void);
```

25.2.10 USBH_OS_WaitNetEvent()

Description

Blocks until the timeout expires or a USBH-event occurs. Called from USBH_MainTask() only. A USBH-event is signaled with USBH_OS_SignalNetEvent() called from an other task or ISR.

Prototype

```
void USBH_OS_WaitNetEvent(unsigned ms);
```

Parameters

Parameter	Description
<code>ms</code>	Timeout in milliseconds.

25.2.11 USBH_OS_SignalISREx()

Description

Wakes the USBH_ISRTask(). Called from ISR.

Prototype

```
void USBH_OS_SignalISREx(U32 DevIndex);
```

Parameters

Parameter	Description
<code>DevIndex</code>	Zero-based index of the host controller that needs attention.

25.2.12 USBH_OS_WaitISR()

Description

Blocks until USBH_OS_SignalISR() is called (from ISR). Called from USBH_ISRTask() only.

Prototype

```
U32 USBH_OS_WaitISR(void);
```

Return value

An ISR mask, where each bit set corresponds to a host controller index.

25.2.13 USBH_OS_AllocEvent()

Description

Allocates and returns an event object.

Prototype

```
USBH_OS_EVENT_OBJ *USBH_OS_AllocEvent(void);
```

Return value

A pointer to a USBH_OS_EVENT_OBJ object on success or NULL on error.

25.2.14 USBH_OS_FreeEvent()

Description

Releases an object event.

Prototype

```
void USBH_OS_FreeEvent(USBH_OS_EVENT_OBJ * pEvent);
```

Parameters

Parameter	Description
pEvent	Pointer to an event object that was returned by USBH_OS_AllocEvent().

25.2.15 USBH_OS_SetEvent()

Description

Sets the state of the specified event object to signalled.

Prototype

```
void USBH_OS_SetEvent(USBH_OS_EVENT_OBJ * pEvent);
```

Parameters

Parameter	Description
<code>pEvent</code>	Pointer to an event object that was returned by <code>USBH_OS_AllocEvent()</code> .

25.2.16 USBH_OS_ResetEvent()

Description

Sets the state of the specified event object to non-signalled.

Prototype

```
void USBH_OS_ResetEvent(USBH_OS_EVENT_OBJ * pEvent);
```

Parameters

Parameter	Description
<code>pEvent</code>	Pointer to an event object that was returned by <code>USBH_OS_AllocEvent()</code> .

25.2.17 USBH_OS_WaitEvent()

Description

Wait for the specific event.

Prototype

```
void USBH_OS_WaitEvent(USBH_OS_EVENT_OBJ * pEvent);
```

Parameters

Parameter	Description
<code>pEvent</code>	Pointer to an event object that was returned by <code>USBH_OS_AllocEvent()</code> .

25.2.18 USBH_OS_WaitEventTimed()

Description

Wait for the specific event within a given timeout.

Prototype

```
int USBH_OS_WaitEventTimed(USBH_OS_EVENT_OBJ * pEvent,
                           U32                MilliSeconds);
```

Parameters

Parameter	Description
pEvent	Pointer to an event object that was returned by USBH_OS_AllocEvent().
MilliSeconds	Timeout in milliseconds.

Return value

- USBH_OS_EVENT_SINGALED
- Event was signaled.
- USBH_OS_EVENT_TIMEOUT
- Timeout occurred.

Chapter 26

Performance & resource usage

This chapter covers the performance and resource usage of emUSB-Host. It contains information about the memory requirements in typical systems which can be used to obtain sufficient estimates for most target systems.

26.1 Memory footprint

emUSB-Host is designed to fit many kinds of embedded design requirements. Several features can be excluded from a build to get a minimal system. The code size depend on the API functions called by the application. The code was compiled for a Cortex-M4 CPU with size optimization. Note that the values are only valid for an average configuration.

26.1.1 ROM usage

To save code memory, support for isochronous USB transfers in emUSB-Host can be disabled via the compile time switch `USBH_SUPPORT_ISO_TRANSFER`. Currently isochronous transfers are required only to handle audio devices.

The following table shows the approximate ROM requirement of emUSB-Host (compiled using the SEGGER compiler with size optimization):

Component	ROM (iso disabled)	ROM (iso enabled)	Remark
USB core	6.2 KBytes		
HUB Support	3.4 KBytes		
CDC	4.5 KBytes		
Vendor class	3.8 KBytes		
CCID	5.2 KBytes		
FT232	4.6 KBytes		
HID Generic	4.9 KBytes		
HID Mouse Keyboard	6.0 KBytes		
MSD	5.2 KBytes		+ sizeof(file system)
MTP	11.7 KBytes		
Printer	4.7 KBytes		
MIDI	4.8 KBytes		
AUDIO		6.7 KBytes	
LAN using ASIX	8.5 KBytes		+ sizeof(emNET)
LAN using CDC-ECM	6.5 KBytes		+ sizeof(emNET)
LAN using RNDIS	6.8 KBytes		+ sizeof(emNET)
VIDEO		7.6 KBytes	
Driver EHCI	4.4 KBytes	6.2 KBytes	
Driver OHCI	5.7 KBytes	6.9 KBytes	
Driver STM32F4 FS	4.0 KBytes	4.8 KBytes	
Driver STM32F4 HS	4.5 KBytes	5.3 KBytes	
Driver STM32F7 HS	4.7 KBytes	5.4 KBytes	
Driver Kinetis FS	2.7 KBytes		
Driver Renesas RX64	4.4 KBytes	5.0 KBytes	
Driver LPC54xxx HS	2.2 KBytes	3.1 KBytes	
Driver LPC54xxx FS	6.2 KBytes	7.3 KBytes	
Driver MUSB	2.3 KBytes	2.7 KBytes	
Driver SAMV70	2.7 KBytes		

26.1.2 RAM usage

The following table shows the average RAM requirement of emUSB-Host. The actual RAM usage may vary depending on the USB host controller used, the memory architecture of the target, the USB devices connected to emUSB-Host and the type of operations performed with that devices.

Component	RAM
emUSB-Host core incl. one driver	3.8 KByte
For each connected generic HID device	2.8 KByte
For each connected CDC ACM device	4.1 KByte
For each connected Vendor (BULK) device	3.5 KByte
For each connected MSD device	2.3 KByte
For each connected Mouse	4.4 KByte
For each connected external HUB	1.9 KByte
For each connected LAN (ASIX) device	11.1 KByte
For each connected LAN (CDC-ECM) device	18.1 KByte
For each connected LAN (RNDIS) device	13.5 KByte

26.2 Performance

The following values have been tested using the CDC protocol. An emPower evaluation board running emUSB-Device CDC class was connected to the host.

System with Synopsys (USB High-Speed) controller:

Description	Speed
Send speed	34.9 MByte/sec
Receive speed	39.0 MByte/sec

System with EHCI (USB High-Speed) controller:

Description	Speed
Send speed	30.9 MByte/sec
Receive speed	36.0 MByte/sec

System with OHCI (USB Full-Speed) controller:

Description	Speed
Send speed	800 KByte/sec
Receive speed	800 KByte/sec

Chapter 27

Glossary

Term	Definition
BSP	Board support package.
CPU	Central Processing Unit. The “brain” of a microcontroller; the part of a processor that carries out instructions.
DMA	Direct Memory Access.
EOT	End Of Transmission.
FIFO	First-In, First-Out.
ISR	Interrupt Service Routine. The routine is called automatically by the processor when an interrupt is acknowledged. ISRs must preserve the entire context of a task (all registers).
MCU	Microcontroller unit.
MMU	Memory managing unit
NULL packet	See ZLP.
PLL	Phase-locked loop, used for clock generation inside a microcontroller.
RTOS	Real-time Operating System.
RTT	Real-Time Transfer. Method to output information from the target microcontroller as well as sending input to the application at a very high speed without affecting the target’s real time behavior.
Scheduler	The program section of an RTOS that selects the active task, based on which tasks are ready to run, their relative priorities, and the scheduling system being used.
Stack	An area of memory with LIFO storage of parameters, automatic variables, return addresses, and other information that needs to be maintained across function calls. In multitasking systems, each task normally has its own stack.
Superloop	A program that runs in an infinite loop and uses no real-time kernel. ISRs are used for real-time parts of the software.
Task	A program running on a processor. A multitasking system allows multiple tasks to execute independently from one another.
Tick	The OS timer interrupt. Usually equals 1 ms.
ZLP	Zero-Length-Packet

Chapter 28

FAQ

This chapter answers some frequently asked questions.

Q: Do I need a real-time operating system (RTOS) to use the emUSB-Host stack?

A: Yes, an RTOS is required.

Q: Is the emUSB-Host API thread-safe?

A: Not generally. Different devices or endpoints can be handled in different tasks without restrictions. For example a task may read from one endpoint or device while another task is writing to another endpoint or device. If accessing the same resource, the user is responsible for locking API calls against each other.

Q: emUSB-Host does not compile because of missing includes in the USBH_MSD_FS.c file.

A: The USBH_MSD_FS.c file is a file system layer for emUSB-Host's MSD module. This layer is written for the SEGGER file system emFile. If you do not have emFile you can use this file as a sample to write a file system layer for your own file system.

Q: Devices connected directly to my Host work, but do not work when connected through a hub.

A: Please check your USBH_X_Config function. In it you should call USBH_ConfigSupportExternalHubs(1) to enable hub support. Also consider restrictions listed in *Host controller specifics* on page 692.

Q: When I enable all logs via USBH_ConfigMsgFilter(USBH_LOG_FILTER_SET_ALL, ...) my devices no longer enumerate.

A: Enabling too many log outputs can drastically influence the timing of the application up to a point where it may no longer function. It is best practice to limit the number of logs only to the ones you are interested in.